INFOSYS SP / DSE CODING ASSESSMENT · 6 DECEMBER 2026

# Data Structures and Algorithms in Java

Daily worksheets, exercises and mock exams for the 29-day plan

## Volume II · Practice Workbook

Twenty-nine chapters, one per day. Each has the day's plan, a spaced-review list, exercises that you complete by hand, a worksheet for every problem, and an end-of-day error log. Hints and answers are at the back.

Read with: **Volume I · Theory** (same chapter numbers)

Name _____

Prepared for Purvam Prajapati · Edition of 7 November 2026

# Contents


1 Java for Problem Solving: Collections, Input and Output ..... 8

2 Prefix Sums, Difference Arrays and Hashing ..... 16

3 Two Pointers and Sliding Windows ..... 23

4 Sorting and Greedy Choice: Intervals ..... 31

5 Greedy Algorithms with Heaps ..... 38

6 Binary Search and Searching on the Answer ..... 46

7 Monotonic Stacks ..... 55

8 Bits, XOR, Tries and Linear Bases ..... 63

9 Brute Force, Stress Testing and the Mock Exam ..... 73

10 Number Theory for Programmers ..... 81

11 Dynamic Programming I: States, Transitions and Knapsacks ..... 90

12 Dynamic Programming II: Two Strings, Edit Distance and Palindromes ..... 99

13 Dynamic Programming III: Increasing Subsequences and Extra Dimensions ..... 108

14 Dynamic Programming IV: Divisors, Intervals and Games ..... 117

15 Graphs I: Breadth-First Search, Depth-First Search and Topological Order ..... 127

16 Analysing a Problem Before You Code ..... 138

17 Graphs II: Shortest Paths, Union-Find and Spanning Trees ..... 146

18 Trees I: Recursion, Parent Arrays and Ancestors ..... 157

19 Trees II: Tree DP, Rerooting and Small-to-Large ..... 169

20 Fenwick Trees and Segment Trees ..... 180

21 Lazy Segment Trees and the K-Parts DP ..... 189

22 Backtracking, Bitmask DP and Digit DP ..... 198

23 Four Problems, Partial Marks and Mock 3 ..... 208

24 Infosys Repeat Problems I ..... 214

25 Mock 4: The Official Infosys Sample Paper ..... 223

26 The Repair Session and the Hard XOR Repeats ..... 231

27 The Exam Playbook and Mock 5 ..... 237

28 Consolidation: Recognising Patterns and Recalling Templates ..... 245

29 Rest and Logistics: The Day Before the Exam ..... 254

A Hints ..... 259

B Answer key ..... 265

C Error log ..... 310

D Mock scores and SP checkpoints ..... 313

E Notes and scratch paper ..... 314

# How to use this workbook

This workbook is Volume II of the course. Volume I, the theory book, teaches each day's ideas; this volume is where you use them, by hand and against a clock. Both volumes have the same 29 chapters, and each chapter number is a day of the 29-day plan, from Saturday 7 November to Saturday 5 December 2026. Write in it. A workbook that stays clean has not been used.

## A day in the workbook

Every chapter has the same parts, in the order you meet them during the day.

1. **Today's plan.** The day's blocks with their times, the plan's notes for the day and why the day matters for Infosys. Write your start time, end time, energy and sleep in the line below the notes; over four weeks they show when you work best.
2. **Spaced review.** The patterns taught 1, 3, 7 and 14 days ago. Cover the theory book, write each template from memory, then compare it with the listing in the chapter shown and tick *Again*, *Shaky* or *Solid*. "Solid" means it was right the first time.
3. **Exercises.** Work them after reading the theory chapter and before the problems. They move from a complete example in the theory book to a faded one with gaps, and then to work of your own: fill the gaps, trace by hand, judge a statement, design a solution, find a bug, size a method from its limits, and write a template from memory.
4. **Problem worksheets.** One per problem of the day. Fill in the top rows *before* you type any code: the pattern, the target complexity from the largest limit (Theory Appendix A) and the key idea in one sentence.
5. **Re-solve from three days ago.** Pick one problem from three days earlier and solve it again without your old code.
6. **End of day.** The error log for the day and two lines of reflection.

On the five mock days (Chapters 9, 16, 23, 25 and 27) the problem worksheets are replaced by the mock sheet: the rules, a planning grid for the first fifteen minutes, a score sheet and a post-mortem. Do that chapter's exercises after the mock, not before.

## Time limits and hints

Each problem has a time limit: Easy 20 minutes, Medium 40, Hard 60. When it runs out, open the hint for that problem in Appendix A. If you are still stuck ten minutes later, read the editorial, close it, and write the solution yourself. Tick how the problem ended on its worksheet: solved alone, after the hint, after the editorial, or not yet. "Not yet" problems go into the error log.

Core problems come first. The bonus problems are the Specialist Programmer level; attempt them only if the core is done and you still feel sharp.

## Answers

Answers to every exercise are in Appendix B. Check them after you finish a whole exercise, not line by line while you work. Every number in them was produced by running code, so if your number differs, look for the mistake in your own work first, then compare the method. Where an exercise has no single right answer, such as writing a template from memory, the answer says what to check.

## The error log

Every mistake goes into the error log: the problem, its topic, the cause and a rule that would have prevented it. Use one of nine causes: misread statement, wrong idea, overflow or modulo, off-by-one, time limit, input reading, edge case, stack overflow, ran out of time. Write the rule so that you can check it against code in ten seconds (“cast the first factor to `long`”), not as an intention (“be careful”). Each entry gets a resolve date three days later. Each chapter ends with a small log, and Appendix C has full pages for the rest. The repair session of Chapter 26 starts from this log.

## The trackers

Two trackers follow this page. The **progress tracker** has a box for every problem of the plan: tick it when the problem is solved, alone or after a hint. Shaded boxes are bonus problems. The **pattern recall tracker** lists all 44 handbook patterns with the days on which each comes back for review. Write A, S or G (again, shaky, solid) in each cell after the review. Any pattern still at A on Day 26 goes into that day’s repair session.

Appendix D collects the scores of the five mocks on one page, next to the checkpoints that say whether you are on track. Appendix E is squared paper for anything that does not fit elsewhere.

## Stop before you burn out

The plan’s signs that you are tired are two problems in a row past their time limit, reading the same line of a statement twice, and pasting code you do not understand. When you notice them, stop starting new problems, do the review block and end the day. Skip the bonus hour freely: the core problems alone cover every pattern. Sleep at least seven hours, especially before a mock.

# Progress tracker

Tick a box for each problem solved (alone or after a hint). Shaded boxes are bonus problems.

| Day | Date | Topic | Core | Bonus | Hours |
|---|---|---|---|---|---|
| 1 | Sat 7 Nov | Set up, Java toolkit, fast input | ☐☐☐☐☐ | ☐☐☐ | |
| 2 | Sun 8 Nov | Prefix sums, difference arrays, hashing | ☐☐☐☐ | ☐☐☐ | |
| 3 | Mon 9 Nov | Two pointers and sliding window | ☐☐☐☐ | ☐☐☐ | |
| 4 | Tue 10 Nov | Sorting and greedy: intervals | ☐☐☐☐ | ☐☐☐ | |
| 5 | Wed 11 Nov | Greedy with heaps | ☐☐☐☐ | ☐☐☐ | |
| 6 | Thu 12 Nov | Binary search, including on the answer | ☐☐☐☐ | ☐☐☐ | |
| 7 | Fri 13 Nov | Monotonic stacks | ☐☐☐☐ | ☐☐☐ | |
| 8 | Sat 14 Nov | Bits and XOR | ☐☐☐☐ | ☐☐☐ | |
| 9 | Sun 15 Nov | Mock 1 + review | ☐☐☐ | | |
| 10 | Mon 16 Nov | Number theory toolkit | ☐☐☐☐ | ☐☐☐ | |
| 11 | Tue 17 Nov | DP I: states, transitions, knapsack | ☐☐☐☐ | ☐☐☐ | |
| 12 | Wed 18 Nov | DP II: two-string DP | ☐☐☐☐ | ☐☐☐ | |
| 13 | Thu 19 Nov | DP III: LIS and DP with an extra dimension | ☐☐☐ | ☐☐☐ | |
| 14 | Fri 20 Nov | DP IV: divisor, interval and game DP | ☐☐☐☐ | ☐☐☐ | |
| 15 | Sat 21 Nov | Graphs I: BFS, DFS, topological order | ☐☐☐☐ | ☐☐☐ | |
| 16 | Sun 22 Nov | Mock 2 + review | ☐☐☐ | | |
| 17 | Mon 23 Nov | Graphs II: Dijkstra, DSU, MST | ☐☐☐☐ | ☐☐☐ | |
| 18 | Tue 24 Nov | Trees I: recursion, parent arrays, LCA | ☐☐☐☐ | ☐☐☐☐ | |
| 19 | Wed 25 Nov | Trees II: tree DP, rerooting, small-to-large | ☐☐☐☐ | ☐☐☐ | |
| 20 | Thu 26 Nov | Fenwick trees and segment trees | ☐☐☐ | ☐☐☐☐ | |
| 21 | Fri 27 Nov | Lazy segment trees and the K-parts DP | ☐☐☐ | ☐☐☐ | |
| 22 | Sat 28 Nov | Backtracking, bitmask DP, digit DP | ☐☐☐☐ | ☐☐☐ | |
| 23 | Sun 29 Nov | Mock 3 (4 problems) + review | ☐☐☐☐ | | |
| 24 | Mon 30 Nov | Infosys repeat problems I | ☐☐☐☐☐ | ☐☐☐☐☐ | |
| 25 | Tue 1 Dec | Mock 4: the official Infosys sample paper | ☐☐☐ | ☐ | |
| 26 | Wed 2 Dec | Repair day + the hard XOR repeat | ☐☐ | ☐ | |
| 27 | Thu 3 Dec | Mock 5: dress rehearsal | ☐☐☐☐ | | |
| 28 | Fri 4 Dec | Consolidate | ☐☐☐ | | |
| 29 | Sat 5 Dec | Rest and logistics | ☐ | | |

# Pattern recall tracker

Each pattern comes back for review 1, 3, 7 and 14 days after it is taught; the day numbers are filled in. Write A (again), S (shaky) or G (solid, good) in each cell after the review. Any pattern still at A on Day 26 goes into the repair session.

| Pattern | Taught | +1 | +3 | +7 | +14 | Notes |
|---|---|---|---|---|---|---|
| Java collections toolkit | 1 | 2 | 4 | 8 | 15 | |
| Fast input and output (FastScanner) | 1 | 2 | 4 | 8 | 15 | |
| Linked list moves | 1 | 2 | 4 | 8 | 15 | |
| Prefix sums + HashMap | 2 | 3 | 5 | 9 | 16 | |
| Difference array (many range adds) | 2 | 3 | 5 | 9 | 16 | |
| Two pointers on a sorted array | 3 | 4 | 6 | 10 | 17 | |
| Sliding window (variable size) | 3 | 4 | 6 | 10 | 17 | |
| Window maximum (monotonic deque) | 3 | 4 | 6 | 10 | 17 | |
| Intervals: sort, then sweep | 4 | 5 | 7 | 11 | 18 | |
| Greedy with a heap | 5 | 6 | 8 | 12 | 19 | |
| Binary search: lower and upper bound | 6 | 7 | 9 | 13 | 20 | |
| Binary search on the answer | 6 | 7 | 9 | 13 | 20 | |
| Monotonic stack | 7 | 8 | 10 | 14 | 21 | |
| Bit tricks and XOR | 8 | 9 | 11 | 15 | 22 | |
| Binary trie for XOR | 8 | 9 | 11 | 15 | 22 | |
| XOR basis (maximum subset XOR) | 8 | 9 | 11 | 15 | 22 | |
| Brute force first, then stress test | 9 | 10 | 12 | 16 | 23 | |
| gcd, fast power, inverse, nCr | 10 | 11 | 13 | 17 | 24 | |
| Sieve, prime factors, square-free kernel | 10 | 11 | 13 | 17 | 24 | |
| DP recipe: state, transition, base, answer | 11 | 12 | 14 | 18 | 25 | |
| Knapsack family | 11 | 12 | 14 | 18 | 25 | |
| Two-string DP (LCS, edit distance) | 12 | 13 | 15 | 19 | 26 | |
| Longest increasing subsequence | 13 | 14 | 16 | 20 | 27 | |
| Interval DP and game DP | 14 | 15 | 17 | 21 | 28 | |
| BFS: fewest edges, grids, many sources | 15 | 16 | 18 | 22 | 29 | |
| DFS without recursion, components | 15 | 16 | 18 | 22 | 29 | |
| Topological order (Kahn) and DAG DP | 15 | 16 | 18 | 22 | 29 | |
| 0-1 BFS | 15 | 16 | 18 | 22 | 29 | |
| Dijkstra | 17 | 18 | 20 | 24 | | |
| Union-find (DSU) and Kruskal | 17 | 18 | 20 | 24 | | |
| Binary tree recursion (LeetCode TreeNode) | 18 | 19 | 21 | 25 | | |
| General trees from a parent array (no recursion) | 18 | 19 | 21 | 25 | | |
| LCA with binary lifting | 18 | 19 | 21 | 25 | | |
| Tree DP (dp[v][state]) | 19 | 20 | 22 | 26 | | |
| Rerooting (an answer for every root) | 19 | 20 | 22 | 26 | | |

| Pattern | Taught | +1 | +3 | +7 | +14 | Notes |
|---|---|---|---|---|---|---|
| Small-to-large merging | 19 | 20 | 22 | 26 | | |
| Euler tour (subtree = range) | 20 | 21 | 23 | 27 | | |
| Fenwick tree (BIT) | 20 | 21 | 23 | 27 | | |
| Segment tree (iterative) | 20 | 21 | 23 | 27 | | |
| Lazy segment tree (range add, range max) | 21 | 22 | 24 | 28 | | |
| Split into K parts (dp[k][i]) | 21 | 22 | 24 | 28 | | |
| Backtracking and subset enumeration | 22 | 23 | 25 | 29 | | |
| Bitmask DP | 22 | 23 | 25 | 29 | | |
| Digit DP | 22 | 23 | 25 | 29 | | |

DAY 1 OF 29 · SATURDAY 7 NOVEMBER 2026 · LEARNING DAY · SETUP

1

# Java for Problem Solving: Collections, Input and Output

*Accounts, the Java collections you will use every day, and how FastScanner replaces Scanner.*

**Read first** · Theory Chapter 1
**Patterns** · Java collections toolkit, Fast input and output (FastScanner), Linked list moves

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 1; type each new listing once and run it | 40 min |
| ☐ | 1.P1 LC 1 · Two Sum | 20 min |
| ☐ | 1.P2 LC 242 · Valid Anagram | 20 min |
| ☐ | 1.P3 LC 20 · Valid Parentheses | 20 min |
| ☐ | 1.P4 LC 347 · Top K Frequent Elements | 40 min |
| ☐ | 1.P5 LC 729 · My Calendar I | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 1.P6, 1.P7, 1.P8* | *60 min* |
| | **Core total (estimate)** | **3.9 h** |

**NOTES FOR THE DAY**

- Submit the Infosys date/location form (deadline **8 Nov**) and choose 6 Dec.
- Create free accounts on LeetCode and Codeforces. In LeetCode's editor, set the language to Java.
- Read Theory Chapter 1 (fast input): why Scanner is slow and how FastScanner works. Type FastScanner once from memory, paste it into OneCompiler, put `3 10 20 30` in the STDIN box and check that it prints 60.
- Read the handbook cards below. From tomorrow, the handbook's "Today's list" tells you what to review.
- Solve the problems in LeetCode's editor: click *Run* first, then *Submit*.

**Why it matters for Infosys.** The exam gives you a plain editor and standard input. Being quick with Java collections and input reading saves minutes on every later problem.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Exercises

Work these after reading Theory Chapter 1 and before the problem worksheets. The first three practise the templates you will type all month (the input reader, the stack check and the ordered-map look-up); the rest test the Java details that silently cost marks and ask you to design and debug small programs of your own.

### EXERCISE 1.1 Fill the gaps: FastScanner

Complete the reader from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 1.6*, type it into OneCompiler, put `3 10 20 30` in the STDIN box and check that it prints 60.

```
String next() throws IOException {
    while (_______________________________) {        // (1) current line used up
        String line = br.readLine();
        if (line == null) _______________;           // (2) end of input
        st = _______________________________;        // (3) split the new line
    }
    return st.nextToken();
}

public static void main(String[] args) throws IOException {
    FastScanner fs = new FastScanner(System.in);
    PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
    int n = fs.nextInt();
    long sum = 0;
    for (int i = 0; i < n; i++) sum += _______________;   // (4)
    out.println(sum);
    ______________;                                      // (5) or nothing is printed
}
```

**Why does the same program work whether the four numbers are on one line or on four lines?**

### EXERCISE 1.2 Trace the stack check

Trace *Theory Listing 1.2* on `([]{()})]`. For each character write the action (push which closer, or pop which closer and whether it matches) and the stack afterwards, from bottom to top. Stop at the row where the method returns.

| char | action | stack (bottom → top) |
|---|---|---|
| ( | | |
| [ | | |
| ] | | |
| { | | |
| ( | | |
| ) | | |
| } | | |
| ) | | |
| ] | | |

**Which branch of the `else if` returns `false` here, and what would the method return for `([]{()})` (the same string without the last character)?**

**EXERCISE 1.3 A faded trace: My Calendar**

Run *Theory Listing 1.4* on the requests below, in order. The first three rows are done. For each request write the floor and ceiling neighbours of its start as start (end), and whether it is accepted.

| request | floorKey (end) | ceilingKey (end) | accepted? |
|---|---|---|---|
| [3, 8) | null | null | yes |
| [8, 12) | 3 (8) | null | yes |
| [1, 4) | null | 3 (8) | no: [3, 8) starts before 4 |
| [12, 15) | | | |
| [10, 13) | | | |
| [0, 3) | | | |
| [15, 16) | | | |
| [14, 16) | | | |

**Write the final contents of the map.**

**EXERCISE 1.4 Concept check: what does Java do?**

For each line, write the value or the outcome, and one line of reason. Do it on paper first; then run the lines to check.

1. `int x = 46341 * 46341;`
2. `-10 % 4` and `Math.floorMod(-10, 4)`
3. `Integer a = 127, b = 127; a == b` and `Integer c = 200, d = 200; c == d`
4. `String t = "ab"; t += "c";` then `t == "abc"` and `t.equals("abc")`
5. `List<Integer> l = new ArrayList<>(Arrays.asList(5, 6, 7)); l.remove(1);` What is `l`?
6. `new ArrayDeque<Integer>().push(null);`
7. `'a' + 'b'` and `(char) ('a' + 25)`
8. Printing a `PriorityQueue` shows its elements in sorted order. True or false?

**EXERCISE 1.5 Complexity and limits**

For each situation, say whether the plan fits in about one second of Java, using *Theory Table 1.2* and the figure of $10^8$–$3 \cdot 10^8$ basic steps per second. Show the estimate.

1. $n = 2 \cdot 10^5$; check every pair $i < j$.
2. $n = 20$; try every subset, doing $O(n)$ work per subset.
3. $n = 10^5$; sort, then one binary search per element.
4. $n = 3\,000$; three nested loops over all triples $(i, j, k)$.
5. A $5\,000 \times 5\,000$ table of `long` values: how much memory?

**EXERCISE 1.6 Design before you code: a parking garage**

A garage has spots numbered $1..m$ ($m \le 10^5$). There are $q \le 2 \cdot 10^5$ events. "Arrive $p$": a car wants the smallest free spot with number $\ge p$; it parks there, or is turned away (print $-1$) if none is free. "Leave $s$": the car in spot $s$ leaves. Choose the data structure, the call that answers "arrive", and the total complexity. Then process these events for $m = 6$ and write the output of each arrival.

`arrive 2, arrive 2, arrive 5, arrive 5, leave 3, arrive 2, arrive 6, arrive 1, leave 6, arrive 4`

**EXERCISE 1.7 Find the bug**

This method should say whether the values $x$ and $y$ occur equally often in $a$. It passes small tests. Find an input on which it returns the wrong answer, explain why, and fix it with one change.

```
static boolean sameFrequency(int[] a, int x, int y) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int v : a) cnt.merge(v, 1, Integer::sum);
    return cnt.get(x) == cnt.get(y);
}
```

**EXERCISE 1.8 Write it from memory: list moves**

Without looking at *Theory Listing 1.8*, write `reverse` and `hasCycle`. Then answer: by *Theory Theorem 1.11*, after how many steps do slow and fast meet on a list with a tail of $\mu = 4$ nodes and a cycle of $\lambda = 6$? With $\mu = 10$ and $\lambda = 3$?

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 5 core problems, 3 bonus.

**1.P1** **LC 1 · Two Sum** EASY CORE — **20 min**

`leetcode.com/problems/two-sum · LeetCode`

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P1*

**1.P2** **LC 242 · Valid Anagram** EASY CORE — **20 min**

`leetcode.com/problems/valid-anagram · LeetCode`

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P2*

### 1.P3 LC 20 · Valid Parentheses EASY CORE

20 min

leetcode.com/problems/valid-parentheses · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 1.P3*

### 1.P4 LC 347 · Top K Frequent Elements MEDIUM CORE

40 min

leetcode.com/problems/top-k-frequent-elements · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 1.P4*

### 1.P5 LC 729 · My Calendar I MEDIUM CORE

40 min

leetcode.com/problems/my-calendar-i · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 1.P5*

### 1.P6 LC 206 · Reverse Linked List EASY BONUS

20 min

leetcode.com/problems/reverse-linked-list · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 1.P6*

**1.P7** **LC 141 · Linked List Cycle** EASY BONUS **20 min**

`leetcode.com/problems/linked-list-cycle · LeetCode`

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P7*

**1.P8** **LC 49 · Group Anagrams** MEDIUM BONUS **40 min**

`leetcode.com/problems/group-anagrams · LeetCode`

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 1.P8*

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 2, Prefix sums, difference arrays, hashing.*

DAY 2 OF 29 · SUNDAY 8 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

2

# Prefix Sums, Difference Arrays and Hashing

*Count subarrays by remembering earlier prefix values in a HashMap.*

**Read first** · Theory Chapter 2
**Patterns** · Prefix sums + HashMap, Difference array (many range adds)

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 2; type each new listing once and run it | 40 min |
| ☐ | 2.P1 LC 560 · Subarray Sum Equals K | 40 min |
| ☐ | 2.P2 LC 974 · Subarray Sums Divisible by K | 40 min |
| ☐ | 2.P3 LC 1524 · Number of Sub-arrays With Odd Sum | 40 min |
| ☐ | 2.P4 LC 525 · Contiguous Array | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 2.P5, 2.P6, 2.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- `P[i] = a[0] + … + a[i-1]`, so `sum(l..r) = P[r+1] - P[l]`. Use `long`.
- Counting subarrays with sum k: at each index, count earlier prefixes equal to `P - k`. Seed the map with `{0: 1}`.
- For "divisible by k", store `Math.floorMod(P, k)`; Java's `%` can be negative.
- Many range adds, read once: `d[l] += v; d[r+1] -= v`, then take a running sum.

**Why it matters for Infosys.** Q1 is often an array problem with a counting twist. Prefix sums + HashMap is one of the most reported Infosys patterns.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Java collections toolkit | Ch. 1 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Fast input and output (FastScanner) | Ch. 1 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Linked list moves | Ch. 1 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 2 and before the problem worksheets. They move from a complete worked example (in the theory book) to a faded one with gaps, and then to questions you answer on your own.

### EXERCISE 2.1 Fill the gaps

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 2.2*.

```
static long countSubarraysWithSum(int[] a, long k) {
    Map<Long, Integer> seen = new HashMap<>();
    ______________________________;          // (1) the empty prefix
    long prefix = 0, count = 0;
    for (int x : a) {
        prefix += x;
        count += ______________________________;   // (2) earlier prefixes that match
        ______________________________;            // (3) record this prefix
    }
    return count;
}
```

**Why must line (2) come before line (3)?**

### EXERCISE 2.2 Trace the prefix-count scan

Trace *Theory Listing 2.2* on $a$ = [3, 4, −7, 3, 1, 3, 1, −4, −2, −2] with $k$ = 7. The map starts as {0: 1}. For each row, write the prefix after reading $a[i]$, the value looked up, how many earlier prefixes matched, and the running total.

| i | a[i] | prefix | prefix − k | found | total |
|---|---|---|---|---|---|
| 0 | 3 | | | | |
| 1 | 4 | | | | |
| 2 | −7 | | | | |
| 3 | 3 | | | | |
| 4 | 1 | | | | |
| 5 | 3 | | | | |

| i | a[i] | prefix | prefix − k | found | total |
|---|---|---|---|---|---|
| 6 | 1 | | | | |
| 7 | −4 | | | | |
| 8 | −2 | | | | |
| 9 | −2 | | | | |

**Row 5 finds more than one match. Which subarrays ending at index 5 have sum 7?**

### EXERCISE 2.3 A faded trace: remainders

Count the subarrays of $a = [4, 5, 0, -2, -3, 1]$ whose sum is divisible by $k = 5$, using *Theory Listing 2.3*. The first three rows are done. Fill in the rest. Use `Math.floorMod`, so every remainder is in 0..4.

| i | a[i] | remainder of prefix | cnt[remainder] before | total | cnt after this row |
|---|---|---|---|---|---|
| 0 | 4 | 4 | 0 | 0 | [1, 0, 0, 0, 1] |
| 1 | 5 | 4 | 1 | 1 | [1, 0, 0, 0, 2] |
| 2 | 0 | 4 | 2 | 3 | [1, 0, 0, 0, 3] |
| 3 | −2 | | | | |
| 4 | −3 | | | | |
| 5 | 1 | | | | |

**At which row would `(prefix + x) % 5` (plain `%`) give a negative value, and what would happen?**

### EXERCISE 2.4 Difference array by hand

An array of six zeros receives four updates: +2 on [0..3], +5 on [2..2], −1 on [1..5] and +3 on [4..5]. Fill in the difference array $d$ (seven slots) after all updates, then the final array $a$ by running sums. Check one entry of $a$ by counting the updates that cover it.

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| d | | | | | | | |
| a | | | | | | | — |

### EXERCISE 2.5 Two-dimensional prefix sums

For the grid below, fill in the prefix table $S$ (with its row and column of zeros), then compute the sum of rows 1..2 and columns 1..2 with the formula of *Theory Theorem 2.4*. Check it by adding the four cells directly.

| g | c0 | c1 | c2 |
|---|---|---|---|
| r0 | 3 | 0 | 1 |
| r1 | 5 | 6 | 3 |
| r2 | 1 | 2 | 0 |

| S | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | | | |
| 2 | 0 | | | |
| 3 | 0 | | | |

**Sum of rows 1..2, columns 1..2 = S[3][3] − S[1][3] − S[3][1] + S[1][1] =**

### EXERCISE 2.6 Concept check

Answer true or false, and give a one-line reason.

1. If every element is positive, a sliding window can count subarrays with sum $k$ in $O(n)$.
2. For "longest subarray with sum $k$", the map should store the latest index of each prefix.
3. `HashMap<Long, Integer>` is a better choice than `int[]` when the keys are remainders $0..k-1$ with $k \le 10^6$.
4. A difference array can answer "what is $a[4]$ now?" between two updates in $O(1)$.
5. The number of odd-sum subarrays equals (number of even prefixes) × (number of odd prefixes), with $P[0]$ counted as even.

### EXERCISE 2.7 Design before you code

A binary string $s$ of length up to $10^5$ is given. Count the substrings that contain as many 0s as 1s. Using *Theory Table 2.1*, say what you store for each prefix, what you look up, and whether you store a count or an index. Then compute the answer for $s =$ `0011010` by hand.

**EXERCISE 2.8** **Find the bug**

This method is meant to solve LeetCode 974, but it fails on some inputs. Find the input type that breaks it, give a failing example, and fix it with one change.

```
static int subarraysDivByKBuggy(int[] a, int k) {
    int[] cnt = new int[k];
    cnt[0] = 1;
    int prefix = 0, total = 0;
    for (int x : a) {
        prefix = (prefix + x) % k;
        if (prefix < 0) break;          // "can't happen"
        total += cnt[prefix];
        cnt[prefix]++;
    }
    return total;
}
```

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**2.P1** **LC 560 · Subarray Sum Equals K** MEDIUM CORE **40 min**

leetcode.com/problems/subarray-sum-equals-k · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P1*

### 2.P2 LC 974 · Subarray Sums Divisible by K MEDIUM CORE

40 min

leetcode.com/problems/subarray-sums-divisible-by-k · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 2.P2*

### 2.P3 LC 1524 · Number of Sub-arrays With Odd Sum MEDIUM CORE

40 min

leetcode.com/problems/number-of-sub-arrays-with-odd-sum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 2.P3*

### 2.P4 LC 525 · Contiguous Array MEDIUM CORE

40 min

leetcode.com/problems/contiguous-array · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 2.P4*

### 2.P5 LC 1109 · Corporate Flight Bookings MEDIUM BONUS

40 min

leetcode.com/problems/corporate-flight-bookings · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 2.P5*

**2.P6** **LC 304 · Range Sum Query 2D - Immutable** MEDIUM BONUS **40 min**

leetcode.com/problems/range-sum-query-2d-immutable · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P6*

**2.P7** **CF 466C · Number of Ways** CF 1700 BONUS **45 min**

codeforces.com/problemset/problem/466/C · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 2.P7*

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 3, Two pointers and sliding window.*

DAY 3 OF 29 · MONDAY 9 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

3

# Two Pointers and Sliding Windows

*Grow the window on the right; shrink it on the left while it is invalid.*

**Read first** · Theory Chapter 3
**Patterns** · Two pointers on a sorted array, Sliding window (variable size), Window maximum (monotonic deque)

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 3; type each new listing once and run it | 40 min |
| ☐ | 3.P1 LC 3 · Longest Substring Without Repeating Characters | 40 min |
| ☐ | 3.P2 LC 1004 · Max Consecutive Ones III | 40 min |
| ☐ | 3.P3 LC 424 · Longest Repeating Character Replacement | 40 min |
| ☐ | 3.P4 LC 15 · 3Sum | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 3.P5, 3.P6, 3.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- A sliding window works when shrinking a valid window keeps it valid.
- Template: `for r: add(a[r]); while (bad) remove(a[l++]); best = max(best, r - l + 1);`
- "Exactly K" = atMost(K) − atMost(K − 1).
- Sorted array with a target sum: two pointers from both ends; skip equal values to avoid duplicates.

**Why it matters for Infosys.** The Infosys "Andy's vacation" problem (cancel at most K obligations) is LeetCode 1004 on a calendar.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Prefix sums + HashMap | Ch. 2 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Difference array (many range adds) | Ch. 2 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 3 and before the problem worksheets. The first three are the window and deque templates by hand; then you test rules for monotonicity, count "exactly K" windows, design a two-deque window of your own and find a bug that only shows up on some inputs.

### EXERCISE 3.1 Fill the gaps: the window template

Complete the template from memory, then compare with *Theory Listing 3.3*. Each blank is one expression or statement.

```
static int longestAtMostKDistinct(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
    for (int l = 0, r = 0; r < a.length; r++) {
        ________________________________;                    // (1) add a[r]
        while (________________________) {                   // (2) the window is invalid
            if (cnt.merge(a[l], -1, Integer::sum) == 0) _____________;   // (3)
            ______;                                          // (4)
        }
        best = ________________________________;             // (5)
    }
    return best;
}
```

**Which line would you change to count all valid windows instead of finding the longest, and how?**

### EXERCISE 3.2 Trace the monotonic deque

Trace *Theory Listing 3.9* on $a = [3, 1, 5, 2, 2, 6, 0, 4]$ with $k = 3$. Write the **indices** in the deque after each step, front first, and the window maximum once $i \geq 2$.

| i | a[i] | indices removed at back | index removed at front | deque (indices) | max |
|---|---|---|---|---|---|
| 0 | 3 | | | | — |
| 1 | 1 | | | | — |
| 2 | 5 | | | | |
| 3 | 2 | | | | |
| 4 | 2 | | | | |
| 5 | 6 | | | | |
| 6 | 0 | | | | |
| 7 | 4 | | | | |

**At $i = 4$ the new value equals the value at the back. What does the `<=` in the listing do with the older copy, and why is that safe?**

### EXERCISE 3.3 A faded trace: at most two distinct values

Run *Theory Listing 3.3* on $a = [1, 2, 1, 3, 3, 2, 2, 4]$ with $k = 2$. The first three rows are done; fill in the rest.

| r | a[r] | l before → after | counts in the window | length | best |
|---|---|---|---|---|---|
| 0 | 1 | 0 → 0 | {1: 1} | 1 | 1 |
| 1 | 2 | 0 → 0 | {1: 1, 2: 1} | 2 | 2 |
| 2 | 1 | 0 → 0 | {1: 2, 2: 1} | 3 | 3 |
| 3 | 3 | | | | |
| 4 | 3 | | | | |
| 5 | 2 | | | | |
| 6 | 2 | | | | |
| 7 | 4 | | | | |

### EXERCISE 3.4 Concept check: is the rule monotone?

For each rule, say whether it is monotone in the sense of *Theory Definition 3.2* (every sub-window of a valid window is valid), and give a one-line reason or a counterexample. Then say which loop you would use: grow-and-shrink, shrink-while-valid, or neither.

1. The window contains at most 2 zeros.
2. The window's sum is at least 10 (all values positive).
3. The window's sum is exactly 10 (all values positive).
4. The window's maximum minus its minimum is at most 3.
5. The window contains at least one copy of every vowel.
6. The window's sum is at most 10 (values may be negative).

### EXERCISE 3.5 Exactly K by subtraction

Count the subarrays of $a = [2, 2, 1, 3, 1]$ that contain exactly two distinct values, as atMost(2) − atMost(1). For each scan, write $l$ after shrinking and the number of windows added at each $r$. Then list the subarrays to check.

| r | a[r] | atMost(2): l | added | atMost(1): l | added |
|---|---|---|---|---|---|
| 0 | 2 | | | | |
| 1 | 2 | | | | |
| 2 | 1 | | | | |
| 3 | 3 | | | | |
| 4 | 1 | | | | |
| total | | | | | |

**Exactly two distinct values: atMost(2) − atMost(1) =**

### EXERCISE 3.6 Design before you code: a window with two deques

Given $a[0..n-1]$ with $n \le 10^5$ and an integer $d \ge 0$, find the longest subarray whose maximum and minimum differ by at most $d$. Show that the rule is monotone, say what state the window needs so that "max − min" is available in O(1) amortised, describe what happens when $l$ advances, and give the complexity. Then find the answer for $a = [4, 6, 5, 9, 7, 8, 3, 8]$ and $d = 2$.

### EXERCISE 3.7 Find the bug

This version of the template passed both samples it was tried on. Find an input on which it fails, say how it fails, and fix it.

```
static int longestAtMostKDistinctBuggy(int[] a, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    int best = 0;
```

```
    for (int l = 0, r = 0; r < a.length; r++) {
        cnt.merge(a[r], 1, Integer::sum);
        while (cnt.size() > k) {
            cnt.merge(a[l], -1, Integer::sum);
            l++;
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

**EXERCISE 3.8** **Complexity and limits: which tool?**

For each problem, name the technique from *Theory Table 3.1*, the target complexity, and the one fact about the input that decides it.

1. $n \le 10^5$, values in $[-10^4, 10^4]$: count subarrays with sum exactly $k$.
2. $n \le 10^5$, values in $[1, 10^4]$: the shortest subarray with sum $\ge S$.
3. $n \le 3\,000$: count the triples $i < j < k$ with $a[i] + a[j] + a[k] < T$.
4. $n \le 10^6$, $k \le n$: the minimum of every window of size $k$.
5. A string of length $\le 10^5$: the longest substring with at most 3 distinct characters.

**EXERCISE 3.9** **Write it from memory: the window maximum**

Without looking, write `windowMax` with the monotonic deque. Then state, in two sentences, the invariant that makes the front the maximum and the reason the total work is at most $2n$ deque operations.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**3.P1** **LC 3 · Longest Substring Without Repeating Characters** MEDIUM CORE — **40 min**

`leetcode.com/problems/longest-substring-without-repeating-characters` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P1*

**3.P2** **LC 1004 · Max Consecutive Ones III** MEDIUM CORE — **40 min**

`leetcode.com/problems/max-consecutive-ones-iii` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P2*

### 3.P3 LC 424 · Longest Repeating Character Replacement MEDIUM CORE — 40 min

leetcode.com/problems/longest-repeating-character-replacement · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P3*

### 3.P4 LC 15 · 3Sum MEDIUM CORE — 40 min

leetcode.com/problems/3sum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P4*

### 3.P5 LC 992 · Subarrays with K Different Integers HARD BONUS — 60 min

leetcode.com/problems/subarrays-with-k-different-integers · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P5*

### 3.P6 LC 239 · Sliding Window Maximum HARD BONUS — 60 min

leetcode.com/problems/sliding-window-maximum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P6*

**3.P7** **LC 76 · Minimum Window Substring** HARD BONUS **60 min**

`leetcode.com/problems/minimum-window-substring` · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 3.P7*

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 4, Sorting and greedy: intervals.*

DAY 4 OF 29 · TUESDAY 10 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

4

# Sorting and Greedy Choice: Intervals

*Sort by the right key, then make one decision per element. Check the greedy by swapping two choices.*

**Read first** · Theory Chapter 4
**Patterns** · Intervals: sort, then sweep

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 4; type each new listing once and run it | 40 min |
| ☐ | 4.P1 LC 56 · Merge Intervals | 40 min |
| ☐ | 4.P2 LC 435 · Non-overlapping Intervals | 40 min |
| ☐ | 4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups | 40 min |
| ☐ | 4.P4 LC 179 · Largest Number | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 4.P5, 4.P6, 4.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- Exchange argument: if swapping two neighbouring choices never helps, the sort order is optimal.
- Most non-overlapping intervals: sort by END. Fewest platforms or rooms: the busiest moment, found with a sweep.
- Comparators: `Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1]))`, never `x[1] - y[1]`.
- Largest number: compare `b + a` with `a + b` as strings.

**Why it matters for Infosys.** Q2 is often a greedy problem. Minimum platforms and Largest number are both on the most-reported Infosys list.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Two pointers on a sorted array | Ch. 3 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Sliding window (variable size) | Ch. 3 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Window maximum (monotonic deque) | Ch. 3 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Java collections toolkit | Ch. 1 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Fast input and output (FastScanner) | Ch. 1 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Linked list moves | Ch. 1 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 4 and before the problem worksheets. They practise the three interval templates, then ask you to write an exchange argument, refute a false greedy, order strings for the Largest Number problem and design a greedy of your own.

### EXERCISE 4.1 Fill the gaps: merge and schedule

Complete both templates from memory, then compare with *Theory Listing 4.3* and *Theory Listing 4.4*.

```
static int[][] merge(int[][] iv) {
    Arrays.sort(iv, (x, y) -> _________________________________);    // (1) sort key
    List<int[]> out = new ArrayList<>();
    for (int[] cur : iv) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last != null && _______________) last[1] = _______________;   // (2) (3)
        else out.add(new int[]{cur[0], cur[1]});
    }
    return out.toArray(new int[0][]);
}

static int maxNonOverlapping(int[][] iv) {
    Arrays.sort(iv, (x, y) -> _________________________________);    // (4) sort key
    int count = 0;
    long end = Long.MIN_VALUE;
    for (int[] cur : iv) {
        if (_______________) { count++; end = cur[1]; }            // (5) touching allowed
    }
    return count;
}
```

**Why does `merge` sort by start while `maxNonOverlapping` sorts by end? What does `merge` return for [1, 2], [2, 3]?**

### EXERCISE 4.2 Trace the event sweep

Six meetings are closed intervals: [1, 4], [2, 3], [4, 6], [6, 8], [3, 5], [7, 9]. List the twelve events in the order *Theory Listing 4.6* processes them (equal times: starts first), with the running count.

| time | event | busy | best | time | event | busy | best |
|---|---|---|---|---|---|---|---|
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

| time | event | busy | best | time | event | busy | best |
|---|---|---|---|---|---|---|---|
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

**How many rooms are needed? How many if a room freed at time $t$ may be used by a meeting starting at time $t$?**

**EXERCISE 4.3** **A faded trace: the heap of end times**

Run *Theory Listing 4.7* on the same six meetings. They are processed in order of start; the first two rows are done.

| meeting | new room or reuse (freed at) | heap after (sorted) |
|---|---|---|
| [1, 4] | new room | {4} |
| [2, 3] | new room | {3, 4} |
| [3, 5] | | |
| [4, 6] | | |
| [6, 8] | | |
| [7, 9] | | |

**EXERCISE 4.4** **Write an exchange argument**

One machine must run $n$ jobs, one at a time; job $i$ takes $t_i$ minutes. The *completion time* of a job is the minute it finishes. Show with an exchange of two **neighbouring** jobs that running the jobs in increasing order of $t$ minimises the sum of completion times. Then check by listing all six orders of $t = (3, 1, 2)$ with their sums.

### EXERCISE 4.5 Refute a greedy

"To pay an amount with the fewest coins, always take the largest coin that fits." Find the smallest amount for which this fails with coins {1, 3, 4}, and give the greedy's coins and an optimal set. Then name a coin system for which you believe the rule is correct, and say how you would gain confidence without a proof.

### EXERCISE 4.6 Order strings for the Largest Number

Sort "8", "89", "898", "9" and "0" with the comparator of *Theory Section 4.6*. For each neighbouring pair in your answer, show the two concatenations you compared. Then compute $v(x)$ for "89" and "898" and check that it agrees. What would the method return for [0, 0]?

### EXERCISE 4.7 Design before you code: the fewest coins to add

You have coins, sorted, and a target $n \le 10^9$. Find the fewest coins you must **add** (any values you like) so that every total $1..n$ can be paid with a subset of all the coins. Use *Theory Lemma 4.7*: describe the state, the step when the next coin fits, the step when it does not, and why the number of added coins is $O(\log n)$. Then run your method on coins [1, 5, 10] with $n = 20$.

**EXERCISE 4.8** **Concept check: sorting in Java**

True or false, with a one-line reason.

1. `Arrays.sort(int[])` is stable.
2. `Arrays.sort(Integer[])` takes O($n$ log $n$) time in the worst case on every JDK.
3. `(x, y) -> x[0] - y[0]` is a correct comparator whenever all values are between $-10^9$ and $10^9$.
4. Sorting $10^6$ `Integer` objects costs about the same as sorting an `int[]` of the same size.
5. A comparison sort can sort any 5 distinct values with at most 6 comparisons.
6. If a greedy matches a brute force on 3,000 random inputs of size 4, it is correct.

**EXERCISE 4.9** **Two passes by hand**

Apply *Theory Listing 4.8* to $h$ = [2, 5, 5, 0, 3, 4, 1]: write the array after each pass, then check that neighbours differ by at most 1 and that no entry exceeds its cap.

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| *h* | 2 | 5 | 5 | 0 | 3 | 4 | 1 |
| after pass 1 | | | | | | | |
| after pass 2 | | | | | | | |

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**4.P1** **LC 56 · Merge Intervals** MEDIUM CORE — **40 min**

`leetcode.com/problems/merge-intervals` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 4.P1*

### 4.P2 LC 435 · Non-overlapping Intervals MEDIUM CORE — 40 min

leetcode.com/problems/non-overlapping-intervals · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 4.P2*

### 4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups MEDIUM CORE — 40 min

leetcode.com/problems/divide-intervals-into-minimum-number-of-groups · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 4.P3*

### 4.P4 LC 179 · Largest Number MEDIUM CORE — 40 min

leetcode.com/problems/largest-number · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 4.P4*

### 4.P5 LC 1798 · Maximum Number of Consecutive Values You Can Make MEDIUM BONUS — 40 min

leetcode.com/problems/maximum-number-of-consecutive-values-you-can-make · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 4.P5*

**4.P6 LC 452 · Minimum Number of Arrows to Burst Balloons** MEDIUM BONUS — 40 min

leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 4.P6*

**4.P7 LC 135 · Candy** HARD BONUS — 60 min

leetcode.com/problems/candy · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 4.P7*

## Re-solve from three days ago

Pick one problem from Day 1 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 1.P1 LC 1 · Two Sum ☐ 1.P2 LC 242 · Valid Anagram ☐ 1.P3 LC 20 · Valid Parentheses
☐ 1.P4 LC 347 · Top K Frequent Elements ☐ 1.P5 LC 729 · My Calendar I

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 5, Greedy with heaps.*

DAY 5 OF 29 · WEDNESDAY 11 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

5

# Greedy Algorithms with Heaps

*Unlock options in sorted order, keep the unlocked ones in a heap, undo your worst choice when a limit breaks.*

**Read first** · Theory Chapter 5
**Patterns** · Greedy with a heap

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 5; type each new listing once and run it | 40 min |
| ☐ | 5.P1 LC 502 · IPO | 60 min |
| ☐ | 5.P2 LC 1642 · Furthest Building You Can Reach | 40 min |
| ☐ | 5.P3 LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array | 60 min |
| ☐ | 5.P4 LC 1046 · Last Stone Weight | 20 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 5.P5, 5.P6, 5.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- "Best among the options unlocked so far": sort by the unlock key and push options into a `PriorityQueue` as they unlock.
- Regret greedy: take everything, remember it in a heap, and when you break a limit, drop the worst item.
- Max-heap: `new PriorityQueue<>(Collections.reverseOrder())`.
- Fewest "subtract 1 on a subarray" operations to reach zero = Σ max(0, a[i] − a[i−1]).

**Why it matters for Infosys.** The RPG monsters problem (11 reports) is this pattern. The Infosys "make the array zero" problem builds on LeetCode 1526.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Intervals: sort, then sweep | Ch. 4 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Prefix sums + HashMap | Ch. 2 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Difference array (many range adds) | Ch. 2 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 5 and before the problem worksheets. The first three take the heap and the regret greedy apart by hand. The rest ask you to judge statements about `PriorityQueue`, design an unlock-and-heap simulation, find a bug that compiles, choose between approaches under given limits, and finally write the templates from memory.

### EXERCISE 5.1 Fill the gaps: the heap's repairs

Complete the three methods from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 5.1*.

```
int poll() {
    int top = a[0];
    a[0] = _____________;                       // (1) refill the root
    siftDown(0);
    return top;
}

private void siftUp(int i) {
    while (i > 0) {
        int p = _____________;                  // (2) parent of i
        if (a[p] <= a[i]) return;
        swap(i, p);
        i = p;
    }
}

private void siftDown(int i) {
    while (true) {
        int l = _____________, r = l + 1, m = i;  // (3) left child
        if (l < size && a[l] < a[m]) m = l;
        if (_____________________________) m = r;   // (4)
        if (m == i) _____________;                  // (5)
        swap(i, m);
        _____________;                          // (6)
    }
}
```

**Why does `poll` refill the root with the last leaf, and not by moving the smaller child of the root up?**

### EXERCISE 5.2 Trace heapify and one poll

Run *Theory Listing 5.2* on $a$ = [8, 6, 7, 2, 5, 9, 1, 4, 3] ($n$ = 9). For each call, write the index pairs that are swapped and the array afterwards.

| call | value sifted | swaps (index pairs) | array after the call |
|---|---|---|---|
| siftDown(3) | | | |
| siftDown(2) | | | |
| siftDown(1) | | | |
| siftDown(0) | | | |

**Now apply one `poll()` to the heap you built: what does it return, which value moves to the root, which swaps follow, and what is the final array?**

**EXERCISE 5.3** **A faded trace: bricks and ladders**

Run *Theory Listing 5.6* on $h$ = [1, 4, 3, 9, 10, 6, 13, 14, 18, 17, 25] with 7 bricks and 2 ladders. The first three rows are done. Continue until the method returns, and leave the rows after that empty.

| i | h[i] → h[i+1] | climb | ladders on (heap after) | bricks pay | bricks left |
|---|---|---|---|---|---|
| 0 | 1 → 4 | 3 | {3} | – | 7 |
| 1 | 4 → 3 | −1 | skip | | 7 |
| 2 | 3 → 9 | 6 | {3, 6} | – | 7 |
| 3 | 9 → 10 | | | | |
| 4 | 10 → 6 | | | | |
| 5 | 6 → 13 | | | | |
| 6 | 13 → 14 | | | | |
| 7 | 14 → 18 | | | | |
| 8 | 18 → 17 | | | | |
| 9 | 17 → 25 | | | | |

**What does the method return? At which row is a ladder taken back from an earlier climb? What does it return with 3 ladders instead of 2?**

### EXERCISE 5.4 Concept check

True or false, with a one-line reason.

1. In a min-heap stored in an array, the largest value is always at the last index.
2. Bottom-up heapify of $10^6$ values can make $10^7$ swaps.
3. `new PriorityQueue<>(list)` costs $O(n)$, while adding the same $n$ values one at a time can cost $\Theta(n \log n)$.
4. `pq.remove(x)` on a `PriorityQueue` of size $n$ costs $O(\log n)$.
5. Adding 4, 1, 3, 2 to a min-`PriorityQueue` and printing it shows `[1, 2, 3, 4]`, so a `PriorityQueue` keeps its array sorted.
6. `new PriorityQueue<Integer>((p, q) -> q - p)` is a correct max-heap for all `int` values.
7. In the IPO template, a project pushed into the heap stays affordable until the end of the run.
8. In the bricks-and-ladders greedy, a climb whose ladder was taken back never gets a ladder again.

### EXERCISE 5.5 Design before you code: a shortest-job printer

A printer receives $n \le 10^5$ jobs; job $i$ arrives at time arrival[$i$] and takes pages[$i$] time units. Whenever the printer is free, it starts the waiting job with the fewest pages, breaking ties by the smaller job number; if no job is waiting, it stays idle until the next arrival. Output the order in which the jobs are printed.

Describe the unlock key and how the pointer moves, the heap's comparator, what happens when the heap is empty, and the time and memory cost. Then find the order for these jobs (number: arrival, pages): 0: (0, 4), 1: (1, 3), 2: (2, 1), 3: (3, 3), 4: (12, 5), 5: (13, 2), 6: (13, 1).

### EXERCISE 5.6 Find the bug

This version of heapify compiles and visits every internal node once, but it sometimes returns an array that is not a heap. Find an input of length 4 on which it fails, say which step of the proof of *Theory Theorem 5.4* no longer holds, and fix it with one change.

```
// Turn a[0..n-1] into a min-heap: sift down every internal node
static void heapifyForward(int[] a) {
    for (int i = 0; i < a.length / 2; i++) siftDown(a, a.length, i);
}

static void siftDown(int[] a, int n, int i) {
    while (true) {
        int l = 2 * i + 1, r = l + 1, m = i;
        if (l < n && a[l] < a[m]) m = l;
        if (r < n && a[r] < a[m]) m = r;
        if (m == i) return;
        int t = a[i]; a[i] = a[m]; a[m] = t;
        i = m;
    }
}
```

### EXERCISE 5.7 Complexity and limits

For each situation, choose an approach, estimate the number of basic steps, and say whether it fits comfortably in one second (about $10^8$ simple steps in Java).

- (a) IPO with $n = 10^5$ projects and $k = 10^5$.
- (b) $10^5$ “add a value”, $10^5$ “cancel a given value” and $10^5$ “report the smallest” requests, with up to $10^5$ values present, using `PriorityQueue.remove(x)` for the cancels.
- (c) $10^6$ values arrive at once and you need the 20 smallest, in order.
- (d) Bricks and ladders with $n = 10^5$ buildings and 5 ladders.

**EXERCISE 5.8 Write it from memory**

Close both books. Write `add`, `siftUp`, `poll` and `siftDown` of the int min-heap, and then `furthestBuilding`. Afterwards compare line by line with *Theory Listing 5.1* and *Theory Listing 5.6* and mark every difference.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**5.P1 LC 502 · IPO** HARD CORE — **60 min**

`leetcode.com/problems/ipo` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 5.P1*

### 5.P2 LC 1642 · Furthest Building You Can Reach MEDIUM CORE — 40 min

leetcode.com/problems/furthest-building-you-can-reach · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P2*

### 5.P3 LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array HARD CORE — 60 min

leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P3*

### 5.P4 LC 1046 · Last Stone Weight EASY CORE — 20 min

leetcode.com/problems/last-stone-weight · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P4*

### 5.P5 LC 871 · Minimum Number of Refueling Stops HARD BONUS — 60 min

leetcode.com/problems/minimum-number-of-refueling-stops · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P5*

### 5.P6 LC 630 · Course Schedule III HARD BONUS

60 min

leetcode.com/problems/course-schedule-iii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P6*

### 5.P7 CF 1526C2 · Potions (Hard Version) CF 1600 BONUS

45 min

codeforces.com/problemset/problem/1526/C2 · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 5.P7*

## Re-solve from three days ago

Pick one problem from Day 2 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 2.P1 LC 560 · Subarray Sum Equals K ☐ 2.P2 LC 974 · Subarray Sums Divisible by K

☐ 2.P3 LC 1524 · Number of Sub-arrays With Odd Sum ☐ 2.P4 LC 525 · Contiguous Array

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 6, Binary search, including on the answer.*

DAY 6 OF 29 · THURSDAY 12 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

6

# Binary Search and Searching on the Answer

*When "can the answer be x?" is monotone, binary search x and check it greedily.*

**Read first** · Theory Chapter 6
**Patterns** · Binary search: lower and upper bound, Binary search on the answer

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 6; type each new listing once and run it | 40 min |
| ☐ | 6.P1 LC 34 · Find First and Last Position of Element in Sorted Array | 40 min |
| ☐ | 6.P2 LC 875 · Koko Eating Bananas | 40 min |
| ☐ | 6.P3 LC 1011 · Capacity To Ship Packages Within D Days | 40 min |
| ☐ | 6.P4 LC 410 · Split Array Largest Sum | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 6.P5, 6.P6, 6.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- Write `ok(x)` first, usually greedy and O(n), then binary search the smallest x that passes.
- Use `long` bounds and `lo + (hi - lo) / 2`.
- Write lower and upper bound yourself; `Arrays.binarySearch` returns any matching index.
- "Minimise the maximum" and "maximise the minimum" almost always mean binary search on the answer.

**Why it matters for Infosys.** Binary search on the answer is one of the most reported pattern families in the Infosys question bank.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Greedy with a heap | Ch. 5 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Two pointers on a sorted array | Ch. 3 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sliding window (variable size) | Ch. 3 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Window maximum (monotonic deque) | Ch. 3 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 6 and before the problem worksheets. The first three rebuild the two templates by hand, the middle ones ask you to design and debug, and the last ones are retrieval practice.

### EXERCISE 6.1 Fill the gaps

Complete the lower-bound template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 6.1*.

```
// First index with a[i] >= x (a.length if none). a must be sorted.
static int lowerBound(int[] a, int x) {
    int lo = 0, hi = _____________;              // (1)
    while (_____________) {                      // (2)
        int mid = _____________;                 // (3) no overflow
        if (a[mid] >= x) _____________;          // (4)
        else _____________;                      // (5)
    }
    return lo;
}
```

**Why does `hi` start at `a.length` and not at `a.length - 1`? Give an input where the second choice returns a wrong answer.**

### EXERCISE 6.2 Trace two searches

Let $a$ = [2, 4, 4, 6, 6, 6, 9, 11, 15] ($n$ = 9). Trace `upperBound(a, 8)` and `lowerBound(a, 6)` from *Theory Listing 6.1*. Write one row per iteration; leave unused rows empty.

| upperBound(a, 8) | lo | hi | mid | a[mid] | next |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

| lowerBound(a, 6) | lo | hi | mid | a[mid] | next |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

**How many times does 6 occur, using only lower and upper bounds? What could `Arrays.binarySearch(a, 6)` return, and what does `Arrays.binarySearch(a, 5)` return?**

### EXERCISE 6.3 A faded trace: ship capacity

Run `minCapacity` (*Theory Listing 6.5*) on the weights [3, 2, 2, 4, 1, 4] with 3 days. The first rows are done. Complete the trace, then write the greedy days for the answer.

| lo | hi | mid | daysNeeded(mid) | decision |
|---|---|---|---|---|
| 4 | 16 | 10 | 2 | works: hi = 10 |
| 4 | 10 | 7 | 3 | works: hi = 7 |
| | | | | |
| | | | | |
| | | | | |

**Answer and the greedy days for it:**

### EXERCISE 6.4 Concept check

Answer true or false, and give a one-line reason.

1. `Arrays.binarySearch` returns the first index holding the key when the key occurs several times.
2. `lowerBound(a, x)` can return `a.length`.
3. In a "maximise the minimum" search with the branches `lo = mid` and `hi = mid - 1`, the midpoint must be rounded down.
4. If ok($x$) is monotone on a range of $V$ values, the search calls ok at most $\lceil \log_2 V \rceil$ times.
5. For a real-valued answer near $10^9$, `while (hi - lo > 1e-9)` is a safe stopping rule.
6. The value found by "smallest $x$ with count($x$) $\geq k$" might not be an element of the collection.
7. A generous upper bound *hi* (say $10^{18}$ instead of $10^9$) can never make the answer wrong, only slower.

### EXERCISE 6.5 Design before you code: machines

A factory has $n$ machines. Machine $i$ finishes one product every $t[i]$ minutes, and all machines work at the same time. What is the least number of minutes $T$ after which at least $M$ products are finished? Limits: $n \le 2 \cdot 10^5$, $t[i] \le 10^9$, $M \le 10^9$. This problem is not in the plan.

**Write the predicate ok($T$), prove in one line that it is monotone, choose *lo* and *hi* (with a reason why ok(*hi*) is true), give the cost of one check and of the whole search, and name the overflow risk inside the check.**

**Run your search on $t$ = [3, 2, 5] and $M$ = 10. Write each (lo, hi, mid, products) row.**

### EXERCISE 6.6 Find the bug

This method is meant to compute the largest possible smallest piece sum (*Theory Listing 6.6*) but sometimes never returns. It compiles. Find the smallest input on which it hangs, explain the cause with the values of *lo*, *hi* and *mid*, and fix it with one change.

```
static long maxMinPieceBuggy(int[] a, int k) {
    long lo = 0, hi = 0;
    for (int x : a) hi += x;
    while (lo < hi) {
```

```
        long mid = lo + (hi - lo) / 2;
        if (piecesAtLeast(a, mid) >= k) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}

static int piecesAtLeast(int[] a, long s) {
    int pieces = 0;
    long cur = 0;
    for (int x : a) {
        cur += x;
        if (cur >= s) {
            pieces++;
            cur = 0;
        }
    }
    return pieces;
}
```

**EXERCISE 6.7** **The k-th smallest by counting**

Consider the $4 \times 5$ multiplication table (entries $i \cdot j$ for $1 \le i \le 4$, $1 \le j \le 5$). Find its 11th smallest entry with *Theory Listing 6.9* without listing the table.

| x | 5 | 6 | 7 | 8 | 10 |
|---|---|---|---|---|---|
| count(x) = Σ min(5, ⌊x/i⌋) | | | | | |

| lo | hi | mid | count(mid) | decision |
|---|---|---|---|---|
| 1 | 20 | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**count(6) and count(7) are equal. Why does the search return 6 and not 7?**

### EXERCISE 6.8 Complexity and limits

For each situation, give the bounds *lo* and *hi*, whether they must be `long`, and the number of iterations of the search.

1. LeetCode 875: piles up to $10^9$, at most $10^4$ piles.
2. LeetCode 1011: at most $5 \cdot 10^4$ packages of weight at most 500.
3. LeetCode 410: at most 1000 values, each at most $10^6$.
4. A real answer in $[0, 10^9]$ needed to within $10^{-6}$.

### EXERCISE 6.9 Write it from memory

Without looking, write `firstTrue` and `lastTrue` (*Theory Listing 6.4*) with their requirements as comments. Mark the line where the two differ in rounding, and say why.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

### 6.P1 LC 34 · Find First and Last Position of Element in Sorted Array MEDIUM CORE — 40 min

`leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P1*

### 6.P2 LC 875 · Koko Eating Bananas MEDIUM CORE — 40 min

`leetcode.com/problems/koko-eating-bananas` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P2*

### 6.P3 LC 1011 · Capacity To Ship Packages Within D Days MEDIUM CORE — 40 min

`leetcode.com/problems/capacity-to-ship-packages-within-d-days` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P3*

### 6.P4 LC 410 · Split Array Largest Sum HARD CORE — 60 min

`leetcode.com/problems/split-array-largest-sum` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P4*

### 6.P5 LC 1552 · Magnetic Force Between Two Balls MEDIUM BONUS 40 min

leetcode.com/problems/magnetic-force-between-two-balls · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P5*

### 6.P6 LC 2226 · Maximum Candies Allocated to K Children MEDIUM BONUS 40 min

leetcode.com/problems/maximum-candies-allocated-to-k-children · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P6*

### 6.P7 LC 719 · Find K-th Smallest Pair Distance HARD BONUS 60 min

leetcode.com/problems/find-k-th-smallest-pair-distance · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 6.P7*

## Re-solve from three days ago

Pick one problem from Day 3 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 3.P1 LC 3 · Longest Substring Without Repeating Characters ☐ 3.P2 LC 1004 · Max Consecutive Ones III

☐ 3.P3 LC 424 · Longest Repeating Character Replacement ☐ 3.P4 LC 15 · 3Sum

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 7, Monotonic stacks.*

DAY 7 OF 29 · FRIDAY 13 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

7

# Monotonic Stacks

*Next or previous greater/smaller element for every index in O(n), and the contribution trick.*

**Read first** · Theory Chapter 7
**Patterns** · Monotonic stack

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 7; type each new listing once and run it | 40 min |
| ☐ | 7.P1 LC 739 · Daily Temperatures | 40 min |
| ☐ | 7.P2 LC 503 · Next Greater Element II | 40 min |
| ☐ | 7.P3 LC 84 · Largest Rectangle in Histogram | 60 min |
| ☐ | 7.P4 LC 907 · Sum of Subarray Minimums | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 7.P5, 7.P6, 7.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- Keep indices whose answer is unknown on a stack. A bigger value answers all smaller ones on top.
- Contribution: a[i] is the minimum of (i − left) × (right − i) subarrays. Use < on one side and ≤ on the other.
- Use `ArrayDeque` or an `int[]` with a top pointer, never `Stack`.

**Why it matters for Infosys.** "Next greater element" and "remove K digits for the smallest ticket price" both appear in Infosys reports.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Binary search: lower and upper bound | Ch. 6 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search on the answer | Ch. 6 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Intervals: sort, then sweep | Ch. 4 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 7. Keep the stack contents written out at every step; most stack bugs show up as a wrong stack after one particular step, and a hand trace finds that step.

### EXERCISE 7.1 Fill the gaps

Complete the next-greater template. Then compare with *Theory Listing 7.1*.

```
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    ______________________;                          // (1) the answer when none exists
    int[] st = new int[n];
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (__________ && ________________________)   // (2) stack not empty, (3) top is smaller
            ______________________;                  // (4) pop it and record its answer
        ______________________;                      // (5) push i
    }
    return res;
}
```

**What must change in line (3) to get the next greater or equal element? And to get the next smaller element?**

### EXERCISE 7.2 Trace the next-greater stack

Trace *Theory Listing 7.1* on $a$ = [4, 2, 1, 3, 5, 2, 6]. Write the indices popped at each step and the stack (as indices) after the push.

| i | a[i] | popped (their answer is i) | stack after |
|---|---|---|---|
| 0 | 4 | | |
| 1 | 2 | | |
| 2 | 1 | | |
| 3 | 3 | | |
| 4 | 5 | | |
| 5 | 2 | | |
| 6 | 6 | | |

**The result array, and the total number of pops. Why can the total never exceed 7 here?**

### EXERCISE 7.3 A faded contribution table

Compute the sum of subarray minimums of $a$ = [3, 1, 2, 2, 1] with *Theory Theorem 7.6*: left = previous strictly smaller index, right = next smaller-or-equal index. The first two rows are done.

| i | a[i] | left | right | (i − left) · (right − i) | a[i] · count |
|---|---|---|---|---|---|
| 0 | 3 | −1 | 1 | 1 · 1 = 1 | 3 |
| 1 | 1 | −1 | 4 | 2 · 3 = 6 | 6 |
| 2 | 2 | | | | |
| 3 | 2 | | | | |
| 4 | 1 | | | | |

**Total, and the check on the counts. Index 1 holds the value 1 but owns only six subarrays, although more subarrays contain it. Which subarray containing index 1 does it not own, and who owns it?**

### EXERCISE 7.4 A histogram by hand

For the bars $h$ = [2, 4, 3, 5, 1, 3], fill in the previous and next strictly lower bar of each bar, the width of its rectangle and its area (*Theory Theorem 7.4*).

| bar i | h[i] | L(i) | R(i) | width R − L − 1 | area |
|---|---|---|---|---|---|
| 0 | 2 | | | | |
| 1 | 4 | | | | |
| 2 | 3 | | | | |
| 3 | 5 | | | | |
| 4 | 1 | | | | |
| 5 | 3 | | | | |

**Draw the histogram and shade the largest rectangle.**

### EXERCISE 7.5 Concept check

Answer true or false, and give a one-line reason.

1. One iteration of the `for` loop in *Theory Listing 7.1* can perform up to $n - 1$ pops, so the method is $O(n^2)$ in the worst case.
2. For the previous strictly smaller element, pop while `a[top] >= a[i]` and read the top after popping.
3. In the contribution formula, using strict comparisons on both sides over-counts subarrays whose minimum occurs more than once.
4. The sum of subarray ranges equals the sum of subarray maximums minus the sum of subarray minimums.
5. For a circular array, it is wrong to push indices during the second lap.
6. In *Theory Listing 7.7*, if $k$ deletions remain after the scan, they are taken from the front of the stack.

### EXERCISE 7.6 Design before you code: price spans

Each day has a price. The **span** of day $i$ is the number of consecutive days ending at day $i$ (including it) whose price is $\leq$ the price of day $i$. This problem is not in the plan.

**Which variant of *Theory Table 7.1* gives the span? Write the pop condition, the formula for the span from what is left on the stack, and the complexity.**

**Compute the spans for the prices [30, 25, 27, 27, 40, 22, 26].**

### EXERCISE 7.7 Find the bug

This version of the contribution sum compiles and passes every test with distinct values. Find the smallest positive input on which it is wrong, give both answers, and fix it with one character.

```
static long sumMinsBuggy(int[] a) {
    int n = a.length;
    long total = 0;
    for (int i = 0; i < n; i++) {
        int l = i - 1;
        while (l >= 0 && a[l] >= a[i]) l--;          // previous strictly smaller
        int r = i + 1;
        while (r < n && a[r] >= a[i]) r++;           // next strictly smaller
        total += (long) a[i] * (i - l) * (r - i);
    }
    return total;
}
```

### EXERCISE 7.8 Delete three digits

Run *Theory Listing 7.7* on the digits “3141592” with $k = 3$. Write what is popped at each step, the stack after the step and the deletions left.

| digit | popped | stack after | k left |
|---|---|---|---|
| 3 | | | |
| 1 | | | |
| 4 | | | |
| 1 | | | |
| 5 | | | |
| 9 | | | |
| 2 | | | |

**The result. Why is deleting the 9 better than deleting the 5?**

**EXERCISE 7.9** **Write it from memory**

Write the two boundary scans of *Theory Listing 7.5* with `int[]` stacks instead of `ArrayDeque`, and the final sum. Mark which comparison is strict.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**7.P1** **LC 739 · Daily Temperatures** MEDIUM CORE **40 min**

`leetcode.com/problems/daily-temperatures` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P1*

### 7.P2 LC 503 · Next Greater Element II MEDIUM CORE

40 min

leetcode.com/problems/next-greater-element-ii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 7.P2*

### 7.P3 LC 84 · Largest Rectangle in Histogram HARD CORE

60 min

leetcode.com/problems/largest-rectangle-in-histogram · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 7.P3*

### 7.P4 LC 907 · Sum of Subarray Minimums MEDIUM CORE

40 min

leetcode.com/problems/sum-of-subarray-minimums · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 7.P4*

### 7.P5 LC 402 · Remove K Digits MEDIUM BONUS

40 min

leetcode.com/problems/remove-k-digits · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 7.P5*

**7.P6 LC 316 · Remove Duplicate Letters** MEDIUM BONUS **40 min**

leetcode.com/problems/remove-duplicate-letters · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P6*

**7.P7 LC 2104 · Sum of Subarray Ranges** MEDIUM BONUS **40 min**

leetcode.com/problems/sum-of-subarray-ranges · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 7.P7*

## Re-solve from three days ago

Pick one problem from Day 4 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 4.P1 LC 56 · Merge Intervals ☐ 4.P2 LC 435 · Non-overlapping Intervals
☐ 4.P3 LC 2406 · Divide Intervals Into Minimum Number of Groups ☐ 4.P4 LC 179 · Largest Number

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 8, Bits and XOR.*

DAY 8 OF 29 · SATURDAY 14 NOVEMBER 2026 · LEARNING DAY · PHASE 1 · Q1/Q2 TOOLKIT

8

# Bits, XOR, Tries and Linear Bases

*Each bit is independent in a sum of XORs. A trie answers pair questions; a basis answers subset questions.*

**Read first** · Theory Chapter 8
**Patterns** · Bit tricks and XOR, Binary trie for XOR, XOR basis (maximum subset XOR)

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 8; type each new listing once and run it | 40 min |
| ☐ | 8.P1 LC 136 · Single Number | 20 min |
| ☐ | 8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR | 40 min |
| ☐ | 8.P3 LC 421 · Maximum XOR of Two Numbers in an Array | 40 min |
| ☐ | 8.P4 LC 1803 · Count Pairs With XOR in a Range | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 8.P5, 8.P6, 8.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- `x & -x` is the lowest set bit; write `1L << k` when k can reach 31.
- Prefix XOR: `xor(l..r) = P[r+1] ^ P[l]`, exactly like prefix sums.
- Binary trie: walk from the top bit, choosing the opposite bit to maximise the XOR.
- XOR basis: at most one number per highest bit; the maximum subset XOR is built greedily.

**Why it matters for Infosys.** There are two XOR repeats: "at most N/2 elements, max XOR" (13 reports) and "X ≤ K maximising Σ X xor A[i]" (6 reports). The sample Hard problem is XOR-based too.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Monotonic stack | Ch. 7 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Greedy with a heap | Ch. 5 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Java collections toolkit | Ch. 1 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Fast input and output (FastScanner) | Ch. 1 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Linked list moves | Ch. 1 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 8 and before the problem worksheets. The first four run the chapter's tools by hand on 4- and 5-bit numbers, so write every number in binary as well as in decimal: the bits are where the reasoning happens. The rest ask you to decide, design and debug.

### EXERCISE 8.1 Fill the gaps: the two trie walks

Complete the two queries of the binary trie from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 8.4*.

```
int maxXor(int x) {                                  // trie must not be empty
    int node = 0, res = 0;
    for (int b = B - 1; b >= 0; b--) {
        int want = ________________________;          // (1) the bit that makes this XOR bit 1
        if (child[node][want] != 0) {
            ________________________;                 // (2)
            node = child[node][want];
        } else {
            node = ________________________;          // (3)
        }
    }
    return res;
}

long countXorLess(int x, int k) {                    // how many stored y have (x ^ y) < k
    long res = 0;
    int node = 0;
    for (int b = B - 1; b >= 0; b--) {
        int xb = (x >> b) & 1, kb = (k >> b) & 1;
        if (kb == 1) {
            if (child[node][xb] != 0) res += _______________;   // (4)
            node = child[node][xb ^ 1];
        } else {
            node = ________________________;          // (5)
        }
        ________________________;                     // (6)
    }
    return res;
}
```

**At a 1-bit of $k$, why does `countXorLess` add the whole child $x_b$ and then walk into the child $x_b \oplus 1$?**

### EXERCISE 8.2 Trace the trie

Insert 9, 5, 12, 3, 14 and 6 into a trie with $B = 4$ bits (*Theory Listing 8.4* with `B = 4`). Draw it as in the theory chapter's figure: label each node with its prefix and its count, and each edge with its bit. How many nodes does it have, including the root?

Now trace `maxXor(13)` on your trie. Each row is one level: the bit of $x = 13 = 1101_2$, the bit the walk wants, whether that child exists, `res` after the level, and the prefix of the node the walk ends the level on.

| bit b | x_b | want | child exists? | res after | node after |
|---|---|---|---|---|---|
| 3 | | | | | |
| 2 | | | | | |
| 1 | | | | | |
| 0 | | | | | |

Then trace `countXorLess(13, 6)`, with $k = 6 = 0110_2$. Write what the walk adds and where it goes. The walk may stop before bit 0.

| bit b | x_b | k_b | action | count after |
|---|---|---|---|---|
| 3 | | | | |
| 2 | | | | |
| 1 | | | | |
| 0 | | | | |

**Which stored $y$ gives the maximum? Which stored $y$ have $13 \oplus y < 6$? Check both answers by computing all six XORs.**

**EXERCISE 8.3 A faded trace: building a basis**

Insert 26, 11, 17, 7, 28 and 22 into the basis of *Theory Listing 8.6*, using 5 bits. The first two rows are done. In each row, list every reduction "bit $i$: $\oplus\ b[i] \to$ new $x$" and then the outcome: the slot where $x$ is stored, or "false".

| add(x) | x in binary | reductions | outcome |
|---|---|---|---|
| 26 | 11010 | none | b[4] = 11010 (26) |

| add(x) | x in binary | reductions | outcome |
|---|---|---|---|
| 11 | 01011 | none | b[3] = 01011 (11) |
| 17 | 10001 | | |
| 7 | 00111 | | |
| 28 | 11100 | | |
| 22 | 10110 | | |

Now run `max()`. The first row is done.

| i | b[i] | r ⊕ b[i] | take? | r after |
|---|---|---|---|---|
| 4 | 26 | 26 | yes | 26 |
| 3 | | | | |
| 2 | | | | |
| 1 | | | | |
| 0 | | | | |

**The rank is _____, so the subsets of the six numbers produce _____ distinct XORs, and each of them comes from _____ of the 64 subsets. Use *Theory Listing 8.7* to decide whether some subset has XOR 13, and whether some subset has XOR 14. Write two original numbers whose XOR is the maximum.**

## EXERCISE 8.4 Per-bit counting and choosing X ≤ K

Let $A = [1, 3, 13, 6, 10, 2, 8]$, so $n = 7$, and four bits suffice. Fill in, for each bit, the number $c_b$ of elements with the bit set, the number of pairs whose bits differ, and the gains of *Theory Section 8.3* for $X_b = 0$ and $X_b = 1$ (already multiplied by $2^b$).

| bit b | c_b | pairs c_b(n − c_b) | gain if X_b = 0 | gain if X_b = 1 | free choice |
|---|---|---|---|---|---|
| 3 | | | | | |
| 2 | | | | | |
| 1 | | | | | |
| 0 | | | | | |

**Use *Theory Theorem 8.5* to compute the sum of $A[i] \oplus A[j]$ over all pairs $i < j$. With no limit on $X$, which $X$ is best, and what is $\Sigma (X \oplus A[i])$ for it?**

Now $X$ must satisfy $X \le K = 10 = 1010_2$. Following *Theory Lemma 8.6*, fill in one row per candidate and pick the best.

| candidate | X (binary) | X | Σ (X ⊕ A[i]) |
|---|---|---|---|
| X = K | | | |
| branch at bit 3 | | | |
| branch at bit 1 | | | |

### EXERCISE 8.5 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. `1 << 32` evaluates to 4 294 967 296.
2. `-17 >> 1` is −9, while `-17 / 2` is −8.
3. `(x & (x - 1)) == 0` correctly tests "$x$ is a power of two" for every `int` $x$.
4. `Integer.highestOneBit(40)` returns 5.
5. If every value of an array occurs twice, except two different values $u$ and $v$ that occur once each, the XOR of the array is 0.
6. A trie that stores $n$ distinct $B$-bit numbers has exactly $n \cdot B + 1$ nodes.
7. XORing all the non-zero numbers of a basis together gives the maximum subset XOR.
8. In Exercise 8.3, each of the reachable values is the XOR of exactly 4 of the 64 subsets.

### EXERCISE 8.6 Design before you code

A new problem, not in the plan: given $n \le 10^5$ values $0 \le a[i] < 2^{20}$ and a bound $0 \le k \le 10^6$, count the subarrays whose XOR is less than $k$. Use *Theory Table 8.3* to pick the tools. Write what you store, what you query and in which order, the base case, and the time and memory cost. Then compute the answer by hand for $a = [5, 1, 6, 2, 7]$ and $k = 4$, listing the subarrays you count.

### EXERCISE 8.7 Find the bug

This shortened `countXorLess` has lost one line of *Theory Listing 8.4*: its author reasoned that the loop stops after bit 0 anyway. The arrays are filled by `insert` exactly as in the theory listing (with $B = 31$). It compiles, and on the theory chapter's example (stored 2, 3, 5, 6; $x = 4$, $k = 3$) it returns the correct 2. Find a trie with a single stored number and a query on which it is wrong, say what it returns, explain why, and fix it.

```
static int[][] child = new int[100 * 31 + 1][2];   // child[node][bit], 0 = missing; node 0 is the root
static int[] cnt = new int[100 * 31 + 1];

static long countXorLessBuggy(int x, int k) {
    long res = 0;
    int node = 0;
    for (int b = 30; b >= 0; b--) {
        int xb = (x >> b) & 1, kb = (k >> b) & 1;
        if (kb == 1) {
            if (child[node][xb] != 0) res += cnt[child[node][xb]];
            node = child[node][xb ^ 1];
        } else {
            node = child[node][xb];
        }
    }
    return res;
}
```

### EXERCISE 8.8 Complexity and limits

For each task, name the tool from Theory Chapter 8 (or a brute force from Chapter 9), its time and memory, and one trap.

1. $n \le 2 \cdot 10^5$ values below $2^{30}$: the largest XOR of two of them.
2. $n \le 10^6$ values up to $10^9$: the sum of $a[i] \oplus a[j]$ over all pairs $i < j$, modulo $10^9 + 7$.
3. $n \le 10^5$ values up to $10^{18}$: the largest XOR of any subset.

4. $n \le 10^6$ values below $2^{20}$ and $k < 2^{20}$: the number of subarrays with XOR exactly $k$.
5. $N = 20$ values below $2^{20}$: the largest XOR of at most $N/2$ of them.
6. $N = 100$ values below $2^{20}$: the same question.

**EXERCISE 8.9** **Write it from memory**

Close the theory book. Write the `XorBasis` class of *Theory Listing 8.6* (`add` and `max`) and the method `canMake` of *Theory Listing 8.7* from memory, then check them line by line. Mark the two places where a `long` matters.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

### 8.P1 LC 136 · Single Number EASY CORE — 20 min

leetcode.com/problems/single-number · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P1*

### 8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR MEDIUM CORE — 40 min

leetcode.com/problems/count-triplets-that-can-form-two-arrays-of-equal-xor · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P2*

### 8.P3 LC 421 · Maximum XOR of Two Numbers in an Array MEDIUM CORE — 40 min

leetcode.com/problems/maximum-xor-of-two-numbers-in-an-array · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P3*

### 8.P4 LC 1803 · Count Pairs With XOR in a Range HARD CORE — 60 min

leetcode.com/problems/count-pairs-with-xor-in-a-range · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P4*

### 8.P5 LC 2429 · Minimize XOR MEDIUM BONUS — 40 min

leetcode.com/problems/minimize-xor · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P5*

### 8.P6 GfG · Maximum Subset XOR GFG BONUS — 40 min

geeksforgeeks.org/problems/maximum-subset-xor/1 · GeeksforGeeks

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P6*

### 8.P7 LC 1707 · Maximum XOR With an Element From Array HARD BONUS — 60 min

leetcode.com/problems/maximum-xor-with-an-element-from-array · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 8.P7*

## Re-solve from three days ago

Pick one problem from Day 5 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 5.P1 LC 502 · IPO ☐ 5.P2 LC 1642 · Furthest Building You Can Reach
☐ 5.P3 LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array ☐ 5.P4 LC 1046 · Last Stone Weight

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 9, Mock 1 + review.*

DAY 9 OF 29 · SUNDAY 15 NOVEMBER 2026 · MOCK DAY · PHASE 1 · Q1/Q2 TOOLKIT

9

# Brute Force, Stress Testing and the Mock Exam

*3 hours on Phase 1 topics at Q1/Q2/Q3 difficulty, then a full review.*

**Read first** · Theory Chapter 9
**Patterns** · Brute force first, then stress test

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 9.P1 CF 1343C · Alternating Subsequence | 30 min |
| ☐ | 9.P2 CF 1475D · Cleaning the Phone | 60 min |
| ☐ | 9.P3 CF 1251D · Salary Changing | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Read the mock rules. Open each problem, then type your program on the Codeforces *Submit* page (language: Java). Test with the samples on *Custom test* first.
- Before the mock, read the handbook card "Brute force first": it's how you collect partial marks.
- After the mock: enter your scores in the Mocks section, read the editorials, re-code every miss and log each cause.

**Why it matters for Infosys.** This is your first measurement. Compare it with the SP checkpoints.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Bit tricks and XOR | Ch. 8 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary trie for XOR | Ch. 8 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| XOR basis (maximum subset XOR) | Ch. 8 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search: lower and upper bound | Ch. 6 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search on the answer | Ch. 6 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Prefix sums + HashMap | Ch. 2 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Difference array (many range adds) | Ch. 2 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Mock exam: Mock 1 · Phase 1 topics

Q1 ≈ 1200, Q2 ≈ 1800, Q3 ≈ 1900 (Codeforces ratings). Weights 20/30/50. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

### Planning grid (first 15 minutes)

| Q | Limits | Idea | Complexity | Brute-force fallback | Order |
|---|---|---|---|---|---|
| Q1 | | | | | |
| Q2 | | | | | |
| Q3 | | | | | |

### Score sheet

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q1 | **CF 1343C** Alternating Subsequence `codeforces.com/problemset/problem/1343/C` | 1200 | 20 | | | | |
| Q2 | **CF 1475D** Cleaning the Phone `codeforces.com/problemset/problem/1475/D` | 1800 | 30 | | | | |
| Q3 | **CF 1251D** Salary Changing `codeforces.com/problemset/problem/1251/D` | 1900 | 50 | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

**SP checkpoint.** Q1 fully solved; Q2 at least half the tests.

### Post-mortem (about 60 minutes after the mock)

| Q | What went wrong | Cause | Rule that would have prevented it |
|---|---|---|---|
| Q1 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q2 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q3 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |

## Exercises

Do these after the mock and its post-mortem, or the next morning if you are tired. They practise the habits of Theory Chapter 9 on small new problems: writing the slow, obviously correct version, enumerating by hand, shrinking a counterexample, naming the cause of a failure, and reading the limits. None of them uses or hints at a mock problem.

**EXERCISE 9.1** **Write a brute force**

A new problem: there are $n \le 16$ players with skills $s[0..n-1] \ge 1$. A team is a set of at least two players, and it is *balanced* when its strongest skill is at most twice its weakest skill. Count the balanced teams, and give the size of the largest one. Write a brute force in the style of *Theory Listing 9.1*. Keep it obviously correct; speed does not matter.



**Compute both answers by hand for $s = [3, 5, 8, 4, 10]$. (Hint: sort the skills and count the teams by their weakest member.) What does your brute force cost, and up to which $n$ would it pass if the real limit were $n \le 2 \cdot 10^5$?**

**EXERCISE 9.2** **Enumerate by hand**

First the subsets. For the elements $a$ = [4, 7, 9], write each mask in binary, the subset it stands for (element $i$ is in when bit $i$ is 1, as in *Theory Lemma 9.1*), and its sum.

| mask | binary | subset | sum |
|---|---|---|---|
| 0 | | | |
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |

**What does `bestSubsetSum(a, 15)` (*Theory Listing 9.1*) return? Which mask stands for the subset {0, 2, 3} of a 4-element array, and which elements are in mask 22?**

Now the orders. Apply *Theory Listing 9.2* once to each array below. Write the index $i$ found by the first loop (the position just before the longest non-increasing suffix), the index $j$ of the element swapped with $a[i]$, the array after the swap, the result after the reverse, and the return value.

| input | i | j | after swap | result | returns |
|---|---|---|---|---|---|
| [1, 2, 2, 3] | | | | | |
| [1, 3, 2, 2] | | | | | |
| [2, 3, 2, 1] | | | | | |
| [3, 2, 2, 1] | | | | | |

**Starting from [1, 2, 2, 3], how many arrangements does `do { ... } while (nextPermutation(a));` visit, and why is that fewer than 4! = 24?**

**EXERCISE 9.3 Shrink a counterexample**

A new problem: find the largest product of a non-empty subarray. A tempting fast idea copies Kadane's scan from *Theory Listing 9.4*, with products in place of sums. Both methods below compile.

```
static long solve(int[] a) {                       // Kadane's scan, copied for products
    long best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur * a[i]);            // the best product ending at i?
        best = Math.max(best, cur);
    }
    return best;
}

static long brute(int[] a) {                       // every subarray
    long best = Long.MIN_VALUE;
    for (int l = 0; l < a.length; l++) {
        long p = 1;
        for (int r = l; r < a.length; r++) { p *= a[r]; best = Math.max(best, p); }
    }
    return best;
}
```

With these two methods, the stress tester of *Theory Listing 9.4* (same seed, same generator) stops at once with `Mismatch on iteration 1: [-6, -9, -10, 10, 3, 4]` and `fast=120 brute=10800`. Run the shrinker of *Theory Listing 9.5* by hand. Remember its order: in each round it tries deleting one element, from the left, then two neighbours, then halving each value (rounding towards zero, as Java's `/` does), and it restarts the round after every accepted change. The first three accepted changes are done. Continue until no change keeps the disagreement.

| step | accepted change | input after it | fast | brute |
|---|---|---|---|---|
| 0 | first mismatch | [−6, −9, −10, 10, 3, 4] | 120 | 10800 |
| 1 | delete a[3] | [−6, −9, −10, 3, 4] | 54 | 1080 |
| 2 | delete a[3] | [−6, −9, −10, 4] | 54 | 360 |
| 3 | delete a[3] | [−6, −9, −10] | 54 | 90 |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |
| 9 | | | | |
| 10 | | | | |
| 11 | | | | |
| 12 | | | | |

**Why was deleting $a[0]$ rejected in step 1? What does the final input show about the idea, which of the nine causes of *Theory Section 9.6* is it, and how would you repair `solve`?**

### EXERCISE 9.4 Classify the failures

Each line describes a failed submission. Name its cause with the taxonomy of *Theory Section 9.6* (misread statement, wrong idea, overflow / mod, off-by-one, TLE / complexity, input reading, edge case, stack overflow, ran out of time) and write a rule specific enough to check in ten seconds.

1. A binary search computes `int mid = (lo + hi) / 2` with `lo` and `hi` up to $2 \cdot 10^9$, and fails only on the large tests.
2. The statement asks for the answers of the queries on separate lines; the program prints them on one line, separated by spaces.
3. A recursive DFS over a tree given by parent pointers crashes on a test where the tree is a path of $2 \cdot 10^5$ nodes.
4. To keep the most non-overlapping meetings, the program sorts them by start time and takes each one that fits.
5. The program reads $n$ with `nextInt()` and then a name with `nextLine()`; the name comes out empty.
6. A search for the first index with $a[i] \geq x$ starts with `hi = n - 1`; when every element is smaller than $x$, it returns $n - 1$.
7. With $t \leq 10^4$ test cases and $\Sigma\, n \leq 2 \cdot 10^5$, the program allocates `new long[200001]` inside the test-case loop.
8. The task is the second-largest distinct value, or $-1$ if there is none; on [5, 5, 5] the program prints 5.
9. At minute 175, Q3 has no submission: its brute force was ready at minute 140, but you kept optimising it.

### EXERCISE 9.5 From the limits to the complexity

For each set of limits (time limit 1–2 seconds), write the target complexity, the number of steps at the limit, and a technique from *Theory Table 9.2* that fits.

1. $n \le 8$.
2. $n \le 18$.
3. $n \le 400$.
4. $n \le 4000$. Would an $O(n^3)$ idea pass?
5. $t \le 10^4$ test cases with $\Sigma\, n \le 2 \cdot 10^5$. What does clearing a global array of size $2 \cdot 10^5$ in every test case cost?
6. A single number $n \le 10^{12}$.
7. A two-dimensional DP over $5000 \times 5000$ states under a 256 MB memory limit. Does a `long` table fit? An `int` table?

**Finally: an $O(n^2)$ brute force for a problem with $n \le 2 \cdot 10^5$. Up to which $n$ does it finish in about a second?**

### EXERCISE 9.6 Write it from memory

Close the theory book. Write the stress tester of *Theory Listing 9.4* from memory, with a generator for "$n$ between 1 and 8, values between −10 and 10", then check it line by line. Mark the line that makes failures reproducible.

## Re-solve from three days ago

Pick one problem from Day 6 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 6.P1 LC 34 · Find First and Last Position of Element in Sorted Array
- [ ] 6.P2 LC 875 · Koko Eating Bananas
- [ ] 6.P3 LC 1011 · Capacity To Ship Packages Within D Days
- [ ] 6.P4 LC 410 · Split Array Largest Sum

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 10, Number theory toolkit.*

DAY 10 OF 29 · MONDAY 16 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

10

# Number Theory for Programmers

*gcd, sieves, fast power, inverses, nCr, and the square-free kernel.*

**Read first** · Theory Chapter 10
**Patterns** · gcd, fast power, inverse, nCr, Sieve, prime factors, square-free kernel

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 10; type each new listing once and run it | 40 min |
| ☐ | 10.P1 LC 204 · Count Primes | 40 min |
| ☐ | 10.P2 LC 1922 · Count Good Numbers | 40 min |
| ☐ | 10.P3 LC 2447 · Number of Subarrays With GCD Equal to K | 40 min |
| ☐ | 10.P4 LC 2862 · Maximum Element-Sum of a Complete Subset of Indices | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 10.P5, 10.P6, 10.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- `lcm = a / gcd(a, b) * b`: divide first so it doesn't overflow.
- The smallest-prime-factor sieve factorises any $x \le 10^7$ in O(log x).
- Fermat inverse `a^(MOD-2)` when MOD is prime; precompute factorials for O(1) nCr. A non-prime modulus (Infosys has used 10000) needs Pascal's triangle.
- x·y is a perfect square ⟺ kernel(x) = kernel(y), where the kernel keeps the primes with an odd exponent.

**Why it matters for Infosys.** The sample Easy (perfect-square products) and Medium (gcd exactly p) are both number theory at their core.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Brute force first, then stress test | Ch. 9 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Monotonic stack | Ch. 7 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Two pointers on a sorted array | Ch. 3 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sliding window (variable size) | Ch. 3 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Window maximum (monotonic deque) | Ch. 3 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 10 and before the problem worksheets. The first four are traces of the chapter's algorithms, which you should do with pencil and paper only: every intermediate number is small enough. The rest ask you to decide, explain and debug.

### EXERCISE 10.1 Fill the gaps

Complete fast power and the factorial precomputation from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 10.3* and *Theory Listing 10.5*.

```
static long power(long base, long exp, long m) {
    long r = _____________;                // (1) so that m = 1 gives 0
    base %= m;
    if (base < 0) base += m;
    while (exp > 0) {
        if (_____________) r = r * base % m; // (2) the lowest bit of exp is set
        ______________________________;       // (3) square the base
        ______________________________;       // (4) drop the lowest bit
    }
    return r;
}

static void initFactorials(int n) {
    fact = new long[n + 1];
    invFact = new long[n + 1];
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[n] = ______________________________;                 // (5) one Fermat inverse
    for (int i = n; i > 0; i--) invFact[i - 1] = _____________;   // (6)
}
```

**Why is line (6) correct? Write the identity it uses.**

### EXERCISE 10.2 Trace Euclid's algorithm

Run *Theory Listing 10.1* on $a = 2024$, $b = 1717$. Each row is one iteration of the loop. Then answer the questions below.

| iteration | a | b | q = ⌊a/b⌋ | r = a mod b |
|---|---|---|---|---|
| 1 | 2024 | 1717 | | |
| 2 | | | | |

| iteration | a | b | q = ⌊a/b⌋ | r = a mod b |
|---|---|---|---|---|
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |

**gcd(2024, 1717) = ? How many iterations did the loop make? What is lcm(2024, 1717), computed as `a / gcd * b`?**

**EXERCISE 10.3** **A faded trace: the extended algorithm**

*Theory Listing 10.2* returns from the deepest call first. Use the quotients $q$ from Exercise 10.2. Each row computes $x = y'$ and $y = x' - q \cdot y'$ from the row above it $(x', y')$. The first four rows are done. Fill in the rest and check the last row: $2024x + 1717y$ must equal 1.

| call (a, b) | q | x | y | check a·x + b·y |
|---|---|---|---|---|
| (1, 0) | — | 1 | 0 | 1 |
| (2, 1) | 2 | 0 | 1 | 1 |
| (11, 2) | 5 | 1 | −5 | 1 |
| (57, 11) | 5 | −5 | 26 | 1 |
| (125, 57) | 2 | | | |
| (182, 125) | 1 | | | |
| (307, 182) | 1 | | | |
| (1717, 307) | 5 | | | |
| (2024, 1717) | 1 | | | |

**Use the last row to write the inverse of 1717 modulo 2024 as a number in 0..2023.**

**EXERCISE 10.4** **Trace fast power**

Compute $7^{45}$ mod 1000 with *Theory Listing 10.3*. First write 45 in binary. Each row is one iteration: the value of exp at the start, its lowest bit, and then *r*, base and exp after the iteration.

| exp at start | bit | r after | base after | exp after |
|---|---|---|---|---|
| 45 | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**45 in binary = _____. The answer $7^{45}$ mod 1000 = _____. How many multiplications would the naive loop use?**

**EXERCISE 10.5** **Smallest prime factors and kernels by hand**

Fill in `spf[x]` for $2 \le x \le 40$, as *Theory Listing 10.9* would. The cell in row *t* and column +*u* is $x = t + u$. Circle the primes.

| | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 | +8 | +9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | — | — | | | | | | | | |
| 10 | | | | | | | | | | |
| 20 | | | | | | | | | | |
| 30 | | | | | | | | | | |
| 40 | | — | — | — | — | — | — | — | — | — |

Now factorise 756 and 84 with the table, writing the chain of divisions as in the theory chapter. Give each one's factorisation, number of divisors and square-free kernel. Is $756 \cdot 84$ a perfect square? Is $756 \cdot 1176$ one ($1176 = 2^3 \cdot 3 \cdot 7^2$)? Decide from the kernels, without multiplying.

### EXERCISE 10.6 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. If $b$ divides $a$, then $(a / b) \bmod m = (a \bmod m) / (b \bmod m)$.
2. If $\gcd(a, 10\,000) = 1$, then $a^{9998} \bmod 10\,000$ is the inverse of $a$ modulo 10 000.
3. *Theory Listing 10.1* returns $\gcd(0, 0) = 0$ and $\gcd(0, 12) = 12$.
4. The sum of two residues modulo $10^9 + 7$ always fits in an `int`.
5. $x \cdot y$ is a perfect square exactly when $x$ and $y$ have the same set of prime factors.
6. The sieve of Eratosthenes up to $n$ costs $O(n \log n)$, because of the harmonic series.
7. An `int[]` SPF sieve up to $10^7$ needs about 40 MB.

### EXERCISE 10.7 Design before you code

A new problem, not in the plan: given $n \le 10^5$ values $1 \le a[i] \le 10^7$, count the pairs $i < j$ such that $a[i] \cdot a[j]$ is a perfect square. Write the key idea, the data structures, the formula for the answer, and the time and memory cost. Then compute the answer by hand for $a = [2, 8, 3, 12, 18, 5, 50, 27, 1, 4]$.

**EXERCISE 10.8** **Find the bug**

This fast power was typed in a hurry. It compiles and works on small inputs such as `powerBuggy(3, 5, 1000)` = 243, but it fails modulo $10^9 + 7$. Find the smallest kind of input that breaks it, give one failing call with the wrong and the right result, and fix it.

```
static int powerBuggy(int base, int exp, int m) {
    int r = 1;
    base %= m;
    while (exp > 0) {
        if ((exp & 1) == 1) r = r * base % m;
        base = base * base % m;
        exp >>= 1;
    }
    return r;
}
```

**EXERCISE 10.9** **Complexity and limits**

For each task, name the method from Theory Chapter 10, its time and memory, and one trap.

1. Answer $10^5$ queries $C(n, r)$ mod $10^9 + 7$ with $n \le 10^6$.
2. Compute $C(n, r)$ mod 10 000 for all $n \le 1000$.
3. Decide for each of $10^5$ numbers up to $10^7$ whether it is prime.
4. Factorise one number up to $10^{12}$.
5. Compute $a^e$ mod $10^9 + 7$ for $e$ up to $10^{18}$.
6. Compute the square-free kernels of $10^5$ numbers up to $10^9$.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time

is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**10.P1** **LC 204 · Count Primes** MEDIUM CORE — **40 min**

leetcode.com/problems/count-primes · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P1*

**10.P2** **LC 1922 · Count Good Numbers** MEDIUM CORE — **40 min**

leetcode.com/problems/count-good-numbers · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P2*

**10.P3** **LC 2447 · Number of Subarrays With GCD Equal to K** MEDIUM CORE — **40 min**

leetcode.com/problems/number-of-subarrays-with-gcd-equal-to-k · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P3*

**10.P4** **LC 2862 · Maximum Element-Sum of a Complete Subset of Indices** HARD CORE — **60 min**

leetcode.com/problems/maximum-element-sum-of-a-complete-subset-of-indices · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P4*

**10.P5** **LC 1819 · Number of Different Subsequences GCDs** HARD BONUS — 60 min

leetcode.com/problems/number-of-different-subsequences-gcds · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P5*

**10.P6** **LC 2572 · Count the Number of Square-Free Subsets** MEDIUM BONUS — 40 min

leetcode.com/problems/count-the-number-of-square-free-subsets · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P6*

**10.P7** **CF 1497E1 · Square-Free Division (easy version)** CF 1700 BONUS — 45 min

codeforces.com/problemset/problem/1497/E1 · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 10.P7*

## Re-solve from three days ago

Pick one problem from Day 7 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 7.P1 LC 739 · Daily Temperatures ☐ 7.P2 LC 503 · Next Greater Element II

☐ 7.P3 LC 84 · Largest Rectangle in Histogram ☐ 7.P4 LC 907 · Sum of Subarray Minimums

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 11, DP I: states, transitions, knapsack.*

DAY 11 OF 29 · TUESDAY 17 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

11

# Dynamic Programming I: States, Transitions and Knapsacks

*Define the state in words before writing any code.*

**Read first** · Theory Chapter 11
**Patterns** · DP recipe: state, transition, base, answer, Knapsack family

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 11; type each new listing once and run it | 40 min |
| ☐ | 11.P1 LC 198 · House Robber | 40 min |
| ☐ | 11.P2 LC 322 · Coin Change | 40 min |
| ☐ | 11.P3 LC 518 · Coin Change II | 40 min |
| ☐ | 11.P4 LC 416 · Partition Equal Subset Sum | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 11.P5, 11.P6, 11.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- Write it down first: *dp[i]* = …, the transition, the base case, and where the answer is.
- 0/1 knapsack: loop capacity *downwards*. Unbounded: loop it upwards.
- Counting combinations vs orderings: coins in the outer loop for combinations.
- Prefer loops over recursion in Java for deep DP.

**Why it matters for Infosys.** Coin change, subset sum and 0/1 knapsack are among the most-reported Infosys DP problems.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| gcd, fast power, inverse, nCr | Ch. 10 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Sieve, prime factors, square-free kernel | Ch. 10 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Bit tricks and XOR | Ch. 8 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary trie for XOR | Ch. 8 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| XOR basis (maximum subset XOR) | Ch. 8 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Intervals: sort, then sweep | Ch. 4 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 11 and before the problem worksheets. The first four fill DP tables by hand. Do every entry: the point is to feel which earlier entry each one reads, because that is what decides loop directions and loop orders. The rest ask you to state a DP in words, find a bug in a table, and choose a method from the limits.

### EXERCISE 11.1 Fill the gaps

Complete the 0/1 knapsack and the fewest-coins DP from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 11.5* and *Theory Listing 11.6*.

```
static long knapsack01(int[] w, int[] v, int cap) {
    long[] dp = new long[cap + 1];                    // dp[c] = best value with weight <= c
    for (int i = 0; i < w.length; i++)
        for (_______________________________)          // (1) the capacity loop
            dp[c] = _______________________________;    // (2)
    return dp[cap];
}

static int minCoins(int[] coins, int amount) {
    int INF = ______________________;                 // (3)
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, INF);
    ______________________;                           // (4)
    for (int a = 1; a <= amount; a++)
        for (int c : coins) if (c <= a) dp[a] = ______________________;   // (5)
    return _______________________________;           // (6)
}
```

**Which change to blank (1) turns `knapsack01` into the unbounded knapsack? Explain in one sentence why the direction matters.**

### EXERCISE 11.2 Trace the 0/1 knapsack with one array

Run `knapsack01` on four items $(w, v) = (2, 4), (4, 4), (3, 7), (5, 9)$, in this order, with capacity 9. Each row is the array `dp` after the pass for one item. Remember that the pass runs $c$ downwards, so every entry reads the row above it.

| dp[c] after | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| start | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| dp[c] after | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| item (2, 4) | | | | | | | | | | |
| item (4, 4) | | | | | | | | | | |
| item (3, 7) | | | | | | | | | | |
| item (5, 9) | | | | | | | | | | |

**Which items give the answer dp[9], and what is their total weight? Greedy by value per unit of weight takes items in the order (3, 7), (2, 4), (5, 9), (4, 4) while they fit: what does it reach? What would dp[9] be if the capacity loop ran upwards, and which items would that "use"?**

**EXERCISE 11.3 A faded table: coins 1, 4 and 5**

For the coins {1, 4, 5} and the amounts 0..9, fill in five rows: the array of `countCombinations` (*Theory Listing 11.7*) after each pass of its outer loop, the array of `countSequences`, and the array of `minCoins`. The first row and the first four columns are done.

| amount | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| combinations after coin 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| combinations after coin 4 | 1 | 1 | 1 | 1 | | | | | | |
| combinations after coin 5 | 1 | 1 | 1 | 1 | | | | | | |
| ordered sequences | 1 | 1 | 1 | 1 | | | | | | |
| fewest coins | 0 | 1 | 2 | 3 | | | | | | |

**List the combinations for amount 9, and check the sequence count for 9 with the multinomial argument after *Theory Theorem 11.8*. How many coins does "largest coin first" use for amount 8, and how many does the table say?**

### EXERCISE 11.4 Subset sums by hand

Follow *Theory Example 11.2* for the elements [5, 9, 3, 12, 6]: after each element, write the sums that some subset of the elements so far can reach. Each line is the previous line together with the previous line shifted by the new element. The first two lines are done.

| after | reachable sums |
|---|---|
| 5 | 0, 5 |
| 9 | 0, 5, 9, 14 |
| 3 | |
| 12 | |
| 6 | |

**Can these elements be split into two groups with equal sums? What is the smallest possible difference between the two group sums, and which split achieves it? How many subsets have sum 17, and what does `countSubsets` (*Theory Listing 11.8*) return for target 17?**

### EXERCISE 11.5 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. Memoising a recursion whose subproblems never repeat does not reduce the number of calls.
2. Running the capacity loop of `knapsack01` upwards computes the answer of the unbounded knapsack.
3. Swapping the two loops of `minCoins` (coins outside, amounts inside, both upwards) changes the answer.
4. For coins {1, 2, 5} and amount 5, `countCombinations` returns 9.
5. The top-down House Robber of *Theory Listing 11.2* handles $10^6$ houses with Java's default stack.
6. The O($nC$) knapsack is polynomial in the length of the input.
7. `countSubsets(new int[]{0, 0, 3}, 3)` returns 4.
8. A `long[101][100001]` table needs about 80 MB.

**EXERCISE 11.6 Design before you code: spend the grant exactly**

A new problem, not in the plan: a grant of exactly $W \le 10^5$ rupees must be spent in full. There are $n \le 100$ items, item $i$ costs $w[i] \ge 1$ and has usefulness $v[i] \ge 0$, and each item can be bought at most once. Find the largest total usefulness of a set whose prices add up to exactly $W$, or $-1$ if no set does. Write the five parts of the recipe (*Theory Section 11.2*): the state in words, the transition, the base cases, the evaluation order and the answer. Say what changes compared with `knapsack01`, and give the time and memory. Then fill in the final array by hand for the items $(w, v) = (4, 6), (4, 5), (3, 2), (6, 7), (2, 1)$ and $W = 9$.

| c | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| dp[c] at the end | | | | | | | | | | |

**EXERCISE 11.7 Find the bug**

This two-dimensional knapsack was written by copying the loop bounds of the one-array version. It compiles, and on the theory chapter's example (items (1, 1), (3, 4), (4, 5), (5, 7), capacity 7) it returns the correct 9. Find a failing input with three items, say what it returns, explain why, and fix it.

```java
static long knapsackTableBuggy(int[] w, int[] v, int cap) {
    int n = w.length;
    long[][] best = new long[n + 1][cap + 1];
    for (int i = 1; i <= n; i++)
        for (int c = w[i - 1]; c <= cap; c++)
            best[i][c] = Math.max(best[i - 1][c], best[i - 1][c - w[i - 1]] + v[i - 1]);
    return best[n][cap];
}
```

**EXERCISE 11.8 Complexity and limits**

For each task, choose a method from Theory Chapter 11 (or Chapter 9's brute force) and justify it with an operation count. Name one trap.

1. $n \le 100$ items, capacity $\le 10^5$, values up to $10^9$: the largest value with total weight at most the capacity.
2. $n \le 100$ items, weights and capacity up to $10^9$, values at most 1000: the same question.
3. The number of combinations of 100 coin types that make an amount $\le 10^4$, modulo $10^9 + 7$.
4. Can a subset of 200 numbers, each up to $2 \cdot 10^4$, hit a target near $10^6$?
5. House Robber with $n \le 10^6$ houses.
6. $n \le 20$ items with weights, values and capacity up to $10^{15}$.

**EXERCISE 11.9 Write it from memory**

Close the theory book. Write the five recipe lines for the 0/1 knapsack, then `knapsack01`, `knapsackUnbounded` and `countCombinations` from memory. Check them against *Theory Listing 11.5* and *Theory Listing 11.7*, and mark the two loops whose direction or order matters.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**11.P1** **LC 198 · House Robber** MEDIUM CORE — **40 min**

`leetcode.com/problems/house-robber` · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P1*

**11.P2** **LC 322 · Coin Change** MEDIUM CORE — **40 min**

`leetcode.com/problems/coin-change` · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P2*

### 11.P3 LC 518 · Coin Change II MEDIUM CORE — 40 min

leetcode.com/problems/coin-change-ii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P3*

### 11.P4 LC 416 · Partition Equal Subset Sum MEDIUM CORE — 40 min

leetcode.com/problems/partition-equal-subset-sum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P4*

### 11.P5 LC 494 · Target Sum MEDIUM BONUS — 40 min

leetcode.com/problems/target-sum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P5*

### 11.P6 LC 1049 · Last Stone Weight II MEDIUM BONUS — 40 min

leetcode.com/problems/last-stone-weight-ii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P6*

**11.P7** **LC 740 · Delete and Earn** MEDIUM BONUS **40 min**

leetcode.com/problems/delete-and-earn · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 11.P7*

## Re-solve from three days ago

Pick one problem from Day 8 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 8.P1 LC 136 · Single Number ☐ 8.P2 LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR

☐ 8.P3 LC 421 · Maximum XOR of Two Numbers in an Array ☐ 8.P4 LC 1803 · Count Pairs With XOR in a Range

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 12, DP II: two-string DP.*

DAY 12 OF 29 · WEDNESDAY 18 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

12

# Dynamic Programming II: Two Strings, Edit Distance and Palindromes

*dp[i][j] over the prefixes of two strings.*

**Read first** · Theory Chapter 12
**Patterns** · Two-string DP (LCS, edit distance)

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 12; type each new listing once and run it | 40 min |
| ☐ | 12.P1 LC 1143 · Longest Common Subsequence | 40 min |
| ☐ | 12.P2 LC 72 · Edit Distance | 40 min |
| ☐ | 12.P3 LC 516 · Longest Palindromic Subsequence | 40 min |
| ☐ | 12.P4 LC 1312 · Minimum Insertion Steps to Make a String Palindrome | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 12.P5, 12.P6, 12.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- dp[i][j] = answer for the first i characters of s and the first j characters of t.
- Longest palindromic subsequence = LCS(s, reverse(s)); fewest insertions = n − that.
- To print the answer (not just its length), walk the table back from dp[n][m].

**Why it matters for Infosys.** LCS and edit distance are both on the Infosys "very high" list.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| DP recipe: state, transition, base, answer | Ch. 11 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Knapsack family | Ch. 11 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Brute force first, then stress test | Ch. 9 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Greedy with a heap | Ch. 5 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 12 and before the problem worksheets. The first four make you fill tables by hand: three two-string tables and one interval table. Filling a table once by hand is the quickest way to see which neighbour each entry reads and why the walk back works. The rest ask you to judge, design and debug without a printed answer in front of you.

### EXERCISE 12.1 Fill the gaps: walking back

Complete the walk back from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 12.2*.

```
static String lcsString(String s, String t) {
    int[][] dp = lcsTable(s, t);
    StringBuilder sb = new StringBuilder();
    int i = s.length(), j = t.length();
    while (________________________) {                // (1)
        if (________________________) {               // (2)
            sb.append(s.charAt(i - 1));
            ________________________;                 // (3)
        } else if (________________________) {        // (4)
            i--;
        } else {
            j--;
        }
    }
    return ________________________;                  // (5)
}
```

**Why may the loop stop as soon as one of *i* and *j* reaches 0, even if the other has not?**

### EXERCISE 12.2 Fill an LCS table by hand

Fill the LCS table for *s* = `teapot` (rows) and *t* = `potato` (columns) with *Theory Theorem 12.2*. Row 0 and column 0 are done. Then walk back from the corner with the rule of *Theory Listing 12.2* (on a tie, go up) and circle the cells you visit.

| | `""` | p | o | t | a | t | o |
|---|---|---|---|---|---|---|---|
| `""` | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| t | 0 | | | | | | |
| e | 0 | | | | | | |
| a | 0 | | | | | | |

| | "" | p | o | t | a | t | o |
|---|---|---|---|---|---|---|---|
| p | 0 | | | | | | |
| o | 0 | | | | | | |
| t | 0 | | | | | | |

**LCS length = _____. The walk prints _____. Write two other longest common subsequences, and say which tie-break reaches one of them.**

**EXERCISE 12.3 A faded table: edit distance**

Fill the rest of the edit-distance table from $s$ = `cradle` (rows) to $t$ = `ladder` (columns) with *Theory Listing 12.3*. Rows 0 to 2 and column 0 are done.

| | "" | l | a | d | d | e | r |
|---|---|---|---|---|---|---|---|
| "" | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| c | 1 | 1 | 2 | 3 | 4 | 5 | 6 |
| r | 2 | 2 | 2 | 3 | 4 | 5 | 5 |
| a | 3 | | | | | | |
| d | 4 | | | | | | |
| l | 5 | | | | | | |
| e | 6 | | | | | | |

**The edit distance is _____. Walk back from the corner, name each move (match, replace, delete, insert), and apply your edits to `cradle` one at a time to check that you reach `ladder`.**

**EXERCISE 12.4 The interval table for palindromes**

Fill the table dp[$l$][$r$] = LPS length of $s[l..r]$ for $s$ = `cbbdcb` with *Theory Theorem 12.8*. The diagonal (length 1) and the entries of length 2 are done. Fill the remaining entries one diagonal at a time (length 3, then 4, 5 and 6).

| l \ r | 0 c | 1 b | 2 b | 3 d | 4 c | 5 b |
|---|---|---|---|---|---|---|
| 0 c | 1 | 1 | | | | |
| 1 b | — | 1 | 2 | | | |
| 2 b | — | — | 1 | 1 | | |
| 3 d | — | — | — | 1 | 1 | |
| 4 c | — | — | — | — | 1 | 1 |
| 5 b | — | — | — | — | — | 1 |

**LPS = _____, a longest palindromic subsequence is _____, and the fewest insertions are _____ (*Theory Theorem 12.10*). Write one palindrome made from `cbbdcb` with that many insertions.**

**EXERCISE 12.5 Concept check**

Answer true or false, and give a one-line reason or a counterexample.

1. When $s[i-1] \neq t[j-1]$, the LCS entry dp$[i][j]$ always equals dp$[i-1][j-1]$.
2. The edit distance from $s$ to $t$ equals the edit distance from $t$ to $s$.
3. The edit distance is never larger than $\max(n, m)$.
4. *Theory Listing 12.4* can print the edits if you also keep its final row.
5. Every longest common subsequence of $s$ and reverse($s$) is a palindrome.
6. For the palindrome table, the order "$l$ from 0 upwards, $r$ from $l+1$ upwards" is valid.
7. The fewest deletions that leave a palindrome equal the fewest insertions that make one.

**EXERCISE 12.6 Design before you code: the longest common substring**

A **substring** is contiguous. Given $s$ and $t$ of length up to 5000, find the length of the longest string that is a substring of both. First explain in one sentence why the LCS state "best answer for the prefixes" cannot be extended to this problem. Then write the state, the transition, the base cases, the order, the answer, the

time and the memory. Finally compute the answer for $s$ = `xabcdy` and $t$ = `abcxabd`, and compare it with their LCS.

**EXERCISE 12.7 Find the bug**

This two-row edit distance compiles, and it returns the right answer whenever $s$ has at most one character. Find the shortest input on which it fails, say what it returns there, explain the cause, and fix it.

```
static int editDistanceBuggy(String s, String t) {
    int n = s.length(), m = t.length();
    int[] prev = new int[m + 1], cur = new int[m + 1];
    for (int j = 0; j <= m; j++) prev[j] = j;
    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++) {
            if (s.charAt(i - 1) == t.charAt(j - 1)) cur[j] = prev[j - 1];
            else cur[j] = 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
        }
        prev = cur;                       // keep the new row
    }
    return prev[m];
}
```

**EXERCISE 12.8 Complexity and limits**

For each task, choose the method and justify it with an operation count and, where it matters, a memory estimate in bytes. Assume a 256 MB limit and about $10^8$ simple steps per second.

1. $n = m = 5000$: the LCS length only.
2. $n = m = 5000$: print one LCS.
3. $n = m = 10^5$, general lowercase strings: the LCS length. What changes if $s$ and $t$ are both permutations of $1..n$?

4. $|s| = 10^5$, $|t| = 10$: the edit distance.
5. $n \le 1000$: the longest palindromic subsequence.
6. $n = m = 1000$: the number of position pairs of *Theory Listing 12.5*, modulo $10^9 + 7$.

**EXERCISE 12.9** **Write it from memory**

Close the theory book. Write *Theory Listing 12.4* (`editDistanceTwoRows`) from memory, then check it against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**12.P1** **LC 1143 · Longest Common Subsequence** MEDIUM CORE — **40 min**

leetcode.com/problems/longest-common-subsequence · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P1*

**12.P2** **LC 72 · Edit Distance** MEDIUM CORE — **40 min**

leetcode.com/problems/edit-distance · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P2*

**12.P3** **LC 516 · Longest Palindromic Subsequence** MEDIUM CORE — **40 min**

leetcode.com/problems/longest-palindromic-subsequence · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P3*

**12.P4** **LC 1312 · Minimum Insertion Steps to Make a String Palindrome** HARD CORE — **60 min**

leetcode.com/problems/minimum-insertion-steps-to-make-a-string-palindrome · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P4*

**12.P5** **LC 115 · Distinct Subsequences** HARD BONUS — 60 min

leetcode.com/problems/distinct-subsequences · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P5*

**12.P6** **LC 97 · Interleaving String** MEDIUM BONUS — 40 min

leetcode.com/problems/interleaving-string · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P6*

**12.P7** **LC 1092 · Shortest Common Supersequence** HARD BONUS — 60 min

leetcode.com/problems/shortest-common-supersequence · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 12.P7*

## Re-solve from three days ago

Pick one problem from Day 9 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 9.P1 CF 1343C · Alternating Subsequence ☐ 9.P2 CF 1475D · Cleaning the Phone

☐ 9.P3 CF 1251D · Salary Changing

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 13, DP III: LIS and DP with an extra dimension.*

DAY 13 OF 29 · THURSDAY 19 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

13

# Dynamic Programming III: Increasing Subsequences and Extra Dimensions

*LIS in O(n log n), DP over values with a Fenwick tree, and an extra state for a side condition.*

**Read first** · Theory Chapter 13
**Patterns** · Longest increasing subsequence

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 13; type each new listing once and run it | 40 min |
| ☐ | 13.P1 LC 300 · Longest Increasing Subsequence | 40 min |
| ☐ | 13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position | 60 min |
| ☐ | 13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 13.P4, 13.P5, 13.P6* | *60 min* |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- tails + lower bound gives the strict LIS; upper bound gives the non-decreasing one.
- DP over values: dp(x) = 1 + max dp over smaller values, kept in a Fenwick tree of maximums.
- A side condition with few states becomes an extra dimension. The sample Hard problem uses 1024 XOR states.

**Why it matters for Infosys.** Three Infosys repeats are LIS variants: LNDS with XOR ≥ M (13 reports), LIS with a bitwise condition, and mountain-array changes.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Two-string DP (LCS, edit distance) | Ch. 12 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| gcd, fast power, inverse, nCr | Ch. 10 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sieve, prime factors, square-free kernel | Ch. 10 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Binary search: lower and upper bound | Ch. 6 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search on the answer | Ch. 6 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 13 and before the problem worksheets. The first four make you run the chapter's algorithms by hand, which is the fastest way to see why the tails array and the Fenwick tree give the right numbers. The rest ask you to decide, design and debug without a printed answer in front of you.

### EXERCISE 13.1 Fill the gaps: patience sorting

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 13.2*.

```
static int lisStrict(int[] a) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int lo = 0, hi = ______;                  // (1)
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (tails[mid] ______ x) hi = mid;    // (2) strictly increasing
            else lo = mid + 1;
        }
        ______________________;                   // (3)
        ______________________;                   // (4)
    }
    return len;
}
```

**Which single character changes in blank (2) for the longest non-decreasing subsequence, and why?**

### EXERCISE 13.2 Trace patience sorting

Run *Theory Listing 13.2* (strict) on $a$ = [4, 10, 4, 3, 8, 9, 1, 5]. For each element write the position $p$ returned by the binary search, the best length ending at that element ($p + 1$), and `tails` after the step.

| x | position p | p + 1 | tails after |
|---|---|---|---|
| 4 | | | |
| 10 | | | |
| 4 | | | |
| 3 | | | |
| 8 | | | |

| x | position p | p + 1 | tails after |
|---|---|---|---|
| 9 | | | |
| 1 | | | |
| 5 | | | |

**The LIS length is _____. Write one longest increasing subsequence, and explain why the final `tails` is not one.**

**EXERCISE 13.3 A faded table: the mountain**

For $a$ = [2, 5, 3, 6, 4, 1, 7, 2], up[$i$] is the longest strictly increasing subsequence ending at $i$ and down[$i$] the longest strictly decreasing subsequence starting at $i$. The first three columns are done. Fill in the rest, mark the indices that may be a peak (both values at least 2), and compute the fewest removals that leave a mountain.

| i | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| a[i] | 2 | 5 | 3 | 6 | 4 | 1 | 7 | 2 |
| up[i] | 1 | 2 | 2 | | | | | |
| down[i] | 2 | 3 | 2 | | | | | |
| peak allowed? | no | yes | yes | | | | | |
| up + down − 1 | — | 4 | 3 | | | | | |

**Longest mountain = _____, so the fewest removals = _____. Write one longest mountain subsequence.**

**EXERCISE 13.4 The Fenwick tree of maxima by hand**

Compress $a$ = [30, 10, 50, 10, 20, 40] to ranks 1..$V$, then run *Theory Listing 13.6* (strict). For each element write its rank, the query result `prefixMax(fw, rank - 1)`, the new length `cur`, and the array `fw[1..V]` after `raise`. Remember that `raise` at rank $r$ writes nodes $r$, $r$ + lowbit($r$), … up to $V$.

| a[i] | rank | prefixMax(rank − 1) | cur | fw[1..V] after |
|---|---|---|---|---|
| 30 | | | | |
| 10 | | | | |
| 50 | | | | |

| a[i] | rank | prefixMax(rank − 1) | cur | fw[1..V] after |
|---|---|---|---|---|
| 10 | | | | |
| 20 | | | | |
| 40 | | | | |

**Why does the second 10 get `cur` = 1 even though a 10 was seen before?**

## EXERCISE 13.5 Concept check

Answer true or false, and give a one-line reason.

1. After patience sorting finishes, `tails` is a longest increasing subsequence of $a$.
2. For the longest non-decreasing subsequence, each element replaces the first tail strictly greater than it.
3. In the mountain formula, an index with up[$i$] = 1 can be the peak if down[$i$] is large.
4. A Fenwick tree of maxima can process an update that lowers a stored value.
5. For envelopes that nest when both sides are strictly smaller, envelopes of equal width are sorted by height descending.
6. Adding a remainder modulo $K$ as an extra state multiplies the work of the DP by about $K$.

## EXERCISE 13.6 Design before you code: towers of blocks

You have $n \le 10^5$ blocks, each with a width $w$, a height $h$ (both up to $10^9$) and a value $v \ge 1$. A tower is a set of blocks that can be ordered so that both $w$ and $h$ strictly increase from top to bottom. Find the largest total value of a tower. Write the sort order (including ties), what the Fenwick tree stores and is indexed by, the strict or non-strict query, and the complexity. Then compute the answer by hand for the blocks ($w$, $h$, $v$) = (1, 3, 4), (2, 2, 5), (2, 4, 3), (3, 5, 6), (4, 1, 2), (4, 6, 7).

**EXERCISE 13.7** **Find the bug**

This method should return the length of the longest **strictly** increasing subsequence of ranks in 1..$V$. It compiles and passes the sample [3, 1, 2, 4]. Find an input where it fails, say what it returns there, and fix it with one change.

```
static int lisRanks(int[] rank, int V) {
    int[] fw = new int[V + 1];
    int best = 0;
    for (int x : rank) {
        int cur = 1;
        for (int i = x; i > 0; i -= i & -i) cur = Math.max(cur, fw[i] + 1);
        for (int i = x; i <= V; i += i & -i) fw[i] = Math.max(fw[i], cur);
        best = Math.max(best, cur);
    }
    return best;
}
```

**EXERCISE 13.8** **Complexity and limits**

For each task, choose the method from Theory Chapter 13 (or Chapter 9's brute force) and justify it with an operation count.

1. $n \le 3000$: the longest subsequence in which each element is a multiple of the previous one (values may repeat).
2. $n \le 2 \cdot 10^5$, values up to $10^9$: the LIS length.
3. $n \le 2 \cdot 10^5$: the largest total of a strictly increasing subsequence, values up to $10^9$.
4. $N \le 1000$, values at most $N$: the longest non-decreasing subsequence whose XOR is at least $M$ (the sample-paper Hard problem; give the state and the cost, not the code).
5. $n \le 20$, any condition on the subsequence at all.

**EXERCISE 13.9** **Write it from memory**

Close the theory book. Write *Theory Listing 13.4* (`lengthEndingAt` with the `strict` flag) from memory, then check it against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 3 bonus.

### 13.P1 LC 300 · Longest Increasing Subsequence MEDIUM CORE — 40 min

leetcode.com/problems/longest-increasing-subsequence · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P1*

### 13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position HARD CORE — 60 min

leetcode.com/problems/find-the-longest-valid-obstacle-course-at-each-position · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P2*

### 13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array HARD CORE — 60 min

leetcode.com/problems/minimum-number-of-removals-to-make-mountain-array · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P3*

### 13.P4 LC 2370 · Longest Ideal Subsequence MEDIUM BONUS — 40 min

leetcode.com/problems/longest-ideal-subsequence · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P4*

**13.P5** **LC 354 · Russian Doll Envelopes** HARD BONUS — 60 min

leetcode.com/problems/russian-doll-envelopes · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P5*

**13.P6** **LC 673 · Number of Longest Increasing Subsequence** MEDIUM BONUS — 40 min

leetcode.com/problems/number-of-longest-increasing-subsequence · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 13.P6*

## Re-solve from three days ago

Pick one problem from Day 10 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 10.P1 LC 204 · Count Primes ☐ 10.P2 LC 1922 · Count Good Numbers
☐ 10.P3 LC 2447 · Number of Subarrays With GCD Equal to K
☐ 10.P4 LC 2862 · Maximum Element-Sum of a Complete Subset of Indices

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 14, DP IV: divisor, interval and game DP.*

DAY 14 OF 29 · FRIDAY 20 NOVEMBER 2026 · LEARNING DAY · PHASE 2 · NUMBER THEORY AND DP

14

# Dynamic Programming IV: Divisors, Intervals and Games

*Loops over multiples, dp[l][r] by length, and "my score minus yours" for two-player games.*

**Read first** · Theory Chapter 14
**Patterns** · Interval DP and game DP

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 14; type each new listing once and run it | 40 min |
| ☐ | 14.P1 LC 368 · Largest Divisible Subset | 40 min |
| ☐ | 14.P2 LC 486 · Predict the Winner | 40 min |
| ☐ | 14.P3 LC 1547 · Minimum Cost to Cut a Stick | 60 min |
| ☐ | 14.P4 LC 1690 · Stone Game VII | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 14.P5, 14.P6, 14.P7* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- Loop over multiples: `for (m = 2 * v; m <= N; m += v)` costs N·ln N in total.
- Interval DP: `dp[l][r]`, filled by increasing length; $O(n^3)$ for $n \le 500$.
- Two players taking from the ends: `dp[l][r]` = best (my score − opponent's score).

**Why it matters for Infosys.** "Count arrays where each element divides the next" (13 reports) is divisor DP; the hard version is the bonus LC 2338.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Longest increasing subsequence | Ch. 13 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| DP recipe: state, transition, base, answer | Ch. 11 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Knapsack family | Ch. 11 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Monotonic stack | Ch. 7 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 14 and before the problem worksheets. The first four make you fill the chapter's tables by hand: two game tables, a stick table and a table of divisor chains. Fill every cell, even the ones that look obvious, and fill them in the order the code would; in an interval DP the order is half of the idea. The rest ask you to judge, design and debug without a printed answer in front of you.

### EXERCISE 14.1 Fill the gaps: the interval DP template

Complete the template from memory. Each blank is one short expression. Then compare with *Theory Listing 14.3*.

```
static long matrixChain(int[] d) {                    // matrix i is d[i] x d[i+1]
    int n = d.length - 1;
    long[][] dp = new long[n][n];                     // dp[l][l] = 0
    for (int len = ______; len <= n; len++)                       // (1)
        for (int l = 0; l + len - 1 < n; l++) {
            int r = _______________;                              // (2)
            dp[l][r] = _______________;                           // (3)
            for (int k = l; _______________; k++)                 // (4) the last product splits at k
                dp[l][r] = Math.min(dp[l][r],
                        dp[l][k] + _______________ + (long) d[l] * d[k + 1] * d[r + 1]);   // (5)
        }
    return dp[0][n - 1];
}
```

**Why does the outer loop run over the length and not over *l*? What would `d[l] * d[k + 1] * d[r + 1]` give without the `(long)` cast for *d* = [2000, 2000, 2000]?**

### EXERCISE 14.2 Trace two games

First a take-away game. A pile has *s* stones, a move removes 2 or 5 stones, and a player who cannot move loses. Using *Theory Theorem 14.9* (this is *Theory Listing 14.4* with `moves = {2, 5}`), mark each position W if the player to move wins and L if they lose. Position 0 is done.

| s | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| W / L | L | | | | | | | |
| **s** | **8** | **9** | **10** | **11** | **12** | **13** | **14** | **15** |
| W / L | | | | | | | | |

**Which pattern do the losing positions follow? Is 100 a winning position, and if so, which move wins?**

Now the score game of *Theory Theorem 14.10* on $a = [4, 8, 4, 1, 3]$: two players take from either end, and $D(l, r)$ is the best "my total minus yours" for the player to move on $a[l..r]$. Fill in the table by diagonals, shortest segments first. The diagonal is done.

| D(l, r) | r = 0 | r = 1 | r = 2 | r = 3 | r = 4 |
|---|---|---|---|---|---|
| l = 0 | 4 | | | | |
| l = 1 | — | 8 | | | |
| l = 2 | — | — | 4 | | |
| l = 3 | — | — | — | 1 | |
| l = 4 | — | — | — | — | 3 |

**Which end should the first player take, and what are the two totals with perfect play? What are the totals if both players always take the larger end (the left one on a tie)?**

**EXERCISE 14.3 A faded table: cutting a stick**

A stick of length $L = 11$ must be cut at the positions 7, 2, 9 and 3, in any order, and each cut costs the length of the piece it cuts (Figure 14.1). Sort the cuts and add the two ends, so $p = [0, 2, 3, 7, 9, 11]$, and fill the table of *Theory Theorem 14.8* by increasing $j - i$, as in *Theory Example 14.3*. The pieces with nothing to cut ($j = i + 1$) cost 0. The first four rows are done.

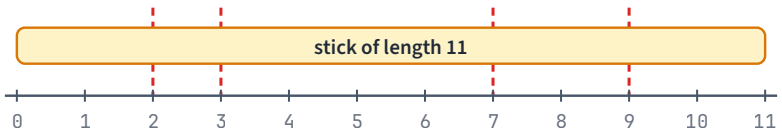



**Figure 14.1** The stick of Exercise 14.3. The dashed lines are the four cuts; the numbers below are positions.

| piece | i, j | candidates dp[i][k] + dp[k][j] | length | dp[i][j] |
|---|---|---|---|---|
| [0, 3] | 0, 2 | k = 1: 0 + 0 | 3 | 3 |
| [2, 7] | 1, 3 | k = 2: 0 + 0 | 5 | 5 |
| [3, 9] | 2, 4 | k = 3: 0 + 0 | 6 | 6 |
| [7, 11] | 3, 5 | k = 4: 0 + 0 | 4 | 4 |

| piece | i, j | candidates dp[i][k] + dp[k][j] | length | dp[i][j] |
|---|---|---|---|---|
| [0, 7] | 0, 3 | | | |
| [2, 9] | 1, 4 | | | |
| [3, 11] | 2, 5 | | | |
| [0, 9] | 0, 4 | | | |
| [2, 11] | 1, 5 | | | |
| [0, 11] | 0, 5 | | | |

**The least total cost is _____. Which cut is made first? Write an optimal order of the four cuts, and the cost of cutting from left to right (2, 3, 7, 9).**

### EXERCISE 14.4 Counting divisor arrays by hand

Count the arrays $b[0..K-1]$ with values in 1..8 in which each element divides the next, with the method of *Theory Theorem 14.5*. First fill cnt($s$, $v$), the number of strictly increasing divisor chains with $s$ values that end at $v$: cnt(1, $v$) = 1, and cnt($s$ + 1, $m$) is the sum of cnt($s$, $v$) over the proper divisors $v$ of $m$. By *Theory Claim 14.4* no chain in 1..8 has more than 4 values.

| v | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | sum |
|---|---|---|---|---|---|---|---|---|---|
| cnt(1, v) | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 8 |
| cnt(2, v) | | | | | | | | | |
| cnt(3, v) | | | | | | | | | |
| cnt(4, v) | | | | | | | | | |

Now combine the chain totals with the binomials for $K = 4$.

| s | chains with s values | C(K − 1, s − 1) | contribution |
|---|---|---|---|
| 1 | 8 | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| total | | | |

**Repeat the last table for $K = 3$. For $K = 100$ with the answer modulo 10000, which binomials do you need, and what is the answer? Why must they come from Pascal's rule and not from factorials?**

**EXERCISE 14.5 Concept check**

Answer true or false, and give a one-line reason.

1. The loop over multiples of *Theory Listing 14.2* makes about $10^9$ steps for $N = 10^6$.
2. A divisor chain of distinct values in $1..10^9$ has at most 30 values.
3. Modulo 10000, $C(K-1, s-1)$ can be computed as a factorial times two modular inverses.
4. For the stick DP, filling $i$ from $m+1$ down to 0 and, for each $i$, $j$ from $i+1$ up to $m+1$ is a valid order.
5. In the take-from-either-end game, the first player should take the larger end whenever the two ends differ.
6. In that game, players who each maximise their own total make the same moves as players who maximise the difference.
7. With an odd number of elements, the first player of that game can still always secure at least half of the total.
8. With moves $\{2, 5\}$, the player to move from 22 stones loses.

**EXERCISE 14.6 Design before you code: merging piles**

A new problem, not in the plan. There are $n \le 400$ piles of stones in a row, each of size at most $10^6$. A move merges two **adjacent** piles into one pile, which takes their place and costs its new size. You merge until one pile remains. Find the least total cost.

Following the tip at the end of Theory Chapter 14, write the meaning of dp[$l$][$r$] in one sentence, the base case, the single operation that splits a segment (the first or the last?), the recurrence, the evaluation order, the time and memory, and the type of the answer. Then fill the table by hand for the piles [7, 5, 4, 7], and compare with the greedy rule "always merge the cheapest adjacent pair".

**EXERCISE 14.7 Find the bug**

This version of *Theory Listing 14.6* runs each pass from right to left. It compiles, and it returns the right differences on both samples of LeetCode 486 (Predict the Winner): −2 for [1, 5, 2] and 222 for [1, 5, 233, 7]. Find a row of three elements on which it fails, say what it returns and what is correct, and fix it with one change.

```
static long bestScoreDifferenceRollingBuggy(int[] a) {
    int n = a.length;
    long[] d = new long[n];
    for (int i = 0; i < n; i++) d[i] = a[i];
    for (int len = 2; len <= n; len++)
        for (int l = n - len; l >= 0; l--) {
            int r = l + len - 1;
            d[l] = Math.max(a[l] - d[l + 1], a[r] - d[l]);
        }
    return n == 0 ? 0 : d[0];
}
```

**EXERCISE 14.8 Complexity and limits**

For each task, choose a method from Theory Chapter 14, give an operation count, and say whether time or memory is the binding limit. *Theory Table 14.2* and *Theory Table 14.1* have most of the numbers.

1. $n \le 1000$ distinct values up to $2 \cdot 10^9$: the largest subset in which every pair divides.
2. The number of divisors of every integer from 1 to $10^6$.
3. Burst Balloons (the setting of the bonus problem LeetCode 312) with $n \le 500$ balloons.
4. $n \le 5000$ numbers in a row, two players taking from the ends: the first player's best margin, with 256 MB of memory.
5. The cheapest order of multiplying $n \le 5000$ matrices.
6. Arrays of length $K \le 10^4$ with values in $1..10^4$, each dividing the next, counted modulo 10000.

7. The take-away game with moves {1, 3, 4} and a pile of up to $10^{18}$ stones.

**EXERCISE 14.9** **Write it from memory**

Close the theory book. Write *Theory Listing 14.5* (the table version) and *Theory Listing 14.6* (one array) from memory, then check them against the listings line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

### 14.P1 LC 368 · Largest Divisible Subset MEDIUM CORE — 40 min

leetcode.com/problems/largest-divisible-subset · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P1*

### 14.P2 LC 486 · Predict the Winner MEDIUM CORE — 40 min

leetcode.com/problems/predict-the-winner · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P2*

### 14.P3 LC 1547 · Minimum Cost to Cut a Stick HARD CORE — 60 min

leetcode.com/problems/minimum-cost-to-cut-a-stick · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P3*

### 14.P4 LC 1690 · Stone Game VII MEDIUM CORE — 40 min

leetcode.com/problems/stone-game-vii · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P4*

**14.P5** **LC 2338 · Count the Number of Ideal Arrays** HARD BONUS — 60 min

leetcode.com/problems/count-the-number-of-ideal-arrays · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P5*

**14.P6** **LC 312 · Burst Balloons** HARD BONUS — 60 min

leetcode.com/problems/burst-balloons · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P6*

**14.P7** **LC 664 · Strange Printer** HARD BONUS — 60 min

leetcode.com/problems/strange-printer · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 14.P7*

## Re-solve from three days ago

Pick one problem from Day 11 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 11.P1 LC 198 · House Robber ☐ 11.P2 LC 322 · Coin Change ☐ 11.P3 LC 518 · Coin Change II
☐ 11.P4 LC 416 · Partition Equal Subset Sum

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 15, Graphs I: BFS, DFS, topological order.*

DAY 15 OF 29 · SATURDAY 21 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

15

# Graphs I: Breadth-First Search, Depth-First Search and Topological Order

*Shortest paths by edges, grids, multi-source BFS, Kahn's algorithm and DP on a DAG.*

**Read first** · Theory Chapter 15
**Patterns** · BFS: fewest edges, grids, many sources, DFS without recursion, components, Topological order (Kahn) and DAG DP, 0-1 BFS

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 15; type each new listing once and run it | 40 min |
| ☐ | 15.P1 LC 200 · Number of Islands | 40 min |
| ☐ | 15.P2 LC 994 · Rotting Oranges | 40 min |
| ☐ | 15.P3 LC 207 · Course Schedule | 40 min |
| ☐ | 15.P4 LC 1091 · Shortest Path in Binary Matrix | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 15.P5, 15.P6, 15.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- BFS finds the fewest edges. For several sources, push them all first with distance 0.
- Mark a node visited when you push it, not when you pop it.
- Kahn's algorithm: repeatedly remove nodes with indegree 0. Leftover nodes mean a cycle.
- Costs of only 0 and 1: a deque replaces the heap (0 → front, 1 → back).

**Why it matters for Infosys.** Recent reports describe the hard question as "shortest path with a twist".

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Interval DP and game DP | Ch. 14 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Two-string DP (LCS, edit distance) | Ch. 12 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Bit tricks and XOR | Ch. 8 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary trie for XOR | Ch. 8 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| XOR basis (maximum subset XOR) | Ch. 8 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Java collections toolkit | Ch. 1 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Fast input and output (FastScanner) | Ch. 1 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Linked list moves | Ch. 1 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 15 and before the problem worksheets. The first four make you run the chapter's traversals by hand: BFS layers on a graph and on a grid, Kahn's counters, and the 0-1 BFS deque. Keep the adjacency lists in exactly the order given, because the order decides which vertex is found first, and write the queue or deque after every step. The rest ask you to judge, design and debug on your own.

### EXERCISE 15.1 Fill the gaps: Kahn's algorithm

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 15.6*.

```
static int[] topoOrder(List<Integer>[] g) {
    int n = g.length;
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++) for (int v : g[u]) _____________;          // (1)
    int[] order = new int[n];
    int head = 0, tail = 0;
    for (int u = 0; u < n; u++) if (_____________) order[tail++] = u;      // (2)
    while (_____________) {                                                // (3)
        int u = order[head++];
        for (int v : g[u]) if (_____________) order[tail++] = v;           // (4)
    }
    return ____________________;                                           // (5)
}
```

**Which positions of `order` play the role of the queue? Why can the test in blank (4) succeed at most once for each vertex?**

### EXERCISE 15.2 Trace BFS: on a graph and on a grid

Run *Theory Listing 15.3* from vertex 0 on the undirected graph of Figure 15.1. The lists are built by *Theory Listing 15.1* from the edges in the order 0–1, 0–2, 0–3, 1–4, 2–4, 3–5, 4–6, 5–6, 6–7, 5–7, so `g[4]` = [1, 2, 6] and `g[5]` = [3, 6, 7]. For each pop, write the vertices discovered, the neighbours that were already known, the queue afterwards and the array `dist`.

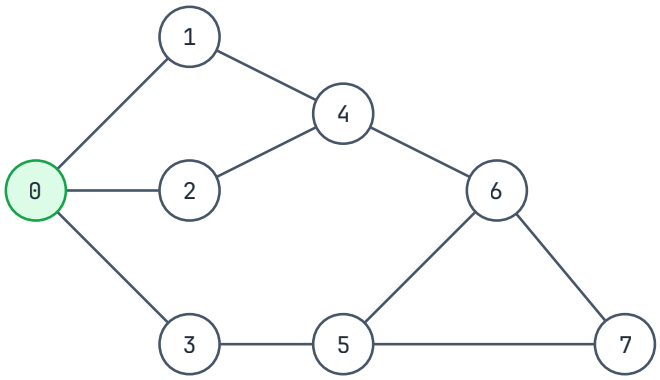



**Figure 15.1** The undirected graph of Exercise 15.2. BFS starts at vertex 0 (green).

| pop | dist[u] | newly discovered | already known | queue after | dist[0..7] |
|---|---|---|---|---|---|
| 0 | 0 | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

**List the BFS layers and the edges of the shortest-path tree (the edge along which each vertex was discovered).**

Now a grid. The two cells marked 0 are sources and # is a wall; moves go up, down, left and right. Using *Theory Theorem 15.4*, write in every open cell its distance to the nearer source, or −1 if no source can reach it.

| | c0 | c1 | c2 | c3 | c4 | c5 |
|---|---|---|---|---|---|---|
| r0 | 0 | | | # | | |
| r1 | | # | | # | | |
| r2 | | # | | | | 0 |
| r3 | | | | # | # | |
| r4 | # | | # | | | # |

**If the sources were rotten oranges and every open cell held a fresh one, what would LeetCode 994 (Rotting Oranges) return? What if the cells you marked −1 were walls instead?**

### EXERCISE 15.3 A faded trace: Kahn's counters

Run *Theory Listing 15.6* on the DAG of Figure 15.2. The edges were added in the order 2→6, 5→6, 5→3, 0→3, 0→4, 6→1, 3→1, 3→4, 1→4, so `g[0]` = [3, 4], `g[3]` = [1, 4] and `g[5]` = [6, 3]. The initial indegrees are [0, 2, 0, 2, 3, 0, 2] and the queue starts as [0, 2, 5]. The first two rows are done.

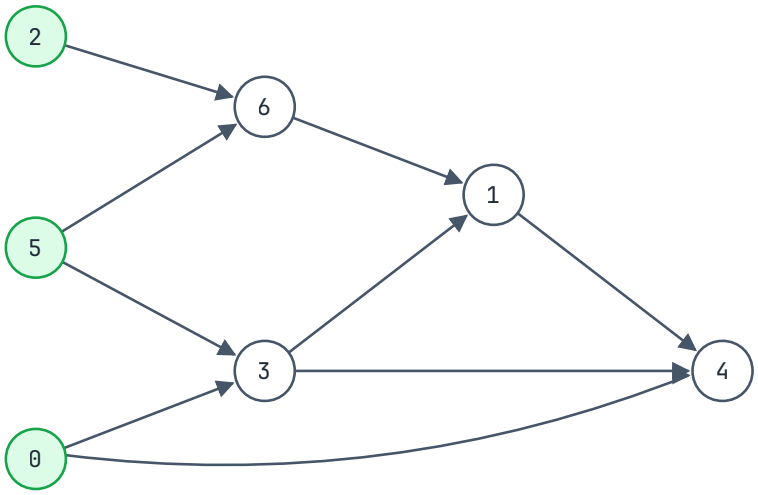



**Figure 15.2** The DAG of Exercise 15.3. The sources 0, 2 and 5 are green.

| take | counters decremented (new value) | queue after | indeg after |
|---|---|---|---|
| 0 | 3 (1), 4 (2) | [2, 5] | [0, 2, 0, 1, 2, 0, 2] |
| 2 | 6 (1) | [5] | [0, 2, 0, 1, 2, 0, 1] |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

**Write the order that is output. Then run *Theory Listing 15.7* in that order: the longest path (in edges) ending at each vertex, and the number of paths from vertex 5 to each vertex. Finally add the edge 4 → 6: which vertices are output now, and what does the method return?**

### EXERCISE 15.4 Trace the 0-1 BFS deque

Run *Theory Listing 15.8* from vertex 0 on the directed graph of Figure 15.3, whose edges are labelled with their costs. The adjacency lists hold the edges in this order: 0→1 (1), 0→2 (1), 0→3 (0), 3→2 (0), 3→1 (1), 1→4 (0), 2→4 (1), 2→5 (1), 4→5 (0). For each pop write the relaxations that succeed (and whether the vertex goes to the front or the back), the deque afterwards, front first, and the array $d$ (∞ for unreached). Some pops change nothing.

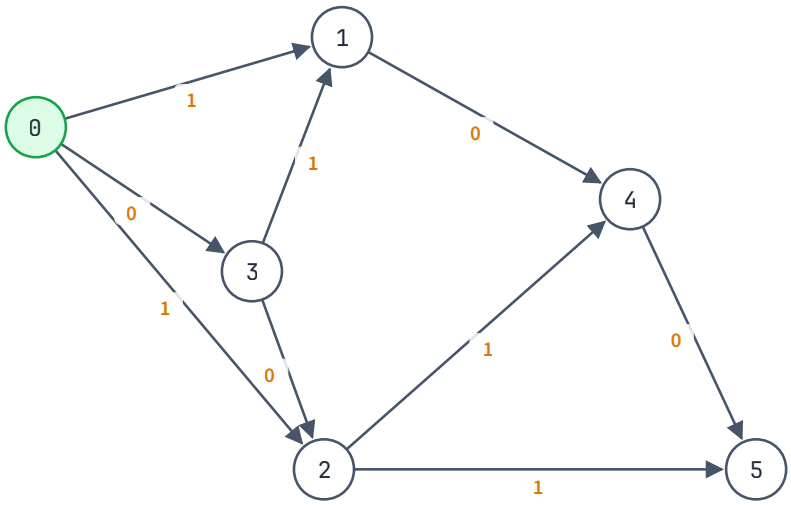



**Figure 15.3** The directed graph of Exercise 15.4. Free edges cost 0, paid edges cost 1; the source is vertex 0.

| pop | d[u] | relaxations that succeed | deque after | d[0..5] |
|---|---|---|---|---|
| 0 | 0 | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

**Which vertex was pushed twice, and which pop was an outdated entry? What distances does plain BFS (*Theory Listing 15.3*, ignoring the costs) report on this graph?**

### EXERCISE 15.5 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. In *Theory Listing 15.3*, the `dist` of a vertex can decrease after it is first set.
2. If the queue of *Theory Listing 15.3* is replaced by a stack (still marking on push), the distances stay correct on every tree, but not on every graph.
3. A graph with $10^5$ vertices and no edges has $10^5$ connected components.
4. A graph that contains a cycle of length 4 cannot be bipartite.
5. If *Theory Listing 15.6* returns `null`, every vertex that was not output lies on a cycle.
6. For the same graph, Kahn's algorithm can output different valid orders if the adjacency lists are stored in a different order.
7. In 0-1 BFS a vertex can be popped three times.
8. Multi-source BFS from $k$ sources costs $k$ times as much as a BFS from one source.

### EXERCISE 15.6 Design before you code: one hammer

A new problem, not in the plan. A robot walks on an $R \times C$ grid ($R$, $C \le 1000$) of open cells (.) and walls (#), one step up, down, left or right at a time. It carries one hammer, which lets it walk through **at most one** wall cell. Find the fewest steps from the top-left cell to the bottom-right cell, or $-1$.

Plain grid BFS keeps one distance per cell, and that is not enough: write down what a vertex of the search must remember, the moves between vertices, the start, where the answer is read, and the time and memory. Then explain on the grid of Figure 15.4 why "one distance per cell, plus a flag saying whether that first arrival used the hammer" gives a wrong answer, and find the correct one by hand.

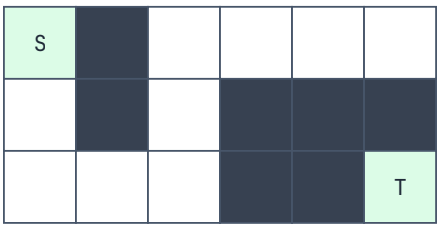



**Figure 15.4** The grid of Exercise 15.6. The robot starts at S and must reach T; dark cells are walls.

### EXERCISE 15.7 Find the bug

This version of *Theory Listing 15.8* copies the rule of plain BFS: a vertex is final as soon as it is reached. It compiles, and it is correct on every graph whose costs are all 1. Find a directed graph with three vertices on which it returns a wrong distance, say which value it returns and what is correct, and fix it.

```
static int[] zeroOneBfsBuggy(List<int[]>[] g, int src) {
    int[] d = new int[g.length];
    Arrays.fill(d, Integer.MAX_VALUE);
    d[src] = 0;
    ArrayDeque<Integer> dq = new ArrayDeque<>();
    dq.add(src);
    while (!dq.isEmpty()) {
        int u = dq.pollFirst();
        for (int[] e : g[u]) {
            int v = e[0];
            if (d[v] != Integer.MAX_VALUE) continue;      // already reached: skip it, as in BFS
            d[v] = d[u] + e[1];
            if (e[1] == 0) dq.addFirst(v);
            else dq.addLast(v);
        }
    }
    return d;
}
```

**EXERCISE 15.8** **Complexity and limits**

For each task, choose a method from Theory Chapter 15, give an operation or memory count, and name one trap.

1. A $1000 \times 1000$ grid with walls: the fewest steps from one corner to every cell.
2. $n = 10^5$ vertices and $m = 10^6$ directed edges: how to store the graph, and what each way costs in memory.
3. A tree of $2 \cdot 10^5$ nodes given as a parent array, possibly a single long path: visit every node.
4. A town of $10^5$ houses and $3 \cdot 10^5$ two-way roads, with 1000 hospitals among the houses: for every house, the fewest roads to the nearest hospital.
5. A grid of $10^6$ cells where each move costs 0 or 1.
6. $10^5$ courses and $2 \cdot 10^5$ prerequisite pairs: can every course be taken?
7. $10^5$ vertices and $2 \cdot 10^5$ directed edges, each of cost 1 or 2: all distances from vertex 0.

**EXERCISE 15.9** **Write it from memory**

Close the theory book. Write *Theory Listing 15.4* (grid BFS from several sources) from memory, including the direction arrays, then check it against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**15.P1 LC 200 · Number of Islands** MEDIUM CORE — **40 min**

`leetcode.com/problems/number-of-islands` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P1*

**15.P2 LC 994 · Rotting Oranges** MEDIUM CORE — **40 min**

`leetcode.com/problems/rotting-oranges` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P2*

**15.P3 LC 207 · Course Schedule** MEDIUM CORE — **40 min**

`leetcode.com/problems/course-schedule` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P3*

**15.P4 LC 1091 · Shortest Path in Binary Matrix** MEDIUM CORE — **40 min**

`leetcode.com/problems/shortest-path-in-binary-matrix` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P4*

**15.P5** **LC 785 · Is Graph Bipartite?** MEDIUM BONUS — 40 min

leetcode.com/problems/is-graph-bipartite · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P5*

**15.P6** **LC 1368 · Minimum Cost to Make at Least One Valid Path in a Grid** HARD BONUS — 60 min

leetcode.com/problems/minimum-cost-to-make-at-least-one-valid-path-in-a-grid · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P6*

**15.P7** **LC 2050 · Parallel Courses III** HARD BONUS — 60 min

leetcode.com/problems/parallel-courses-iii · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 15.P7*

## Re-solve from three days ago

Pick one problem from Day 12 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 12.P1 LC 1143 · Longest Common Subsequence ☐ 12.P2 LC 72 · Edit Distance
☐ 12.P3 LC 516 · Longest Palindromic Subsequence
☐ 12.P4 LC 1312 · Minimum Insertion Steps to Make a String Palindrome

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 16, Mock 2 + review.*

DAY 16 OF 29 · SUNDAY 22 NOVEMBER 2026 · MOCK DAY · PHASE 2 · NUMBER THEORY AND DP

16

# Analysing a Problem Before You Code

*3 hours with number theory and DP added, then a full review.*

**Read first** · Theory Chapter 16
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 16.P1 CF 1367C · Social Distance | 30 min |
| ☐ | 16.P2 CF 1373D · Maximum Sum on Even Positions | 45 min |
| ☐ | 16.P3 CF 1475G · Strange Beauty | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Same rules as Mock 1. Use Codeforces *Custom test* to run the samples before submitting.
- After the mock: scores, editorials, re-code, error log.
- Compare with the SP checkpoints. Below them? Drop next week's bonus problems and redo DP core problems from Days 11–14.

**Why it matters for Infosys.** Mock 2 is the checkpoint for Q1 + Q2 fully solved.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| BFS: fewest edges, grids, many sources | Ch. 15 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| DFS without recursion, components | Ch. 15 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Topological order (Kahn) and DAG DP | Ch. 15 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| 0-1 BFS | Ch. 15 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Longest increasing subsequence | Ch. 13 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Brute force first, then stress test | Ch. 9 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Prefix sums + HashMap | Ch. 2 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Difference array (many range adds) | Ch. 2 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Mock exam: Mock 2 · number theory and DP added

Q1 ≈ 1300, Q2 ≈ 1600, Q3 ≈ 1900. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

### Planning grid (first 15 minutes)

| Q | Limits | Idea | Complexity | Brute-force fallback | Order |
|---|---|---|---|---|---|
| Q1 | | | | | |
| Q2 | | | | | |
| Q3 | | | | | |

### Score sheet

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q1 | **CF 1367C** Social Distance<br>codeforces.com/problemset/problem/1367/C | 1300 | 20 | | | | |
| Q2 | **CF 1373D** Maximum Sum on Even Positions<br>codeforces.com/problemset/problem/1373/D | 1600 | 30 | | | | |
| Q3 | **CF 1475G** Strange Beauty<br>codeforces.com/problemset/problem/1475/G | 1900 | 50 | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

**SP checkpoint.** Q1 + Q2 full, weighted score ≥ 50/100.

### Post-mortem (about 60 minutes after the mock)

| Q | What went wrong | Cause | Rule that would have prevented it |
|---|---|---|---|
| Q1 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q2 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q3 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |

## Exercises

Do these after the mock and its post-mortem, or the next morning if you are tired. They practise the habits of Theory Chapter 16 on small new problems: stripping a story down to its model, bounding loops whose inner part varies, budgeting time and memory, choosing types, and testing the edge cases before you submit. None of them uses or hints at a mock problem.

**EXERCISE 16.1** **From story to model**

For each story, write the model of *Theory Definition 16.1* (objects, question type, limits), the complexity the limits allow (*Theory Table 16.2*), the technique you would try first and its chapter (*Theory Table 16.6*), and one trap.

1. A shop has recorded its profit on each of $n \le 2 \cdot 10^5$ days; a day's profit is between $-10^9$ and $10^9$. The owner wants to know how many periods of consecutive days had a total profit of exactly $K$.
2. A teacher has $n \le 10^5$ students and a list of $m \le 2 \cdot 10^5$ pairs of students who quarrel. She has two rooms. Can she place every student in a room so that no quarrelling pair shares a room? Print YES or NO.
3. $n \le 10^5$ documents with up to $10^9$ pages each must be printed in their given order on $k \le n$ printers. Each printer receives one block of consecutive documents. Make the largest number of pages given to any printer as small as possible.
4. A build tool knows $n \le 10^5$ modules and $m \le 2 \cdot 10^5$ rules of the form "module $x$ must be built before module $y$". Print an order in which every module can be built, or IMPOSSIBLE.

**Story 1:**

**Story 2:**

**Story 3:**

**Story 4:**

---

### EXERCISE 16.2 Aggregate counting

Each method counts the steps of its inner loop. For each one, give a bound in terms of $n$ (the length of the array, or the argument) and the event you charge the work to, as in *Theory Theorem 16.1*. Then count the steps by hand for the input shown: `loopA` with $a$ = [3, 1, 4, 1, 5, 9, 2, 6] and $k$ = 8, `loopD` with the same $a$, and the others with $n$ = 1000.

```
static long loopA(int[] a, int k) {            // every a[i] >= 1
    long steps = 0, sum = 0;
    int l = 0;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum > k) {
            sum -= a[l];
            l++;
            steps++;
        }
    }
    return steps;
}

static long loopB(int n) {
    long steps = 0;
    for (int len = 1; len < n; len *= 2)
        for (int i = 0; i < len; i++) steps++;
    return steps;
}

static long loopC(int n) {
    long steps = 0;
    for (int i = 0; i < n; i++) {
        int j = 0;
        while (j < i) {
            j++;
            steps++;
        }
    }
    return steps;
}

static long loopD(int[] a) {
    long steps = 0;
    ArrayDeque<Integer> st = new ArrayDeque<>();
    for (int x : a) {
        while (!st.isEmpty() && st.peek() <= x) {
            st.pop();
```

```
            steps++;
        }
        st.push(x);
        steps++;
    }
    return steps;
}

static long loopE(int n) {                      // a binary counter, incremented n times
    int[] bits = new int[32];
    long flips = 0;
    for (int t = 0; t < n; t++) {
        int i = 0;
        while (bits[i] == 1) {
            bits[i] = 0;
            i++;
            flips++;
        }
        bits[i] = 1;
        flips++;
    }
    return flips;
}
```

| method | bound | the work is charged to | count on the given input |
|---|---|---|---|
| loopA | | | |
| loopB | | | |
| loopC | | | |
| loopD | | | |
| loopE | | | |

**One of the five is not linear. Which one, and what is the difference from `loopA`? For `loopE`, why is the total below $2n$ although a single increment can flip many bits?**

**EXERCISE 16.3 Operation and memory budgets**

The limits are 1 second and 256 MB. For each idea, estimate the operations or the bytes with *Theory Table 16.3* and *Theory Table 16.4*, say whether it fits, and name a better choice where it does not (or where it only just fits).

1. A `HashMap<Integer, Integer>` that counts $2 \cdot 10^6$ values, each at most $10^6$.
2. $10^6$ edges stored as `int[1000000][2]`.
3. A DP table `long[5000][5000]` in which row $i$ reads only row $i - 1$.
4. Sorting $10^6$ values stored as `Integer[]`.
5. $3 \cdot 10^8$ steps of `x * y % MOD`.
6. A double loop over all pairs $i < j$, with $n = 10^5$.
7. A recursive DFS on a tree of $10^5$ nodes that may be a single path.

| idea | operations or bytes | fits? | better choice |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |

**EXERCISE 16.4** **int or long?**

In each line, `a`, `b` and `n` are `int`s, `u`, `v` and `w` are `long`s, and `MOD` is `1_000_000_007L`. Say whether the line overflows or gives a wrong value, write the value Java actually computes for the values given, and fix the line. Check yourself against *Theory Table 16.5* and the pitfall after it.

1. `long p = a * b;` with $a = b = 100\,000$.
2. `int mid = (a + b) / 2;` with $a = 1.5 \cdot 10^9$ and $b = 2 \cdot 10^9$.
3. `int t = a + b;` with $a = b = 10^9$.
4. `int s = 0; for (int x : arr) s += x;` on $2 \cdot 10^5$ values, all equal to $10^9$.
5. `long r = u * v * w % MOD;` with $u, v, w = 999\,999\,999$, $999\,999\,998$, $999\,999\,997$.
6. `long d = (a - b) % MOD;` with $a = 3$ and $b = 5$.
7. `int m = Math.abs(a);` with $a = -2^{31}$ (`Integer.MIN_VALUE`).
8. `long pairs = n * (n - 1) / 2;` with $n = 10^5$.

| line | wrong? | Java computes | fix |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| 8 | | | |

**EXERCISE 16.5** **Design the edge cases**

A new problem: given $n \geq 1$ integers with $|a[i]| \leq 10^9$ and $1 \leq k \leq n$, print the largest sum of $k$ consecutive elements. The samples are [2, −1, 3, 4, −2] with $k = 2$ (answer 7) and [1, 2, 3, 1, 2, 3] with $k = 3$ (answer 6). The solution below passes both samples.

```
static long maxWindowSumBuggy(int[] a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    long best = 0;
    for (int i = k; i < a.length; i++) {
        sum += a[i] - a[i - k];
        best = Math.max(best, sum);
    }
    return best;
}
```

Run it by hand on the six arrays that *Theory Listing 16.5* produces for $n = 5$ and $maxV = 10^9$, with $k = 1$ for the first and $k = 3$ for the others. Write the correct answer and what the method returns.

| input | k | correct answer | method returns |
|---|---|---|---|
| [$10^9$] | 1 | | |
| [7, 7, 7, 7, 7] | 3 | | |
| [1, 2, 3, 4, 5] | 3 | | |
| [5, 4, 3, 2, 1] | 3 | | |
| [$10^9$, $10^9$, $10^9$, $10^9$, $10^9$] | 3 | | |
| [$-10^9$, $10^9$, $-10^9$, $10^9$, $-10^9$] | 3 | | |

**Which tests fail? Name the two bugs and the edge case of *Theory Section 16.6* that exposes each, and fix the method. Which case from the checklist does the generator not produce, and which wrong version would it catch here?**

## Re-solve from three days ago

Pick one problem from Day 13 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- ☐ 13.P1 LC 300 · Longest Increasing Subsequence
- ☐ 13.P2 LC 1964 · Find the Longest Valid Obstacle Course at Each Position
- ☐ 13.P3 LC 1671 · Minimum Number of Removals to Make Mountain Array

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 17, Graphs II: Dijkstra, DSU, MST.*

DAY 17 OF 29 · MONDAY 23 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

17

# Graphs II: Shortest Paths, Union-Find and Spanning Trees

*Weighted shortest paths, expanded state graphs, union-find and Kruskal.*

**Read first** · Theory Chapter 17
**Patterns** · Dijkstra, Union-find (DSU) and Kruskal

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 17; type each new listing once and run it | 40 min |
| ☐ | 17.P1 LC 743 · Network Delay Time | 40 min |
| ☐ | 17.P2 LC 1631 · Path With Minimum Effort | 40 min |
| ☐ | 17.P3 LC 1584 · Min Cost to Connect All Points | 40 min |
| ☐ | 17.P4 LC 1976 · Number of Ways to Arrive at Destination | 40 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 17.P5, 17.P6, 17.P7* | *60 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- Dijkstra with `PriorityQueue<long[]>`; skip stale entries (`if (d != dist[u]) continue`).
- Expanded state: a node becomes (vertex, stops used) or (vertex, coupon used).
- Counting shortest paths: when you find an equal distance, add the ways.
- Kruskal = sort edges + DSU.

**Why it matters for Infosys.** "Connect cities at minimum cost" (MST) and Dijkstra are both repeat topics.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Interval DP and game DP | Ch. 14 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| gcd, fast power, inverse, nCr | Ch. 10 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sieve, prime factors, square-free kernel | Ch. 10 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Two pointers on a sorted array | Ch. 3 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sliding window (variable size) | Ch. 3 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Window maximum (monotonic deque) | Ch. 3 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 17 and before the problem worksheets. The first four make you run Dijkstra's heap, the union-find arrays and Kruskal's scan by hand. Write down every pair and every array exactly as the code would hold it: the stale pairs and the path-halving changes are where most mistakes hide. The rest ask you to judge, design and debug on your own.

### EXERCISE 17.1 Fill the gaps: Dijkstra with lazy deletion

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 17.2*.

```
static long[] dijkstra(List<int[]>[] g, int src) {       // g[u] holds {to, weight}
    long[] d = new long[g.length];
    Arrays.fill(d, Long.MAX_VALUE);
    ______________________;                                          // (1)
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> _______________);   // (2)
    pq.add(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[1];
        if (______________________) continue;                        // (3) stale entry
        for (int[] e : g[u]) {
            long nd = _______________;                               // (4)
            if (nd < d[e[0]]) {
                d[e[0]] = nd;
                ______________________________;                      // (5)
            }
        }
    }
    return d;
}
```

**Why is blank (2) not `(int) (x[0] - y[0])`? Give two distances that this version compares the wrong way round.**

### EXERCISE 17.2 Trace Dijkstra's heap

Run *Theory Listing 17.2* from vertex 0 on the directed graph of Figure 17.1. The adjacency lists hold the edges in this order: 0 → 1 (9), 0 → 2 (2), 2 → 1 (3), 2 → 3 (5), 2 → 4 (9), 1 → 3 (1), 1 → 4 (9), 3 → 4 (2), 3 → 5 (9), 4 → 5 (2). Each row is one pair taken from the heap. Write whether it settles its vertex or is stale, the relaxations that succeed, the array *d* after the row, and the pairs left in the heap, smallest first. The heap is empty after the tenth pair. No two pairs in the heap ever have the same distance, so the order is fixed.

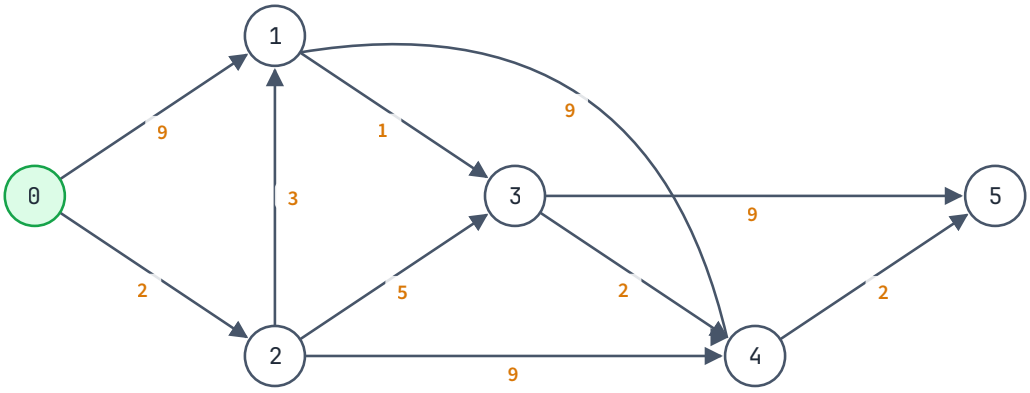



**Figure 17.1** The directed graph of Exercise 17.2. Edge labels are weights; the source is vertex 0.

| step | pair taken | settle / stale | relaxations that succeed | d[0..5] after | heap afterwards |
|---|---|---|---|---|---|
| 1 | (0, 0) | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |
| 9 | | | | | |
| 10 | | | | | |

**How many pairs were pushed, and how many of them were stale? Compare with the bound $E + 1$ of the theory chapter. Which edges form the shortest-path tree?**

### EXERCISE 17.3 Union-find arrays by hand

Start with nine singletons 0..8 and apply the nine unions below with *Theory Listing 17.6* (union by size, path halving; on equal sizes the root of the first argument stays the root). After each union write its result, the whole `parent` array, and the size of every root.

| operation | result | parent[0..8] | sizes of the roots |
|---|---|---|---|
| start | — | [0, 1, 2, 3, 4, 5, 6, 7, 8] | all 1 |
| union(0, 1) | | | |
| union(3, 2) | | | |
| union(2, 0) | | | |
| union(4, 5) | | | |
| union(6, 7) | | | |
| union(5, 7) | | | |
| union(8, 1) | | | |
| union(7, 1) | | | |
| union(0, 7) | | | |

**In which rows did a `find` change `parent` by path halving, and which entry did it change? Draw the final forest.**

**EXERCISE 17.4 A faded trace: Kruskal's scan**

Run `mst` from *Theory Listing 17.6* on the undirected graph of Figure 17.2. The input lists the edges in the order 0–1, 1–2, 0–3, 1–4, 2–5, 3–4, 4–5, 0–4, 1–5, 2–6, 5–6. Java's object sort is stable, so equal weights keep that order. The first three rows are done. Fill in the rest.

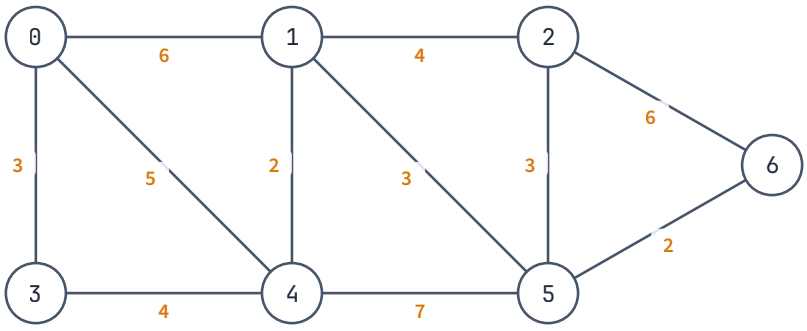



**Figure 17.2** The undirected graph of Exercise 17.4 with its edge weights.

| row | edge | weight | already in one group? | kept? | total | groups left |
|---|---|---|---|---|---|---|
| 1 | 1–4 | 2 | no | keep | 2 | 6 |

| row | edge | weight | already in one group? | kept? | total | groups left |
|---|---|---|---|---|---|---|
| 2 | 5–6 | 2 | no | keep | 4 | 5 |
| 3 | 0–3 | 3 | no | keep | 7 | 4 |
| 4 | | | | | | |
| 5 | | | | | | |
| 6 | | | | | | |
| 7 | | | | | | |
| 8 | | | | | | |
| 9 | | | | | | |
| 10 | | | | | | |
| 11 | | | | | | |

**The MST weighs _____. Use *Theory Theorem 17.11* to find the minimax cost from 0 to 6. Is the shortest path from 0 to 6 (by total weight) the same route?**

**EXERCISE 17.5 Concept check**

Answer true or false, and give a one-line reason or a counterexample.

1. Dijkstra's algorithm may mark a vertex as finished when it is first pushed, as BFS does.
2. If the stale check is deleted from *Theory Listing 17.2*, some distances come out wrong.
3. With one zero-weight edge, the counting rule of *Theory Listing 17.4* can return a wrong count.
4. Updating the array in place during $K + 1$ Bellman–Ford rounds gives the cheapest route with at most $K + 1$ flights.
5. Two elements are in the same union-find group when `parent[a] == parent[b]`.
6. With union by size, a `find` on 9 elements never visits more than 4 vertices, even without path halving.
7. If two edges of a connected graph have the same weight, the minimum spanning tree is never unique.
8. Kruskal's algorithm, unlike Dijkstra's, still works when some weights are negative.

**EXERCISE 17.6 Design before you code: a charging stop**

A new problem, not in the plan. An electric van drives from city $s$ to city $t$ on $n \le 10^5$ cities joined by $m \le 2 \cdot 10^5$ two-way roads with travel times up to $10^9$. Some cities have a charging station, and the van must pass through at least one of them on the way (it may drive through a city more than once). Find the least total travel time, or $-1$. Using *Theory Definition 17.3*, write the state, the edges of the new graph, the start vertex, the answer, and the time and memory. Then solve the example of Figure 17.3 by hand, with $s = 0$, $t = 5$ and stations at 2 and 6.

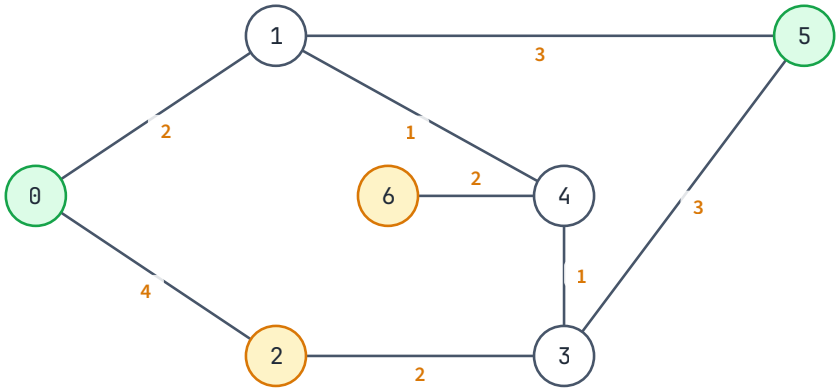



**Figure 17.3** The road map of Exercise 17.6. Stations are amber; the van starts at 0 and must reach 5.

### EXERCISE 17.7 Find the bug

This method is meant to count shortest paths for LeetCode 1976. It compiles, and on the graph of *Theory Example 17.2* it returns the correct count 3 for vertex 4. Find a small undirected graph on which it returns a wrong count, say which value it returns and what is correct, and fix it with one line.

```
static long[] countWaysBuggy(List<int[]>[] g, int src, long mod) {
    int n = g.length;
    long[] d = new long[n], ways = new long[n];
    Arrays.fill(d, Long.MAX_VALUE);
    d[src] = 0;
    ways[src] = 1;
    PriorityQueue<long[]> pq = new PriorityQueue<>((x, y) -> Long.compare(x[0], y[0]));
    pq.add(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] cur = pq.poll();
        int u = (int) cur[1];
        for (int[] e : g[u]) {
            int v = e[0];
            long nd = d[u] + e[1];
            if (nd < d[v]) {
                d[v] = nd;
                ways[v] = ways[u];
                pq.add(new long[]{nd, v});
            } else if (nd == d[v]) {
                ways[v] = (ways[v] + ways[u]) % mod;
            }
        }
    }
    return ways;
}
```

### EXERCISE 17.8 Complexity and limits

For each task, choose the method from Theory Chapter 17 (use *Theory Table 17.1* and *Theory Table 17.2*), give an operation count, and name one trap.

1. $V = 10^5$ cities, $E = 2 \cdot 10^5$ one-way roads with lengths up to $10^9$: all distances from city 0.
2. A $1000 \times 1000$ grid: stepping into a free cell costs 0, into a wall cell costs 1 (you break it). The fewest walls broken from one corner to the other.
3. 2000 points in the plane; connect all of them at the least total Manhattan length.
4. $V = 500$, $E = 5000$, some weights negative but no negative cycle: all distances from vertex 0.
5. $n = 100$ cities, $10^4$ flights, at most $K = 50$ stops: the cheapest price from $s$ to $t$.
6. $2 \cdot 10^5$ cables are added one by one; after each, print the number of connected groups of computers.
7. $10^5$ edges and $10^5$ queries "can $a$ reach $b$ using only edges of weight $\le T$?".

**EXERCISE 17.9** **Write it from memory**

Close the theory book. Write the `DSU` class (union by size and path halving) and Kruskal's `mst` from *Theory Listing 17.6*, then check them against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

### 17.P1 LC 743 · Network Delay Time MEDIUM CORE — 40 min

leetcode.com/problems/network-delay-time · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P1*

### 17.P2 LC 1631 · Path With Minimum Effort MEDIUM CORE — 40 min

leetcode.com/problems/path-with-minimum-effort · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P2*

### 17.P3 LC 1584 · Min Cost to Connect All Points MEDIUM CORE — 40 min

leetcode.com/problems/min-cost-to-connect-all-points · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P3*

### 17.P4 LC 1976 · Number of Ways to Arrive at Destination MEDIUM CORE — 40 min

leetcode.com/problems/number-of-ways-to-arrive-at-destination · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P4*

**17.P5** **LC 787 · Cheapest Flights Within K Stops** MEDIUM BONUS **40 min**

`leetcode.com/problems/cheapest-flights-within-k-stops` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P5*

**17.P6** **LC 2290 · Minimum Obstacle Removal to Reach Corner** HARD BONUS **60 min**

`leetcode.com/problems/minimum-obstacle-removal-to-reach-corner` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P6*

**17.P7** **LC 2421 · Number of Good Paths** HARD BONUS **60 min**

`leetcode.com/problems/number-of-good-paths` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 17.P7*

## Re-solve from three days ago

Pick one problem from Day 14 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 14.P1 LC 368 · Largest Divisible Subset ☐ 14.P2 LC 486 · Predict the Winner

☐ 14.P3 LC 1547 · Minimum Cost to Cut a Stick ☐ 14.P4 LC 1690 · Stone Game VII

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 18, Trees I: recursion, parent arrays, LCA.*

DAY 18 OF 29 · TUESDAY 24 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

18

# Trees I: Recursion, Parent Arrays and Ancestors

*LeetCode TreeNode recursion, general trees without recursion, and binary lifting.*

**Read first** · Theory Chapter 18
**Patterns** · Binary tree recursion (LeetCode TreeNode), General trees from a parent array (no recursion), LCA with binary lifting

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 18; type each new listing once and run it | 40 min |
| ☐ | 18.P1 LC 543 · Diameter of Binary Tree | 20 min |
| ☐ | 18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree | 40 min |
| ☐ | 18.P3 LC 124 · Binary Tree Maximum Path Sum | 60 min |
| ☐ | 18.P4 LC 2246 · Longest Path With Different Adjacent Characters | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 18.P5, 18.P6, 18.P7, 18.P8* | *60 min* |
| | **Core total (estimate)** | **4.1 h** |

**NOTES FOR THE DAY**

- Binary trees: decide what one node returns to its parent; paths that bend go into a "best" holder.
- General trees with $10^5$ nodes: build the BFS order and walk it backwards. No recursion, no stack overflow.
- Binary lifting: `up[k][v] = up[k-1][up[k-1][v]]` ; dist(a, b) = depth[a] + depth[b] − 2·depth[lca].

**Why it matters for Infosys.** Both trees in the sample paper arrive as parent arrays with n = $10^5$. A recursive DFS on a $10^5$-node chain crashes in Java.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Dijkstra | Ch. 17 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Union-find (DSU) and Kruskal | Ch. 17 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| BFS: fewest edges, grids, many sources | Ch. 15 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| DFS without recursion, components | Ch. 15 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Topological order (Kahn) and DAG DP | Ch. 15 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| 0-1 BFS | Ch. 15 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| DP recipe: state, transition, base, answer | Ch. 11 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Knapsack family | Ch. 11 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Intervals: sort, then sweep | Ch. 4 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 18 and before the problem worksheets. The first four make you produce a BFS order, a recursion's return values and a binary-lifting table by hand. These are the three things the day's problems are built from, and each is easy to get subtly wrong: a parent array in an unusual order, a branch that should be dropped, a jump that passes the root. The rest ask you to judge, design and debug on your own.

### EXERCISE 18.1 Fill the gaps: binary lifting

Complete the constructor and the LCA query from memory. Each blank is one short expression. Then compare with *Theory Listing 18.8*.

```
LCA(int[] par, int[] order) {                          // par[root] = -1, order = BFS order
    int n = par.length, lg = 1;
    while (_______________) lg++;                                  // (1)
    LOG = lg;
    up = new int[LOG][n];
    depth = new int[n];
    for (int v : order) {
        up[0][v] = ________________________;                       // (2) the root points to itself
        depth[v] = par[v] == -1 ? 0 : depth[par[v]] + 1;
        for (int k = 1; k < LOG; k++) up[k][v] = _______________;       // (3)
    }
}

int lca(int a, int b) {
    if (depth[a] < depth[b]) {
        int t = a;
        a = b;
        b = t;
    }
    a = kthAncestor(a, _______________);                           // (4) same depth first
    if (a == b) return a;
    for (int k = LOG - 1; k >= 0; k--) {
        if (________________________) {                            // (5)
            a = up[k][a];
            b = up[k][b];
        }
    }
    return _______________;                                        // (6)
}
```

**Why does the constructor loop over `order` instead of $v = 0, 1, \ldots, n - 1$? What goes wrong otherwise?**

**EXERCISE 18.2 Trace the BFS order and the subtree sizes**

Take the parent array par = [3, 3, 0, −1, 8, 1, 1, 4, 0, 5] (ten vertices, root 3). First run `bfsOrder` from *Theory Listing 18.3*; `childrenOf` lists each vertex's children in increasing order. Each row of the first table is one iteration of the `while` loop. Then run the backward loop of `subtreeSizes`.

| iteration | u = order[head] | children appended | order[0..tail − 1] afterwards |
|---|---|---|---|
| 1 | 3 | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| 8 | | | |
| 9 | | | |
| 10 | | | |

| i | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|
| v = order[i] | | | | | | | | | |
| size[v] (final) | | | | | | | | | |
| par[v] | | | | | | | | | |
| size[par[v]] after | | | | | | | | | |

**Write the final `size` and `depth` arrays (depths by one forward pass). A classmate walks $v = 9, 8, \ldots, 0$ instead of the BFS order. Which values of `size` come out wrong, and why?**

**EXERCISE 18.3 Trace a recursion: maximum path sum**

Run `maxPathSum` from *Theory Listing 18.1* on the binary tree of Figure 18.1. The rows are listed in the order in which the calls to `gain` finish (post-order). For each node write the two values returned by its children's calls (0 for a missing child), $l$ and $r$ after dropping negatives, the bend `node.val + l + r`, the value returned, and `best` afterwards.

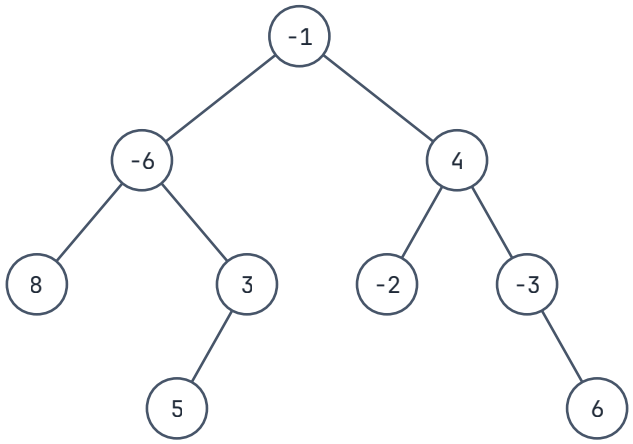



**Figure 18.1** The binary tree of Exercise 18.3. Labels are node values.

| node | gain(left) | gain(right) | l | r | bend | returns | best |
|---|---|---|---|---|---|---|---|
| 8 | | | | | | | |
| 5 | | | | | | | |
| 3 | | | | | | | |
| −6 | | | | | | | |
| −2 | | | | | | | |
| 6 | | | | | | | |
| −3 | | | | | | | |
| 4 | | | | | | | |
| −1 | | | | | | | |

**The answer is _____, the path _____. At which node does it bend? What is the diameter of this tree in edges (*Theory Theorem 18.4*), and which path achieves it?**

**EXERCISE 18.4** **A faded table: binary lifting**

Take par = [−1, 0, 1, 2, 3, 4, 5, 0, 7, 8, 9, 3] (Figure 18.2). Here $n = 12$, so LOG = 4. The BFS order is [0, 1, 7, 2, 8, 3, 9, 4, 11, 10, 5, 6]. The depths, row up[0] and the first half of row up[1] are filled in. Complete the table with *Theory Lemma 18.10*.

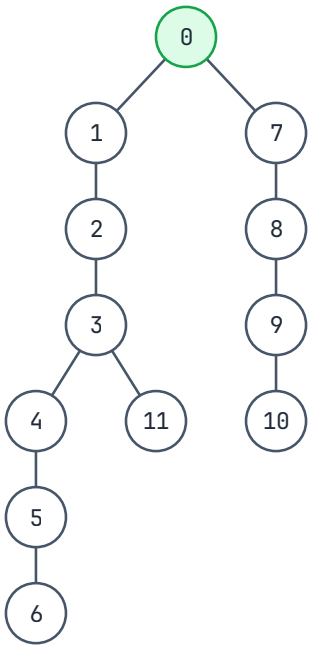



**Figure 18.2** The tree of Exercise 18.4, rooted at 0. Vertex 6 is the deepest (depth 6).

| v | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| depth | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 1 | 2 | 3 | 4 | 4 |
| up[0] | 0 | 0 | 1 | 2 | 3 | 4 | 5 | 0 | 7 | 8 | 9 | 3 |
| up[1] | 0 | 0 | 0 | 1 | 2 | 3 | | | | | | |
| up[2] | | | | | | | | | | | | |
| up[3] | | | | | | | | | | | | |

Now trace `lca(6, 10)`: write the lift of 6 to the depth of 10 (the jumps used), then, for $k = 3, 2, 1, 0$, the two entries compared and whether the loop jumps.

| step | a | b | entries compared | jump? |
|---|---|---|---|---|
| lift 6 by _____ | | 10 | — | — |
| k = 3 | | | | |
| k = 2 | | | | |
| k = 1 | | | | |

| step | a | b | entries compared | jump? |
|---|---|---|---|---|
| k = 0 | | | | |

**lca(6, 10) = _____ and dist(6, 10) = _____. Also give lca(6, 11), lca(5, 9) with the jumps the loop makes, `kthAncestor(11, 3)` and `kthAncestor(10, 5)`.**

### EXERCISE 18.5 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. A parent array of length $n$ with exactly one entry $-1$, and every other entry a valid vertex number, always describes a tree.
2. In `gain`, a node could return `node.val + l + r` to its parent instead of `node.val + Math.max(l, r)`.
3. In `maxPathSum`, `best` may start at 0, because a single node is already a path.
4. When $q$ lies in the subtree of $p$, the recursive `lca` of *Theory Listing 18.2* returns $p$ without visiting $q$.
5. In `levelOrder`, the loop `for (int k = 0; k < q.size(); k++)` still produces one list per level.
6. For $n = 10^5$ the `LCA` constructor chooses LOG = 17, and the table takes about 6.8 MB.
7. With the entry and exit times of *Theory Listing 18.6*, $u$ is an ancestor of $v$ exactly when tin[$u$] < tin[$v$] and tout[$v$] < tout[$u$].
8. On a chain of $10^5$ vertices, the sum of all pairwise distances fits in an `int`.

### EXERCISE 18.6 Design before you code: the heaviest edge on a path

A new problem, not in the plan. A tree has $n \le 10^5$ vertices given as a parent array, and the edge from $v$ to par[$v$] has weight $w$[$v$] $\le 10^9$. Answer $q \le 10^5$ queries: the largest edge weight on the path between $a$ and $b$. Extend binary lifting: say what a second table mx[$k$][$v$] stores, its recurrence, how the query collects the answer during the two phases of `lca`, and the time and memory. Then, for the tree of Figure 18.3, write row mx[1] and answer the queries (6, 4), (7, 8) and (8, 2).

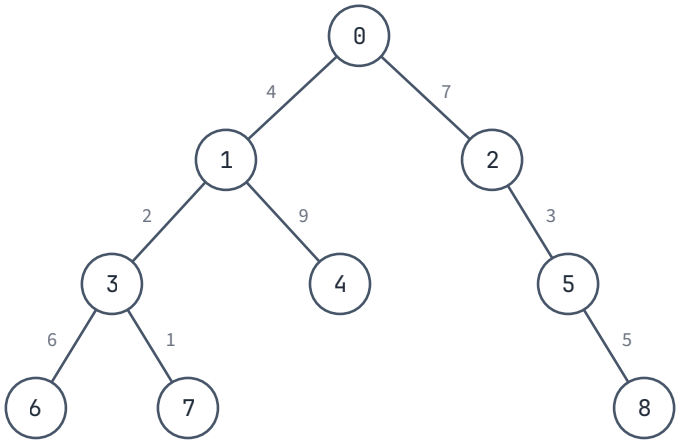



**Figure 18.3** The weighted tree of Exercise 18.6, rooted at 0. Edge labels are the weights w[v] of the edges to the parents.

**EXERCISE 18.7** **Find the bug**

This method computes subtree sizes from a parent array. It compiles, and on the tree of *Theory Figure 18.1* it returns [6, 3, 2, 1, 1, 1]. (a) Describe an input with $n = 10^5$ on which it fails, and what happens. (b) A classmate replaces the recursion by the loop `for (int v = n - 1; v >= 0; v--) if (par[v] != -1) size[par[v]] += size[v];` (after filling `size` with 1). Give a parent array with three vertices on which that loop is wrong. (c) Write the fix.

```java
@SuppressWarnings("unchecked")
static int[] subtreeSizesRec(int[] par) {
    int n = par.length, root = -1;
    List<Integer>[] kids = new List[n];
    for (int v = 0; v < n; v++) kids[v] = new ArrayList<>();
    for (int v = 0; v < n; v++) {
        if (par[v] == -1) root = v;
        else kids[par[v]].add(v);
    }
    int[] size = new int[n];
    fill(root, kids, size);
    return size;
}

static void fill(int v, List<Integer>[] kids, int[] size) {
    size[v] = 1;
    for (int c : kids[v]) {
        fill(c, kids, size);
        size[v] += size[c];
    }
}
```

**EXERCISE 18.8 Complexity and limits**

For each task, choose the method from Theory Chapter 18 and justify it with an operation count.

1. A parent array with $n = 10^5$ and $10^5$ queries "distance between $a$ and $b$".
2. The same tree with only 10 such queries.
3. The sum of the distances over all pairs of vertices, $n = 10^5$.
4. LeetCode's `TreeNode` tree with up to $10^4$ nodes: the diameter.
5. $10^5$ queries "is $u$ an ancestor of $v$?" on a tree with $n = 10^5$.
6. $n = 10^4$, edge weights in 1..26, $2 \cdot 10^4$ queries "how many edges of each weight lie on the path from $a$ to $b$?".
7. A tree with $n = 10^5$ given as an undirected edge list: the depth of every vertex from vertex 0.

**EXERCISE 18.9** **Write it from memory**

Close the theory book. Write `bfsOrderArrays` and `depths` from *Theory Listing 18.4*, then a subtree-size loop that uses the order (as in *Theory Listing 18.3*). Check them against the listings line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 4 bonus.

### 18.P1 LC 543 · Diameter of Binary Tree EASY CORE

20 min

leetcode.com/problems/diameter-of-binary-tree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P1*

### 18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree MEDIUM CORE

40 min

leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P2*

### 18.P3 LC 124 · Binary Tree Maximum Path Sum HARD CORE

60 min

leetcode.com/problems/binary-tree-maximum-path-sum · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P3*

### 18.P4 LC 2246 · Longest Path With Different Adjacent Characters HARD CORE

60 min

leetcode.com/problems/longest-path-with-different-adjacent-characters · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P4*

### 18.P5 LC 102 · Binary Tree Level Order Traversal MEDIUM BONUS

40 min

leetcode.com/problems/binary-tree-level-order-traversal · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P5*

### 18.P6 LC 1483 · Kth Ancestor of a Tree Node HARD BONUS

60 min

leetcode.com/problems/kth-ancestor-of-a-tree-node · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P6*

### 18.P7 LC 2477 · Minimum Fuel Cost to Report to the Capital MEDIUM BONUS

40 min

leetcode.com/problems/minimum-fuel-cost-to-report-to-the-capital · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P7*

### 18.P8 LC 2846 · Minimum Edge Weight Equilibrium Queries in a Tree HARD BONUS

60 min

leetcode.com/problems/minimum-edge-weight-equilibrium-queries-in-a-tree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 18.P8*

## Re-solve from three days ago

Pick one problem from Day 15 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 15.P1 LC 200 · Number of Islands ☐ 15.P2 LC 994 · Rotting Oranges ☐ 15.P3 LC 207 · Course Schedule
☐ 15.P4 LC 1091 · Shortest Path in Binary Matrix

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 19, Trees II: tree DP, rerooting, small-to-large.*

DAY 19 OF 29 · WEDNESDAY 25 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

19

# Trees II: Tree DP, Rerooting and Small-to-Large

*dp[v][state] from the children, answers for every root in O(n), and merging the smaller set into the larger.*

**Read first** · Theory Chapter 19
**Patterns** · Tree DP (dp[v][state]), Rerooting (an answer for every root), Small-to-large merging

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 19; type each new listing once and run it | 40 min |
| ☐ | 19.P1 LC 337 · House Robber III | 40 min |
| ☐ | 19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label | 40 min |
| ☐ | 19.P3 LC 834 · Sum of Distances in Tree | 60 min |
| ☐ | 19.P4 LC 3068 · Find the Maximum Sum of Node Values | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 19.P5, 19.P6, 19.P7* | *60 min* |
| | **Core total (estimate)** | **4.4 h** |

**NOTES FOR THE DAY**

- Tree DP: a few states per node (taken / not taken), combined from the children.
- Rerooting: pass 1 computes subtree answers; pass 2 moves the root to each child with a formula.
- Small-to-large: keep the bigger child set and pour the smaller sets into it. This is exactly the sample Easy problem.

**Why it matters for Infosys.** The sample Easy is small-to-large, and the sample Complex changes both ends of an edge (like LC 3068). This is the most valuable day for SP.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Binary tree recursion (LeetCode TreeNode) | Ch. 18 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| General trees from a parent array (no recursion) | Ch. 18 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| LCA with binary lifting | Ch. 18 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Two-string DP (LCS, edit distance) | Ch. 12 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Greedy with a heap | Ch. 5 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 19 and before the problem worksheets. The first four make you run a tree DP, the two passes of rerooting and a small-to-large merge by hand, each on a tree given as a parent array and its BFS order, exactly as the code sees it. Keep the order of the rows: in every one of these loops a value is only correct at the moment the loop reaches it. The rest ask you to judge, design and debug on your own.

### EXERCISE 19.1 Fill the gaps: rerooting

Complete the sum-of-distances template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 19.4*.

```
static long[] sumOfDistances(int[] par, int[] order) {   // par from parentsFromEdges, BFS order
    int n = par.length;
    int[] size = new int[n];
    long[] down = new long[n];
    ______________________;                                          // (1)
    for (int i = _____________; i > 0; i--) {                        // (2) pass 1
        int v = order[i], p = par[v];
        size[p] += size[v];
        down[p] += ________________________;                         // (3)
    }
    long[] ans = new long[n];
    _______________________________;                                 // (4)
    for (int i = 1; i < n; i++) {                                    // pass 2
        int v = order[i];
        ans[v] = ________________________________;                   // (5)
    }
    return ans;
}
```

**Blank (5) uses size[*v*] from the tree rooted at `order[0]`. Why is that still the right number when the root moves from par[*v*] to *v*?**

### EXERCISE 19.2 Trace a tree DP: take or skip

Run *Theory Listing 19.1* on the tree par = [−1, 0, 0, 1, 1, 2, 5, 5, 2] with weights $w$ = [3, 2, 5, 4, 1, 6, 2, 3, 4] (Figure 19.1). Its BFS order is [0, 1, 2, 3, 4, 5, 8, 6, 7], so the loop reaches the vertices in the order of the rows below. For each vertex write take[$v$] and skip[$v$] at the moment the loop reaches it, and what it adds to its parent's two values.

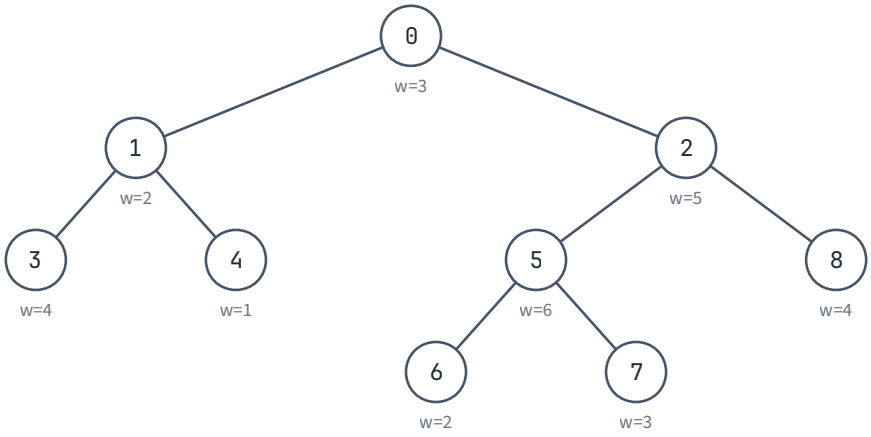



**Figure 19.1** The tree of Exercise 19.2. Notes give the weights.

| v | w[v] | take[v] | skip[v] | adds to take[par] | adds to skip[par] |
|---|---|---|---|---|---|
| 7 | 3 | | | | |
| 6 | 2 | | | | |
| 8 | 4 | | | | |
| 5 | 6 | | | | |
| 4 | 1 | | | | |
| 3 | 4 | | | | |
| 2 | 5 | | | | |
| 1 | 2 | | | | |
| 0 | 3 | | — | — | — |

**The best weight is _____. Recover the set itself by walking the BFS order forwards: a vertex whose parent is taken must be skipped; otherwise take it if take[$v$] > skip[$v$]. Which vertex has take = skip, and why does that tie not matter here?**

### EXERCISE 19.3 A faded trace: the two passes of rerooting

Run *Theory Listing 19.4* on par = [−1, 0, 1, 1, 0, 4, 4, 6, 6] (Figure 19.2). The BFS order is [0, 1, 4, 2, 3, 5, 6, 7, 8]. Pass 1 walks it backwards; the first four columns are done. Pass 2 walks it forwards; the first two columns are done.

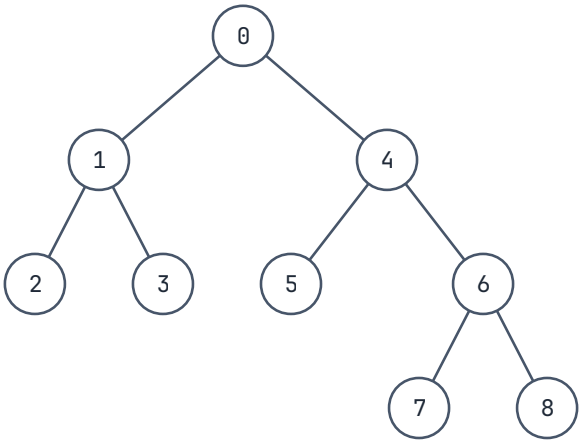



**Figure 19.2** The tree of Exercise 19.3, rooted at 0.

| **pass 1: v** | **8** | **7** | **6** | **5** | **3** | **2** | **4** | **1** | **0** |
|---|---|---|---|---|---|---|---|---|---|
| size[v] | 1 | 1 | 3 | 1 | | | | | |
| down[v] | 0 | 0 | 2 | 0 | | | | | |

| **pass 2: v** | **0** | **1** | **4** | **2** | **3** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| parent | — | 0 | 0 | | | | | | |
| ans[v] | 16 | 19 | 15 | | | | | | |

**Which vertex has the smallest sum of distances? Check its value directly by listing its distances to the other eight vertices.**

### EXERCISE 19.4 Trace a small-to-large merge

Run *Theory Listing 19.7* on par = [−1, 0, 0, 1, 1, 2, 2, 4, 4] with keys [5, 7, 5, 8, 5, 9, 5, 7, 5] (Figure 19.3). The BFS order is 0..8, so the loop runs $v = 8, 7, \ldots, 0$. For each vertex write the count of its own key already in its map (the pairs it closes with vertices below it), its map after adding its key, and what happens next: handed over to the parent, or merged (which map moves, how many entries, the cross pairs). Then write pairs[par[$v$]] afterwards.

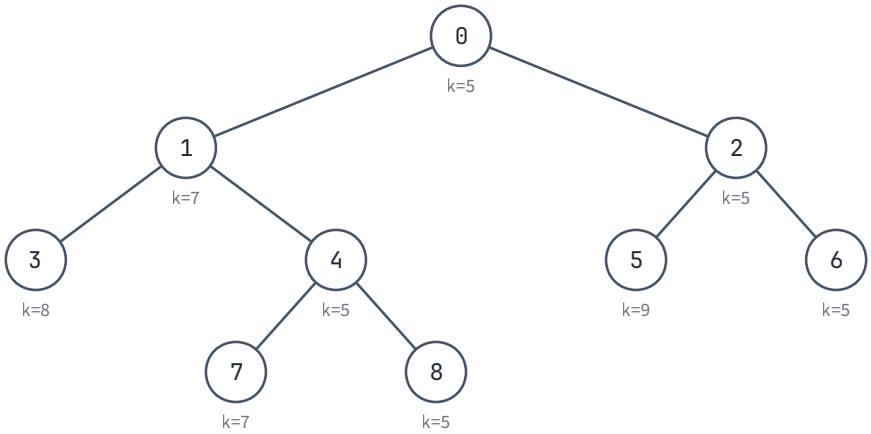



**Figure 19.3** The tree of Exercise 19.4. Notes give the keys (k).

| v | key | own | map of v after adding its key | handed / merged: what moves, cross | pairs[par] after |
|---|---|---|---|---|---|
| 8 | 5 | | | | |
| 7 | 7 | | | | |
| 6 | 5 | | | | |
| 5 | 9 | | | | |
| 4 | 5 | | | | |
| 3 | 8 | | | | |
| 2 | 5 | | | | |
| 1 | 7 | | | | |
| 0 | 5 | | | — (root) | — |

**Write the final `pairs` array. At which merge did the references swap, and how many entries moved instead of how many? Check pairs[0] by counting equal keys directly.**

### EXERCISE 19.5 Concept check

Answer true or false, and give a one-line reason or a counterexample.

1. In the independent-set DP, take[$v$] adds max(take[$c$], skip[$c$]) over the children $c$.
2. Pass 2 of rerooting may walk the BFS order backwards, as long as pass 1 is complete.
3. In *Theory Listing 19.2*, `gain[p]` can end up negative.
4. In *Theory Listing 19.5*, the line `up[v] = 1 + Math.max(up[p], down1[p])` (without the “sideways” check) gives the same eccentricities.

5. If the child's set is larger but you copy it into the parent's smaller set anyway, *Theory Listing 19.6* returns wrong answers.
6. On a star with centre 0 and 9 leaves, *Theory Listing 19.6* moves 8 entries in total.
7. On the path 0–1–2–3, the edge operations of LeetCode 3068 can flip exactly the vertices {1, 3}, but never exactly {0, 1, 2}.
8. `new int[n][26]` for $n = 10^5$ needs about 10 MB for the counters alone.

**EXERCISE 19.6 Design before you code: where to put the warehouse**

A new problem, not in the plan. A tree has $n \le 10^5$ towns (given as an edge list) and town $x$ orders $w[x] \le 10^4$ parcels a day. A warehouse at town $r$ costs $\Sigma\, w[x] \cdot \text{dist}(r, x)$ parcel-roads per day. Find the cost for every possible $r$ and the best town. Adapt *Theory Listing 19.4*: what replaces size[v], what does down[v] mean now, what are the two transitions, and why does the answer need `long`? Then compute every cost for the tree of Figure 19.4.

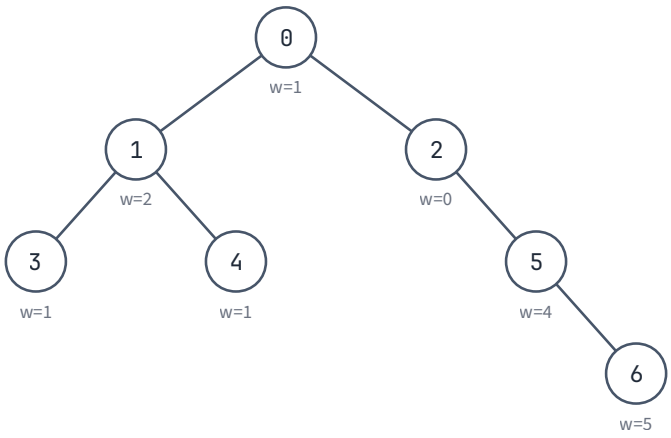



**Figure 19.4** The towns of Exercise 19.6. Notes give the parcels w[x].

**EXERCISE 19.7** **Find the bug**

This method should return the number of distinct colours in every subtree. It compiles, and for par = [−1, 0, 0] with colours [1, 2, 2] it returns the correct [2, 1, 1]. Find a tree with four vertices on which it is wrong, say what it returns there and what is correct, and fix it with one line.

```
static int[] distinctBuggy(int[] par, int[] order, int[] color) {
    int n = par.length;
    int[] ans = new int[n];
    @SuppressWarnings("unchecked")
    HashSet<Integer>[] set = new HashSet[n];
    for (int i = n - 1; i >= 0; i--) {
        int v = order[i];
        if (set[v] == null) set[v] = new HashSet<>();
        set[v].add(color[v]);
        ans[v] = set[v].size();
        int p = par[v];
        if (p == -1) continue;
        if (set[p] == null) {
            set[p] = set[v];
        } else {
            HashSet<Integer> big = set[p], small = set[v];
            if (big.size() < small.size()) {
                HashSet<Integer> t = big;
                big = small;
                small = t;
            }
            big.addAll(small);
        }
        set[v] = null;
    }
    return ans;
}
```

**EXERCISE 19.8** **Complexity and limits**

For each task, choose the technique with *Theory Table 19.1* and justify it with an operation count.

1. $n = 10^5$: for every vertex, the number of distinct values (up to $10^9$) in its subtree.
2. $n = 2 \cdot 10^5$ given as an edge list: for every vertex, the sum of its distances to all others.
3. $n = 10^5$ vertices labelled with lowercase letters: for every vertex, how many vertices of its subtree carry its letter.
4. $n = 10^5$ with weights: the largest total weight of vertices with no two adjacent.
5. $n = 3 \cdot 10^4$: a classmate runs a BFS from every vertex to get all sums of distances. Is that fast enough?
6. $n = 2 \cdot 10^4$: you may XOR both ends of any edge with $k$, as often as you like; maximise the sum.
7. $n = 10^5$: for every vertex, the distance to the farthest vertex.

**EXERCISE 19.9** **Write it from memory**

Close the theory book. Write `equalKeyPairs` (*Theory Listing 19.7*) from memory, with the hand-over of the first child's map, the swap of references and the cross count. Check it against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**19.P1** **LC 337 · House Robber III** MEDIUM CORE — **40 min**

`leetcode.com/problems/house-robber-iii · LeetCode`

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P1*

### 19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label MEDIUM CORE — 40 min

leetcode.com/problems/number-of-nodes-in-the-sub-tree-with-the-same-label · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P2*

### 19.P3 LC 834 · Sum of Distances in Tree HARD CORE — 60 min

leetcode.com/problems/sum-of-distances-in-tree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P3*

### 19.P4 LC 3068 · Find the Maximum Sum of Node Values HARD CORE — 60 min

leetcode.com/problems/find-the-maximum-sum-of-node-values · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P4*

### 19.P5 LC 2858 · Minimum Edge Reversals So Every Node Is Reachable HARD BONUS — 60 min

leetcode.com/problems/minimum-edge-reversals-so-every-node-is-reachable · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P5*

**19.P6** **LC 2003 · Smallest Missing Genetic Value in Each Subtree** HARD BONUS — 60 min

leetcode.com/problems/smallest-missing-genetic-value-in-each-subtree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P6*

**19.P7** **CF 600E · Lomsat gelral** CF 2300 BONUS — 75 min

codeforces.com/problemset/problem/600/E · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 19.P7*

## Re-solve from three days ago

Pick one problem from Day 16 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 16.P1 CF 1367C · Social Distance ☐ 16.P2 CF 1373D · Maximum Sum on Even Positions
☐ 16.P3 CF 1475G · Strange Beauty

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 20, Fenwick trees and segment trees.*

DAY 20 OF 29 · THURSDAY 26 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

20

# Fenwick Trees and Segment Trees

*Point updates with range queries for sum, min and gcd, coordinate compression, and Euler tours.*

**Read first** · Theory Chapter 20
**Patterns** · Fenwick tree (BIT), Segment tree (iterative), Euler tour (subtree = range)

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 20; type each new listing once and run it | 40 min |
| ☐ | 20.P1 LC 307 · Range Sum Query - Mutable | 40 min |
| ☐ | 20.P2 LC 315 · Count of Smaller Numbers After Self | 60 min |
| ☐ | 20.P3 LC 2407 · Longest Increasing Subsequence II | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 20.P4, 20.P5, 20.P6, 20.P7* | *60 min* |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Fenwick: point add plus prefix sum. "Count smaller to the right": scan right to left over compressed values.
- The iterative segment tree handles any associative operation: change `op` and `ID`.
- An Euler tour turns every subtree into one range, so subtree questions become range questions.

**Why it matters for Infosys.** The sample Medium is a gcd segment tree plus one counter. CF 914D (bonus) is the closest public problem to it.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Tree DP (dp[v][state]) | Ch. 19 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Rerooting (an answer for every root) | Ch. 19 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Small-to-large merging | Ch. 19 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Dijkstra | Ch. 17 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Union-find (DSU) and Kruskal | Ch. 17 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Longest increasing subsequence | Ch. 13 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search: lower and upper bound | Ch. 6 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary search on the answer | Ch. 6 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 20 and before the problem worksheets. The first four make you run the Fenwick loops, the segment-tree query and the Euler tour by hand, with the lowbit values and node indices written out, because most bugs in these structures are index bugs. The rest ask you to judge, design and debug on your own.

### EXERCISE 20.1 Fill the gaps: the Fenwick tree

Complete the template from memory. Each blank is one short expression. Then compare with *Theory Listing 20.1*.

```
static final class Fenwick {
    final long[] t;

    Fenwick(int n) { t = new long[______]; }                // (1)

    void add(int i, long v) {
        for (; ______; i += ______) t[i] += v;              // (2) (3)
    }

    long sum(int i) {                                       // a[1] + ... + a[i]
        long s = 0;
        for (; ______; i -= i & -i) s += t[i];              // (4)
        return s;
    }

    long sum(int l, int r) { return ______; }               // (5)
}
```

**Why does the update loop *add* lowbit(*i*) while the query loop *subtracts* it? Answer in terms of responsibility ranges.**

### EXERCISE 20.2 Trace a Fenwick tree by hand

Put $a$ = [2, 7, 1, 8, 2, 8, 1, 8, 2, 8, 4, 5] at positions 1..12. Fill in lowbit($i$), the responsibility range and $t[i]$ for every node.

| i | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| lowbit(i) | | | | | | | | | | | | |
| range | | | | | | | | | | | | |
| t[i] | | | | | | | | | | | | |

Now run `add(6, -3)` and then the two queries below on the updated tree. For each call, list the nodes in the order the loop visits them, with lowbit of each, and the values read or written.

| call | nodes visited (lowbit) | values read or changed |
|---|---|---|
| add(6, −3) | | |
| sum(11) | | |
| sum(4) | | |

**sum(5, 11) = _____. Check it by adding $a$[5..11] directly (remember that $a$[6] has changed).**

**EXERCISE 20.3 A faded trace: the iterative query on n = 10**

A minimum tree (*Theory Listing 20.5* with `op = Math.min` and `ID = Long.MAX_VALUE`) is built on $a$ = [6, 3, 8, 5, 9, 2, 7, 4, 1, 6], so $n$ = 10 and the leaves are $t$[10..19] = $a$[0..9]. The internal nodes are:

| node | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| t | 1 | 1 | 2 | 1 | 3 | 5 | 2 | 4 | 1 |

Trace `query(1, 8)`, the minimum of $a$[1..7]. The first row is done; ∞ stands for the identity.

| l | r | l odd? | r odd? | resL | resR |
|---|---|---|---|---|---|
| 11 | 18 | yes: take t[11] = 3 | no | 3 | ∞ |
| | | | | | |
| | | | | | |
| | | loop ends | | | |

**Answer = _____. Which positions of $a$ does node 2 combine, and in which order? Use *Theory Theorem 20.7* to explain why no query ever uses node 2 for this $n$.**

### EXERCISE 20.4 An Euler tour by hand

A tree has `par` = [3, 3, −1, 2, 2, 4, 1, 4] and values $w$ = [5, 1, 4, 2, 6, 3, 7, 2]. Run *Theory Listing 20.8*: children are visited in increasing order of their numbers. Fill in tin and tout, and the vertex at each position of the tour.

| vertex v | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| tin[v] | | | | | | | | |
| tout[v] | | | | | | | | |

| position p | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| vertex with tin = p | | | | | | | | |

**subtreeSum(3) = _____ and subtreeSum(4) = _____. Is 3 an ancestor of 6? Is 4 an ancestor of 6? Show the two comparisons each time. After `set(1, 10)`, which Fenwick position is updated, and what is subtreeSum(3)?**

### EXERCISE 20.5 Concept check

Answer true or false, and give a one-line reason.

1. A Fenwick tree can answer the maximum of $a[l..r]$ under arbitrary point updates.
2. The iterative segment tree's `query` is correct for every $n \geq 1$, not only for powers of two.
3. The identity of a gcd segment tree is 1.
4. `sum(65535)` on a Fenwick tree with $n \geq 65\,535$ reads 16 nodes.
5. The handbook's `set`, which recomputes a parent as `op(t[i], t[i ^ 1])`, is correct for the maximum-subarray merge.
6. In an Euler tour, tin[$u$] < tin[$v$] implies that $u$ is an ancestor of $v$.
7. Storing `Long.MAX_VALUE` at a position of a minimum tree removes that position from every answer.

### EXERCISE 20.6 Design before you code: best trade in a range

An array holds the prices of $n \le 10^5$ days. Two operations arrive, $q \le 10^5$ in total: "the price of day $i$ becomes $x$" and "what is the largest profit $a[j] - a[i]$ with $l \le i \le j \le r$?" (buy first, sell later or on the same day, so the answer is at least 0). Design a segment tree for it. Write what a node stores, the merge of a left segment $A$ with a right segment $B$, the identity, whether the merge is commutative, and the cost.

**For $a$ = [7, 1, 5, 3, 6, 4, 2, 8], write the node of the left half $a$[0..3], the node of the right half $a$[4..7] and the root. Then answer the query for $a$[1..5]. Finally set $a$[1] = 9 and answer the query for the whole array.**

### EXERCISE 20.7 Find the bug

This sum query was copied from the iterative template with one change. It compiles and returns the right answer for `query(0, 4)` on $a$ = [1, 2, 3, 4]. Find a query on the same array where it fails, say what it returns, explain the cause and fix it.

```
static long querySum(long[] t, int n, int l, int r) {   // sum of a[l..r-1]
    long resL = 0, resR = 0;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
        if ((l & 1) == 1) resL = resL + t[l++];
        if ((r & 1) == 1) resR = t[r--] + resR;
    }
    return resL + resR;
}
```

**EXERCISE 20.8 Complexity and limits**

For each task, choose a structure from Theory Chapter 20 (or Chapter 2, or say "Chapter 21") and justify it with an operation count or a reason.

1. $n, q \le 10^5$: range-sum queries on an array that never changes.
2. $n, q \le 10^5$: "set $a[i] = x$" and range-sum queries, interleaved.
3. $n, q \le 2 \cdot 10^5$: point updates and range-minimum queries.
4. $n, q \le 10^5$: "add $v$ to $a[l..r]$" and "what is $a[i]$ now?", interleaved.
5. $n \le 10^5$, values up to $10^9$: the number of pairs $i < j$ with $a[i] > a[j]$.
6. A tree of $n \le 10^5$ vertices given as a parent array (it may be a path): change a vertex's value, or ask for the sum over the subtree of a vertex.
7. $n, q \le 10^5$: "add $v$ to $a[l..r]$" and "what is the minimum of $a[l..r]$?", interleaved.

**EXERCISE 20.9 Write it from memory**

Close the theory book. Write the iterative segment tree *Theory Listing 20.5* (constructor, `set` and `query`) from memory, with `ID` and `op` for a maximum tree. Then check it against the listing line by line.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 4 bonus.

**20.P1** **LC 307 · Range Sum Query - Mutable** MEDIUM CORE — **40 min**

`leetcode.com/problems/range-sum-query-mutable` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 20.P1*

**20.P2** **LC 315 · Count of Smaller Numbers After Self** HARD CORE — **60 min**

`leetcode.com/problems/count-of-smaller-numbers-after-self` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 20.P2*

**20.P3** **LC 2407 · Longest Increasing Subsequence II** HARD CORE — **60 min**

`leetcode.com/problems/longest-increasing-subsequence-ii` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 20.P3*

**20.P4** **CF 914D · Bash and a Tough Math Puzzle** CF 1900 BONUS — **60 min**

`codeforces.com/problemset/problem/914/D` · Codeforces

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet — *Appendix A, hint 20.P4*

**20.P5** **LC 2286 · Booking Concert Tickets in Groups** HARD BONUS — 60 min

leetcode.com/problems/booking-concert-tickets-in-groups · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P5*

**20.P6** **LC 3165 · Maximum Sum of Subsequence With Non-adjacent Elements** HARD BONUS — 60 min

leetcode.com/problems/maximum-sum-of-subsequence-with-non-adjacent-elements · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P6*

**20.P7** **LC 2322 · Minimum Score After Removals on a Tree** HARD BONUS — 60 min

leetcode.com/problems/minimum-score-after-removals-on-a-tree · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 20.P7*

## Re-solve from three days ago

Pick one problem from Day 17 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 17.P1 LC 743 · Network Delay Time ☐ 17.P2 LC 1631 · Path With Minimum Effort
☐ 17.P3 LC 1584 · Min Cost to Connect All Points ☐ 17.P4 LC 1976 · Number of Ways to Arrive at Destination

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 21, Lazy segment trees and the K-parts DP.*

DAY 21 OF 29 · FRIDAY 27 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

21

# Lazy Segment Trees and the K-Parts DP

*Range updates, then the split-into-K-parts DP that Infosys reuses for its hardest problems.*

**Read first** · Theory Chapter 21
**Patterns** · Lazy segment tree (range add, range max), Split into K parts (dp[k][i])

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 21; type each new listing once and run it | 40 min |
| ☐ | 21.P1 LC 1335 · Minimum Difficulty of a Job Schedule | 60 min |
| ☐ | 21.P2 LC 813 · Largest Sum of Averages | 40 min |
| ☐ | 21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 21.P4, 21.P5, 21.P6* | *60 min* |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Lazy propagation: store a pending add on a node and push it down before visiting its children.
- K parts: `dp[k][i] = best over j of dp[k-1][j] + cost(j, i-1)`. $O(K \cdot n^2)$ gets the small tests.
- When moving i changes cost by a range add over j (distinct counts do), keep dp + cost in a lazy segment tree.

**Why it matters for Infosys.** The Infosys "gift boxes" problem is CF 833B and "k segments, minimum cost" is CF 1527E, with the same limits. Both are today's bonus problems.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Euler tour (subtree = range) | Ch. 20 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Fenwick tree (BIT) | Ch. 20 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Segment tree (iterative) | Ch. 20 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary tree recursion (LeetCode TreeNode) | Ch. 18 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| General trees from a parent array (no recursion) | Ch. 18 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| LCA with binary lifting | Ch. 18 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Interval DP and game DP | Ch. 14 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Monotonic stack | Ch. 7 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 21 and before the problem worksheets. The first three make you run the lazy tree and one DP layer by hand: write every tag down, because the tags are where the mistakes hide. The rest ask you to reason about tags, design a new range-add cost, find a bug that compiles, and choose a method from the limits.

### EXERCISE 21.1 Fill the gaps: push-down and range add

Complete the two methods of the lazy max tree from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 21.1*.

```
private void push(int x) {
    if (lz[x] == 0) return;
    for (int c = 2 * x; c <= 2 * x + 1; c++) {
        ______________________;              // (1)
        ______________________;              // (2)
    }
    ______________________;                  // (3)
}

private void add(int x, int lo, int hi, int l, int r, long v) {
    if (r < lo || hi < l) return;
    if (______________________) {            // (4) fully covered
        mx[x] += v;
        lz[x] += v;
        return;
    }
    ______________________;                  // (5)
    int mid = (lo + hi) >>> 1;
    add(2 * x, lo, mid, l, r, v);
    add(2 * x + 1, mid + 1, hi, l, r, v);
    ______________________;                  // (6)
}
```

**Line (5) must come before the two recursive calls. What goes wrong at line (6) if it is left out?**

### EXERCISE 21.2 Trace range adds by hand

An array of eight zeros is stored in *Theory Listing 21.1* (`n = 8`). Run the three operations below in order. After each one, write every internal node as “mx / tag” (for example “2 / 0”), list the covered nodes, and count the calls (every node the recursion is called on, including outside ones). For the query, also write the answer.

| node | after add(1, 6, 2) | after add(0, 3, 1) | after max(2, 5) |
|---|---|---|---|
| [0,7] | | | |
| [0,3] | | | |
| [4,7] | | | |
| [0,1] | | | |
| [2,3] | | | |
| [4,5] | | | |
| [6,7] | | | |
| covered nodes | | | |
| calls | | | |

**After the second update, node [2,3] stores 2, although $a[2] = a[3] = 3$. Which tag accounts for the difference, and when is it passed down?**

**EXERCISE 21.3 A faded trace: one layer of the gift-box DP**

Run layer $k = 2$ of *Theory Listing 21.6* on $a = [2, 1, 2, 3, 1, 3, 1]$. Layer 1 gave dp[1] = [−∞, 1, 2, 2, 3, 3, 3, 3], which is loaded into the tree as $T[0..7]$. For each $i$, the new element $a[i-1]$ adds 1 to the split points after its previous copy (*Theory Lemma 21.7*); then dp[2][$i$] is the maximum of $T[0..i-1]$. The first three rows are done.

| i | a[i − 1] | +1 on j = | T[0..7] after the add | dp[2][i] |
|---|---|---|---|---|
| 1 | 2 | 0..0 | −∞, 1, 2, 2, 3, 3, 3, 3 | −∞ |
| 2 | 1 | 0..1 | −∞, 2, 2, 2, 3, 3, 3, 3 | 2 |
| 3 | 2 | 1..2 | −∞, 3, 3, 2, 3, 3, 3, 3 | 3 |
| 4 | 3 | | | |
| 5 | 1 | | | |
| 6 | 3 | | | |
| 7 | 1 | | | |

**The answer for $K = 2$ is _____. Write one split into two boxes that achieves it, and say which split point $j$ it corresponds to.**

### EXERCISE 21.4 Concept check

Answer true or false, and give a one-line reason.

1. After any operation of *Theory Listing 21.1*, `mx[x]` is the true maximum of node $x$'s range for every node $x$.
2. A query may skip `push`, because it does not change any value of the array.
3. One operation can call more than two nodes on the same level.
4. Arrays of size $2n$ are enough for a recursive segment tree on $n$ elements.
5. In the K-parts DP, filling row $k$ in any order of $i$ is fine, as long as row $k - 1$ is complete.
6. If all values of $a$ are different, the gift-box answer is $n$ for every $K \le n$.
7. In *Theory Listing 21.6*, `tree.add(last[x] + 1, i - 1, 1)` may be replaced by `tree.add(Math.max(0, last[x]), i - 1, 1)`, because one extra position does no harm.

### EXERCISE 21.5 Compose the tags

Use *Theory Claim 21.5* and *Theory Listing 21.2*.

1. A node of *Theory Listing 21.3* covers four elements and receives, in this order and with no push in between: add 2, set 4, add 3, set 1, add 6. Write the single tag it holds and its maximum.
2. An element equals 10. What is it after "add 1, then set 5", and after "set 5, then add 1"?
3. In *Theory Listing 21.2* with $n = 6$, all zeros, the whole array receives +2 and then +5. What are the root's tag and stored sum?
4. In *Theory Listing 21.3* with $n = 4$, the left child [0,1] holds "add 7" when the root receives "set 5" over the whole array. Then `max(0, 1)` runs. What tag does [0,1] hold after the root's push, and what does the query return?

### EXERCISE 21.6 Design before you code: values that occur once

Split $a$ into exactly $K$ consecutive non-empty parts. The score of a part is the number of values that occur **exactly once** in it; maximise the total score. Limits: $n \le 35\,000$, $K \le 50$.

Let the new element $x = a[i-1]$ have its latest earlier copy at $p_1$ and the copy before that at $p_2$ ($-1$ when missing). Write $\Delta(j)$, the change in the score of the part $a[j..i-1]$ compared with $a[j..i-2]$, for every split point $j \le i-1$, as range adds. Say what the tree stores, what you must remember per value, and the complexity. Then, for $a = [1, 2, 1, 1, 3, 2]$, give the range adds when $a[3] = 1$ arrives ($i = 4$) and the answer for $K = 2$.

### EXERCISE 21.7 Find the bug

This version of the lazy max tree compiles, passes the sample below, and gives correct answers in every test that uses only point updates. Find an input with at most two operations on which it is wrong, say what it returns, and fix it.

```
static final class LazyMaxBuggy {
    final int n;
    final long[] mx, lz;

    LazyMaxBuggy(int n) {
        this.n = n;
        mx = new long[4 * n];
        lz = new long[4 * n];
    }

    private void push(int x) {
        if (lz[x] == 0) return;
        for (int c = 2 * x; c <= 2 * x + 1; c++) {
            mx[c] += lz[x];
```

```
            lz[c] += lz[x];
        }
        lz[x] = 0;
    }

    void add(int l, int r, long v) { add(1, 0, n - 1, l, r, v); }

    private void add(int x, int lo, int hi, int l, int r, long v) {
        if (r < lo || hi < l) return;
        if (l <= lo && hi <= r) {
            mx[x] += v;
            lz[x] += v;
            return;
        }
        push(x);
        int mid = (lo + hi) >>> 1;
        add(2 * x, lo, mid, l, r, v);
        add(2 * x + 1, mid + 1, hi, l, r, v);
        mx[x] = Math.max(mx[2 * x], mx[2 * x + 1]);
    }

    long max(int l, int r) { return max(1, 0, n - 1, l, r); }

    private long max(int x, int lo, int hi, int l, int r) {   // "a query changes nothing"
        if (r < lo || hi < l) return Long.MIN_VALUE;
        if (l <= lo && hi <= r) return mx[x];
        int mid = (lo + hi) >>> 1;
        return Math.max(max(2 * x, lo, mid, l, r), max(2 * x + 1, mid + 1, hi, l, r));
    }
}
```

The sample, with `n = 6`: add(2, 2, 4), add(0, 0, 1), max(0, 5) → 4, add(1, 4, 3), max(0, 5) → 7, max(3, 4) → 3. The buggy tree prints all three answers correctly.

**EXERCISE 21.8 Complexity and limits**

For each task, choose a method from Theory Chapter 21 (or Chapter 2) and justify it with an operation count.

1. Gift boxes with $n = 35\,000$ and $K = 50$.
2. Gift boxes with $n \le 300$ and $K \le 50$.
3. $10^5$ range additions on an array of $10^5$ zeros, then print the final array once.
4. $n = 10^5$ and $2 \cdot 10^5$ operations that mix "add $v$ to $a[l..r]$" and "print the sum of $a[l..r]$".
5. The "$k$ segments, minimise the sum of (last − first occurrence)" repeat with $n \le 35\,000$ and $k \le 100$.
6. Any K-parts problem with $n \le 20$, when you are not sure your recurrence is right.

**EXERCISE 21.9** **Write it from memory**

Close the theory book. Write the class `LazyMax` of *Theory Listing 21.1* from memory, including the constructor and both public methods, then check it line by line. Mark every place where you call `push`.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 3 bonus.

### 21.P1 LC 1335 · Minimum Difficulty of a Job Schedule HARD CORE

60 min

leetcode.com/problems/minimum-difficulty-of-a-job-schedule · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 21.P1*

### 21.P2 LC 813 · Largest Sum of Averages MEDIUM CORE

40 min

leetcode.com/problems/largest-sum-of-averages · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 21.P2*

### 21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II HARD CORE

60 min

leetcode.com/problems/subarrays-distinct-element-sum-of-squares-ii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 21.P3*

### 21.P4 CF 833B · The Bakery CF 2200 BONUS

75 min

codeforces.com/problemset/problem/833/B · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases<br>tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet

*Appendix A, hint 21.P4*

**21.P5 LC 732 · My Calendar III** HARD BONUS — 60 min

leetcode.com/problems/my-calendar-iii · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P5*

**21.P6 CF 1527E · Partition Game** CF 2500 BONUS — 75 min

codeforces.com/problemset/problem/1527/E · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 21.P6*

## Re-solve from three days ago

Pick one problem from Day 18 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 18.P1 LC 543 · Diameter of Binary Tree ☐ 18.P2 LC 236 · Lowest Common Ancestor of a Binary Tree
☐ 18.P3 LC 124 · Binary Tree Maximum Path Sum ☐ 18.P4 LC 2246 · Longest Path With Different Adjacent Characters

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 22, Backtracking, bitmask DP, digit DP.*

DAY 22 OF 29 · SATURDAY 28 NOVEMBER 2026 · LEARNING DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES

22

# Backtracking, Bitmask DP and Digit DP

*Brute force for partial marks, dp[mask] for n ≤ 20, and counting numbers digit by digit.*

**Read first** · Theory Chapter 22
**Patterns** · Backtracking and subset enumeration, Bitmask DP, Digit DP

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 22; type each new listing once and run it | 40 min |
| ☐ | 22.P1 LC 78 · Subsets | 40 min |
| ☐ | 22.P2 LC 1986 · Minimum Number of Work Sessions to Finish the Tasks | 40 min |
| ☐ | 22.P3 LC 1879 · Minimum XOR Sum of Two Arrays | 60 min |
| ☐ | 22.P4 LC 902 · Numbers At Most N Given Digit Set | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 22.P5, 22.P6, 22.P7* | *60 min* |
| | **Core total (estimate)** | **4.4 h** |

**NOTES FOR THE DAY**

- Choose, recurse, un-choose. A bitmask loop enumerates every subset for n ≤ 20.
- Bitmask DP: `dp[mask]` or `dp[mask][last]`, masks in increasing order; $2^{20} \cdot 20 \approx 2 \cdot 10^7$.
- Digit DP: state (position, tight, extra); answer for [L, R] = f(R) − f(L − 1).

**Why it matters for Infosys.** Reported recent hard problems include "tree + DP + backtracking + bitwise" and a bitmask-DP graph problem.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Lazy segment tree (range add, range max) | Ch. 21 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Split into K parts (dp[k][i]) | Ch. 21 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Tree DP (dp[v][state]) | Ch. 19 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Rerooting (an answer for every root) | Ch. 19 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Small-to-large merging | Ch. 19 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| BFS: fewest edges, grids, many sources | Ch. 15 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| DFS without recursion, components | Ch. 15 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Topological order (Kahn) and DAG DP | Ch. 15 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| 0-1 BFS | Ch. 15 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Bit tricks and XOR | Ch. 8 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Binary trie for XOR | Ch. 8 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| XOR basis (maximum subset XOR) | Ch. 8 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 22 and before the problem worksheets. The first four make you run the chapter's three searches by hand: a backtracking tree, a table indexed by masks and a digit-by-digit count. The rest ask you to judge, design, debug and size solutions without a printed answer in front of you.

### EXERCISE 22.1 Fill the gaps: the assignment DP

Complete the template from memory. Each blank is one short expression or statement. Then compare with *Theory Listing 22.3*.

```
static int minAssignment(int[][] cost) {
    int n = cost.length;
    int[] dp = new int[1 << n];
    Arrays.fill(dp, Integer.MAX_VALUE);
    ______________________;                          // (1) base case
    for (int mask = 0; mask < (1 << n); mask++) {
        if (dp[mask] == Integer.MAX_VALUE) continue;
        int worker = ________________________;       // (2) the next worker to serve
        if (worker == n) continue;
        for (int job = 0; job < n; job++) {
            if (________________________) continue;  // (3) job already taken
            int next = ________________________;     // (4)
            dp[next] = Math.min(dp[next], dp[mask] + cost[worker][job]);
        }
    }
    return ______________________;                   // (5)
}
```

**Why may the outer loop visit the masks in plain increasing order? Name the theorem and give its one-line reason.**

### EXERCISE 22.2 Trace a backtracking tree

Run *Theory Listing 22.1* on $a = [1, 3, 4, 6, 7]$ with $T = 8$. List the nodes in the order the search visits them: the chosen values, the remaining sum, and what the node does ("tries ...", "stops at $x > r$", "solution", "no values left").

| node | chosen | remain | what happens |
|---|---|---|---|
| 1 | {} | 8 | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| 8 | | | |
| 9 | | | |
| 10 | | | |
| 11 | | | |
| 12 | | | |

**Which subsets are printed? How many nodes would the search visit without the `break`, and why is the `break` safe here?**

**EXERCISE 22.3 A faded table: work sessions**

Fill dp[*mask*] of *Theory Listing 22.5* for the durations [5, 2, 4, 3] and $T = 7$. A mask is written with task 3 on the left, so 0101 means tasks 0 and 2 (durations 5 and 4). Each column holds the pair reached when task $j$ is placed last, computed from the best pair of the mask without $j$; "best" is the smallest pair in lexicographic order. The first five rows are done.

| mask | j = 0 | j = 1 | j = 2 | j = 3 | best |
|---|---|---|---|---|---|
| 0001 | (1,5) | – | – | – | (1,5) |
| 0010 | – | (1,2) | – | – | (1,2) |
| 0011 | (1,7) | (1,7) | – | – | (1,7) |
| 0100 | – | – | (1,4) | – | (1,4) |
| 0101 | (2,5) | – | (2,4) | – | (2,4) |

| mask | j = 0 | j = 1 | j = 2 | j = 3 | best |
|---|---|---|---|---|---|
| 0110 | | | | | |
| 0111 | | | | | |
| 1000 | | | | | |
| 1001 | | | | | |
| 1010 | | | | | |
| 1011 | | | | | |
| 1100 | | | | | |
| 1101 | | | | | |
| 1110 | | | | | |
| 1111 | | | | | |

**The fewest sessions = _____. Write the sessions of one optimal schedule.**

**EXERCISE 22.4 Digit DP by hand**

Count the integers $x$ in [0, 5372] that contain no digit 3. Leading zeros do no harm here (0 is not 3), so pad every $x$ to four digits and follow the tight path of 5372 as in *Theory Theorem 22.13*. At each position, count the allowed digits smaller than the digit of $N$; each of them opens a block of free completions, in which every remaining position has 9 allowed digits.

| position | digit of N | allowed smaller digits | free completions | contribution | tight path continues? |
|---|---|---|---|---|---|
| 0 | 5 | | $9^3$ = | | |
| 1 | 3 | | $9^2$ = | | |
| 2 | 7 | | $9^1$ = | | |
| 3 | 2 | | $9^0$ = | | |
| end | | | | $N$ itself: | |

**$f$(5372) = _____. Now compute $f$(999) the same way, and the number of integers in [1000, 5372] with no digit 3.**

### EXERCISE 22.5 Concept check

Answer true or false, and give a one-line reason.

1. In *Theory Listing 22.1* on a sorted array, replacing `break` by `continue` makes the search visit more nodes.
2. The permutation tree of 10 items has fewer than $10^7$ nodes.
3. If $mask_1 < mask_2$ as numbers, then $mask_1$ is a submask of $mask_2$.
4. In the assignment DP, dp[*mask*] depends on the order in which the jobs of *mask* were taken.
5. The loop `for (s = mask; s > 0; s = (s - 1) & mask)` visits the empty submask last.
6. In digit DP, the tight states may be memoised in a separate table without changing the answer.
7. For the range $[0, R]$, the formula $f(R) - f(L-1)$ needs $f(-1) = 0$.

### EXERCISE 22.6 Design before you code: pairing students

An even number $n \le 20$ of students must be split into pairs for a project, and putting students $i$ and $j$ together costs $c[i][j] = c[j][i]$. Find the least total cost. Write the state, the transition, the evaluation order and the complexity. Explain why it is enough to pair the **lowest-numbered unpaired** student at each step, and compare your state count with the number of ways to split 20 students into pairs. Then solve this instance by hand:

| c | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | – | 4 | 7 | 3 | 6 | 5 |
| 1 | 4 | – | 2 | 8 | 5 | 6 |
| 2 | 7 | 2 | – | 6 | 3 | 9 |
| 3 | 3 | 8 | 6 | – | 4 | 2 |
| 4 | 6 | 5 | 3 | 4 | – | 7 |
| 5 | 5 | 6 | 9 | 2 | 7 | – |

### EXERCISE 22.7 Find the bug

This method should count the $x$ in $[0, N]$ whose digit sum is divisible by $D$. It compiles, and for $D = 2$ it returns the right answer for every $N$ from 0 to 19 and for $N = 99$. Find an input with $N < 30$ where it fails, say what it returns there and what it should return, explain the cause, and fix it.

```
static char[] dig;
static long[][] memo;
static int mod;

static long countUpToBuggy(long N, int D) {
    dig = Long.toString(N).toCharArray();
    mod = D;
    memo = new long[dig.length][D];
    for (long[] row : memo) Arrays.fill(row, -1);
    return go(0, 0, true);
}

static long go(int pos, int rem, boolean tight) {
    if (pos == dig.length) return rem == 0 ? 1 : 0;
    if (memo[pos][rem] != -1) return memo[pos][rem];
    int limit = tight ? dig[pos] - '0' : 9;
    long res = 0;
    for (int d = 0; d <= limit; d++) res += go(pos + 1, (rem + d) % mod, tight && d == limit);
    memo[pos][rem] = res;
    return res;
}
```

### EXERCISE 22.8 Complexity and limits

For each task, choose a method from Theory Chapter 22 (or say that none fits) and justify it with a count of steps or bytes.

1. $n \le 8$ items: print every order of the items in which no two neighbours differ by exactly 1.
2. $n \le 20$ machines and 20 jobs: assign the jobs one-to-one at minimum total cost.

3. $n \le 18$ cities with distances up to $10^6$: the shortest path that visits every city once. Which element type does the table need, and how much memory does it take?
4. $n \le 16$ items and a given test `ok(group)`: split the items into the fewest groups that pass the test.
5. $R \le 10^{18}$: count the $x$ in $[L, R]$ that are divisible by 7 and whose digit sum is divisible by 5.
6. $n \le 40$ values: decide whether some subset has sum exactly $S$.

**EXERCISE 22.9** **Write it from memory**

Close the theory book. Write *Theory Listing 22.5* (`minSessions`, with dp[*mask*] = (sessions, time used in the last one)) from memory, then check it line by line. Mark the line that would break the method if the second part of the comparison were left out.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 4 core problems, 3 bonus.

**22.P1** **LC 78 · Subsets** MEDIUM CORE — **40 min**

leetcode.com/problems/subsets · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P1*

**22.P2** **LC 1986 · Minimum Number of Work Sessions to Finish the Tasks** MEDIUM CORE — **40 min**

leetcode.com/problems/minimum-number-of-work-sessions-to-finish-the-tasks · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P2*

**22.P3** **LC 1879 · Minimum XOR Sum of Two Arrays** HARD CORE — **60 min**

leetcode.com/problems/minimum-xor-sum-of-two-arrays · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P3*

**22.P4** **LC 902 · Numbers At Most N Given Digit Set** HARD CORE — **60 min**

leetcode.com/problems/numbers-at-most-n-given-digit-set · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P4*

**22.P5** **LC 51 · N-Queens** HARD BONUS — 60 min

leetcode.com/problems/n-queens · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P5*

**22.P6** **LC 847 · Shortest Path Visiting All Nodes** HARD BONUS — 60 min

leetcode.com/problems/shortest-path-visiting-all-nodes · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P6*

**22.P7** **LC 2827 · Number of Beautiful Integers in the Range** HARD BONUS — 60 min

leetcode.com/problems/number-of-beautiful-integers-in-the-range · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 22.P7*

## Re-solve from three days ago

Pick one problem from Day 19 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 19.P1 LC 337 · House Robber III ☐ 19.P2 LC 1519 · Number of Nodes in the Sub-Tree With the Same Label
☐ 19.P3 LC 834 · Sum of Distances in Tree ☐ 19.P4 LC 3068 · Find the Maximum Sum of Node Values

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 23, Mock 3 (4 problems) + review.*

DAY 23 OF 29 · SUNDAY 29 NOVEMBER 2026 · MOCK DAY · PHASE 3 · GRAPHS, TREES, SEGMENT TREES 23

# Four Problems, Partial Marks and Mock 3

*Full SP-level mock with a 4th problem, as in recent in-person rounds. This date is also the fallback exam slot.*

**Read first** · Theory Chapter 23
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 23.P1 CF 1426D · Non-zero Segments | 45 min |
| ☐ | 23.P2 CF 1689C · Infected Tree | 45 min |
| ☐ | 23.P3 CF 1187E · Tree Painting | 75 min |
| ☐ | 23.P4 CF 1913D · Array Collapse | 75 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Four problems in 3 hours. Aim for full marks on 2–3 and a brute force on the rest.
- After the mock: scores, editorials, re-code, error log.

**Why it matters for Infosys.** The SP L3 report was 3 full solutions plus 6/12 tests on the 4th. Practise collecting partial marks on Q4 here.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Backtracking and subset enumeration | Ch. 22 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Bitmask DP | Ch. 22 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Digit DP | Ch. 22 | 1 day ago | ☐ Again ☐ Shaky ☐ Solid |
| Euler tour (subtree = range) | Ch. 20 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Fenwick tree (BIT) | Ch. 20 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Segment tree (iterative) | Ch. 20 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Brute force first, then stress test | Ch. 9 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Mock exam: Mock 3 · four problems

Q1 ≈ 1500, Q2 ≈ 1600, Q3 ≈ 2100, Q4 ≈ 2100. Collect partial marks on Q4. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

### Planning grid (first 15 minutes)

| Q | Limits | Idea | Complexity | Brute-force fallback | Order |
|---|---|---|---|---|---|
| Q1 | | | | | |
| Q2 | | | | | |
| Q3 | | | | | |
| Q4 | | | | | |

### Score sheet

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q1 | **CF 1426D** Non-zero Segments<br>`codeforces.com/problemset/problem/1426/D` | 1500 | 20 | | | | |
| Q2 | **CF 1689C** Infected Tree<br>`codeforces.com/problemset/problem/1689/C` | 1600 | 30 | | | | |
| Q3 | **CF 1187E** Tree Painting<br>`codeforces.com/problemset/problem/1187/E` | 2100 | 50 | | | | |
| Q4 | **CF 1913D** Array Collapse<br>`codeforces.com/problemset/problem/1913/D` | 2100 | partial | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

**SP checkpoint.** Weighted score ≥ 60 and a brute force submitted on every problem.

### Post-mortem (about 60 minutes after the mock)

| Q | What went wrong | Cause | Rule that would have prevented it |
|---|---|---|---|
| Q1 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q2 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q3 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q4 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |

## Exercises

Do these after the mock and its post-mortem, not before: they use the methods of Theory Chapter 23 on small examples of their own, and none of them is about the four mock problems. The first two practise planning, the next two debugging, and the last one checks the rules.

**EXERCISE 23.1** **Order the work by marks per minute**

The reading phase of a paper gives five pieces of work. Times are measured from the start of work.

| Piece | Minutes $p$ | Expected marks $w$ |
|---|---|---|
| A | 30 | 18 |
| B | 20 | 9 |
| C | 45 | 22.5 |
| D | 15 | 10.5 |
| E | 60 | 12 |

| Piece | A | B | C | D | E |
|---|---|---|---|---|---|
| $w/p$ | | | | | |
| position in the ratio order | | | | | |
| finishing time in the ratio order | | | | | |

**Compute Σ $w \cdot C$ for the ratio order and for the order A, B, C, D, E. Then give the expected marks secured by minutes 60, 90 and 120 in both orders. Which piece is unfinished if you have 150 working minutes?**

**Does *Theory Lemma 23.1* decide the pair D and B on its own? Why?**

## EXERCISE 23.2 Design the tiers before you code

A new problem: a rooted tree with $n \le 2 \cdot 10^5$ vertices is given as a parent array, each vertex has a value up to $10^9$, and $q \le 2 \cdot 10^5$ queries $(u, k)$ ask for the sum of the values of $u$ and of its first $k$ ancestors ($k$ is at most the depth of $u$). Without writing code, design three tiers: for each, the restriction it needs, its complexity, how the program detects that the restriction holds, and the cost at the largest input it accepts.

**Which tier would you write first in a mock, and why?**

## EXERCISE 23.3 Find the bug: subtree sizes from a parent array

This method passes the samples of a problem whose parent array has the root at 0 and every parent smaller than its child. The hidden tests do not promise the second property.

```
static int[] subtreeSizes(int[] par) {        // par[root] = -1, root = 0
    int n = par.length;
    int[] size = new int[n];
    Arrays.fill(size, 1);
    for (int v = n - 1; v >= 1; v--) size[par[v]] += size[v];
    return size;
}
```

**Give a tree with three vertices on which it fails, the array it returns and the correct one.**

**Write one runtime check, in the style of *Theory Listing 23.2*, that would have turned every wrong result into a runtime error.**

**How do you fix the method?**

### EXERCISE 23.4 Shrink a failing tree by hand

The buggy diameter of *Theory Example 23.2* (the longest path through the root only) disagrees with the true diameter on par = [−1, 0, 1, 1, 3, 1, 2, 6]. Run *Theory Listing 23.3* by hand: it tries the leaves from the highest number down, and after a kept deletion it starts again on the new tree. For each try, write the parent array after the deletion, both diameters and whether the deletion is kept.

| Try | Leaf deleted | Parent array after deletion | Buggy | True | Kept? |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |
| 5 | | | | | |

**The final tree is _____. Why does the shrinker stop there?**

### EXERCISE 23.5 Concept check

Answer true or false, and give a one-line reason.

1. If the rest of Q3 needs 50 more minutes for 8.75 expected marks and the Q4 brute force needs 15 minutes for 9, the brute force should come first.
2. The minutes already spent on a problem should raise its priority, because they would otherwise be wasted.
3. Java `assert` statements run on Codeforces.
4. Debug lines printed with `System.err.println` change the Codeforces verdict.

5. A submission that sends every input with $n \le 3000$ to a correct brute force can lose marks compared with sending everything to a correct fast tier.
6. Under Plan B, Mock 3 is taken on Saturday 28 November.

## Re-solve from three days ago

Pick one problem from Day 20 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 20.P1 LC 307 · Range Sum Query - Mutable
- [ ] 20.P2 LC 315 · Count of Smaller Numbers After Self
- [ ] 20.P3 LC 2407 · Longest Increasing Subsequence II

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 24, Infosys repeat problems I.*

DAY 24 OF 29 · MONDAY 30 NOVEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

24

# Infosys Repeat Problems I

*The most reported Infosys problems, written as full programs with one value per line and checked with the stress tester.*

**Read first** · Theory Chapter 24
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 24; type each new listing once and run it | 40 min |
| ☐ | 24.P1 RPG: maximum monsters defeated | 30 min |
| ☐ | 24.P2 Andy's vacation (cancel ≤ K obligations) | 30 min |
| ☐ | 24.P3 Gym: fewest exercises to drain energy | 20 min |
| ☐ | 24.P4 GfG · Wine Buying and Selling | 30 min |
| ☐ | 24.P5 Arrays where each element divides the next (mod 10000) | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 24.P6, 24.P7, 24.P8, 24.P9, 24.P10* | *60 min* |
| | **Core total (estimate)** | **3.9 h** |

**NOTES FOR THE DAY**

- For each problem without a judge: write the brute force, then the fast solution, then stress-test until thousands of random cases match. OneCompiler runs both.
- Read the input exactly as the card says: one value per line, in the order of the samples.
- At most 40 minutes per problem before you open the folded approach.

**Why it matters for Infosys.** Several of these were reported in 4–6 different years.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Lazy segment tree (range add, range max) | Ch. 21 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Split into K parts (dp[k][i]) | Ch. 21 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Dijkstra | Ch. 17 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Union-find (DSU) and Kruskal | Ch. 17 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| gcd, fast power, inverse, nCr | Ch. 10 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Sieve, prime factors, square-free kernel | Ch. 10 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Work these after reading Theory Chapter 24 and before the problem worksheets. The first five make you run the chapter's programs by hand on new inputs, from reading the input to the last line of the answer. The last three ask you to find a bug, judge some claims and write a stress test from memory, which is the routine you will use on every problem today that has no judge.

### EXERCISE 24.1 Fill the gaps: reading one value per line

Complete `main` of *Theory Listing 24.5* from memory. The input is $N$, $K$, $M$ and then the $M$ days, one value per line. Each blank is one short expression or statement.

```
public static void main(String[] args) throws IOException {
    FastScanner fs = new FastScanner(_______________);      // (1)
    int N = fs.nextInt();
    int K = _______________;                                // (2)
    int M = fs.nextInt();
    int[] cnt = new int[_______________];                   // (3) days are 1..N
    for (int i = 0; i < M; i++) _________________________;  // (4)
    System.out.println(longestFreeRun(N, K, cnt));
}
```

The first sample arrives as the eight lines 10, 2, 5, 6, 9, 3, 2, 7.

**Write the values of $N$, $K$, $M$ and cnt[1..10] after reading. What changes if all eight values arrive on one line, separated by spaces?**

### EXERCISE 24.2 Trace the monsters greedy

Run *Theory Listing 24.2* on $e = 4$, powers [9, 4, 1, 20, 6, 5, 13] and bonuses [3, 0, 2, 7, 1, 4, 2]. First sort the pairs by power, then fill one row per fight until the walk stops.

| power | bonus | e before | fight or stop | e after |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |

| power | bonus | e before | fight or stop | e after |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**The program prints _____. What is the smallest starting experience $e$ with which all seven monsters are beaten? Explain without running the walk again.**

### EXERCISE 24.3 Wine by hand

For $a = [3, -5, 4, -1, 2, -3]$ (+ buys, − sells), fill in the running sum $P_i$ after each house, the work $|P_i|$ at the gap after it, and the direction in which the bottles cross that gap (← towards house 0, → away from it). Then add up the work, as *Theory Theorem 24.4* says.

| gap after house | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $P_i$ | | | | | |
| $\lvert P_i \rvert$ | | | | | |
| direction | | | | | |

**Total work = _____. Why is there no gap after house 5?**

### EXERCISE 24.4 A faded table: divisor chains for N = 12

Fill in the chain counts $\mathrm{ch}(s, v)$ of *Theory Theorem 24.5* for $N = 12$: the number of strictly increasing divisor chains with $s$ values that end at $v$. The first row and the first six columns of rows 2 and 3 are done. Each entry is the sum of the row above over the proper divisors of $v$.

| $s \setminus v$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 12 |
| 2 | 0 | 1 | 1 | 2 | 1 | 3 | | | | | | | |
| 3 | 0 | 0 | 0 | 1 | 0 | 2 | | | | | | | |
| 4 | | | | | | | | | | | | | |

**For $K = 5$, write $C(4, s-1)$ for $s = 1..4$ and the number of arrays of length 5 with values in 1..12 in which each element divides the next. Why is row 4 non-zero at only two values of $v$?**

## EXERCISE 24.5 Candidates for X ≤ K

Take $A = [0, 1, 6, 3, 2]$ and $K = 11 = 1011_2$. Fill in the bit counts and shares of *Theory Listing 24.8*, then the candidates: $X = K$ and one candidate for each set bit $b$ of $K$ (copy $K$ above $b$, put 0 at $b$, choose the lower bits by the larger share).

| bit $b$ | ones $c_b$ | keep[$b$] = $c_b \cdot 2^b$ | flip[$b$] = $(5 - c_b) \cdot 2^b$ |
|---|---|---|---|
| 3 | | | |
| 2 | | | |
| 1 | | | |
| 0 | | | |

| candidate | $X$ in binary | $X$ | sum of $X \oplus A[i]$ |
|---|---|---|---|
| $X = K$ | 1011 | 11 | |
| branch at bit 3 | | | |
| branch at bit 1 | | | |
| branch at bit 0 | | | |

**The answer is _____ at $X$ = _____. Which $X$ would the free choice of every bit give, and why is it not allowed?**

## EXERCISE 24.6 Find the bug

This version of the vacation window passes both samples of the plan (it prints 5 and 3). Find an input on which it fails, say what it prints there and what it should print, and fix it.

```
static int longestFreeRunBuggy(int N, long K, int[] cnt) {
    long used = 0;
    int best = 0;
    for (int l = 1, r = 1; r <= N; r++) {
        if (cnt[r] > 0) used++;                  // day r joins the window
        while (used > K) if (cnt[l++] > 0) used--;
        best = Math.max(best, r - l + 1);
```

```
    }
    return best;
}
```

**EXERCISE 24.7 Concept check**

Answer true or false, and give a one-line reason.

1. 10000 is prime, so C($K-1$, $s-1$) modulo 10000 can be computed with Fermat inverses.
2. Modulo 10000, the number 3 has an inverse.
3. The monsters greedy stays correct when some bonuses are negative.
4. If $x < y$ and their highest set bits differ, then $(x \,\&\, y) \cdot 2 < (x \mid y)$.
5. In the gym problem, using the smallest drains first also gives the fewest exercises.
6. In the wine problem with $n = 10^5$ and $|a[i]| \le 10^4$, the answer always fits in an `int`.
7. With `FastScanner`, input with one value per line and input with all values on one line are read the same way.

**EXERCISE 24.8 Write it from memory: a stress test**

Close the theory book. In the layout of *Theory Listing 24.1*, write a `stress()` method for the gym problem that compares `fewestExercises` (*Theory Listing 24.3*) with `gymBrute` (*Theory Listing 24.4*) on 20,000 random inputs and prints the first mismatch. Choose the generator's ranges yourself and say why they are legal inputs.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 5 core problems, 5 bonus.

**24.P1 RPG: maximum monsters defeated** SELF-JUDGE CORE — **30 min**

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P1*

**24.P2 Andy's vacation (cancel ≤ K obligations)** SELF-JUDGE CORE — **30 min**

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P2*

### 24.P3 Gym: fewest exercises to drain energy SELF-JUDGE CORE — 20 min

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P3*

### 24.P4 GfG · Wine Buying and Selling GFG CORE — 30 min

geeksforgeeks.org/problems/wine-buying-and-selling/1 · GeeksforGeeks

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P4*

### 24.P5 Arrays where each element divides the next (mod 10000) SELF-JUDGE CORE — 60 min

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P5*

### 24.P6 LIS with (x & y)·2 < (x | y) SELF-JUDGE BONUS — 45 min

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P6*

### 24.P7 Smallest base with identical digits SELF-JUDGE BONUS

45 min

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P7*

### 24.P8 Choose X ≤ K maximising Σ X xor A[i] SELF-JUDGE BONUS

45 min

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P8*

### 24.P9 CF 1338A · Powered Addition CF 1500 BONUS

45 min

codeforces.com/problemset/problem/1338/A · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P9*

### 24.P10 CF 1514C · Product 1 Modulo N CF 1600 BONUS

45 min

codeforces.com/problemset/problem/1514/C · Codeforces

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 24.P10*

## Re-solve from three days ago

Pick one problem from Day 21 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 21.P1 LC 1335 · Minimum Difficulty of a Job Schedule ☐ 21.P2 LC 813 · Largest Sum of Averages

☐ 21.P3 LC 2916 · Subarrays Distinct Element Sum of Squares II

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 25, Mock 4: the official Infosys sample paper.*

DAY 25 OF 29 · TUESDAY 1 DECEMBER 2026 · MOCK DAY · PHASE 4 · EXAM SIMULATION

25

# Mock 4: The Official Infosys Sample Paper

*Infosys's own paper under exam rules. Grade yourself with the samples and your brute force, then open the walkthroughs.*

**Read first** · Theory Chapter 25
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 25.P1 Sample paper · Easy (tree beauty) | 45 min |
| ☐ | 25.P2 Sample paper · Medium (gcd under updates) | 45 min |
| ☐ | 25.P3 Sample paper · Hard (LNDS with XOR ≥ M) | 60 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 25.P4* | *30 min* |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Download the sample PDF and copy the sample inputs.
- Write each solution in OneCompiler; run it with the samples in the STDIN box.
- Solve Easy, Medium and Hard in 3 hours. Attempt Complex only for partial marks.
- Grade: samples, plus a stress test against your own brute force. Only then read the walkthroughs in Theory Chapter 25.

**Why it matters for Infosys.** This is the closest thing to the real paper you will have.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Backtracking and subset enumeration | Ch. 22 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Bitmask DP | Ch. 22 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Digit DP | Ch. 22 | 3 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Binary tree recursion (LeetCode TreeNode) | Ch. 18 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| General trees from a parent array (no recursion) | Ch. 18 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| LCA with binary lifting | Ch. 18 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| DP recipe: state, transition, base, answer | Ch. 11 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Knapsack family | Ch. 11 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Mock exam: Mock 4 · official Infosys sample paper

Infosys's own paper. Run it in OneCompiler with the PDF's samples, grade with your stress tester, then open the walk-throughs. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- ☐ **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- ☐ **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- ☐ **By minute 150** every problem has a submission, even if it is only a brute force.
- ☐ **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

### Planning grid (first 15 minutes)

| Q | Limits | Idea | Complexity | Brute-force fallback | Order |
|---|---|---|---|---|---|
| Q1 | | | | | |
| Q2 | | | | | |
| Q3 | | | | | |
| Q4 | | | | | |

### Score sheet

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q1 | **Infosys** Sample paper · Easy (tree beauty) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf` | — | 20 | | | | |
| Q2 | **Infosys** Sample paper · Medium (gcd under updates) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf` | — | 30 | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q3 | **Infosys** Sample paper · Hard (LNDS with XOR ≥ M) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf` | — | 50 | | | | |
| Q4 | **Infosys** Sample paper · Complex (partial marks only) `rec-test.infosys.com/iwa-web/public/instructions/Sample_Test_SP_DSE.pdf` | — | partial | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

**SP checkpoint.** Compare with the walkthroughs in Theory Chapter 25 only after grading yourself.

### Post-mortem (about 60 minutes after the mock)

| Q | What went wrong | Cause | Rule that would have prevented it |
|---|---|---|---|
| Q1 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q2 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q3 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q4 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |

## Exercises

**PITFALL** **Do these only after the mock**

Every exercise below uses the solutions in *Theory Section 25.4*. Sit the paper first, fill in the score sheet and the post-mortem above, and only then start here.

These exercises turn the walkthroughs into something you can reproduce without the book. The first four make you run each solution by hand on an input that is not a sample, so that you see every number the program computes. The last two ask you to write the brute forces from memory and to classify the mistakes you made today.

**EXERCISE 25.1** **Square-free kernels by hand**

Complete the table. The kernel is the product of the primes with an odd exponent (*Theory Definition 25.3*). For the four large values the factorisation is given; use *Theory Claim 25.2* to see what the program's square test does with the cofactor.

| value | factorisation | kernel |
|---|---|---|
| 72 | | |

| value | factorisation | kernel |
|---|---|---|
| 50 | | |
| 98 | | |
| 45 | | |
| 20 | | |
| 8 | | |
| 1 | | |
| 49 | | |
| 3 054 243 | $3 \cdot 1009^2$ | |
| 1 032 247 | $1013 \cdot 1019$ | |
| 4 128 988 | $2^2 \cdot 1013 \cdot 1019$ | |
| 1 018 081 | $1009^2$ | |

**Group the twelve values by kernel. How many unordered pairs of them have a product that is a perfect square?**

### EXERCISE 25.2 A faded merge trace

The tree below has 8 nodes with parents par[2..8] = 1, 1, 2, 2, 3, 6, 6 (node 1 is the root) and values 2, 5, 8, 20, 45, 3, 12, 18 on nodes 1 to 8. *Theory Listing 25.2* processes the merges in the order shown. Recall that the parent's map is the larger one unless it has fewer keys than the child's, in which case the two are swapped and the parent's entries are poured into the child's map. The first four rows are done; fill in the rest.

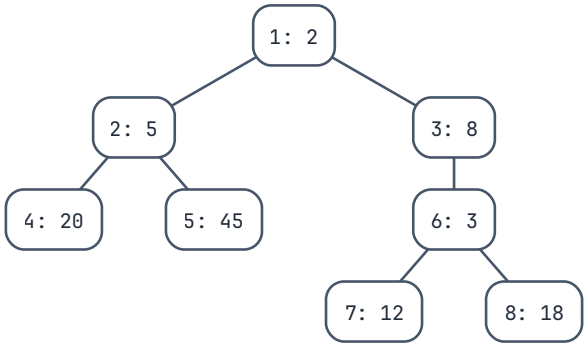



**Figure 25.1** The tree of the exercise. Each box reads node: value.

| child → parent | child's map | parent's map before | swap? | cross | pairs[parent] after | parent's map after |
|---|---|---|---|---|---|---|
| 7 → 6 | {3:1} | {3:1} | no | 1 | 1 | {3:2} |
| 8 → 6 | {2:1} | {3:2} | no | 0 | 1 | {2:1, 3:2} |
| 4 → 2 | {5:1} | {5:1} | no | 1 | 1 | {5:2} |
| 5 → 2 | {5:1} | {5:2} | no | 2 | 3 | {5:3} |
| 6 → 3 | | | | | | |
| 2 → 1 | | | | | | |
| 3 → 1 | | | | | | |

**Give beauty($v$) for every node and the program's answer. At which merge does the swap happen, and why?**

### EXERCISE 25.3 A gcd-tree trace

Run *Theory Listing 25.3* by hand with $n = 3$, $p = 3$, the array [18, 30, 4] and the updates (3, 45), (1, 9), (2, 7), (3, 12). The tree has 4 leaves $t$[4..7]; non-multiples of 3 and the padding leaf store 0. For each update write the leaves, $t$[2], $t$[3], $g = t$[1], `notDiv`, the rule that decides ($g \neq p$, `notDiv` > 0, $n \geq 8$, or drop-one) and the answer.

| update | array after | leaves t[4..7] | t[2] | t[3] | g | notDiv | rule | answer |
|---|---|---|---|---|---|---|---|---|
| start | [18, 30, 4] | 18 30 0 0 | 6 | 0 | 6 | 1 | — | — |
| (3, 45) | | | | | | | | |
| (1, 9) | | | | | | | | |
| (2, 7) | | | | | | | | |
| (3, 12) | | | | | | | | |

**The program's output is _____. For each update that reaches the drop-one check, write the three gcds it computes.**

### EXERCISE 25.4 The XOR-DP table on a new input

Run *Theory Listing 25.4* on $A$ = [1, 3, 1, 2, 2] with $M = 3$. The largest value is 3, so $B = 4$ and the XOR values are 0..3. Each row is `cand[0..3]` for one element: the longest non-decreasing subsequence that ends at

that element with each XOR (0 for none). The first two rows are done. Then fill in the best length for each XOR over all elements.

| element | cand[0] | cand[1] | cand[2] | cand[3] |
|---|---|---|---|---|
| A[0] = 1 | 0 | 1 | 0 | 0 |
| A[1] = 3 | 0 | 0 | 2 | 1 |
| A[2] = 1 | | | | |
| A[3] = 2 | | | | |
| A[4] = 2 | | | | |
| best | | | | |

**The answer is _____. Write one subsequence that achieves it. The longest non-decreasing subsequence of $A$ has length 4; why is it not the answer?**

### EXERCISE 25.5 Write the brute forces from memory

Close the theory book. For each problem, write the brute force you would submit for partial marks, as Java or as precise pseudocode, then its cost and the largest input it finishes in about a second (compare with *Theory Section 25.3*).

**Easy (tree beauty): every subtree, every pair, exact square test.**

**Medium (gcd under updates): every proper subset after every update.**

**Hard (LNDS with XOR ≥ M): every subsequence by bitmask.**

**Complex: every matching, every root-to-leaf path.**

### EXERCISE 25.6 Classify the failures

Each line describes a failure seen on this paper. Give its cause from the error log (misread, idea, overflow, off-by-one, TLE, input, edge, stack, time) and a one-sentence rule that would have prevented it. Then add your own failures from today's mock.

1. The Medium program read $p$ after the array, as the format text says, and printed wrong answers on all three samples.
2. An Easy solution with a recursive depth-first search passed the samples and threw `StackOverflowError` on a chain of $10^5$ nodes.
3. The Easy brute force multiplied two `int` values and missed pairs such as 999 999 937 · 999 999 937.
4. A Hard version wrote to `fw[x]` inside the first loop and printed 3 for [1, 1] with $M = 1$.
5. A Medium version without the drop-one check answered YES for an array of length 1 whose only value is $p$.
6. A Medium version counted an update as YES when the multiples of $p$ had gcd $2p$.
7. Seventy minutes went into the Complex problem, and the Hard problem was left without a submission.

## Re-solve from three days ago

Pick one problem from Day 22 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 22.P1 LC 78 · Subsets
- [ ] 22.P2 LC 1986 · Minimum Number of Work Sessions to Finish the Tasks
- [ ] 22.P3 LC 1879 · Minimum XOR Sum of Two Arrays
- [ ] 22.P4 LC 902 · Numbers At Most N Given Digit Set

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 26, Repair day + the hard XOR repeat.*

DAY 26 OF 29 · WEDNESDAY 2 DECEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

26

# The Repair Session and the Hard XOR Repeats

*Fix what the mocks exposed, then the hardest XOR repeat.*

**Read first** · Theory Chapter 26
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 26; type each new listing once and run it | 40 min |
| ☐ | 26.P1 Max XOR choosing at most N/2 elements | 60 min |
| ☐ | 26.P2 Pairs with XOR < K and a prime sum | 45 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| ☐ | *Bonus hour (optional, only if the core is done and you feel sharp): 26.P3* | *45 min* |
| | **Core total (estimate)** | **3.8 h** |

**NOTES FOR THE DAY**

- First 90 minutes: re-solve every error-log entry dated 3+ days ago, without notes.
- Then the problems below.
- This is a lighter day on purpose. Stop when the core is done.

**Why it matters for Infosys.** This late, most of the score gain comes from removing your own repeated mistakes, not from new topics.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Tree DP (dp[v][state]) | Ch. 19 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Rerooting (an answer for every root) | Ch. 19 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Small-to-large merging | Ch. 19 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Two-string DP (LCS, edit distance) | Ch. 12 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Start with the repair worksheet: it is the first 90 minutes of the day, before any new material. Then work the XOR exercises, which move from the knapsack template with gaps to the table by hand, the trie walk, a bug and a design of your own. The last exercise checks the facts that decide which method to use.

### EXERCISE 26.1 Repair worksheet

Open the error log (Appendix C and the end-of-day tables) and the pattern recall tracker at the front of this workbook. Follow *Theory Section 26.1*: list the entries dated at least three days ago and the patterns still at A, then choose three.

| | Problem and date | Cause | Old rule | Template and reason from memory: differences found | Re-solve: minutes, result | New rule (checkable in ten seconds) |
|---|---|---|---|---|---|---|
| 1 | | | | | | |
| 2 | | | | | | |
| 3 | | | | | | |

**Which of the three re-solves failed, and what is its new re-solve date? If none failed, which entry would you choose for Day 29, and why?**

### EXERCISE 26.2 Fill the gaps: the XOR knapsack

Complete the knapsack and the answer loop from memory, then compare with *Theory Listing 26.1* and *Theory Listing 26.2*.

```
static final int INF = ________________________;                // (1)

static int[] fewestElements(int[] a, int B) {
    int size = 1 << B;
    int[] cur = new int[size], next = new int[size];
    Arrays.fill(cur, INF);
    ______________________;                                      // (2)
    for (int v : a) {
        for (int x = 0; x < size; x++)
            next[x] = ________________________;                  // (3)
        ______________________;                                  // (4) the new row becomes current
    }
    return cur;
}

// answer: the largest x reachable with at most cap = n / 2 elements
for (int x = (1 << B) - 1; x > 0; x--) if (_______________) return x;   // (5)
```

**Why is blank (1) not `Integer.MAX_VALUE`?**

### EXERCISE 26.3 The knapsack table by hand

Fill in the table of $f_i$ for $A = [3, 5, 6, 1]$ ($N = 4$, cap 2, $B = 3$), using *Theory Theorem 26.1*. The first two rows are done; write ∞ where no subset reaches $x$.

| | x = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| f0 | 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| f1 (a = 3) | 0 | ∞ | ∞ | 1 | ∞ | ∞ | ∞ | ∞ |
| f2 (a = 5) | | | | | | | | |
| f3 (a = 6) | | | | | | | | |
| f4 (a = 1) | | | | | | | | |

**The answer is _____, made by the elements _____. What is the rank of the four values, and which branch of `solve` in *Theory Listing 26.2* runs? Is the answer below the basis maximum?**

### EXERCISE 26.4 A trie walk with ranges

The input of *Theory Listing 26.4* is $n = 8$, $K = 5$ and $A = [4, 1, 6, 3, 2, 7, 5, 12]$. Sorted, $s = [1, 2, 3, 4, 5, 6, 7, 12]$, and $B = 4$ ($12 < 16$). Trace the query for $x = 7 = 0111_2$, when $s[0..5]$ are in the trie. $K = 0101_2$.

| bit | x_b | k_b | action | subtree taken: prefix, range of s | sums tested (mark primes) |
|---|---|---|---|---|---|
| 3 | | | | | |
| 2 | | | | | |
| 1 | | | | | |
| 0 | | | | | |

**The count was 3 before this query. What is it after? Which $y$ in $s[0..5]$ are never looked at, and why? What does the program print for the whole input?**

### EXERCISE 26.5 Find the bug

This version of the capped maximum compiles and passes the second sample, [1, 2, 4, 7] → 7. Run it by hand on the first sample, [1, 2] with $B = 2$: write the row after the first pass, the value it returns, and the cause. Then fix it with one change.

```
static int maxXorCapped(int[] a, int B) {
    int size = 1 << B, cap = a.length / 2;
    int[] cur = new int[size], next = new int[size];
    Arrays.fill(cur, Integer.MAX_VALUE);
    cur[0] = 0;
    for (int v : a) {
        for (int x = 0; x < size; x++) next[x] = Math.min(cur[x], cur[x ^ v] + 1);
        int[] t = cur; cur = next; next = t;
    }
    for (int x = size - 1; x > 0; x--) if (cur[x] <= cap) return x;
    return 0;
}
```

### EXERCISE 26.6 Design before you code: counting capped subsets

$N \le 40$ values below $2^{10}$, a target $T$ and a cap $c \le N$ are given. Count the subsets (the empty one included) with XOR exactly $T$ and at most $c$ elements, modulo $10^9 + 7$. Write the state, the transition, an evaluation order that lets you update one table in place, the answer and the cost. Then compute the answer for $A = [1, 2, 3, 1]$, $T = 0$ and $c = 2$, and again for $c = 3$.

### EXERCISE 26.7 Concept check and limits

Answer true or false, and give a one-line reason.

1. With values below $2^{20}$ and $N \ge 40$, the basis maximum is always the answer to the capped problem.
2. If the rank exceeds $N/2$, the answer is smaller than the basis maximum.
3. The knapsack for $N = 38$ and $B = 20$ needs a table of $38 \times 2^{20}$ `int`s.
4. In *Theory Listing 26.4*, the values below every trie node form a range of the sorted array.
5. When $K$ is larger than every XOR, the trie program is much faster than the double loop.

6. A sieve up to $\max(A)$ is enough for the prime-sum test.
7. A tree with 15 vertices has far fewer than $2^{14}$ matchings.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 2 core problems, 1 bonus.

**26.P1 Max XOR choosing at most N/2 elements** SELF-JUDGE CORE — **60 min**

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P1*

**26.P2 Pairs with XOR < K and a prime sum** SELF-JUDGE CORE — **45 min**

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P2*

**26.P3 Sample Complex: brute force for N ≤ 15** SELF-JUDGE BONUS **45 min**

No online judge · solve in OneCompiler · Self-judge

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 26.P3*

## Re-solve from three days ago

Pick one problem from Day 23 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 23.P1 CF 1426D · Non-zero Segments ☐ 23.P2 CF 1689C · Infected Tree ☐ 23.P3 CF 1187E · Tree Painting
☐ 23.P4 CF 1913D · Array Collapse

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 27, Mock 5: dress rehearsal.*

DAY 27 OF 29 · THURSDAY 3 DECEMBER 2026 · MOCK DAY · PHASE 4 · EXAM SIMULATION

27

# The Exam Playbook and Mock 5

*Start at your real reporting time, 4 problems, paper notes for the first 15 minutes, exact exam rules.*

**Read first** · Theory Chapter 27
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 27.P1 CF 1385D · a-Good String | 45 min |
| ☐ | 27.P2 CF 1446B · Catching Cheaters | 60 min |
| ☐ | 27.P3 CF 1324F · Maximum White Subtree | 60 min |
| ☐ | 27.P4 CF 1433F · Zero Remainder Sum | 75 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **4.0 h** |

**NOTES FOR THE DAY**

- Start at the same clock time as your reporting time on 6 Dec.
- No music, no snacks at the desk, phone out of the room.
- Afternoon: review only. No new problems after this mock.

**Why it matters for Infosys.** This is the last full rehearsal. After today, protect your confidence and your sleep.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Euler tour (subtree = range) | Ch. 20 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Fenwick tree (BIT) | Ch. 20 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Segment tree (iterative) | Ch. 20 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Longest increasing subsequence | Ch. 13 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

# Mock exam: Mock 5 · dress rehearsal

Q1 ≈ 1500, Q2 ≈ 1800, Q3 ≈ 1800, Q4 ≈ 2100. Start at your real reporting time. One sitting of 180 minutes, phone in another room, the same rules as the real assessment.

- [ ] **Minutes 0–15.** Read every problem. For each, write the limits, the idea, its complexity and a brute-force fallback in the planning grid below.
- [ ] **Solve the problem you are surest of first**, which is not always Q1. Test on the samples (Codeforces Custom test or OneCompiler) before submitting.
- [ ] **By minute 150** every problem has a submission, even if it is only a brute force.
- [ ] **Minutes 165–180.** Make sure every submission compiles. Do not touch code that already passes.

## Planning grid (first 15 minutes)

| Q | Limits | Idea | Complexity | Brute-force fallback | Order |
|---|---|---|---|---|---|
| Q1 | | | | | |
| Q2 | | | | | |
| Q3 | | | | | |
| Q4 | | | | | |

## Score sheet

| Q | Problem | Rating | Weight | Start | End | % tests | Marks |
|---|---|---|---|---|---|---|---|
| Q1 | **CF 1385D** a-Good String<br>codeforces.com/problemset/problem/1385/D | 1500 | 20 | | | | |
| Q2 | **CF 1446B** Catching Cheaters<br>codeforces.com/problemset/problem/1446/B | 1800 | 30 | | | | |
| Q3 | **CF 1324F** Maximum White Subtree<br>codeforces.com/problemset/problem/1324/F | 1800 | 50 | | | | |
| Q4 | **CF 1433F** Zero Remainder Sum<br>codeforces.com/problemset/problem/1433/F | 2100 | partial | | | | |
| | Weighted score = Σ weight × % tests passed | | | | | | |

**SP checkpoint.** Weighted score ≥ 65.

### Post-mortem (about 60 minutes after the mock)

| Q | What went wrong | Cause | Rule that would have prevented it |
|---|---|---|---|
| Q1 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q2 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q3 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |
| Q4 | | misread · idea · overflow · off-by-one · TLE · input · edge · stack · time | |

## Exercises

Work Exercises 27.1 and 27.2 before the mock, ideally the evening before or in the hour before your reporting time, so that the playbook and the checklist are fresh when the timer starts. Work the rest in the afternoon review, after you have graded the mock. Theory Chapter 27 has the time table (*Theory Table 27.1*), the rules (*Theory Table 27.2*) and the checklist (*Theory Section 27.4*).

**EXERCISE 27.1** **Write your playbook card**

Fill this space as the card you will rehearse today and carry in your head on 6 December. Keep it to what fits here. It needs the time table for three and for four problems, six to eight decision rules, the pre-submission checklist in your own short words, and your three personal rules: open your error log, find the three causes that appear most often, and copy the rule you wrote for each.

**My playbook**

**EXERCISE 27.2 Run the checklist on a submission**

A problem says: the input has $t$ test cases ($t \le 10^4$); each has $n$ (with the sum of $n$ at most $2 \cdot 10^5$) and then $n$ values up to $10^9$, and the samples put every value on its own line. For each test case, print the number of pairs $i < j$ with $a[i] = a[j]$. This submission compiles and looks reasonable.

```
import java.io.*;
import java.util.*;

public class Main {
    static Map<Integer, Integer> seen = new HashMap<>();

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            int n = Integer.parseInt(br.readLine().trim());
            StringTokenizer st = new StringTokenizer(br.readLine());
```

```
            int pairs = 0;
            for (int i = 0; i < n; i++) {
                int x = Integer.parseInt(st.nextToken());
                int before = seen.getOrDefault(x, 0);
                pairs += before;                  // x pairs with every earlier copy
                seen.put(x, before + 1);
            }
            out.println(pairs);
        }
    }
}
```

Go through the nine items of the checklist in order. For every item that fails, write the item, an input that shows the failure and what the program does on it, and the fix.

**EXERCISE 27.3** **Which rule applies?**

For each situation, write what the playbook tells you to do and which row of *Theory Table 27.2* or *Theory Table 27.1* says so.

1. Minute 15 of a three-problem paper. Q3 looks like a tree DP you have done before (sure: 4 of 5). Q1 is a long story you have not yet understood (sure: 2 of 5).
2. Minute 52. Your surest problem passes the samples but gives a wrong answer on your all-equal edge case, and you cannot see why. You wrote its brute force at minute 20.
3. Minute 100 of a three-problem paper. Q2 still fails its second sample, and you have not written a brute force for it.
4. Minute 128 of a four-problem paper. Q1 and Q2 pass everything. Q3 has a promising idea that you estimate at 40 minutes of work; Q4 is untouched.
5. Minute 140 of a three-problem paper. Q3's brute force was submitted at minute 132, and the full solution is half written.
6. Minute 168. You suddenly see how to make your accepted Q2 twice as fast.

### EXERCISE 27.4 Marks arithmetic

*Part 1.* Compute the weighted score 20 · Q1 + 30 · Q2 + 50 · Q3 for each outcome and say whether it meets the Mock 5 checkpoint of 65. Q4's share is recorded but does not enter the score.

| outcome | Q1 | Q2 | Q3 | Q4 | weighted score | ≥ 65? |
|---|---|---|---|---|---|---|
| (a) | 100 % | 100 % | 20 % | 10 % | | |
| (b) | 100 % | 60 % | 40 % | 0 % | | |
| (c) | 100 % | 100 % | 40 % | 30 % | | |
| (d) | 50 % | 100 % | 50 % | 0 % | | |

*Part 2.* In outcome (a), what share of Q3 would have reached the checkpoint?

*Part 3.* Apply *Theory Claim 27.1* with $T = 40$ minutes left, a brute force that takes $b = 15$ minutes and earns $\beta = 15$ marks, and a full solution worth $F = 50$. For which $f$ does brute force first earn less, and by how much? Fill in the two plans' marks for $f = 20$, 25, 30, 40, 41 and 60.

### EXERCISE 27.5 Paper notes for a new problem

A shop has $n \le 10^5$ items with prices up to $10^9$, and $q \le 10^5$ customers arrive, each with a budget up to $10^9$. For each customer, print how many items cost at most the budget. Write the row of paper notes for this problem in three minutes, with every field of *Theory Section 27.6*: limits, idea, complexity, brute-force fall-back with the largest sizes it finishes, edge cases, and how sure you are.

**EXERCISE 27.6 Post-mortem: name the cause and write the rule**

Each line describes a lost problem from a mock. Name its cause with one of the nine causes of Theory Chapter 9 and write a rule that would have prevented it, specific enough to check in ten seconds.

1. The answer was printed with 0-based positions; the statement counted from 1.
2. A recursive DFS crashed on test 9, a path of $2 \cdot 10^5$ vertices.
3. Q3 had no submission at the end; Q2 was debugged until minute 170.
4. Correct output on all samples; "Wrong answer on test 14", where $n = 2 \cdot 10^5$ and every value is $10^9$.
5. A greedy that sorts by deadline was proved wrong by the stress tester after the mock.
6. The program read $p$ after the array because the format text said so; the samples have $p$ right after $n$.

## Re-solve from three days ago

Pick one problem from Day 24 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

- [ ] 24.P1 RPG: maximum monsters defeated
- [ ] 24.P2 Andy's vacation (cancel ≤ K obligations)
- [ ] 24.P3 Gym: fewest exercises to drain energy
- [ ] 24.P4 GfG · Wine Buying and Selling
- [ ] 24.P5 Arrays where each element divides the next (mod 10000)

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 28, Consolidate.*

DAY 28 OF 29 · FRIDAY 4 DECEMBER 2026 · LEARNING DAY · PHASE 4 · EXAM SIMULATION

28

# Consolidation: Recognising Patterns and Recalling Templates

*Templates from memory, a speed drill, and error-log re-solves. No new topics.*

**Read first** · Theory Chapter 28
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | Read Theory Chapter 28; type each new listing once and run it | 40 min |
| ☐ | 28.P1 LC 217 · Contains Duplicate | 15 min |
| ☐ | 28.P2 LC 881 · Boats to Save People | 15 min |
| ☐ | 28.P3 LC 2300 · Successful Pairs of Spells and Potions | 15 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **3.3 h** |

**NOTES FOR THE DAY**

- Handbook in recall mode: write every template from memory, timed. Target: under 40 minutes for the 15 you rated weakest.
- Re-solve 4 error-log problems.
- Speed drill: the 3 problems below, each in under 15 minutes.
- Read the exam-day playbook. Stop by 8 pm.

**Why it matters for Infosys.** Writing templates quickly and without errors is what saves time on the real Q1.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Lazy segment tree (range add, range max) | Ch. 21 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Split into K parts (dp[k][i]) | Ch. 21 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Interval DP and game DP | Ch. 14 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Today has no new material, so the exercises are the consolidation itself. Exercises 28.1 and 28.2 are the recall sitting of *Theory Section 28.4*; do them first, with a timer and the theory book closed. Exercises 28.3 and 28.4 train recognition with the map (*Theory Table 28.1* to *Theory Table 28.4*) and the limits figure (*Theory Figure 28.1*). Exercise 28.5 goes with the speed drill, Exercise 28.6 chooses the four re-solves, and Exercise 28.7 is the paper for Day 29.

### EXERCISE 28.1 The exam file from memory

Start a timer. Write the exam file of Theory Chapter 27 without looking: FastScanner, one `PrintWriter`, a `solve` and a `brute` method side by side, and a `main` that reads $t$ test cases, each with $n$ and then $n$ values. Stop the timer when you believe it compiles.



**CHECKLIST Check, in this order**

- `import java.io.*;` and `import java.util.*;`, and `main` declared `throws IOException`.
- `PrintWriter` wraps a `BufferedWriter` around `new OutputStreamWriter(System.out)`.

- `next()` refills the tokenizer in a `while` loop and returns `null` at the end of the input.
- `nextInt()` and `nextLong()` parse the result of `next()`.
- `out.flush()` once, after the loop over the test cases.
- `solve` and `brute` take the same parameters, so switching between them changes one word.
- Java 8 only: no `var`, no `List.of`.

**Minutes taken, and every checklist item you missed:**

## EXERCISE 28.2 The fifteen weakest templates, timed

Open the pattern recall tracker at the front of this workbook and list the 15 cards rated lowest. Write all 15 templates on separate paper in one sitting, each from its title alone, then compare each with its theory listing. For the templates in *Theory Table 28.5*, check the detail in its last column first.

Start time _____ End time _____ Total minutes _____ (target: 40)

| | Template | Ch. | Differences found (wrong line, missing line, different but correct) | Rating |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |
| 6 | | | | |
| 7 | | | | |
| 8 | | | | |
| 9 | | | | |
| 10 | | | | |
| 11 | | | | |
| 12 | | | | |
| 13 | | | | |
| 14 | | | | |
| 15 | | | | |

**Which three templates took longest, and which of them go on tonight's one-page sheet?**

### EXERCISE 28.3 Which technique?

Each line is a new problem, reduced to its question and its limits. For each, name the technique and the chapter, the signal that decided it, and the complexity. Use the limit first, then the words (*Theory Section 28.3*). Give yourself about a minute per line.

1. The prices of $n \le 2 \cdot 10^5$ days are given. For each day, find the most recent earlier day with a strictly lower price.
2. $n \le 10^5$ codes are given. Count the contiguous blocks whose codes XOR to zero.
3. $n \le 14$ sights and the travel time between every pair are given. Starting at sight 0, visit every sight once in the least total time.
4. A company of $n \le 10^5$ employees is given as a parent array, with a salary for each. For every employee, print the total salary of that employee and everyone below.
5. $n \le 10^5$ machines; machine $i$ makes one item every $t[i] \le 10^9$ seconds. How soon can they make $m \le 10^9$ items together?
6. $n \le 10^5$ towns and $m \le 2 \cdot 10^5$ roads with tolls between 1 and $10^9$. Print the cheapest total toll from town 1 to every town.
7. $n \le 10^5$ values and $q \le 10^5$ operations, each either "add $v$ to every value in positions $l..r$" or "print the largest value in $l..r$".
8. Given $L \le R \le 10^{18}$, count the numbers in $[L, R]$ in which no two neighbouring digits are equal.
9. $n \le 300$ piles in a row. Merging two neighbouring piles costs their total size. Merge everything into one pile at the least total cost.
10. Two strings of length up to 3000. Find the fewest single-character insertions and deletions (no replacements) that turn the first into the second.
11. $n \le 10^5$ people and $q \le 10^5$ events "$x$ and $y$ become friends". After each event, print the number of friend groups.
12. A tree of $n \le 10^5$ nodes, each with a colour. For every node, print the number of distinct colours in its subtree.

**EXERCISE 28.4** **From the limits to the complexity**

For each set of limits, write the target complexity, the number of steps at the limit, and the technique family with its chapters, as in *Theory Figure 28.1*. Compute the steps; do not guess them.

| | Limits | Target complexity | Steps at the limit | Family and chapters |
|---|---|---|---|---|
| 1 | $n \le 16$; the answer depends on which items are chosen | | | |
| 2 | $n \le 400$; a segment's answer comes from splitting it | | | |
| 3 | two strings, each of length $\le 3000$ | | | |
| 4 | $n, q \le 10^5$; point updates and range queries | | | |
| 5 | $t \le 10^4$ test cases, $\Sigma\, n \le 2 \cdot 10^5$ | | | |
| 6 | one number $N \le 10^{12}$ | | | |
| 7 | $n \le 9$; the answer depends on the order of the items | | | |
| 8 | $n \le 2000$; all pairs | | | |

**For line 4, how many steps would the brute force take? For line 5, what does clearing an array of $2 \cdot 10^5$ entries in every test case cost in total?**

**EXERCISE 28.5** **Boats: a trace and a greedy that passes the samples**

*Part 1.* Trace *Theory Listing 28.2* on the weights [4, 1, 3, 2, 3, 5, 6] with *limit* = 6. Then check the answer against two lower bounds: ⌈total weight / *limit*⌉ and ⌈*n*/2⌉.

| boat | i | j | p[i] + p[j] | boat holds |
|---|---|---|---|---|
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |
| 4 | | | | |
| 5 | | | | |

*Part 2.* A classmate submits this greedy instead. It returns 1, 3 and 4 on the three samples of LeetCode 881, which are the expected answers.

```
static int boatsLightestFirst(int[] people, int limit) {
    Arrays.sort(people);
    int i = 0, boats = 0;
    while (i < people.length) {
        boolean pair = i + 1 < people.length && people[i] + people[i + 1] <= limit;
        i += pair ? 2 : 1;                  // the two lightest together if they fit
        boats++;
    }
    return boats;
}
```

Find an input with four people on which it returns the wrong number of boats. Name the cause from the error log's list and write the rule that would have caught it before submission.

**EXERCISE 28.6** **Choose four re-solves**

Here is someone else's error log on the morning of Day 28. Entry 7 failed its re-solve on Day 26 and is due on Day 29. Apply the rule of *Theory Section 28.6*: entries due on Day 29 first, then the highest scores (2 points for a cause that appears at least twice, 2 for a sample-paper topic, 1 for no clean re-solve yet), with ties going to the more recent entry.

| | Problem | Topic | Cause | Re-solved | Logged on day | Score |
|---|---|---|---|---|---|---|
| 1 | LC 907 Sum of Subarray Minimums | monotonic stack | overflow / mod | no | 7 | |

| | Problem | Topic | Cause | Re-solved | Logged on day | Score |
|---|---|---|---|---|---|---|
| 2 | LC 1519 Number of Nodes in the Sub-Tree With the Same Label | trees | stack over-flow | no | 19 | |
| 3 | LC 410 Split Array Largest Sum | binary search | off-by-one | yes | 6 | |
| 4 | LC 315 Count of Smaller Numbers After Self | Fenwick tree | off-by-one | no | 20 | |
| 5 | LC 1143 Longest Common Subsequence | two-string DP | edge case | yes | 12 | |
| 6 | LC 2447 Number of Subarrays With GCD Equal to K | number the-ory | wrong idea | no | 10 | |
| 7 | LC 787 Cheapest Flights Within K Stops | shortest paths | wrong idea | no | 17 | due |
| 8 | LC 2338 Count the Number of Ideal Arrays | number the-ory | overflow / mod | yes | 14 | |

**The four re-solves, in order. Then apply the rule to your own log and write your four.**

**EXERCISE 28.7 Tonight's paper for Day 29**

Before 8 pm, write the four things that *Theory Section 28.7* asks for, as short as you can. This is the reading for tomorrow; anything not written here will not be reviewed.

**Template details rated Again or Shaky today, with their chapters; rules of today's failed re-solves; the error-log rules; anything from the speed drill that overran**

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 3 core problems, 0 bonus.

**28.P1 LC 217 · Contains Duplicate** EASY CORE — **15 min**

`leetcode.com/problems/contains-duplicate` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P1*

**28.P2 LC 881 · Boats to Save People** MEDIUM CORE — **15 min**

`leetcode.com/problems/boats-to-save-people` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P2*

**28.P3 LC 2300 · Successful Pairs of Spells and Potions** MEDIUM CORE — **15 min**

`leetcode.com/problems/successful-pairs-of-spells-and-potions` · LeetCode

| Pattern | Target O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 28.P3*

## Re-solve from three days ago

Pick one problem from Day 25 and solve it again without looking at your old code. If it takes longer than the first time, add it to the error log.

☐ 25.P1 Sample paper · Easy (tree beauty) ☐ 25.P2 Sample paper · Medium (gcd under updates)

☐ 25.P3 Sample paper · Hard (LNDS with XOR ≥ M)

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow: Chapter 29, Rest and logistics.*

DAY 29 OF 29 · SATURDAY 5 DECEMBER 2026 · REST DAY · PHASE 4 · EXAM SIMULATION

29

# Rest and Logistics: The Day Before the Exam

*One warm-up, pack, plan the route, sleep.*

**Read first** · Theory Chapter 29
**Patterns** · —

## Today's plan

| | Block | Time |
|---|---|---|
| ☐ | Handbook review: recall the patterns listed below from memory | 15 min |
| ☐ | 29.P1 LC 1480 · Running Sum of 1d Array | 5 min |
| ☐ | Review: error log, re-solve one problem from 3 days ago | 20 min |
| | **Core total (estimate)** | **1.2 h** |

**NOTES FOR THE DAY**

- One easy warm-up (below) so the workflow feels routine.
- Pack exactly what the Infosys invite lists. Confirm the venue and reporting time.
- Plan to leave early. Lay out tomorrow's things tonight.
- Sleep 7–8 hours. Don't solve problems late at night.

**Why it matters for Infosys.** You have done the work. A rested brain is worth more on Q3 than one more problem tonight.

Start time _____ End time _____ Energy now (1–5) _____ Hours slept _____

## Spaced review: write these from memory

Cover the theory book. For each pattern, write the template on the next blank page or in an editor, then compare it with the listing in the theory chapter shown and rate yourself honestly. "Again" means you looked; "Solid" means it was right first time.

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| Backtracking and subset enumeration | Ch. 22 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Bitmask DP | Ch. 22 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Digit DP | Ch. 22 | 7 days ago | ☐ Again ☐ Shaky ☐ Solid |
| BFS: fewest edges, grids, many sources | Ch. 15 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| DFS without recursion, components | Ch. 15 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |
| Topological order (Kahn) and DAG DP | Ch. 15 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

| Pattern | Taught | Interval | Rating |
|---|---|---|---|
| 0-1 BFS | Ch. 15 | 14 days ago | ☐ Again ☐ Shaky ☐ Solid |

## Exercises

Today's exercises are light on purpose. None of them is a new problem. Exercises 29.1 and 29.2 settle the logistics and the timetable for tonight and tomorrow morning, Exercise 29.3 is the one-page sheet of *Theory Section 29.3*, and Exercises 29.4 and 29.5 are a short quiz to read through, not to labour over. Finish before the evening; the plan's rule is no problems late at night.

### EXERCISE 29.1 The logistics checklist

Take out the Infosys invite, the admit card and the e-mail from Infosys. For each row, first write where the answer comes from: **S** if the plan's sources state it (*Theory Table 29.1*), **I** if only the invite, the admit card or the e-mail can settle it. Then write your answer and tick the row when you have checked it.

| Question | S or I | My answer | ✓ |
|---|---|---|---|
| Date of the assessment | | | |
| Reporting time | | | |
| Venue, building and room | | | |
| Identity documents and printouts to bring | | | |
| Everything else the invite lists | | | |
| Phones, calculators, smartwatches | | | |
| Whether an ordinary watch is allowed | | | |
| Whether scratch paper is provided | | | |
| Whether food or water may come in | | | |
| Languages offered | | | |

### EXERCISE 29.2 The backward plan

*Part 1.* Suppose the reporting time is 08:30, the route takes 65 minutes, you choose a 40-minute margin, and the morning routine takes 80 minutes. Using *Theory Section 29.5*, find the planned arrival, the leaving time, the wake-up time, the bedtime for eight hours of sleep and for seven, and the start of the hour without problems or screens.

*Part 2.* Now with your own numbers from Exercise 29.1 and today's route check.

| **Reporting time** *R* | **Margin** *M* | **Travel** *T* | **Morning** *B* | **Leave** | **Wake** | **Bed (8 hours)** |
|---|---|---|---|---|---|---|
| | | | | | | |

**EXERCISE 29.3** **Your one-page template sheet**

Write your sheet on this page, from tonight's paper of Workbook Exercise 28.7. The top: the playbook in one line per block and your three personal rules. The bottom: only the deciding lines of the templates you rated *Again* or *Shaky*, each with its chapter number. Then read it once, and once more tomorrow morning.

**EXERCISE 29.4 A last pattern quiz**

For each short statement, name the technique and the chapter. Read the limit first.

1. A string of length up to $10^5$. Find the longest substring that uses at most two distinct letters.
2. $n \le 10^5$ build steps, some of which must come before others. Print any valid order, or say that none exists.
3. $n \le 20$ weights, each up to $10^9$. Split them into two groups whose totals are as close as possible.
4. A tree with $n \le 10^5$ nodes and $q \le 10^5$ queries: the number of edges between $u$ and $v$.
5. $n \le 10^5$ workers, each available from time $s$ to time $e$ (times up to $10^9$), and $q \le 10^5$ moments. For each moment, count the workers available.
6. $n \le 10^5$ numbers up to $10^6$. Count the pairs whose gcd is exactly 1.

**EXERCISE 29.5 The first minutes, in order**

Put these actions in the order in which you will do them tomorrow (Theory Chapter 27 and *Theory Section 29.6*).

(a) At minute 15, open the problem you ranked surest. (b) Hand in the phone and any smartwatch as instructed. (c) Choose Java and find where the samples run. (d) Read every problem: the output section first, then the limits and the samples. (e) Write one row of notes per problem. (f) Rank the problems by how sure you are. (g) Type the exam file from memory, or fill in the function stub. (h) Check the input reading against the samples before writing the logic.

## Problem worksheets

Fill the top rows *before* you type code: name the pattern, turn the largest limit into a target complexity (Theory Appendix A), and write the key idea in one sentence. Time limits: Easy 20, Medium 40, Hard 60 minutes. When the time is up, open the hint; if still stuck after 10 more minutes, read the editorial, close it, and write the solution yourself. 1 core problems, 0 bonus.

**29.P1** **LC 1480 · Running Sum of 1d Array** EASY CORE **5 min**

leetcode.com/problems/running-sum-of-1d-array · LeetCode

| Pattern | Target<br>O( ) |
|---|---|
| Key idea | |
| Edge cases tested | |

Time used _____ min ☐ Solved alone ☐ After hint ☐ After editorial ☐ Not yet *Appendix A, hint 29.P1*

## End of day

| Problem | What went wrong | Cause | Rule for next time | Re-solve on |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |

**One thing I understand today that I did not understand yesterday:**

**One thing that is still shaky (re-read it in the theory book tomorrow):**

*Tomorrow is the exam.*

APPENDIX A

# Hints

One hint per problem, taken from the 29-day plan. Open it only when the time limit on the worksheet has run out.

### Day 1 · Set up, Java toolkit, fast input

1.P1 **LC 1 · Two Sum.** One pass: before storing a[i], look for target − a[i] in the map.

1.P2 **LC 242 · Valid Anagram.** Count letters in an int[26]; one string adds, the other subtracts.

1.P3 **LC 20 · Valid Parentheses.** ArrayDeque as a stack: push the closer you expect, pop and compare.

1.P4 **LC 347 · Top K Frequent Elements.** HashMap of counts, then a min-heap of size k (or buckets by count).

1.P5 **LC 729 · My Calendar I.** TreeMap start → end. Check floorKey(start) and ceilingKey(start) for overlap.

1.P6 **LC 206 · Reverse Linked List.** Three pointers: prev, cur, next.

1.P7 **LC 141 · Linked List Cycle.** Slow moves 1, fast moves 2; they meet only if there is a cycle.

1.P8 **LC 49 · Group Anagrams.** Key = the sorted letters of the word.

### Day 2 · Prefix sums, difference arrays, hashing

2.P1 **LC 560 · Subarray Sum Equals K.** Prefix + HashMap seeded with {0: 1}.

2.P2 **LC 974 · Subarray Sums Divisible by K.** Key = Math.floorMod(prefix, k); equal remainders make a valid subarray.

2.P3 **LC 1524 · Number of Sub-arrays With Odd Sum.** Count odd and even prefixes seen so far.

2.P4 **LC 525 · Contiguous Array.** Treat 0 as −1; store the FIRST index of each prefix value.

2.P5 **LC 1109 · Corporate Flight Bookings.** Difference array.

2.P6 **LC 304 · Range Sum Query 2D - Immutable.** 2D prefix: P[i+1][j+1] = P[i][j+1] + P[i+1][j] − P[i][j] + a[i][j].

2.P7 **CF 466C · Number of Ways.** Prefix sums: count split points for one third of the total, then for two thirds.

### Day 3 · Two pointers and sliding window

3.P1 **LC 3 · Longest Substring Without Repeating Characters.** Window with a count (or last-seen index) per character.

3.P2 **LC 1004 · Max Consecutive Ones III.** Window with at most K zeros: the vacation pattern.

3.P3 **LC 424 · Longest Repeating Character Replacement.** Valid while window length − (most frequent count) ≤ k.

3.P4 **LC 15 · 3Sum.** Sort, fix i, two pointers on the rest, skip duplicates.

3.P5 **LC 992 · Subarrays with K Different Integers.** Exactly K = atMost(K) − atMost(K − 1).

3.P6 **LC 239 · Sliding Window Maximum.** Monotonic deque of indices.

3.P7 **LC 76 · Minimum Window Substring.** Counts of what you need vs what the window has; shrink while valid.

### Day 4 · Sorting and greedy: intervals

4.P1 **LC 56 · Merge Intervals.** Sort by start; extend the last interval while they overlap.

4.P2 **LC 435 · Non-overlapping Intervals.** Sort by end; answer = n − (most you can keep).

4.P3 **LC 2406 · Divide Intervals Into Minimum Number of Groups.** This is "minimum platforms": sweep sorted starts and ends.

4.P4 **LC 179 · Largest Number.** Sort strings with (b + a).compareTo(a + b). All zeros → "0".

4.P5 **LC 1798 · Maximum Number of Consecutive Values You Can Make.** Sort; if a coin > reach + 1, stop, else reach += coin.

4.P6 **LC 452 · Minimum Number of Arrows to Burst Balloons.** Sort by end; touching balloons share an arrow.

4.P7 **LC 135 · Candy.** Two passes: left to right, then right to left.

### Day 5 · Greedy with heaps

5.P1 **LC 502 · IPO.** Sort projects by capital; heap of profits you can afford.

5.P2 **LC 1642 · Furthest Building You Can Reach.** Min-heap of the climbs given ladders; the smallest pays bricks.

5.P3 **LC 1526 · Minimum Number of Increments on Subarrays to Form a Target Array.** Answer = sum of the positive rises a[i] − a[i−1].

5.P4 **LC 1046 · Last Stone Weight.** Max-heap: poll two, push the difference.

5.P5 **LC 871 · Minimum Number of Refueling Stops.** Heap of fuel at stations you have passed; refuel from the biggest when stuck.

5.P6 **LC 630 · Course Schedule III.** Sort by deadline; max-heap of durations; drop the longest when over time.

5.P7 **CF 1526C2 · Potions (Hard Version).** Drink every potion; when health goes negative, undo the most negative one so far.

### Day 6 · Binary search, including on the answer

6.P1 **LC 34 · Find First and Last Position of Element in Sorted Array.** Lower bound of x, and lower bound of x + 1, minus 1.

6.P2 **LC 875 · Koko Eating Bananas.** Smallest speed with total hours ≤ h. Hours for a pile = ceil(pile / speed).

6.P3 **LC 1011 · Capacity To Ship Packages Within D Days.** Smallest capacity whose greedy day count ≤ days.

6.P4 **LC 410 · Split Array Largest Sum.** Same check as ship capacity: count parts greedily.

6.P5 **LC 1552 · Magnetic Force Between Two Balls.** Largest gap that still places m balls (round mid up).

6.P6 **LC 2226 · Maximum Candies Allocated to K Children.** Largest amount x with Σ pile / x ≥ k.

6.P7 **LC 719 · Find K-th Smallest Pair Distance.** Binary search the distance; count pairs ≤ d with two pointers.

### Day 7 · Monotonic stacks

7.P1 **LC 739 · Daily Temperatures.** Next greater element to the right; answer = index difference.

7.P2 **LC 503 · Next Greater Element II.** Circular: loop i from 0 to 2n − 1 and use i % n.

7.P3 **LC 84 · Largest Rectangle in Histogram.** Previous and next smaller bar give the width for each bar.

7.P4 **LC 907 · Sum of Subarray Minimums.** Contribution counting, then mod $10^9+7$.

7.P5 **LC 402 · Remove K Digits.** Greedy stack: pop while the top is bigger than the current digit.

7.P6 **LC 316 · Remove Duplicate Letters.** Stack + last index of each letter + "already in stack" flags.

7.P7 **LC 2104 · Sum of Subarray Ranges.** Sum of maximums minus sum of minimums, both by contribution.

### Day 8 · Bits and XOR

8.P1 **LC 136 · Single Number.** XOR everything; pairs cancel.

8.P2 **LC 1442 · Count Triplets That Can Form Two Arrays of Equal XOR.** a == b means xor(i..k) == 0: equal prefix XORs.

8.P3 **LC 421 · Maximum XOR of Two Numbers in an Array.** Binary trie; for each x, walk towards the opposite bits.

8.P4 **LC 1803 · Count Pairs With XOR in a Range.** Trie with counts: pairs with XOR < high + 1 minus pairs with XOR < low.

8.P5 **LC 2429 · Minimize XOR.** Greedy on bits: match num1's high bits first, then fill low bits.

8.P6 **GfG · Maximum Subset XOR.** XOR basis (Gaussian elimination over bits).

8.P7 **LC 1707 · Maximum XOR With an Element From Array.** Offline: sort queries by limit, insert numbers into the trie as they qualify.

### Day 10 · Number theory toolkit

10.P1 **LC 204 · Count Primes.** Sieve of Eratosthenes up to n − 1.

10.P2 **LC 1922 · Count Good Numbers.** 5^(even positions) × 4^(odd positions) with fast power mod $10^9+7$.

10.P3 **LC 2447 · Number of Subarrays With GCD Equal to K.** For each start, extend while gcd is a multiple of k; stop early otherwise.

10.P4 **LC 2862 · Maximum Element-Sum of a Complete Subset of Indices.** Group indices by square-free kernel; the best group sum wins.

10.P5 **LC 1819 · Number of Different Subsequences GCDs.** For each g, gcd of all multiples of g present in the array; count when it equals g.

10.P6 **LC 2572 · Count the Number of Square-Free Subsets.** Numbers ≤ 30: bitmask of the 10 primes, knapsack over masks.

10.P7 **CF 1497E1 · Square-Free Division (easy version).** Square-free kernel, then greedy cuts when a kernel repeats.

### Day 11 · DP I: states, transitions, knapsack

11.P1 **LC 198 · House Robber.** Take or skip: best[i] = max(best[i−1], best[i−2] + a[i−1]).

11.P2 **LC 322 · Coin Change.** dp[a] = min over coins of dp[a − c] + 1; start with a large INF.

11.P3 **LC 518 · Coin Change II.** Combinations: coins in the outer loop, amount in the inner loop.

11.P4 **LC 416 · Partition Equal Subset Sum.** Boolean subset sum to total / 2, looping the sum downwards.

11.P5 **LC 494 · Target Sum.** Count subsets with sum (total + target) / 2.

11.P6 **LC 1049 · Last Stone Weight II.** Split into two groups as equal as possible: subset sum up to total / 2.

11.P7 **LC 740 · Delete and Earn.** Sum points per value, then House Robber over values.

### Day 12 · DP II: two-string DP

12.P1 **LC 1143 · Longest Common Subsequence.** Match → diagonal + 1, else the better of up and left.

12.P2 **LC 72 · Edit Distance.** Insert, delete, replace = left, up, diagonal + 1.

12.P3 **LC 516 · Longest Palindromic Subsequence.** dp[l][r] on one string, or LCS with its reverse.

12.P4 **LC 1312 · Minimum Insertion Steps to Make a String Palindrome.** n − longest palindromic subsequence.

12.P5 **LC 115 · Distinct Subsequences.** dp[i][j] = dp[i−1][j] + (match ? dp[i−1][j−1] : 0). Use long.

12.P6 **LC 97 · Interleaving String.** dp[i][j] = can the first i of s1 and j of s2 form the first i + j of s3.

12.P7 **LC 1092 · Shortest Common Supersequence.** Build the LCS table, then walk back to print.

### Day 13 · DP III: LIS and DP with an extra dimension

13.P1 **LC 300 · Longest Increasing Subsequence.** Write the O(n log n) tails version.

13.P2 **LC 1964 · Find the Longest Valid Obstacle Course at Each Position.** Non-decreasing LIS: upper bound; the answer at i is the position + 1.

13.P3 **LC 1671 · Minimum Number of Removals to Make Mountain Array.** LIS ending at i from the left and from the right; peak needs both ≥ 2.

13.P4 **LC 2370 · Longest Ideal Subsequence.** dp over the last letter: 26 states.

13.P5 **LC 354 · Russian Doll Envelopes.** Sort by width ascending, height descending; LIS on heights.

13.P6 **LC 673 · Number of Longest Increasing Subsequence.** For each i, keep (length, count).

### Day 14 · DP IV: divisor, interval and game DP

14.P1 **LC 368 · Largest Divisible Subset.** Sort; LIS-style DP where j → i is allowed if a[i] % a[j] == 0.

14.P2 **LC 486 · Predict the Winner.** dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]).

14.P3 **LC 1547 · Minimum Cost to Cut a Stick.** Add 0 and n as cuts; interval DP over cut indices.

14.P4 **LC 1690 · Stone Game VII.** Same game DP; the score of a move is the sum of what is left (prefix sums).

14.P5 **LC 2338 · Count the Number of Ideal Arrays.** Divisor chains of s distinct values × C(n − 1, s − 1).

14.P6 **LC 312 · Burst Balloons.** Choose the LAST balloon to burst in (l, r).

14.P7 **LC 664 · Strange Printer.** dp[l][r]; when s[k] == s[l], the two prints can merge.

### Day 15 · Graphs I: BFS, DFS, topological order

15.P1 **LC 200 · Number of Islands.** Each unvisited land cell starts a BFS/DFS that marks its island.

15.P2 **LC 994 · Rotting Oranges.** Multi-source BFS from every rotten orange; the answer is the last layer.

15.P3 **LC 207 · Course Schedule.** Kahn's algorithm; possible iff every course gets processed.

15.P4 **LC 1091 · Shortest Path in Binary Matrix.** BFS with 8 directions; distance counts cells, so start at 1.

15.P5 **LC 785 · Is Graph Bipartite?.** BFS colouring; a neighbour with the same colour means no.

15.P6 **LC 1368 · Minimum Cost to Make at Least One Valid Path in a Grid.** 0-1 BFS: following the arrow costs 0, changing it costs 1.

15.P7 **LC 2050 · Parallel Courses III.** Topological order; finish[v] = time[v] + max finish of prerequisites.

### Day 17 · Graphs II: Dijkstra, DSU, MST

17.P1 **LC 743 · Network Delay Time.** Dijkstra from k; the answer is the largest distance (−1 if any is unreachable).

17.P2 **LC 1631 · Path With Minimum Effort.** Dijkstra where a path costs its largest step.

17.P3 **LC 1584 · Min Cost to Connect All Points.** MST over all pairs (Kruskal, or Prim in $O(n^2)$).

17.P4 **LC 1976 · Number of Ways to Arrive at Destination.** Dijkstra plus ways[]; distances need long.

17.P5 **LC 787 · Cheapest Flights Within K Stops.** State = (city, stops), or Bellman-Ford for k + 1 rounds.

17.P6 **LC 2290 · Minimum Obstacle Removal to Reach Corner.** 0-1 BFS: an obstacle cell costs 1.

17.P7 **LC 2421 · Number of Good Paths.** Process nodes by value; DSU joins neighbours with value ≤ current.

### Day 18 · Trees I: recursion, parent arrays, LCA

18.P1 **LC 543 · Diameter of Binary Tree.** Height of each side; the diameter bends at some node.

18.P2 **LC 236 · Lowest Common Ancestor of a Binary Tree.** Return the node if it is p or q; if both sides return something, this is the LCA.

18.P3 **LC 124 · Binary Tree Maximum Path Sum.** Return the best downward sum; drop negative branches.

18.P4 **LC 2246 · Longest Path With Different Adjacent Characters.** Parent array input: reverse BFS order, keep the two best child chains.

18.P5 **LC 102 · Binary Tree Level Order Traversal.** BFS; process q.size() nodes per level.

18.P6 **LC 1483 · Kth Ancestor of a Tree Node.** Binary lifting table.

18.P7 **LC 2477 · Minimum Fuel Cost to Report to the Capital.** Subtree sizes: each edge needs ceil(size / seats) cars.

18.P8 **LC 2846 · Minimum Edge Weight Equilibrium Queries in a Tree.** LCA plus counts of each weight (1..26) on the root path.

### Day 19 · Trees II: tree DP, rerooting, small-to-large

19.P1 **LC 337 · House Robber III.** Each node returns {best if robbed, best if not}.

19.P2 **LC 1519 · Number of Nodes in the Sub-Tree With the Same Label.** Count labels per subtree: int[26] per node, merged from children (edges given: root it first).

19.P3 **LC 834 · Sum of Distances in Tree.** Rerooting: ans[child] = ans[parent] − size[child] + (n − size[child]).

19.P4 **LC 3068 · Find the Maximum Sum of Node Values.** Any even number of nodes can be XORed; greedy by gain, or dp[v][parity].

19.P5 **LC 2858 · Minimum Edge Reversals So Every Node Is Reachable.** Rerooting over edge directions: reversals from 0, then adjust per edge.

19.P6 **LC 2003 · Smallest Missing Genetic Value in Each Subtree.** Only the path from the node with value 1 to the root matters; or small-to-large sets.

19.P7 **CF 600E · Lomsat gelral.** Small-to-large maps of colour counts, tracking the maximum count and its sum.

### Day 20 · Fenwick trees and segment trees

20.P1 **LC 307 · Range Sum Query - Mutable.** Fenwick or segment tree; update stores the difference.

20.P2 **LC 315 · Count of Smaller Numbers After Self.** Compress values; scan from the right, query the Fenwick tree, then add.

20.P3 **LC 2407 · Longest Increasing Subsequence II.** Segment tree of max over values; query [x − k, x − 1].

20.P4 **CF 914D · Bash and a Tough Math Puzzle.** Gcd segment tree; count how many positions break the gcd (stop after 2).

20.P5 **LC 2286 · Booking Concert Tickets in Groups.** Segment tree of max and sum; descend to the first row with room.

20.P6 **LC 3165 · Maximum Sum of Subsequence With Non-adjacent Elements.** Each node stores 4 values: best with or without its two ends.

20.P7 **LC 2322 · Minimum Score After Removals on a Tree.** Euler tour times decide whether one removed subtree contains the other.

### Day 21 · Lazy segment trees and the K-parts DP

21.P1 **LC 1335 · Minimum Difficulty of a Job Schedule.** dp[day][i] over the last day's start; cost = max of that part.

21.P2 **LC 813 · Largest Sum of Averages.** dp[k][i] with prefix sums for the average of a part.

21.P3 **LC 2916 · Subarrays Distinct Element Sum of Squares II.** Lazy segment tree over start positions: a new element adds 1 to a range of starts.

21.P4 **CF 833B · The Bakery.** The Infosys "gift boxes": dp[k][i] with a lazy max segment tree (handbook card).

21.P5 **LC 732 · My Calendar III.** Lazy segment tree (range add, global max), or a TreeMap sweep.

21.P6 **CF 1527E · Partition Game.** Infosys "k segments min cost": range add of i − prev[a[i]] over earlier split points.

### Day 22 · Backtracking, bitmask DP, digit DP

22.P1 **LC 78 · Subsets.** Bitmask loop, or choose / skip recursion.

22.P2 **LC 1986 · Minimum Number of Work Sessions to Finish the Tasks.** dp[mask] = (sessions, time used in the last one), compared in that order.

22.P3 **LC 1879 · Minimum XOR Sum of Two Arrays.** Assignment with dp[mask]: the next element of nums1 picks an unused element of nums2.

22.P4 **LC 902 · Numbers At Most N Given Digit Set.** Shorter lengths: |D|^len. Same length: digit by digit with tight.

22.P5 **LC 51 · N-Queens.** Backtracking with column and two diagonal flags.

22.P6 **LC 847 · Shortest Path Visiting All Nodes.** BFS over (node, visited mask).

22.P7 **LC 2827 · Number of Beautiful Integers in the Range.** Digit DP: (pos, even − odd, remainder mod k, tight, started).

### Day 24 · Infosys repeat problems I

24.P1 **RPG: maximum monsters defeated.** Sort by power; walk while experience ≥ power.

24.P2 **Andy's vacation (cancel ≤ K obligations).** Count obligations per day; sliding window over days.

24.P3 **Gym: fewest exercises to drain energy.** Sort drains descending; use each twice.

24.P4 **GfG · Wine Buying and Selling.** Answer = sum of |prefix sum|.

24.P5 **Arrays where each element divides the next (mod 10000).** Divisor chains × Pascal's triangle mod 10000.

24.P6 **LIS with (x & y)·2 < (x | y).** The condition means the highest set bits differ.

24.P7 **Smallest base with identical digits.** Try bases ≤ $10^6$, then the two-digit case M = d·(B + 1).

24.P8 **Choose X ≤ K maximising Σ X xor A[i].** Per-bit greedy under K, like digit DP.

24.P9 **CF 1338A · Powered Addition.** The idea behind Infosys "terrain days".

24.P10 **CF 1514C · Product 1 Modulo N.** Infosys "product 1 mod N".

### Day 26 · Repair day + the hard XOR repeat

26.P1 **Max XOR choosing at most N/2 elements.** XOR knapsack: fewest elements for each XOR value below $2^{20}$.

26.P2 **Pairs with XOR < K and a prime sum.** Sieve once to 2·max(A); trie for many pairs.

26.P3 **Sample Complex: brute force for N ≤ 15.** Enumerate matchings for N ≤ 15, check every root-to-leaf path.

### Day 28 · Consolidate

28.P1 **LC 217 · Contains Duplicate.** Speed drill: HashSet.

28.P2 **LC 881 · Boats to Save People.** Speed drill: sort + two pointers.

28.P3 **LC 2300 · Successful Pairs of Spells and Potions.** Speed drill: sort + lower bound.

### Day 29 · Rest and logistics

29.P1 **LC 1480 · Running Sum of 1d Array.** 5-minute warm-up.

APPENDIX B

# Answer key

Answers to the exercises. Check your work after you finish a section, not during it.

## Chapter 1 · Java for Problem Solving: Collections, Input and Output

**Exercise 1.1.** (1) `st == null || !st.hasMoreTokens()`; (2) `return null`; (3) `new StringTokenizer(line)`; (4) `fs.nextLong()`; (5) `out.flush()`. The `while` loop in `next()` keeps reading lines until the tokenizer has a word, so a line break is treated like a space. Whether the four numbers share a line or have one line each, `next()` returns them in the same order.

**Exercise 1.2.**

| char | action | stack (bottom → top) |
|---|---|---|
| ( | push ) | ) |
| [ | push ] | )] |
| ] | pop ], matches | ) |
| { | push } | )} |
| ( | push ) | )}) |
| ) | pop ), matches | )} |
| } | pop }, matches | ) |
| ) | pop ), matches | (empty) |
| ] | stack is empty: return `false` | (empty) |

The `expect.isEmpty()` half of the condition returns `false`: a closer arrived when no opener was waiting. Without the last character the loop ends with an empty stack and the method returns `true`.

**Exercise 1.3.** [12, 15): floor 8 (12), ceiling none, accepted (touching 12 is allowed). [10, 13): floor 8 (12), ceiling 12 (15); rejected because [8, 12) ends after 10. [0, 3): floor none, ceiling 3 (8), accepted because 3 is not less than 3. [15, 16): floor 12 (15), ceiling none, accepted. [14, 16): floor 12 (15), ceiling 15 (16); rejected because [12, 15) ends after 14. Final map: {0: 3, 3: 8, 8: 12, 12: 15, 15: 16}.

**Exercise 1.4.** (1) −2,147,479,015: the true product 2,147,488,281 exceeds $2^{31} - 1$ and wraps by $2^{32}$. (2) −2 and 2: `%` takes the sign of the left operand, `floorMod` does not. (3) `true` and `false`: 127 is in the `Integer` cache −128..127, 200 is not, so two different objects are created. (4) `false` and `true`: `t` is built at run time, so it is a new object with the same text. (5) [5, 7]: `remove(1)` with an `int` argument removes index 1; `remove(Integer.valueOf(1))` would remove a value. (6) It throws a `NullPointerException`: `ArrayDeque` refuses `null`. (7) 195 (an `int`: 97 + 98) and `'z'`. (8) False: `toString` shows the internal heap array; only the first element is guaranteed to be the smallest.

**Exercise 1.5.** (1) $2 \cdot 10^5 \cdot (2 \cdot 10^5 - 1)/2 = 19{,}999{,}900{,}000 \approx 2 \cdot 10^{10}$ pairs: far too slow. (2) $2^{20} \cdot 20 = 20{,}971{,}520 \approx 2 \cdot 10^7$ steps: fits. (3) About $10^5 \cdot \log_2 10^5 \approx 1.7 \cdot 10^6$ for the sort and as much again for the searches: fits easily. (4) $3000^3 = 2.7 \cdot 10^{10}$ triples (or about $4.5 \cdot 10^9$ if only $i < j < k$): too slow; $n \le 500$ is the usual limit for $O(n^3)$. (5) $25 \cdot 10^6$ cells × 8 bytes = $2 \cdot 10^8$ bytes, about 200 MB: too much for most limits.

**Exercise 1.6.** Keep the free spots in a `TreeSet<Integer>`, initially $1..m$. "Arrive $p$" is `ceiling(p)`: if it is `null`, print −1; otherwise remove that spot and print it. "Leave $s$" is `add(s)`. Each event costs $O(\log m)$, so the total is $O(m \log m + q \log m)$. For the events given the arrivals print 2, 3, 5, 6, 3, −1, 1, 4. (After `leave 3` the free spots are {1, 3, 4}; the arrival preferring 6 finds no free spot ≥ 6; after `leave 6` the arrival preferring 4 takes 4.)

**Exercise 1.7.** `cnt.get(x) == cnt.get(y)` compares two `Integer` objects by reference. Counts up to 127 come from the `Integer` cache, so equal counts are the same object and the test passes. With $x = 1$ and $y = 2$ each occurring 128 times the method returns `false` (it returns `true` with 127 each). Fix: compare values, for example `return cnt.getOrDefault(x, 0).intValue() == cnt.getOrDefault(y, 0).intValue();`, which also handles a value that does not occur at all.

**Exercise 1.8.** Compare your code with *Theory Listing 1.8* line by line, especially the order `nxt = head.next; head.next = prev; prev = head; head = nxt;` and the check `fast != null && fast.next != null`. The meeting step is the smallest multiple of λ that is at least max(μ, 1): for μ = 4, λ = 6 it is 6; for μ = 10, λ = 3 it is 12.

## Chapter 2 · Prefix Sums, Difference Arrays and Hashing

**Exercise 2.1.** (1) `seen.put(0L, 1);` (2) `seen.getOrDefault(prefix - k, 0);` (3) `seen.merge(prefix, 1, Integer::sum)`. The look-up must come first so that the current prefix is never matched with itself; otherwise, for $k = 0$, every position adds a phantom empty subarray.

**Exercise 2.2.**

| i | a[i] | prefix | prefix − k | found | total |
|---|---|---|---|---|---|
| 0 | 3 | 3 | −4 | 0 | 0 |
| 1 | 4 | 7 | 0 | 1 | 1 |
| 2 | −7 | 0 | −7 | 0 | 1 |
| 3 | 3 | 3 | −4 | 0 | 1 |
| 4 | 1 | 4 | −3 | 0 | 1 |
| 5 | 3 | 7 | 0 | 2 | 3 |
| 6 | 1 | 8 | 1 | 0 | 3 |
| 7 | −4 | 4 | −3 | 0 | 3 |
| 8 | −2 | 2 | −5 | 0 | 3 |
| 9 | −2 | 0 | −7 | 0 | 3 |

The answer is 3. At row 5 the value 0 has been seen twice (the empty prefix and the prefix after index 2), so two subarrays end there: $a[0..5] = 3 + 4 - 7 + 3 + 1 + 3$ and $a[3..5] = 3 + 1 + 3$. The third is $a[0..1] = 3 + 4$, found at row 1.

**Exercise 2.3.** Rows 3–5: remainders 2, 4, 0; counts found before 0, 3, 1; totals 3, 6, 7; `cnt` after row 5 is [2, 0, 1, 0, 4]. The answer is 7, which matches LeetCode 974's first example. With plain `%`, row 4 computes (2 + (−3)) % 5 = −1, and `cnt[-1]` throws an `ArrayIndexOutOfBoundsException`. `Math.floorMod(-1, 5)` is 4, the correct key.

**Exercise 2.4.** $d = [2, -1, 5, -5, 1, 0, -2]$ and $a = [2, 1, 6, 1, 2, 2]$. Check $a[2]$: +2 (from [0..3]), +5, −1, so 6.

**Exercise 2.5.** $S$ rows 1–3 are [0, 3, 3, 4], [0, 8, 14, 18], [0, 9, 17, 21]. The query is 21 − 4 − 9 + 3 = 11, and directly 6 + 3 + 2 + 0 = 11.

**Exercise 2.6.** (1) True: with positive values the window sum is monotone in each pointer (Chapter 3). (2) False: store the *first* index; later indices give shorter subarrays. (3) False: an `int[]` of size $k$ is faster and has no boxing or worst case. (4) False: $a[4]$ is a prefix sum of $d$, which costs $O(n)$ to recompute; use a Fenwick tree. (5) True: a subarray's sum is odd exactly when its two end prefixes have different parity.

**Exercise 2.7.** Map 0 → −1 and 1 → +1; a substring is balanced when its sum is 0, so its two end prefixes are equal. Store each prefix value with a **count**, look up the **same** value, seed {0: 1}. For `0011010` the prefixes are 0, −1, −2, −1, 0, −1, 0, −1. The value −1 occurs 4 times and 0 occurs 3 times, so the answer is C(4, 2) + C(3, 2) = 6 + 3 = 9.

**Exercise 2.8.** Negative elements break it. With $a = [-1, 1]$ and $k = 5$ the prefix remainder becomes −1, the loop stops, and the method returns 0 instead of 1 (the subarray [−1, 1] has sum 0). Fix: `prefix = Math.floorMod(prefix + x, k);` and delete the `if`.

## Chapter 3 · Two Pointers and Sliding Windows

**Exercise 3.1.** (1) `cnt.merge(a[r], 1, Integer::sum)`; (2) `cnt.size() > k`; (3) `cnt.remove(a[l])`; (4) `l++`; (5) `Math.max(best, r - l + 1)`. To count all valid windows, replace line (5) by `count += r - l + 1` (with a `long count`), because every window $[l', r]$ with $l \le l' \le r$ is valid.

**Exercise 3.2.**

| i | a[i] | removed at back | removed at front | deque (indices) | max |
|---|---|---|---|---|---|
| 0 | 3 | — | — | 0 | — |
| 1 | 1 | — | — | 0 1 | — |
| 2 | 5 | 1, 0 | — | 2 | 5 |
| 3 | 2 | — | — | 2 3 | 5 |
| 4 | 2 | 3 | — | 2 4 | 5 |
| 5 | 6 | 4, 2 | — | 5 | 6 |
| 6 | 0 | — | — | 5 6 | 6 |
| 7 | 4 | 6 | — | 5 7 | 6 |

The output is [5, 5, 5, 6, 6, 6]. In this input no index ever leaves at the front, because each front is removed at the back by a larger value before it expires. At $i = 4$ the `<=` removes index 3: the newer copy at index 4 has the same value and stays in the window longer, so the older copy can never be needed.

**Exercise 3.3.** $r = 3$: $l$ 0 → 2, counts {1: 1, 3: 1}, length 2, best 3. $r = 4$: $l$ stays 2, {1: 1, 3: 2}, length 3, best 3. $r = 5$: $l$ 2 → 3, {2: 1, 3: 2}, length 3, best 3. $r = 6$: $l$ stays 3, {2: 2, 3: 2}, length 4, best 4. $r = 7$: $l$ 3 → 5, {2: 2, 4: 1}, length 3, best 4. The answer is 4, the window [3, 3, 2, 2].

**Exercise 3.4.** (1) Monotone: a sub-window has no more zeros; grow-and-shrink. (2) Not monotone: [4, 6] is valid but its sub-window [4] is not. Extending a valid window keeps it valid, so shrink-while-valid finds the shortest. (3) Not monotone ([4, 6] is valid, [4] is not) and not closed under extension ([4, 6, 1] is invalid). Neither loop works directly; count it as (windows with sum ≤ 10) − (windows with sum ≤ 9), two monotone scans, or use prefix sums. (4) Monotone: a sub-window's maximum is no larger and its minimum no smaller; grow-and-shrink (with two deques, Exercise 3.6). (5) Not monotone, but closed under extension; shrink-while-valid with need/have counters. (6) Not monotone: [20, −15] has sum 5, its sub-window [20] has sum 20; neither loop, use prefix sums.

**Exercise 3.5.** atMost(2): $l$ = 0, 0, 0, 2, 2 and additions 1, 2, 3, 2, 3, total 11. atMost(1): $l$ = 0, 0, 2, 3, 4 and additions 1, 2, 1, 1, 1, total 6. Exactly two distinct values: 11 − 6 = 5. They are [2, 2, 1], [2, 1], [1, 3], [1, 3, 1] and [3, 1].

**Exercise 3.6.** Removing an end element cannot raise the maximum or lower the minimum, so sub-windows of valid windows are valid: grow-and-shrink applies. Keep two monotonic deques of indices: one with decreasing values (its front is the window maximum) and one with increasing values (its front is the minimum). When $a[r]$ enters, pop dominated indices from the back of each and append $r$. While front-max − front-min > $d$, advance $l$ and pop the front of either deque if its index is now < $l$. Each index enters and leaves each deque once, so the total is $O(n)$. For the example the answer is 3, for instance [4, 6, 5] or [9, 7, 8].

**Exercise 3.7.** The method never removes a key whose count drops to 0, so `cnt.size()` never decreases. As soon as the array contains more than $k$ distinct values, the `while` loop keeps advancing $l$ past $r$ and off the end of the array. For $a$ = [1, 2, 3] and $k = 2$ it throws `ArrayIndexOutOfBoundsException` at $r = 2$. The samples it passed, such as [1, 2, 1, 2] and [5, 5, 5] with $k = 2$, never exceed $k$ distinct values. Fix: `if (cnt.merge(a[l], -1, Integer::sum) == 0) cnt.remove(a[l]);`.

**Exercise 3.8.** (1) Negative values break windows: prefix sums with a hash map, $O(n)$ expected. (2) All values positive, so "sum ≥ $S$" survives extension: shrink-while-valid window, $O(n)$. (3) Order does not matter: sort, fix $i$, then count pairs with sum < $T - a[i]$ using `countPairsAtMost` on the suffix; $O(n^2) \approx 4.5 \cdot 10^6$ steps. (4) Fixed window size:

monotonic deque with the comparison flipped, O($n$). (5) "At most 3 distinct" is monotone: grow-and-shrink with an `int[]` of counts, O($n$).

**Exercise 3.9.** Compare with *Theory Listing 3.9*. Invariant: the deque holds indices of the current window in increasing order with strictly decreasing values, and every window index not in it is dominated by a later, at least as large index. Each index is added once and removed at most once, so there are at most 2$n$ deque operations.

## Chapter 4 · Sorting and Greedy Choice: Intervals

**Exercise 4.1.** (1) `Integer.compare(x[0], y[0]);` (2) `cur[0] <= last[1];` (3) `Math.max(last[1], cur[1]);` (4) `Integer.compare(x[1], y[1]);` (5) `cur[0] >= end`. Merging needs every interval that can touch the current block to arrive while that block is the last one, which sorting by start guarantees. Scheduling needs the choice that leaves the most room, which is the earliest end (*Theory Theorem 4.3*). `merge` returns [1, 3] for [1, 2], [2, 3], because touching intervals merge.

**Exercise 4.2.** The events, in order, with busy after each: 1 start (1), 2 start (2), 3 start (3), 3 end (2), 4 start (3), 4 end (2), 5 end (1), 6 start (2), 6 end (1), 7 start (2), 8 end (1), 9 end (0). The best is 3, so 3 rooms are needed (for example at time 3, meetings [1, 4], [2, 3] and [3, 5] all contain 3). If a room freed at $t$ can be reused at $t$, ends sort first and the maximum drops to 2.

**Exercise 4.3.** [3, 5]: the earliest-freed room ends at 3, which is not < 3, so a new room; heap {3, 4, 5}. [4, 6]: reuse the room freed at 3; {4, 5, 6}. [6, 8]: reuse the room freed at 4; {5, 6, 8}. [7, 9]: reuse the room freed at 5; {6, 8, 9}. Three rooms, matching Exercise 4.2.

**Exercise 4.4.** Take any order with neighbours $i$ before $j$ and $t_i > t_j$, and let $T$ be the time when the first of them starts. Every other job finishes at the same time in both orders. In the given order the two finish at $T + t_i$ and $T + t_i + t_j$; after swapping, at $T + t_j$ and $T + t_j + t_i$. The sum changes by $t_j - t_i < 0$, so the swap strictly improves it. An optimal order therefore has no such pair, which means it is sorted by $t$ (and ties do not matter). For $t$ = (3, 1, 2) the six orders give 3 1 2 → 13, 3 2 1 → 14, 1 3 2 → 11, 1 2 3 → 10, 2 3 1 → 13, 2 1 3 → 11; the sorted order 1, 2, 3 gives the minimum 10.

**Exercise 4.5.** Amount 6: the greedy takes 4 + 1 + 1 (three coins), but 3 + 3 uses two. (Amount 10 also fails: 4 + 4 + 1 + 1 against 4 + 3 + 3.) For coins {1, 5, 10, 25} the rule agreed with an exact dynamic programme (Chapter 11) for every amount up to 500. That is the way to gain confidence: compare the greedy with an exact method on every small input, which still is evidence, not proof.

**Exercise 4.6.** The order is 9, 89, 898, 8, 0, giving 98989880. The comparisons between neighbours: 9 + 89 = 989 > 899; 89 + 898 = 89898 > 89889; 898 + 8 = 8988 > 8898; 8 + 0 = 80 > 08. $\nu(89) = 89/99 \approx 0.898990$ and $\nu(898) = 898/999 \approx 0.898899$, so "89" comes first, as the concatenations say. For [0, 0] the sorted strings start with "0", so the method returns "0", not "00".

**Exercise 4.7.** State: *reach*, meaning every total 0..*reach* can be paid with the coins used and added so far. If the next coin $c$ satisfies $c \le reach + 1$, use it: *reach* += $c$ (*Theory Lemma 4.7*). Otherwise the total *reach* + 1 cannot be paid, so add the coin *reach* + 1, which extends the range furthest: *reach* = 2 · *reach* + 1. Stop when *reach* ≥ $n$. Each added coin more than doubles *reach*, so at most about $\log_2 n + 1$ coins are added; the loop is O($m$ + log $n$) for $m$ coins. For [1, 5, 10] and $n$ = 20: use 1 (reach 1), add 2 (reach 3), add 4 (reach 7), use 5 (reach 12), use 10 (reach 22). Two coins are added, which a brute force over all choices of added coins confirms.

**Exercise 4.8.** (1) False as a statement about the algorithm (quicksort is not stable), but it cannot be observed, because equal `int` values are identical. (2) True: object sorts use TimSort, O($n$ log $n$) in the worst case. (3) False: the difference can reach $2 \cdot 10^9$, which exceeds $2^{31} - 1$ and overflows. (4) False: boxing and comparator calls made it about five times slower in this book's measurements (410–570 ms against 11–91 ms). (5) False: $\log_2(5!) = \log_2 120 \approx 6.9$, so some input needs at least 7 comparisons. (6) False: some false greedies only fail on larger inputs (Theory review question 3 needed 9 or more intervals); test sizes up to about 10 and prove the rule.

**Exercise 4.9.** After pass 1: [2, 3, 4, 0, 1, 2, 1]. After pass 2: [2, 2, 1, 0, 1, 2, 1]. Neighbours differ by at most 1 everywhere, and each entry is at most its cap (for example index 2 is 1 ≤ 5, forced down by the 0 at index 3).

## Chapter 5 · Greedy Algorithms with Heaps

**Exercise 5.1.** (1) `a[--size]`; (2) `(i - 1) / 2`; (3) `2 * i + 1`; (4) `r < size && a[r] < a[m]`; (5) `return`; (6) `i = m`. The last leaf is the only element that can leave without breaking the shape: the array stays contiguous, so the tree stays complete and the index arithmetic stays valid. Moving children up instead would leave a hole wherever the chain of promotions ends, usually in the middle of the array.

**Exercise 5.2.**

| call | value sifted | swaps (index pairs) | array after the call |
|---|---|---|---|
| siftDown(3) | 2 | none | [8, 6, 7, 2, 5, 9, 1, 4, 3] |
| siftDown(2) | 7 | 2↔6 | [8, 6, 1, 2, 5, 9, 7, 4, 3] |
| siftDown(1) | 6 | 1↔3, 3↔8 | [8, 2, 1, 3, 5, 9, 7, 4, 6] |
| siftDown(0) | 8 | 0↔2, 2↔6 | [1, 2, 7, 3, 5, 9, 8, 4, 6] |

Five swaps in total. `poll()` returns 1 and moves the last leaf 6 to the root, giving [6, 2, 7, 3, 5, 9, 8, 4]. The 6 is swapped with 2 (indices 0↔1), then with 3 (1↔3), then with 4 (3↔7), which leaves [2, 3, 7, 4, 5, 9, 8, 6].

**Exercise 5.3.**

| i | h[i] → h[i+1] | climb | ladders on (heap after) | bricks pay | bricks left |
|---|---|---|---|---|---|
| 3 | 9 → 10 | 1 | {3, 6} | 1 | 6 |
| 4 | 10 → 6 | −4 | skip | | 6 |
| 5 | 6 → 13 | 7 | {6, 7} | 3 | 3 |
| 6 | 13 → 14 | 1 | {6, 7} | 1 | 2 |
| 7 | 14 → 18 | 4 | {6, 7} | 4 | −2 |

The method returns 7 at row 7, so rows 8 and 9 stay empty: building 7 (height 14) is reached, building 8 is not. At row 5 the ladder on the first climb (3) is taken back and given to the climb of 7. At row 3 the new climb of 1 is the smallest of three, so it pays bricks at once. With 3 ladders the method returns 9: at $i = 9$ the climb of 8 takes the ladder from the climb of 4, which then needs 4 bricks when only 2 are left. A brute force over all ladder assignments gives 7 and 9 as well.

**Exercise 5.4.** (1) False: [1, 3, 2] is a heap with its largest value at index 1. The largest value is always in some leaf, but not necessarily the last one. (2) False: heapify makes fewer than $2n$ swaps (*Theory Theorem 5.5*), and for $n = 10^6$ at most 999,993. (3) True: the collection constructor heapifies; on a decreasing input, $10^6$ calls of `add` made 17,951,445 swaps. (4) False: it searches the array linearly, $O(n)$; see *Theory Table 5.1*. (5) False: the sorted print is a coincidence. Adding 5, 1, 4, 2, 3 prints `[1, 2, 4, 5, 3]`. (6) False: for $2 \cdot 10^9$ and $-2 \cdot 10^9$ the subtraction overflows to 294,967,296, and the queue polls $-2 \cdot 10^9$ first. Use `Collections.reverseOrder()` or `Integer.compare(q, p)`. (7) True: profits are non-negative, so the capital never falls below the requirement of a project already unlocked. (8) True: the climb left the heap because at least $L$ climbs as large as it were present, and the set of the $L$ largest only gains larger climbs later, so it never returns (and the code never pushes it again).

**Exercise 5.5.** Sort the job numbers by arrival (the unlock key) and keep a pointer into that order, plus the current time $T$ as a `long`. Each round: if the heap is empty and the next arrival is later than $T$, set $T$ to that arrival (idle); push every job with arrival ≤ $T$; poll the heap's top, print it, and add its pages to $T$. The comparator is `(x, y) -> pages[x] != pages[y] ? Integer.compare(pages[x], pages[y]) : Integer.compare(x, y)`. Each job is pushed and polled once: $O(n \log n)$ time, $O(n)$ memory. For the seven jobs the order is 0, 2, 1, 3, 4, 6, 5. Job 0 prints 0–4; at 4 the waiting jobs are 1, 2, 3, and job 2 (1 page) goes first, 4–5; jobs 1 and 3 both have 3 pages, so job 1 prints 5–8 and job 3 prints 8–11; the printer idles from 11 to 12; job 4 prints 12–17; then job 6 (1 page) prints 17–18 and job 5 prints 18–20. This method agreed with a direct $O(n^2)$ simulation on 5,000 random inputs.

**Exercise 5.6.** On [2, 3, 4, 1] it returns [2, 1, 4, 3]: the call for index 0 finds both children (3 and 4) larger and stops, and only then does the call for index 1 bring the 1 up to index 1, below the root 2. (With repeated values, [1, 1, 1, 0] fails in the same way.) On 1,000 random arrays of length 1 to 10 the forward version failed on 413. The proof of *Theory Theorem 5.4* needs both child subtrees of $i$ to be heaps when $i$ is processed, which is the starting condition of *Theory Lemma 5.3*. Going forward, index $i$ is processed before its subtrees are repaired. The fix is the order of the loop: `for (int i = a.length / 2 - 1; i >= 0; i--)`, which gives [1, 2, 4, 3].

**Exercise 5.7.** (a) *Theory Listing 5.4*: a sort of O($n$ log $n$), about $1.7 \cdot 10^6$ comparisons, then at most $n$ pushes and $k$ polls, $(n + k) \log_2 n \approx 3.3 \cdot 10^6$ heap steps. It fits easily. (b) Each `remove(x)` scans up to $10^5$ elements, so $10^5$ cancels can cost $10^5 \cdot 10^5 = 10^{10}$ steps: far too slow. Use lazy deletion (*Theory Listing 5.3*), O(log $n$) amortised per request, about $3 \cdot 10^5 \cdot 17 \approx 5 \cdot 10^6$ steps, or a `TreeMap` of counts. (c) Heapify, for example `new PriorityQueue<>(list)`, makes at most 999,993 swaps for this $n$, and 20 polls at most $20 \cdot 19 = 380$ more. Adding the values one at a time can take up to 17,951,445 swaps, and sorting about $2 \cdot 10^7$ comparisons; both also fit, but heapify does the least work. (d) The heap never holds more than 6 climbs, so *Theory Listing 5.6* costs O($n$ log $L$), about $10^5 \cdot \log_2 6 \approx 2.6 \cdot 10^5$ steps. It fits easily.

**Exercise 5.8.** Check your version against this list: the array doubles when full; `add` writes at index `size` and sifts up from it; `poll` saves `a[0]`, writes `a[--size]` into it and sifts down from 0; `siftUp` stops at the root or when the parent is not larger; `siftDown` picks the smaller child, checks `r < size` before reading `a[r]`, and stops when neither child is smaller; `furthestBuilding` skips climbs ≤ 0, pushes every positive climb, polls into the bricks when the heap holds more climbs than there are ladders, and returns $i$ (not $i + 1$) when the bricks go negative.

## Chapter 6 · Binary Search and Searching on the Answer

**Exercise 6.1.** (1) `a.length`; (2) `lo < hi`; (3) `(lo + hi) >>> 1` (or `lo + (hi - lo) / 2`); (4) `hi = mid`; (5) `lo = mid + 1`. The answer $n$ ("every element is smaller than $x$") must be inside the range of possible results. With `hi = a.length - 1`, the array [0] and $x = 1$ return 0, although no element is ≥ 1 and the correct answer is 1.

**Exercise 6.2.** upperBound($a$, 8): (lo, hi, mid, a[mid]) = (0, 9, 4, 6) → lo = 5; (5, 9, 7, 11) → hi = 7; (5, 7, 6, 9) → hi = 6; (5, 6, 5, 6) → lo = 6; result 6. lowerBound($a$, 6): (0, 9, 4, 6) → hi = 4; (0, 4, 2, 4) → lo = 3; (3, 4, 3, 6) → hi = 3; result 3. The value 6 occurs upperBound($a$, 6) − lowerBound($a$, 6) = 6 − 3 = 3 times. `Arrays.binarySearch(a, 6)` may return any of 3, 4, 5 (the JDK returns 4 here, its first midpoint); `Arrays.binarySearch(a, 5)` returns −4, because the insertion point of 5 is 3.

**Exercise 6.3.** The remaining rows are (4, 7, 5, 4 days) → fails, lo = 6; (6, 7, 6, 3 days) → works, hi = 6; then lo = hi = 6 and the loop stops. The answer is 6, with the greedy days [3, 2], [2, 4], [1, 4].

**Exercise 6.4.** (1) False: it returns *some* matching index; the documentation gives no guarantee which. (2) True: when every element is smaller than $x$. (3) False: it must be rounded up, or `lo = mid` makes no progress when $hi = lo + 1$. (4) True: each call at least halves the range of candidates (*Theory Theorem 6.4*). (5) False: neighbouring doubles near $10^9$ are about $1.2 \cdot 10^{-7}$ apart, so the gap may never drop below $10^{-9}$; use a fixed number of iterations. (6) False: *Theory Theorem 6.10* shows it is always an element. (7) False in general: a huge $hi$ can overflow sums or products inside the check (for example `mid * count`), which turns a correct predicate into a wrong one.

**Exercise 6.5.** ok($T$) = "$\Sigma \lfloor T/t[i] \rfloor \geq M$". Each term is non-decreasing in $T$, so if $T$ works, $T + 1$ works. $lo = 0$ (or 1) and $hi = \min(t) \cdot M$: the fastest machine alone makes $M$ products in that time, so ok($hi$) is true. One check is O($n$); the search makes about $\log_2(10^{18}) \approx 60$ checks, so O(60 $n$) ≈ $1.2 \cdot 10^7$ steps. Overflow: $hi$ can be $10^{18}$, so the bounds are `long`, and the sum $\Sigma\, T/t[i]$ can exceed `long` when $T$ is large and many $t[i]$ are small; stop adding as soon as the sum reaches $M$. On $t = [3, 2, 5]$, $M = 10$, with $lo = 0$ and $hi = 20$: (0, 20, 10, 10 products) → hi = 10; (0, 10, 5, 4) → lo = 6; (6, 10, 8, 7) → lo = 9; (9, 10, 9, 8) → lo = 10. The answer is $T = 10$: after 10 minutes the machines have made 3 + 5 + 2 = 10 products, after 9 only 8.

**Exercise 6.6.** The smallest failing input is $a = [1]$ with $k = 1$ (correct answer 1). Then $lo = 0$, $hi = 1$ and the rounded-down midpoint is 0; `piecesAtLeast(a, 0)` = 1 ≥ $k$, so `lo = mid` sets $lo$ to 0 again and the loop repeats for ever. In general the method hangs as soon as the range has two values, because $lo$ always passes. Fix: `long mid = lo + (hi - lo + 1) / 2;`.

**Exercise 6.7.** count(5) = 9, count(6) = 11, count(7) = 11, count(8) = 13, count(10) = 15. The search: (1, 20, 10, 15) → hi = 10; (1, 10, 5, 9) → lo = 6; (6, 10, 8, 13) → hi = 8; (6, 8, 7, 11) → hi = 7; (6, 7, 6, 11) → hi = 6; answer 6. The sorted table is [1, 2, 2, 3, 3, 4, 4, 4, 5, 6, 6, 8, 8, 9, 10, 12, 12, 15, 16, 20], whose 11th element is 6. The search looks for the *smallest x* with count(*x*) ≥ 11; 7 is not an entry of the table, so count does not change between 6 and 7.

**Exercise 6.8.** (1) *lo* = 1, *hi* = $10^9$ (the largest pile); bounds fit in `int` but the total of hours needs `long`; 30 iterations. (2) *lo* = 500 at most (the heaviest package), *hi* = the total ≤ $2.5 \cdot 10^7$; `int` would do but `long` is the safe default; $\lceil \log_2 2.5 \cdot 10^7 \rceil = 25$ iterations. (3) *lo* = max ≤ $10^6$, *hi* = total ≤ $10^9$; at most 30 iterations; sums inside the check reach $10^9$, close to the `int` limit, so use `long`. (4) A fixed number of iterations: $10^9 / 10^{-6} = 10^{15} \approx 2^{50}$, so 50 halvings are enough in exact arithmetic; use 100 to be safe.

**Exercise 6.9.** Compare with *Theory Listing 6.4*. `firstTrue` uses `mid = lo + (hi - lo) / 2` with `hi = mid` / `lo = mid + 1`; `lastTrue` uses `mid = lo + (hi - lo + 1) / 2` with `lo = mid` / `hi = mid - 1`. The rounding must keep *mid* strictly away from the end that its branch assigns, so that every branch shrinks the range.

## Chapter 7 · Monotonic Stacks

**Exercise 7.1.** (1) `Arrays.fill(res, -1)`; (2) `top > 0`; (3) `a[st[top - 1]] < a[i]`; (4) `res[st[--top]] = i`; (5) `st[top++] = i`. For next greater or equal, pop on `a[st[top - 1]] <= a[i]`; for next smaller, pop on `a[st[top - 1]] > a[i]`.

**Exercise 7.2.** Popped at each step: none, none, none, {2, 1}, {3, 0}, none, {5, 4}. Stacks after: [0], [0, 1], [0, 1, 2], [0, 3], [4], [4, 5], [6]. The result is [4, 3, 3, 4, 6, 6, −1] with 6 pops. Each index is pushed once and can be popped at most once, so 7 pushes allow at most 7 pops; the last index is never popped here.

**Exercise 7.3.** Rows 2–4: (left 1, right 3, count 1 · 1 = 1, contribution 2); (left 1, right 4, count 2 · 1 = 2, contribution 4); (left −1, right 5, count 5 · 1 = 5, contribution 5). The total is 3 + 6 + 2 + 4 + 5 = 20, and the counts 1 + 6 + 1 + 2 + 5 add to 15 = 5 · 6/2, the number of subarrays. A brute force also gives 20. Index 1 does not own any subarray that also contains index 4, for example [1, 2, 2, 1] or the whole array: the value 1 occurs again at index 4, and the rightmost minimum owns the subarray, so index 4 does.

**Exercise 7.4.** L = [−1, 0, 0, 2, −1, 4], R = [4, 2, 4, 4, 6, 6], widths [4, 1, 3, 1, 6, 1], areas [8, 4, 9, 5, 6, 3]. The largest rectangle has area 9: height 3 over bars 1..3.

**Exercise 7.5.** (1) False: each index is pushed once and popped at most once, so the total number of pops is at most *n* and the method is O(*n*). (2) True: everything ≥ *a*[*i*] is removed, so the top is strictly smaller. (3) True: such a subarray is owned by every position of its minimum. (4) True: the sum of differences is the difference of the sums. (5) False: it changes no answer, only adds work (Theory Chapter 7, review question 4). (6) False: the stack is non-decreasing, so the best deletion is the last character; they come off the end.

**Exercise 7.6.** The span needs the previous strictly greater price: pop while `p[top] <= p[i]`; then the span is *i* + 1 if the stack is empty and *i* − top otherwise; push *i*. O(*n*) time and memory, by the push–pop count. For [30, 25, 27, 27, 40, 22, 26] the spans are [1, 1, 2, 3, 5, 1, 2].

**Exercise 7.7.** Any array with a repeated minimum fails; the smallest positive one is [1, 1]. The true sum is 1 + 1 + 1 = 3; the buggy method gives 4, because the subarray [1, 1] is owned by both positions. On [3, 1, 2, 2, 1] it gives 24 instead of 20. Fix: make one side non-strict, for example `while (r < n && a[r] > a[i]) r++;` on the right.

**Exercise 7.8.** 3: push, stack 3, k 3. 1: pop 3, stack 1, k 2. 4: push, stack 14. 1: pop 4, stack 11, k 1. 5: push, stack 115. 9: push, stack 1159. 2: pop 9, stack 1152, k 0. The result is 1152. Spending the third deletion on the 9 gives 1152; spending it on the 5 would give 1192. The two first differ at the third digit, where 5 < 9, so 1152 is smaller. The greedy deletes the 9 because 9 > 2 is the first descent of 11592.

**Exercise 7.9.** Compare with *Theory Listing 7.5*. The left scan pops on `a[top] >= a[i]` (so left is strictly smaller) and the right-to-left scan pops on `a[top] > a[i]` (so right is smaller or equal); the opposite pairing (non-strict on the left, strict on the right) is also correct. The same strictness on both sides is never correct.

## Chapter 8 · Bits, XOR, Tries and Linear Bases

**Exercise 8.1.** (1) `((x >> b) & 1) ^ 1`; (2) `res |= 1 << b`; (3) `child[node][want ^ 1]`; (4) `cnt[child[node][xb]]`; (5) `child[node][xb]`; (6) `if (node == 0) break;`. Every $y$ in the child $x_b$ agrees with $x$ at bit $b$, so $x \oplus y$ has 0 there, while $k$ has 1, and the two agree on all higher bits (the walk has followed $k$'s bits so far). By *Theory Lemma 8.4*, all of them are smaller than $k$. In the child $x_b \oplus 1$ the XOR bit is 1, equal to $k$'s, so those numbers are still undecided and the walk continues there.

**Exercise 8.2.** The trie has 19 nodes: the root, 2 nodes at depth 1 (prefix 0 with count 3, prefix 1 with count 3), 4 at depth 2 (00: 1, 01: 2, 10: 1, 11: 2), 6 at depth 3 and 6 leaves. The bound $6 \cdot 4 + 1 = 25$ is not reached because the numbers share prefixes.

| bit b | x_b | want | child exists? | res after | node after |
|---|---|---|---|---|---|
| 3 | 1 | 0 | yes | 8 = 1000 | 0 |
| 2 | 1 | 0 | yes | 12 = 1100 | 00 |
| 1 | 0 | 1 | yes | 14 = 1110 | 001 |
| 0 | 1 | 0 | no | 14 = 1110 | 0011 |

`maxXor(13)` = 14, reached at $y = 3 = 0011$.

| bit b | x_b | k_b | action | count after |
|---|---|---|---|---|
| 3 | 1 | 0 | go to child 1 (prefix 1) | 0 |
| 2 | 1 | 1 | add child 1 (prefix 11, count 2), go to child 0 (prefix 10) | 2 |
| 1 | 0 | 1 | add child 0 (prefix 100, count 1), go to child 1: missing, stop | 3 |
| 0 | — | — | not reached | 3 |

`countXorLess(13, 6)` = 3. The six XORs are $13 \oplus 9 = 4$, $13 \oplus 5 = 8$, $13 \oplus 12 = 1$, $13 \oplus 3 = 14$, $13 \oplus 14 = 3$ and $13 \oplus 6 = 11$. The maximum 14 comes from $y = 3$, and the three XORs below 6 come from 9, 12 and 14. The count 2 at bit 2 is the subtree 11 = {12, 14}, and the count 1 at bit 1 is the subtree 100 = {9}.

**Exercise 8.3.**

| add(x) | x in binary | reductions | outcome |
|---|---|---|---|
| 17 | 10001 | bit 4: ⊕ 26 → 01011 (11); bit 3: ⊕ 11 → 00000 | false (17 = 26 ⊕ 11) |
| 7 | 00111 | none | b[2] = 00111 (7) |
| 28 | 11100 | bit 4: ⊕ 26 → 00110 (6); bit 2: ⊕ 7 → 00001 (1) | b[0] = 00001 (1) |
| 22 | 10110 | bit 4: ⊕ 26 → 01100 (12); bit 3: ⊕ 11 → 00111 (7); bit 2: ⊕ 7 → 00000 | false (22 = 26 ⊕ 11 ⊕ 7) |

| i | b[i] | r ⊕ b[i] | take? | r after |
|---|---|---|---|---|
| 3 | 11 | 17 | no | 26 |
| 2 | 7 | 29 | yes | 29 |
| 1 | 0 | — | (empty slot) | 29 |
| 0 | 1 | 28 | no | 29 |

The maximum subset XOR is $29 = 11101_2$. The rank is 4, so there are $2^4 = 16$ distinct XORs, each from $2^{6-4} = 4$ subsets; a brute force over all 64 subsets confirms both numbers and the maximum. `canMake(13)`: 01101 → ⊕ 11 → 00110 → ⊕ 7 → 00001 → ⊕ 1 → 0, so yes (for example 17 ⊕ 28 = 13). `canMake(14)`: 01110 → ⊕ 11 → 00101 → ⊕ 7 →

00010, and $b[1]$ is empty, so 2 remains and the answer is no. Two original numbers with XOR 29 are 26 and 7 (or 11 and 22).

**Exercise 8.4.**

| bit b | c_b | pairs | gain if X_b = 0 | gain if X_b = 1 | free choice |
|---|---|---|---|---|---|
| 3 | 3 | 12 | 24 | 32 | 1 |
| 2 | 2 | 10 | 8 | 20 | 1 |
| 1 | 4 | 12 | 8 | 6 | 0 |
| 0 | 3 | 12 | 3 | 4 | 1 |

The sum over pairs is $12 \cdot 8 + 10 \cdot 4 + 12 \cdot 2 + 12 \cdot 1 = 172$, which matches adding the 21 XORs directly. With no limit the best $X$ is $1101_2 = 13$, with value $32 + 20 + 8 + 4 = 64$, but $13 > 10$.

| candidate | X (binary) | X | Σ (X ⊕ A[i]) |
|---|---|---|---|
| X = K | 1010 | 10 | 32 + 8 + 6 + 3 = 49 |
| branch at bit 3 | 0101 | 5 | 24 + 20 + 8 + 4 = 56 |
| branch at bit 1 | 1001 | 9 | 32 + 8 + 8 + 4 = 52 |

The best is $X = 5$ with 56. A brute force over $X = 0..10$ agrees. Giving up bit 3 costs 8, but it frees bit 2, which gains 12.

**Exercise 8.5.** (1) False: Java takes the shift distance mod 32, so `1 << 32` is `1 << 0` = 1; write `1L << 32`. (2) True: `>>` rounds down ($\lfloor -8.5 \rfloor = -9$), while integer division rounds towards zero. (3) False: it is also true for 0 and for `Integer.MIN_VALUE`, whose bits are a single 1 followed by 31 zeros (and MIN − 1 wraps to MAX); require $x > 0$. (4) False: it returns the *value* 32; the index 5 is `31 - Integer.numberOfLeadingZeros(40)`. (5) False: by *Theory Claim 8.2* the XOR is $u \oplus v$, and $u \oplus v = 0$ would force $u = v$ by law (5) of *Theory Theorem 8.1*. (6) False: $n \cdot B + 1$ is an upper bound; the trie of Exercise 8.2 has 19 nodes, not 25. (7) False: in Exercise 8.3, $26 \oplus 11 \oplus 7 \oplus 1 = 23$, but the maximum is 29; the greedy must skip $b[3]$ and $b[0]$. (8) True: the rank is 4, and each reachable value comes from $2^{n-r} = 2^2 = 4$ subsets (*Theory Section 8.5*, "What the basis cannot answer").

**Exercise 8.6.** $\text{xor}(a[l..r]) = X[r+1] \oplus X[l]$, so count the pairs $p < q$ of prefix indices with $X[p] \oplus X[q] < k$. Keep a binary trie with counts of the prefixes seen so far, seeded with the empty prefix $X[0] = 0$. For $q = 1..n$: compute $X[q]$, add `countXorLess(X[q], k)`, and only then insert $X[q]$, so that no prefix is paired with itself. Since $10^6 < 2^{20}$, $B = 20$ bits suffice for both the values and $k$. Time: $n + 1$ inserts and $n$ queries of 20 steps, about $4 \cdot 10^6$. Memory: at most $20(n+1) + 1 \approx 2 \cdot 10^6$ nodes, about 24 MB in the flat layout of *Theory Listing 8.5* with counts. The answer can reach $n(n+1)/2 \approx 5 \cdot 10^9$, so it needs a `long`.

For the example the prefixes are $X = [0, 5, 4, 2, 0, 7]$. The pairs with XOR below 4 are (0, 3) with 2, (0, 4) with 0, (1, 2) with 1, (1, 5) with 2, (2, 5) with 3 and (3, 4) with 2. So the answer is 6: the subarrays $a[0..2]$, $a[0..3]$, $a[1..1]$, $a[1..4]$, $a[2..4]$ and $a[3..3]$. A brute force agrees, and the trie solution agreed with a brute force on 7000 random arrays.

**Exercise 8.7.** The `if (node == 0) break;` line is missing. Store only the number 2: `countXorLessBuggy(0, 1)` returns 1, but $0 \oplus 2 = 2$ is not less than 1, so the answer is 0. For bits 30..2, $k_b = 0$ and the walk follows the zeros of 2. At bit 1, $k_b = 0$ and $x_b = 0$, but 2 has a 1 there, so child 0 is missing and `node` becomes 0, which is the index of the **root**. At bit 0, $k_b = 1$, and the walk adds the count of the root's child 0, which holds the whole trie. (Another example: store only 0 and query (0, 3); it returns 2 instead of 1.) Fix: put `if (node == 0) break;` back after the `if`/`else`. On 5000 random small tries the buggy version was wrong 1987 times and the fixed one never.

**Exercise 8.8.** (1) Binary trie with $B = 30$: insert everything, then query every number, about $2 \cdot 2 \cdot 10^5 \cdot 30 = 1.2 \cdot 10^7$ steps. There are at most $2 \cdot 10^5 \cdot 30 + 1 = 6\,000\,001$ nodes: 48 MB for the flat `next` array of *Theory Listing 8.5* (`maxXor` needs no counts), against about 170 MB for `int[nodes][2]`. Trap: sizing the arrays below $n \cdot B + 1$. (2) Per-bit counting (*Theory Listing 8.3*), $30 \cdot 10^6 = 3 \cdot 10^7$ steps and O(1) extra memory. Trap: the exact sum can reach about $(n^2/4)(2^{30} - 1) \approx 2.7 \cdot 10^{20}$, beyond `Long.MAX_VALUE` $\approx 9.2 \cdot 10^{18}$. `pairs` (at most $2.5 \cdot 10^{11}$) fits in a `long`, but

`pairs << b` does not, so add `pairs % MOD * ((1L << b) % MOD) % MOD` instead. (3) XOR basis: $10^{18} < 2^{60}$, so the 61 slots of *Theory Listing 8.6* suffice; about $10^5 \cdot 61 \approx 6 \cdot 10^6$ steps and 61 `long`s. Trap: read the values as `long`, and test bits with `(x >> i) & 1`, never `1 << i`. (4) Prefix XOR with a count array `int[1 << 20]` (4 MB) in place of the `HashMap` of *Theory Listing 8.2*: every prefix and `prefix ^ k` are below $2^{20}$, so they are valid indices. About $10^6$ steps. Trap: the count can reach about $5 \cdot 10^{11}$, so keep the total in a `long`. (5) The rank can be $20 > N/2 = 10$, so the basis may use too many elements. Brute force over all $2^{20}$ subsets with a size check (Chapter 9): about $20 \cdot 2^{20} \approx 2 \cdot 10^7$ steps. (6) The rank is at most $20 \le N/2 = 50$, so the basis maximum is the XOR of at most 20 original numbers (*Theory Claim 8.10*) and is the answer: about $100 \cdot 21$ steps.

**Exercise 8.9.** Compare with *Theory Listing 8.6* and *Theory Listing 8.7*. The two places where a `long` matters are the array `long[] b` (values below $2^{61}$) and the parameter `long x` with the bit test `(x >> i) & 1`; with an `int` shift, `1 << i` would wrap for $i \ge 32$.

## Chapter 9 · Brute Force, Stress Testing and the Mock Exam

**Exercise 9.1.** A brute force over all masks, as in *Theory Listing 9.1*:

```
static long[] balancedTeams(int[] s) {              // {count, size of the largest}; O(2^n * n)
    int n = s.length;
    long count = 0, largest = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        if (Integer.bitCount(mask) < 2) continue;
        int lo = Integer.MAX_VALUE, hi = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++)
            if ((mask & (1 << i)) != 0) { lo = Math.min(lo, s[i]); hi = Math.max(hi, s[i]); }
        if (hi <= 2L * lo) {
            count++;
            largest = Math.max(largest, Integer.bitCount(mask));
        }
    }
    return new long[]{count, largest};
}
```

For [3, 5, 8, 4, 10], sort to 3, 4, 5, 8, 10 and count the teams by their weakest member. With 3 weakest, the others must be at most 6, so they come from {4, 5}: $2^2 - 1 = 3$ teams. With 4 weakest: {5, 8}, 3 teams. With 5 weakest: {8, 10}, 3 teams. With 8 weakest: {10}, 1 team. With 10 weakest: none. That makes 10 balanced teams, and the largest has 3 players ({3, 4, 5}, {4, 5, 8} or {5, 8, 10}). The brute force returns {10, 3}. It costs $O(2^n \cdot n)$: $2^{16} \cdot 16 \approx 10^6$ steps here, and $2^{20} \cdot 20 \approx 2 \cdot 10^7$ for $n = 20$, so with $n \le 2 \cdot 10^5$ it would pass the tests with $n$ up to about 20. (The counting by weakest member, done with a sort and a pointer, gives an $O(n \log n)$ solution; it agreed with the brute force on 5000 random arrays.)

**Exercise 9.2.**

| mask | binary | subset | sum |
|---|---|---|---|
| 0 | 000 | {} | 0 |
| 1 | 001 | {4} | 4 |
| 2 | 010 | {7} | 7 |
| 3 | 011 | {4, 7} | 11 |
| 4 | 100 | {9} | 9 |
| 5 | 101 | {4, 9} | 13 |
| 6 | 110 | {7, 9} | 16 |
| 7 | 111 | {4, 7, 9} | 20 |

`bestSubsetSum(a, 15)` returns 13, from mask 5. The subset {0, 2, 3} is mask $2^0 + 2^2 + 2^3 = 13$, and $22 = 10110_2$ stands for {1, 2, 4}.

| input | i | j | after swap | result | returns |
|---|---|---|---|---|---|
| [1, 2, 2, 3] | 2 | 3 | [1, 2, 3, 2] | [1, 2, 3, 2] | true |
| [1, 3, 2, 2] | 0 | 3 | [2, 3, 2, 1] | [2, 1, 2, 3] | true |
| [2, 3, 2, 1] | 0 | 1 | [3, 2, 2, 1] | [3, 1, 2, 2] | true |
| [3, 2, 2, 1] | −1 | — | — | [1, 2, 2, 3] | false |

In the second row the rightmost element larger than 1 is the second 2, at index 3; in the third row the only element larger than 2 in the suffix is the 3. The loop visits 12 arrangements, from [1, 2, 2, 3] to [3, 2, 2, 1]. The two 2s are equal, so swapping them gives the same arrangement, and 4!/2! = 12 distinct arrangements remain.

**Exercise 9.3.**

| step | accepted change | input after it | fast | brute |
|---|---|---|---|---|
| 4 | halve a[0] | [−3, −9, −10] | 27 | 90 |
| 5 | halve a[0] | [−1, −9, −10] | 9 | 90 |
| 6 | halve a[0] | [0, −9, −10] | 0 | 90 |
| 7 | halve a[1] | [0, −4, −10] | 0 | 40 |
| 8 | halve a[1] | [0, −2, −10] | 0 | 20 |
| 9 | halve a[1] | [0, −1, −10] | 0 | 10 |
| 10 | halve a[2] | [0, −1, −5] | 0 | 5 |
| 11 | halve a[2] | [0, −1, −2] | 0 | 2 |
| 12 | halve a[2] | [0, −1, −1] | 0 | 1 |

On [0, −1, −1] no deletion and no halving keeps the disagreement, so the shrinker stops; running *Theory Listing 9.5* gives the same twelve steps. In step 1, deleting $a[0]$ gives [−9, −10, 10, 3, 4], where both methods return 10800, so the change is rejected (and likewise for $a[1]$ and $a[2]$). The final input shows the flaw: after the 0, `cur` is 0, and at the first −1 the scan keeps max(−1, 0) = 0 and throws away the product −1 ending there, which the next −1 would have turned into the answer 1. A negative product can become the largest one. The cause is a **wrong idea**. Repair: keep both the largest and the smallest product ending at $i$:

```
static long solveFixed(int[] a) {                 // largest and smallest product ending at i
    long best = a[0], hi = a[0], lo = a[0];
    for (int i = 1; i < a.length; i++) {
        long x = a[i];
        long h = Math.max(x, Math.max(hi * x, lo * x));
        lo = Math.min(x, Math.min(hi * x, lo * x));
        hi = h;
        best = Math.max(best, hi);
    }
    return best;
}
```

It agreed with `brute` on 100 000 random arrays from the same generator.

**Exercise 9.4.** (1) Overflow / mod: `lo + hi` exceeds $2^{31} - 1$. Rule: write `lo + (hi - lo) / 2`, or use `long` for every search bound. (2) Misread statement (output format). Rule: compare your output with the first sample character by character before submitting. (3) Stack overflow. Rule: any recursion that can be deeper than $10^4$ becomes a loop with an explicit stack. (4) Wrong idea: the earliest start can be a long meeting that blocks many short ones; the exchange argument works for the earliest *end* (Chapter 4). Rule: prove a greedy or stress-test it before submitting. (5) Input reading. Rule: after `nextInt()`, call `nextLine()` once to finish the line, or read every token with FastScanner. (6) Off-by-one: the search range must be half-open, $[0, n)$, so that "not found" can return $n$. Rule: lower-bound searches start with `hi = n`. (7) TLE / complexity: $10^4 \cdot 2 \cdot 10^5 = 2 \cdot 10^9$ cells allocated and zeroed.

Rule: allocate once at the largest size and clear only the first $n + 1$ entries in each test case. (8) Edge case: all values equal. Rule: before submitting, run $n = 1$, all values equal and the extreme values. (9) Ran out of time. Rule: submit the brute force as soon as it passes the samples, and by minute 150 every problem has a submission.

**Exercise 9.5.** (1) $O(n! \cdot n)$: $8 \cdot 8! = 322\,560$ steps; try all orders. (2) $O(2^n \cdot n)$: $18 \cdot 2^{18} \approx 4.7 \cdot 10^6$; all subsets or a bitmask DP. (3) $O(n^3)$: $400^3 = 6.4 \cdot 10^7$; triple loops or interval DP. (4) $O(n^2)$: $4000^2 = 1.6 \cdot 10^7$; all pairs or a 2-D DP. $O(n^3)$ would need $6.4 \cdot 10^{10}$ steps, hundreds of seconds, so no. (5) $O(n \log n)$ per test case, which sums to about $2 \cdot 10^5 \cdot 18 \approx 3.5 \cdot 10^6$ steps; clearing $2 \cdot 10^5$ entries in each of $10^4$ test cases costs $2 \cdot 10^9$ writes, a time-limit failure on its own. (6) $O(\sqrt{n}) = 10^6$ steps (trial division) or $O(\log n)$ (binary search, fast power). (7) $2.5 \cdot 10^7$ states: as `long` that is 200 MB, too close to 256 MB once the JVM's own memory is counted; as `int` 100 MB, which fits; if each row reads only the previous one, two rows suffice. An $O(n^2)$ brute force does about $10^8$ simple steps per second, so it finishes up to $n \approx 10^4$.

**Exercise 9.6.** Compare with *Theory Listing 9.4*. The line that makes failures reproducible is `Random rnd = new Random(1)`. A fixed seed produces the same inputs on every run, so you can rerun the failing case after a change.

## Chapter 10 · Number Theory for Programmers

**Exercise 10.1.** (1) `1 % m`; (2) `(exp & 1) == 1`; (3) `base = base * base % m`; (4) `exp >>= 1`; (5) `inverse(fact[n])`, that is `power(fact[n], MOD - 2, MOD)`; (6) `invFact[i] * i % MOD`. Line (6) uses $(i-1)!^{-1} = i \cdot (i!)^{-1}$, because $(i-1)! \cdot i \cdot (i!)^{-1} = i! \cdot (i!)^{-1} \equiv 1$ (*Theory Theorem 10.10*). It needs $n <$ MOD so that $n!$ has an inverse.

**Exercise 10.2.**

| iteration | a | b | q | r |
|---|---|---|---|---|
| 1 | 2024 | 1717 | 1 | 307 |
| 2 | 1717 | 307 | 5 | 182 |
| 3 | 307 | 182 | 1 | 125 |
| 4 | 182 | 125 | 1 | 57 |
| 5 | 125 | 57 | 2 | 11 |
| 6 | 57 | 11 | 5 | 2 |
| 7 | 11 | 2 | 5 | 1 |
| 8 | 2 | 1 | 2 | 0 |

After the 8th iteration $b = 0$ and $a = 1$, so gcd(2024, 1717) = 1: the numbers are coprime ($2024 = 2^3 \cdot 11 \cdot 23$ and $1717 = 17 \cdot 101$). Eight iterations. lcm = 2024 / 1 · 1717 = 3 475 208.

**Exercise 10.3.** For the first blank row, the row above gives $x' = -5$ and $y' = 26$, so $x = y' = 26$ and $y = x' - q \cdot y' = -5 - 2 \cdot 26 = -57$. The remaining rows:

| call | q | x | y | check |
|---|---|---|---|---|
| (125, 57) | 2 | 26 | −57 | 125 · 26 − 57 · 57 = 1 |
| (182, 125) | 1 | −57 | 83 | −10 374 + 10 375 = 1 |
| (307, 182) | 1 | 83 | −140 | 25 481 − 25 480 = 1 |
| (1717, 307) | 5 | −140 | 783 | −240 380 + 240 381 = 1 |
| (2024, 1717) | 1 | 783 | −923 | 1 584 792 − 1 584 791 = 1 |

So $1717 \cdot (-923) \equiv 1 \pmod{2024}$, and the inverse is −923 + 2024 = 1101. `modInverse(1717, 2024)` returns 1101.

**Exercise 10.4.** $45 = 101101_2$, so the loop runs 6 times.

| exp at start | bit | r after | base after | exp after |
|---|---|---|---|---|
| 45 | 1 | 7 | 49 | 22 |
| 22 | 0 | 7 | 401 | 11 |
| 11 | 1 | 807 | 801 | 5 |
| 5 | 1 | 407 | 601 | 2 |
| 2 | 0 | 407 | 201 | 1 |
| 1 | 1 | 807 | 401 | 0 |

$7^{45} \bmod 1000 = 807$ (Java's `BigInteger.modPow` agrees). The naive loop uses 44 multiplications; fast power uses 10 (6 squarings and 4 multiplications into $r$).

**Exercise 10.5.** Every even $x$ has spf 2. The odd composites are 9 → 3, 15 → 3, 21 → 3, 25 → 5, 27 → 3, 33 → 3, 35 → 5 and 39 → 3. The primes (spf[$x$] = $x$) are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31 and 37.

756 (spf 2) → 378 (spf 2) → 189 (spf 3) → 63 (spf 3) → 21 (spf 3) → 7 (spf 7) → 1, so $756 = 2^2 \cdot 3^3 \cdot 7$, with $(2 + 1)(3 + 1)(1 + 1) = 24$ divisors and kernel $3 \cdot 7 = 21$. $84 = 2^2 \cdot 3 \cdot 7$, with 12 divisors and kernel 21. The kernels are equal, so $756 \cdot 84$ is a square: $63\,504 = 252^2$. The kernel of $1176 = 2^3 \cdot 3 \cdot 7^2$ is $2 \cdot 3 = 6 \neq 21$, so $756 \cdot 1176$ is not a square.

**Exercise 10.6.** (1) False: $6 / 2 = 3 \equiv 3 \pmod 4$, but $(6 \bmod 4) / (2 \bmod 4) = 2 / 2 = 1$. Divide by multiplying with an inverse. (2) False: for $a = 3$, $3^{9998} \bmod 10\,000 = 8889$ and $3 \cdot 8889 \equiv 6667$; Fermat needs a prime modulus. (3) True: with $b = 0$ the loop does not run and returns $|a|$; gcd(0, 12) runs one iteration and returns 12. (4) True: at most $2\,000\,000\,012 < 2\,147\,483\,647$. (5) False: 12 and 18 both have prime set {2, 3}, but $12 \cdot 18 = 216$ is not a square; compare kernels (3 and 2). (6) False: the inner loop runs only for primes, and $\Sigma\, 1/p$ grows like ln ln $n$, so the cost is $O(n \log \log n)$. (7) True: $10^7 \cdot 4$ bytes = 40 MB.

**Exercise 10.7.** Key idea: $a[i] \cdot a[j]$ is a square exactly when $\ker(a[i]) = \ker(a[j])$, so count values by kernel. Build an SPF sieve to $10^7$ (40 MB, $O(V \log \log V)$), compute each kernel in $O(\log V)$, count kernels in a `HashMap<Integer, Integer>` (or a second `int[]` if memory allows), and answer $\Sigma\, c(c - 1)/2$ over the counts $c$, in `long`. Total $O(V \log \log V + n \log V)$. For the example the kernels are 2, 2, 3, 3, 2, 5, 2, 3, 1, 1: kernel 2 occurs 4 times (6 pairs), kernel 3 three times (3 pairs), kernel 1 twice (1 pair), kernel 5 once. The answer is 10, which matches a brute force over all 45 pairs.

**Exercise 10.8.** `r * base` and `base * base` are computed in `int` and overflow as soon as a product exceeds 2 147 483 647. With exponent 2 the first failing base is 46 341 ($46\,341^2 = 2\,147\,488\,281$). For example `powerBuggy(100000, 2, 1_000_000_007)` returns 410 065 401, while the correct value $10^{10} \bmod (10^9 + 7)$ is 999 999 937. Fix: declare `r` and `base` as `long` (as in *Theory Listing 10.3*) and cast the result, or write `r = (int) ((long) r * base % m)` and `base = (int) ((long) base * base % m)`.

**Exercise 10.9.** (1) Factorials and inverse factorials up to $10^6$ (*Theory Listing 10.5*): $O(10^6)$ once, O(1) per query, two `long` arrays of 8 MB each. Trap: reduce after each of the two multiplications in `nCr`. (2) Pascal's triangle (*Theory Listing 10.6*): about $5 \cdot 10^5$ additions and $1001^2$ entries, 8 MB as `long` (4 MB as `int`). Trap: Fermat inverses do not exist modulo 10 000. (3) Boolean sieve to $10^7$: about $2.3 \cdot 10^7$ steps and 10 MB, then O(1) per number. Trap: 0 and 1 are not prime. (4) Trial division up to $\sqrt{(10^{12})} = 10^6$ iterations. Trap: the cofactor left at the end is a prime factor too. (5) Fast power: 60 iterations. Trap: reduce the base first and keep products in `long`. (6) Divide out the 168 primes below 1000, then test the cofactor for being a square (*Theory Listing 10.12*): at most 168 divisions per value, under $2 \cdot 10^7$ in total. Trap: use an exact integer square root.

## Chapter 11 · Dynamic Programming I: States, Transitions and Knapsacks

**Exercise 11.1.** (1) `int c = cap; c >= w[i]; c--`; (2) `Math.max(dp[c], dp[c - w[i]] + v[i])`; (3) `Integer.MAX_VALUE / 2`; (4) `dp[0] = 0`; (5) `Math.min(dp[a], dp[a - c] + 1)`; (6) `dp[amount] >= INF ? -1 : dp[amount]`. For the unbounded knapsack, blank (1) becomes `int c = w[i]; c <= cap; c++`. Going down, dp[$c - w[i]$] has not yet been updated in this pass, so it does not contain item $i$ (*Theory Theorem 11.5*); going up, it may already contain item $i$, which can then be taken again (*Theory Theorem 11.6*).

**Exercise 11.2.**

| dp[c] after | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| item (2, 4) | 0 | 0 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
| item (4, 4) | 0 | 0 | 4 | 4 | 4 | 4 | 8 | 8 | 8 | 8 |
| item (3, 7) | 0 | 0 | 4 | 7 | 7 | 11 | 11 | 11 | 11 | 15 |
| item (5, 9) | 0 | 0 | 4 | 7 | 7 | 11 | 11 | 13 | 16 | 16 |

The answer is dp[9] = 16, from the items (3, 7) and (5, 9), of total weight 8: the best set need not fill the bag. In the last pass dp[9] = max(15, old dp[4] + 9) = max(15, 7 + 9) = 16. A brute force over the 16 subsets agrees, and this is the only optimal set. The ratio greedy takes (3, 7) and (2, 4), cannot fit (5, 9) in the remaining 4, and adds (4, 4): weight 9, value 15. With the loop running upwards the result is the unbounded optimum, dp[9] = 21, which "uses" (3, 7) three times.

**Exercise 11.3.**

| amount | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| combinations after coin 4 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 3 | 3 |
| combinations after coin 5 | 1 | 1 | 1 | 1 | 2 | 3 | 3 | 3 | 4 | 5 |
| ordered sequences | 1 | 1 | 1 | 1 | 2 | 4 | 6 | 8 | 11 | 17 |
| fewest coins | 0 | 1 | 2 | 3 | 1 | 1 | 2 | 3 | 2 | 2 |

The five combinations for 9 are {4, 5}, {4, 4, 1}, {5, 1, 1, 1, 1}, {4, 1, 1, 1, 1, 1} and nine 1s. They can be ordered in 2!/(1! 1!) = 2, 3!/(2! 1!) = 3, 5!/(1! 4!) = 5, 6!/(1! 5!) = 6 and 1 ways, and 2 + 3 + 5 + 6 + 1 = 17. For amount 8, "largest coin first" takes 5 + 1 + 1 + 1, four coins, while the table gives 2 (4 + 4).

**Exercise 11.4.**

| after | reachable sums |
|---|---|
| 3 | 0, 3, 5, 8, 9, 12, 14, 17 |
| 12 | 0, 3, 5, 8, 9, 12, 14, 15, 17, 20, 21, 24, 26, 29 |
| 6 | 0, 3, 5, 6, 8, 9, 11, 12, 14, 15, 17, 18, 20, 21, 23, 24, 26, 27, 29, 30, 32, 35 |

The total is 35, which is odd, so no equal split exists. The largest reachable sum not above 17 is 17 itself, so the smallest difference is $35 - 2 \cdot 17 = 1$, for example {5, 9, 3} (17) against {12, 6} (18). Exactly two subsets have sum 17, {5, 9, 3} and {5, 12}, and `countSubsets` returns 2. A brute force over all 32 subsets agrees on every number. Note that the last line is symmetric: $s$ is reachable exactly when $35 - s$ is, because the complement of a subset is a subset.

**Exercise 11.5.** (1) True: a memo saves work only when the same state is asked for a second time; merge sort is the standard example. (2) True: that is *Theory Theorem 11.6*. (3) False: a minimum over all orders equals the minimum over all multisets; the swapped version agreed with `minCoins` on 3000 random cases. (4) False: it returns 4 ({5}, {2, 2, 1}, {2, 1, 1, 1}, {1, 1, 1, 1, 1}); 9 is the number of ordered sequences. (5) False: the recursion is one call deep per house, and in the theory chapter's measurement it overflowed between about $10^4$ and $1.7 \cdot 10^4$ houses; use the loop or rolling variables. (6) False: it is pseudo-polynomial. $C = 10^9$ takes ten digits to write but $10^9$ table entries. (7) True: each zero doubles the count, giving {3}, {0, 3}, {0′, 3} and {0, 0′, 3}. (8) True: $101 \cdot 100\,001 \cdot 8$ bytes ≈ 80.8 MB.

**Exercise 11.6.** State: dp[$c$] = the largest usefulness of a set of the items seen so far whose prices add up to **exactly** $c$, or $-\infty$ if there is none. Base: dp[0] = 0 and dp[$c$] = $-\infty$ for $c \geq 1$. Transition, for each item, with $c$ running downwards from $W$ to $w[i]$: if dp[$c - w[i]$] $\neq -\infty$, then dp[$c$] = max(dp[$c$], dp[$c - w[i]$] + $v[i]$). Answer: dp[$W$], or −1 if it is $-\infty$. Compared with `knapsack01`, only the base changes, plus the test that skips $-\infty$: starting from all zeros lets every

entry pretend that the unused budget was spent, which is the "at most" problem. Represent −∞ by a sentinel such as `Long.MIN_VALUE / 4` and skip it, so that adding $v$ can never overflow. Time $O(nW) = 10^7$ steps, memory $O(W)$.

| c | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| dp[c] at the end | 0 | −∞ | 1 | 2 | 6 | 3 | 7 | 8 | 11 | 9 |

The answer is 9, from (3, 2) + (6, 7) or from (4, 6) + (3, 2) + (2, 1). Under "at most 9" the answer would be 11, from the two items of price 4 (total 8). The exact version agreed with a brute force over all subsets on 5000 random cases.

**Exercise 11.7.** The loop starts at $c = w[i-1]$, so the entries best[$i$][$c$] with $c < w[i-1]$ are never written and stay 0, instead of copying best[$i-1$][$c$]. A later, lighter item reads one of them. Items (1, 1), (2, 1), (1, 3) with capacity 2: the method returns 3, but items 0 and 2 together weigh 2 and are worth 4. Row 2 leaves best[2][1] = 0 instead of 1, and row 3 computes best[3][2] = max(best[2][2], best[2][1] + 3) = max(1, 3) = 3. Fix: run $c$ from 0 and copy first, as in *Theory Listing 11.4*: `best[i][c] = best[i - 1][c]; if (w[i - 1] <= c) best[i][c] = Math.max(best[i][c], best[i - 1][c - w[i - 1]] + v[i - 1]);`. The one-array version may start at $w[i]$ because there an untouched entry already holds the previous row's value. On 5000 random inputs the buggy version was wrong 1079 times and the fixed one never.

**Exercise 11.8.** (1) `knapsack01` with one array: $100 \cdot 10^5 = 10^7$ steps and 0.8 MB. Trap: the values add up to $10^{11}$, so `dp` must be `long`. (2) Swap the roles: dp[$x$] = the least weight that reaches value exactly $x$, for $x \le 100 \cdot 1000 = 10^5$, again $10^7$ steps; the answer is the largest $x$ with dp[$x$] ≤ capacity. Trap: weights add up to $10^{11}$, so use `long` and a large INF. (3) `countCombinations` with the coins in the outer loop and `% MOD` after each addition: $100 \cdot 10^4 = 10^6$ steps. Trap: with the loops the other way round it counts sequences. (4) The 64-bit version *Theory Listing 11.10*: $200 \cdot 15\,626 \approx 3.1 \cdot 10^6$ word operations, against $2 \cdot 10^8$ for the boolean loop (about 7 ms against 230 ms in the theory chapter's test). Trap: `>>>`, not `>>`. (5) Rolling variables (*Theory Listing 11.3*): $10^6$ steps and O(1) memory. Trap: the top-down version overflows the stack. (6) Brute force over all $2^{20}$ subsets (Chapter 9): about $20 \cdot 2^{20} \approx 2 \cdot 10^7$ steps; a table over capacities up to $10^{15}$ is impossible. Trap: sums up to $2 \cdot 10^{16}$ need `long`.

**Exercise 11.9.** Compare with *Theory Listing 11.5* and *Theory Listing 11.7*, and with the recipe in *Theory Section 11.2*. The two loops that matter are the capacity loop of `knapsack01` (downwards) against `knapsackUnbounded` (upwards), and the loop order of `countCombinations` (coins outside).

## Chapter 12 · Dynamic Programming II: Two Strings, Edit Distance and Palindromes

**Exercise 12.1.** (1) `i > 0 && j > 0`; (2) `s.charAt(i - 1) == t.charAt(j - 1)`; (3) `i--; j--;` (both); (4) `dp[i - 1][j] >= dp[i][j - 1]`; (5) `sb.reverse().toString()`. When $i = 0$ or $j = 0$, one prefix is empty and dp[$i$][$j$] = 0, so no further character can be common; by *Theory Claim 12.3* the collected characters already form an LCS.

**Exercise 12.2.**

| | "" | p | o | t | a | t | o |
|---|---|---|---|---|---|---|---|
| "" | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| t | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| e | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| a | 0 | 0 | 0 | 1 | 2 | 2 | 2 |
| p | 0 | 1 | 1 | 1 | 2 | 2 | 2 |
| o | 0 | 1 | 2 | 2 | 2 | 2 | 3 |
| t | 0 | 1 | 2 | 3 | 3 | 3 | 3 |

The LCS length is 3. The walk visits (6,6) up, (5,6) match `o`, (4,5) up, (3,5) left, (3,4) match `a`, (2,3) up, (1,3) match `t`, and prints `tao`. The other longest common subsequences are `pot` and `tat`. At (6,6) both neighbours hold 3.

Going left there and keeping the rule afterwards gives `tat`; breaking every tie to the left (at (6,6) and again at (5,4), where both neighbours hold 2) gives `pot`, which uses positions 3, 4, 5 of `teapot` and 0, 1, 2 of `potato`.

**Exercise 12.3.**

| | `""` | l | a | d | d | e | r |
|---|---|---|---|---|---|---|---|
| a | 3 | 3 | 2 | 3 | 4 | 5 | 6 |
| d | 4 | 4 | 3 | 2 | 3 | 4 | 5 |
| l | 5 | 4 | 4 | 3 | 3 | 4 | 5 |
| e | 6 | 5 | 5 | 4 | 4 | 3 | 4 |

The edit distance is 4. One walk back (trying match, then replace, then delete, then insert) gives, from the corner: insert `r`, match `e`, replace `l` by `d`, match `d`, match `a`, replace `r` by `l`, delete `c`. Applied from left to right: `cradle` → `radle` (delete c) → `ladle` (replace r by l) → `ladde` (replace the second l by d) → `ladder` (insert r). The two-row version also returns 4.

**Exercise 12.4.**

| l \ r | 0 c | 1 b | 2 b | 3 d | 4 c | 5 b |
|---|---|---|---|---|---|---|
| 0 c | 1 | 1 | 2 | 2 | 4 | 4 |
| 1 b | — | 1 | 2 | 2 | 2 | 3 |
| 2 b | — | — | 1 | 1 | 1 | 3 |
| 3 d | — | — | — | 1 | 1 | 1 |
| 4 c | — | — | — | — | 1 | 1 |
| 5 b | — | — | — | — | — | 1 |

LPS = 4, and the only longest palindromic subsequence is `cbbc` (a brute force over all 64 subsequences agrees; so does LCS(`cbbdcb`, `bcdbbc`) = 4). The corner is max(dp[1][5], dp[0][4]) = max(3, 4) = 4 because the ends `c` and `b` differ; dp[0][4] = dp[1][3] + 2 = 2 + 2 because its ends are both `c`. The fewest insertions are 6 − 4 = 2. Following the proof of *Theory Theorem 12.10*: drop the last `b` and add a copy in front, make `cbbdc` a palindrome by keeping the ends `c` and turning `bbd` into `dbbd`. The result is `bcdbbdcb`, a palindrome of length 8 that contains `cbbdcb`.

**Exercise 12.5.** (1) False: for `ab` and `ba`, dp[2][2] = 1 but dp[1][1] = 0. (2) True: each edit is undone by one edit (insert and delete swap, a replacement is reversed), so a script from $s$ to $t$ reversed is a script from $t$ to $s$ of the same length. (3) True: replace the first $\min(n, m)$ characters where they differ and insert or delete the remaining $|n - m|$; at most $\max(n, m)$ edits. A test on 5000 random pairs confirmed $|n - m| \le \text{distance} \le \max(n, m)$. (4) False: the walk back needs every row, and only the last two survive. (5) False: for `bacba`, `acb` is a longest common subsequence of the string and its reverse `abcab`. (6) False: dp[0][$r$] reads dp[1][$r$] before row 1 is filled; on `badcab` it returns 2 instead of 5. (7) True: both are $n - \text{LPS}(s)$.

**Exercise 12.6.** The best common substring of two prefixes may end anywhere, so knowing its length does not tell you whether it can be extended by the next pair of equal characters; the state must say where the substring **ends**. State: e[$i$][$j$] = length of the longest common suffix of $s[0..i-1]$ and $t[0..j-1]$, that is, of the longest common substring ending exactly at $s[i-1]$ and $t[j-1]$. Transition: e[$i$][$j$] = e[$i-1$][$j-1$] + 1 if $s[i-1] = t[j-1]$, else 0. Base: row 0 and column 0 are 0. Order: row by row. Answer: the **maximum over all entries**, not e[$n$][$m$]. Time $O(nm) = 2.5 \cdot 10^7$ for 5000 × 5000; memory two rows of `int`, about 40 KB. For `xabcdy` and `abcxabd` the table reaches 3 twice: at (3, 6) for `xab` (positions 0..2 of $s$, 3..5 of $t$) and at (4, 3) for `abc`. So the answer is 3, while the LCS is 4 (for example `abcd` or `xabd`): a subsequence may skip characters, a substring may not.

**Exercise 12.7.** The shortest failing input is $s$ = `aa`, $t$ = `a`: it returns 2 instead of 1. After row 1, `prev = cur;` makes both names refer to the same array, [1, 0]. In row 2, `cur[0] = 2` also overwrites `prev[0]`, which the diagonal still needs, so the match gives `cur[1] = prev[0]` = 2. In general, from row 2 on, `prev[j - 1]` has already been over-

written by the current row's `cur[j - 1]` when it is read. With one character in *s* only one pass runs, so nothing is read after the aliasing, which is why short tests pass. On 4000 random pairs of length up to 6 it was wrong 2188 times; for example `abc` against `abc` gives 2 instead of 0. Fix: swap the references, `int[] tmp = prev; prev = cur; cur = tmp;`.

**Exercise 12.8.** (1) *Theory Listing 12.1* with two rows: $2.5 \cdot 10^7$ steps (about 0.15 s measured), 40,008 bytes; the full `int` table would also fit (100,040,004 bytes) but uses most of the limit for nothing. (2) The walk needs the whole table: a `short` table (50,020,002 bytes, about 50 MB) or one direction byte per cell (25,010,001 bytes) next to two rows. (3) $10^{10}$ steps, about 60 s even at the measured speed, so no O(*nm*) DP can work. If both strings are permutations of 1..*n*, replace each value of *t* by its position in *s*; a common subsequence is then an increasing subsequence of those positions, and the LIS methods of Chapter 13 give the answer in O($n \log n$). (4) Put the short string across the top: $100{,}001 \times 11 \approx 1.1 \cdot 10^6$ cells, and even the full table is 4,400,044 bytes. (5) The interval table: 500,500 entries in use, a $1001 \times 1001$ `int` array of 4,008,004 bytes; or LCS with the reverse, $10^6$ steps. (6) *Theory Listing 12.5* with every entry reduced modulo $10^9 + 7$, adding the modulus back after the subtraction; $10^6$ steps, and the `long` table is 8,016,008 bytes (two rows also work).

**Exercise 12.9.** Compare with *Theory Listing 12.4*. The lines that matter are `cur[0] = i;` at the start of each row, the `prev[j - 1]` in both branches, and the three-line swap at the end of the row.

## Chapter 13 · Dynamic Programming III: Increasing Subsequences and Extra Dimensions

**Exercise 13.1.** (1) `len`; (2) `>=`; (3) `tails[lo] = x`; (4) `if (lo == len) len++`. For the non-decreasing version blank (2) becomes `>`: equal values may follow each other, so *x* must go after every tail equal to it, which is the first tail strictly greater than *x* (the upper bound).

**Exercise 13.2.**

| x | position p | p + 1 | tails after |
|---|---|---|---|
| 4 | 0 | 1 | [4] |
| 10 | 1 | 2 | [4, 10] |
| 4 | 0 | 1 | [4, 10] |
| 3 | 0 | 1 | [3, 10] |
| 8 | 1 | 2 | [3, 8] |
| 9 | 2 | 3 | [3, 8, 9] |
| 1 | 0 | 1 | [1, 8, 9] |
| 5 | 1 | 2 | [1, 5, 9] |

The LIS length is 3; *Theory Listing 13.3* returns 3, 8, 9 (4, 8, 9 also works). The final `tails` = [1, 5, 9] is not a subsequence: 1 and 5 come after 9 in *a*. The second 4 lands on the existing 4 and changes nothing, which is the strict version at work. The non-decreasing version on the same array gives length 4 (for example 4, 4, 8, 9).

**Exercise 13.3.** up = [1, 2, 2, 3, 3, 1, 4, 2] and down = [2, 3, 2, 3, 2, 1, 2, 1]. A peak is allowed at indices 1, 2, 3, 4 and 6, with up + down − 1 = 4, 3, 5, 4, 5. The longest mountain has length 5, so the fewest removals are 8 − 5 = 3. Two examples: 2, 5, 6, 4, 1 (peak 6 at index 3) and 2, 3, 6, 7, 2 (peak 7 at index 6). Indices 0, 5 and 7 cannot be peaks because one side has length 1.

**Exercise 13.4.** Ranks [3, 1, 5, 1, 2, 4], $V = 5$.

| a[i] | rank | prefixMax(rank − 1) | cur | fw[1..V] after |
|---|---|---|---|---|
| 30 | 3 | 0 | 1 | [0, 0, 1, 1, 0] |
| 10 | 1 | 0 | 1 | [1, 1, 1, 1, 0] |
| 50 | 5 | 1 | 2 | [1, 1, 1, 1, 2] |

| a[i] | rank | prefixMax(rank − 1) | cur | fw[1..V] after |
|---|---|---|---|---|
| 10 | 1 | 0 | 1 | [1, 1, 1, 1, 2] |
| 20 | 2 | 1 | 2 | [1, 2, 1, 2, 2] |
| 40 | 4 | 2 | 3 | [1, 2, 1, 3, 2] |

The answer is 3 (10, 20, 40). The second 10 has rank 1, and its query covers ranks 1..0, an empty range, so it cannot extend the first 10: in a strictly increasing subsequence equal values may not follow each other. Note how raising rank 3 also wrote node 4, and raising rank 1 wrote nodes 1, 2 and 4.

**Exercise 13.5.** (1) False: for [3, 4, 1] it ends as [1, 4], which is not a subsequence. (2) True: equal values may follow each other, so $x$ goes after all tails equal to it. (3) False: the peak may not be the first element of the mountain, so up[$i$] ≥ 2 is required. (4) False: a node stores a maximum and cannot recover the second-largest value after the largest is lowered. (5) True: then two envelopes of equal width can never both be in a strictly increasing run of heights. (6) True: every state of the plain DP is split into $K$ states, and each transition is done once per remainder.

**Exercise 13.6.** Sort by $w$ ascending and, for equal $w$, by $h$ descending, so that two blocks of equal width can never both be chosen. Compress the heights to ranks. The Fenwick tree is indexed by height rank and stores, at each node, the largest tower value ending with a height in its range (a `long` sum, not a length). For each block, query the prefix maximum over ranks below its own (strict), add $v$, and raise its rank. Time O($n$ log $n$), memory O($n$). For the six blocks the answer is 20, for example (1, 3, 4) + (2, 4, 3) + (3, 5, 6) + (4, 6, 7) = 4 + 3 + 6 + 7 = 20; a brute force over all subsets agrees.

**Exercise 13.7.** The query starts at `x` instead of `x - 1`, so it includes elements with the **same** rank and builds a non-decreasing subsequence. For ranks [1, 1] (two equal values) it returns 2 instead of 1. Fix: `for (int i = x - 1; i > 0; i -= i & -i)`.

**Exercise 13.8.** (1) The quadratic DP of *Theory Listing 13.1* with "$a[i]$ % $a[j]$ == 0" in place of "<": about $3000^2/2 = 4.5 \cdot 10^6$ pairs. (2) Patience sorting, *Theory Listing 13.2*: about $2 \cdot 10^5 \cdot 18 \approx 3.5 \cdot 10^6$ comparisons. (3) Compression plus a Fenwick tree of maximum sums (`long`), *Theory Listing 13.7*: O($n$ log $n$). (4) State (last value, XOR so far); since values are below 1024, there are 1024 XOR states, each with a Fenwick tree over values: about $1000 \cdot 1024 \cdot 10 \approx 10^7$ steps for the queries and as many for the updates, about $2 \cdot 10^7$ in all. (5) Brute force over all $2^{20} \approx 10^6$ subsets, checking each in O($n$): about $2 \cdot 10^7$ steps.

**Exercise 13.9.** Compare with *Theory Listing 13.4*. The two places that matter are the comparison `strict ? tails[mid] >= x : tails[mid] > x` and the line `end[i] = lo + 1` after the update.

## Chapter 14 · Dynamic Programming IV: Divisors, Intervals and Games

**Exercise 14.1.** (1) `2`; (2) `l + len - 1`; (3) `Long.MAX_VALUE`; (4) `k < r`; (5) `dp[k + 1][r]`. The recurrence reads dp[$l$][$k$] and dp[$k$ + 1][$r$], two shorter segments inside [$l$, $r$]. By *Theory Lemma 14.6* they are ready when every shorter segment has been filled first. With "$l$ from 0 up" as the outer loop, dp[$k$ + 1][$r$] lies in a row that is filled later, so it is still 0 when it is read. Without the cast the product is computed in `int`: $2000 \cdot 2000 \cdot 2000 = 8 \cdot 10^9$ wraps around to −589 934 592, and the method returns that negative number. With the cast it returns 8 000 000 000.

**Exercise 14.2.** The positions 0 to 15 are L L W W L W W L L W W L W W L L. The losing positions are exactly those with $s$ mod 7 equal to 0, 1 or 4, which a program confirmed up to 1000. Positions 0 and 1 have no move. From a losing position both moves lead to a winning one: from 7, the moves reach 5 and 2, both W. The pattern must repeat: each entry depends only on the five entries before it, and entries 7–11 (L L W W L) equal entries 0–4, so by induction every entry equals the one 7 places earlier. $100 = 14 \cdot 7 + 2$ is winning: remove 2 and leave $98 = 14 \cdot 7$, a losing position.

| D(l, r) | r = 0 | r = 1 | r = 2 | r = 3 | r = 4 |
|---|---|---|---|---|---|
| l = 0 | 4 | 4 | 0 | 1 | 2 |
| l = 1 | — | 8 | 4 | 5 | 6 |

| D(l, r) | r = 0 | r = 1 | r = 2 | r = 3 | r = 4 |
|---|---|---|---|---|---|
| l = 2 | — | — | 4 | 3 | 2 |
| l = 3 | — | — | — | 1 | 2 |
| l = 4 | — | — | — | — | 3 |

At the corner, taking the 4 gives $4 - D(1, 4) = 4 - 6 = -2$, and taking the 3 gives $3 - D(0, 3) = 3 - 1 = 2$. So the first player takes the **smaller** end, the 3. The sum is 20 and $D = 2$, so the totals are $(20 + 2)/2 = 11$ and 9. One optimal line: 3, then the opponent takes 1 (since $1 - D(0, 2) = 1$ is better than $4 - D(1, 3) = -1$), then 4, 8, 4. If both players always take the larger end, the moves are 4, 8, 4, 3, 1 and the first player loses 9 to 11. An exhaustive minimax search agrees with the table; on 4000 random rows the table, the rolling version and the exhaustive search all agreed.

**Exercise 14.3.**

| piece | i, j | candidates dp[i][k] + dp[k][j] | length | dp[i][j] |
|---|---|---|---|---|
| [0, 7] | 0, 3 | k = 1: 0 + 5; k = 2: 3 + 0 | 7 | 10 |
| [2, 9] | 1, 4 | k = 2: 0 + 6; k = 3: 5 + 0 | 7 | 12 |
| [3, 11] | 2, 5 | k = 3: 0 + 4; k = 4: 6 + 0 | 8 | 12 |
| [0, 9] | 0, 4 | k = 1: 0 + 12; k = 2: 3 + 6; k = 3: 10 + 0 | 9 | 18 |
| [2, 11] | 1, 5 | k = 2: 0 + 12; k = 3: 5 + 4; k = 4: 12 + 0 | 9 | 18 |
| [0, 11] | 0, 5 | k = 1: 0 + 18; k = 2: 3 + 12; k = 3: 10 + 4; k = 4: 18 + 0 | 11 | 25 |

The least total cost is 25. The best candidate at the top is $k = 3$, so the first cut is at $p[3] = 7$. Following the best $k$ in each piece gives the order 7, 3, 2, 9, costing $11 + 7 + 3 + 4 = 25$ (the orders 7, 3, 9, 2 and 7, 9, 3, 2 also cost 25). Cutting from left to right costs $11 + 9 + 8 + 4 = 32$. A search over all 24 orders confirms the minimum 25 (the worst order costs 34), and the DP matched that search on 3000 random sticks.

**Exercise 14.4.**

| v | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | sum |
|---|---|---|---|---|---|---|---|---|---|
| cnt(2, v) | 0 | 1 | 1 | 2 | 1 | 3 | 1 | 3 | 12 |
| cnt(3, v) | 0 | 0 | 0 | 1 | 0 | 2 | 0 | 3 | 6 |
| cnt(4, v) | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |

For example cnt(3, 8) = 3 counts (1, 2, 8), (1, 4, 8) and (2, 4, 8), and cnt(4, 8) = 1 counts (1, 2, 4, 8). For $K = 4$ the contributions are $8 \cdot C(3, 0) + 12 \cdot C(3, 1) + 6 \cdot C(3, 2) + 1 \cdot C(3, 3) = 8 + 36 + 18 + 1 =$ **63**. For $K = 3$: $8 \cdot 1 + 12 \cdot 2 + 6 \cdot 1 + 1 \cdot 0 =$ **38**. A brute force over all $8^4 = 4096$ and $8^3 = 512$ arrays gives the same two numbers. For $K = 100$ you need C(99, 0), C(99, 1), C(99, 2) and C(99, 3), which are 1, 99, 4851 and 156 849. The total is $8 + 1188 + 29\,106 + 156\,849 = 187\,151$, so the answer is 7151; the direct DP of *Theory Section 14.3*, run modulo 10000, also gives 7151. $10000 = 2^4 \cdot 5^4$ is not prime, so numbers such as 2 and 5 have no inverse modulo 10000; worse, $n! \equiv 0 \pmod{10000}$ for every $n \geq 20$, so a factorial formula would divide 0 by 0. Pascal's rule only adds, so it works for any modulus.

**Exercise 14.5.** (1) False: $\Sigma \lfloor N/v \rfloor = 13\,970\,034 \approx 1.4 \cdot 10^7$ for $N = 10^6$ (*Theory Table 14.1*). (2) True: each step at least doubles the value, so $k$ values need $2^{k-1} \leq 10^9$, that is $k \leq 30$; the chain 1, 2, 4, ..., $2^{29} = 536\,870\,912$ has exactly 30. (3) False: 10000 is not prime, so the inverses do not all exist; use Pascal's rule. (4) True: dp[$i$][$j$] reads dp[$i$][$k$] (same $i$, smaller $j$, filled earlier in this pass) and dp[$k$][$j$] (larger $i$, an earlier pass), as in *Theory Lemma 14.6*. (5) False: in Exercise 14.2 the first player must take the 3, the smaller end; taking the 4 loses by 2. (6) True: the two totals add up to the fixed sum, so "own − other" is 2 · own − sum, and both goals pick the same moves (*Theory Theorem 14.10*). (7) False: on [1, 5, 2] the difference is −2, so the first player gets 3 of 8. The colouring argument needs an even number of elements. (8) True: 22 mod 7 = 1. The two moves reach 20 and 17, which are both winning (20 → 15 and 17 → 15, with 15 losing).

**Exercise 14.6.** dp[$l$][$r$] = the least cost of merging piles $l$..$r$ into a single pile; dp[$l$][$l$] = 0. Split on the **last** merge: it joins a pile made from $l$..$k$ with a pile made from $k$ + 1..$r$, for some $l \le k < r$. Its cost is sum($l$..$r$) whatever $k$ is, because the final pile always contains all the stones; before it, the two sides were merged independently. So dp[$l$][$r$] = sum($l$..$r$) + min over $k$ of (dp[$l$][$k$] + dp[$k$ + 1][$r$]), with the sum taken from a prefix-sum array (Chapter 2). Fill by increasing length. Time O($n^3$): $(n+1)n(n-1)/6$ = 10 666 600 triples for $n$ = 400. Memory: a `long[400][400]`, 1.28 MB. Use `long`: the total is at most 399 merges of at most $4 \cdot 10^8$ stones, about $1.6 \cdot 10^{11}$, which does not fit an `int`. For [7, 5, 4, 7] the prefix sums are 0, 7, 12, 16, 23. Then dp[0][1] = 12, dp[1][2] = 9, dp[2][3] = 11; dp[0][2] = 16 + min(0 + 9, 12 + 0) = 25; dp[1][3] = 16 + min(0 + 11, 9 + 0) = 25; dp[0][3] = 23 + min(0 + 25, 12 + 11, 25 + 0) = **46**: merge 7 + 5, then 4 + 7, then 12 + 11, for 12 + 11 + 23. The greedy rule merges the cheapest pair 5 + 4 first (cost 9), leaving [7, 9, 7]; both pairs now cost 16, and either choice leaves two piles that cost 23 to join, a total of 48. The DP matched a brute force over all merge orders on 3000 random rows.

**Exercise 14.7.** It fails on [2, 1, 3]: it returns 3, while the correct difference is 2 (exhaustive search agrees). In the pass for length 2, $l$ = 1 comes first and overwrites `d[1]` with $D(1, 2) = \max(1 - 3, 3 - 1) = 2$. Then $l$ = 0 should compute $D(0, 1) = \max(2 - D(1, 1), 1 - D(0, 0)) = \max(1, -1) = 1$, but `d[1]` no longer holds $D(1, 1) = 1$, so it computes $\max(2 - 2, 1 - 2) = 0$. The last pass then gives $\max(2 - 2, 3 - 0) = 3$ instead of $\max(2 - 2, 3 - 1) = 2$. Fix: run the pass from left to right, `for (int l = 0; l + len - 1 < n; l++)`, so that `d[l + 1]` still holds the previous length when `d[l]` is overwritten. Rows of one or two elements always pass, because each pass has at most one step. On 5000 random rows of up to 8 values the buggy method was wrong on 1493.

**Exercise 14.8.** (1) *Theory Listing 14.1*: 1000 · 999/2 = 499 500 divisibility tests, instant. Values up to $2 \cdot 10^9$ still fit an `int` (the largest is 2 147 483 647). (2) *Theory Listing 14.2*: 13 970 034 steps. Trial division up to $\sqrt{v}$ for every $v$ would cost $\Sigma \lfloor \sqrt{v} \rfloor$ = 666 167 500 steps, about 50 times more. (3) Split DP on the last balloon, O($n^3$). With the two padding balloons there are 502 positions, and the triples $l < k < r$ number C(502, 3) = 20 958 500: time is fine, and the table has $502^2 \approx 2.5 \cdot 10^5$ entries. (4) The game DP has 12 502 500 states, so time is fine, but a `long[5000][5000]` needs 200 000 000 bytes: memory binds. Use *Theory Listing 14.6*, 40 000 bytes. (5) 20 833 332 500 triples, about $2 \cdot 10^{10}$: time binds, and the plan has no faster method for it. (6) Stars and bars. A chain has at most $\lfloor \log 10^4 \rfloor + 1 = 14$ values, so 13 passes of the loop over multiples compute all chain counts, each at most 93 668 steps, about $1.2 \cdot 10^6$ in all. The 14 binomials C($K$ − 1, $s$ − 1) come from Pascal's rule with $r < 14$, about $10^4 \cdot 14 = 1.4 \cdot 10^5$ additions. The direct DP would cost $K \cdot 93\,668 \approx 9.4 \cdot 10^8$ steps. (7) A table of $10^{18}$ entries is impossible. Print the table up to 100, see the period 7 (losing exactly when $s$ mod 7 is 0 or 2), prove it by induction, and answer in O(1): $10^{18} \bmod 7 = 1$, a winning position.

**Exercise 14.9.** Compare with *Theory Listing 14.5* and *Theory Listing 14.6*. The lines that matter are `dp[i][i] = a[i]`, the outer loop over the length, the maximum of `a[l] - dp[l + 1][r]` and `a[r] - dp[l][r - 1]`, and, in the rolling version, the left-to-right pass of Exercise 14.7.

## Chapter 15 · Graphs I: Breadth-First Search, Depth-First Search and Topological Order

**Exercise 15.1.** (1) `indeg[v]++`; (2) `indeg[u] == 0`; (3) `head < tail`; (4) `--indeg[v] == 0`; (5) `tail == n ? order : null`. The positions `head..tail - 1` are the queue: they hold vertices that have been output but whose edges have not been processed yet; positions before `head` are finished. A counter only goes down, by one per edge into $v$, so it passes through the value 0 exactly once, after the last of those edges has been processed. That is why each vertex is appended at most once, even with duplicate edges.

**Exercise 15.2.**

| pop | dist[u] | newly discovered | already known | queue after | dist[0..7] |
|---|---|---|---|---|---|
| 0 | 0 | 1, 2, 3 | — | [1, 2, 3] | [0, 1, 1, 1, −1, −1, −1, −1] |
| 1 | 1 | 4 | 0 | [2, 3, 4] | [0, 1, 1, 1, 2, −1, −1, −1] |
| 2 | 1 | — | 0, 4 | [3, 4] | unchanged |
| 3 | 1 | 5 | 0 | [4, 5] | [0, 1, 1, 1, 2, 2, −1, −1] |
| 4 | 2 | 6 | 1, 2 | [5, 6] | [0, 1, 1, 1, 2, 2, 3, −1] |

| pop | dist[u] | newly discovered | already known | queue after | dist[0..7] |
|---|---|---|---|---|---|
| 5 | 2 | 7 | 3, 6 | [6, 7] | [0, 1, 1, 1, 2, 2, 3, 3] |
| 6 | 3 | — | 4, 5, 7 | [7] | unchanged |
| 7 | 3 | — | 6, 5 | [] | unchanged |

The layers are {0}, {1, 2, 3}, {4, 5} and {6, 7}. The tree edges are 0–1, 0–2, 0–3, 1–4, 3–5, 4–6 and 5–7. Vertex 4 is seen again from 2, and 7 from 6, but neither is queued twice, because both were marked when they were pushed.

| | c0 | c1 | c2 | c3 | c4 | c5 |
|---|---|---|---|---|---|---|
| r0 | 0 | 1 | 2 | # | 3 | 2 |
| r1 | 1 | # | 3 | # | 2 | 1 |
| r2 | 2 | # | 3 | 2 | 1 | 0 |
| r3 | 3 | 4 | 4 | # | # | 1 |
| r4 | # | 5 | # | −1 | −1 | # |

The cells (4, 3) and (4, 4) are open but surrounded by walls and the border, so they keep −1. LeetCode 994 would return −1, because two fresh oranges never rot. If those two cells were walls, it would return 5, the largest distance, reached at (4, 1). Separate runs from each source confirm the table: (0, 4) is 3 steps from the right source but 8 from the left one, and (3, 1) is 4 from the left source and 5 from the right one. One multi-source run gives the smaller value in every cell at once.

**Exercise 15.3.**

| take | counters decremented (new value) | queue after | indeg after |
|---|---|---|---|
| 5 | 6 (0), 3 (0) | [6, 3] | [0, 2, 0, 0, 2, 0, 0] |
| 6 | 1 (1) | [3] | [0, 1, 0, 0, 2, 0, 0] |
| 3 | 1 (0), 4 (1) | [1] | [0, 0, 0, 0, 1, 0, 0] |
| 1 | 4 (0) | [4] | [0, 0, 0, 0, 0, 0, 0] |
| 4 | — | [] | unchanged |

The order is 0, 2, 5, 6, 3, 1, 4. The longest paths ending at vertices 0..6 are [0, 2, 0, 1, 3, 0, 1]; vertex 4 has 3, for example 5 → 6 → 1 → 4. The numbers of paths from 5 are [0, 2, 0, 1, 3, 1, 1]: vertex 4 is reached by 5 → 6 → 1 → 4, 5 → 3 → 1 → 4 and 5 → 3 → 4. With the extra edge 4 → 6 the graph has the cycle 6 → 1 → 4 → 6. The output stops after 0, 2, 5, 3 (4 of 7 vertices, with the counters of 1, 4 and 6 stuck at 1) and the method returns `null`.

**Exercise 15.4.**

| pop | d[u] | relaxations that succeed | deque after | d[0..5] |
|---|---|---|---|---|
| 0 | 0 | 1 back (1), 2 back (1), 3 front (0) | [3, 1, 2] | [0, 1, 1, 0, ∞, ∞] |
| 3 | 0 | 2 front (0); 3→1 gives 1, not better | [2, 1, 2] | [0, 1, 0, 0, ∞, ∞] |
| 2 | 0 | 4 back (1), 5 back (1) | [1, 2, 4, 5] | [0, 1, 0, 0, 1, 1] |
| 1 | 1 | none (1→4 gives 1, not better) | [2, 4, 5] | unchanged |
| 2 | 0 | none (the outdated entry) | [4, 5] | unchanged |
| 4 | 1 | none (4→5 gives 1, not better) | [5] | unchanged |
| 5 | 1 | none | [] | [0, 1, 0, 0, 1, 1] |

Vertex 2 was pushed twice: at the back with cost 1 from 0, then at the front with cost 0 through the free edges 0 → 3 → 2. Its second pop, the old entry from the back, changes nothing. Plain BFS counts edges and reports [0, 1, 1, 1, 2, 2]: it is wrong for vertices 2, 3, 4 and 5. A Bellman–Ford check gives [0, 1, 0, 0, 1, 1].

**Exercise 15.5.** (1) False: it is set once, when the vertex is pushed, and never changed; *Theory Theorem 15.3* proves that this first value is already the distance. (2) True. In a tree, every neighbour of $v$ except its parent is a child, which can only be discovered through $v$, so $v$ is discovered from its parent and gets the right value by induction; on 3000 random trees the stack version always agreed with BFS. On the 5-cycle 0–1, 0–2, 2–3, 3–4, 4–1 it gives vertex 4 the value 3, although 0–1–4 has length 2. (3) True: each vertex is its own component, and *Theory Listing 15.5* starts $10^5$ rounds. (4) False: the 4-cycle is coloured 0, 1, 0, 1; only odd cycles prevent a two-colouring (*Theory Theorem 15.7*). (5) False: in the graph 0 → 1, 1 → 0, 1 → 2, vertex 2 is on no cycle, but its counter never reaches 0, so it is not output either. (6) True: with `g[5]` = [3, 6] instead of [6, 3], Exercise 15.3 outputs 0, 2, 5, 3, 6, 1, 4, which is also a topological order. (7) False: each vertex is pushed at most twice (*Theory Theorem 15.11*), so it is popped at most twice. (8) False: all sources go into one queue and each vertex is still pushed once, so the cost stays $O(n + m)$.

**Exercise 15.6.** A vertex is a pair (cell, $h$), where $h = 0$ if the hammer is unused and $h = 1$ if it has been used: $2RC$ vertices. From $(r, c, h)$ a move to a neighbouring open cell keeps $h$; a move into a wall cell is allowed only from $h = 0$ and leads to $h = 1$. Every move costs 1, so plain BFS on the pairs applies, with `dist[2][R][C]`. Start at (0, 0, 0) and answer min(dist[$R - 1$][$C - 1$][0], dist[$R - 1$][$C - 1$][1]), ignoring −1 values. Time: $2RC = 2 \cdot 10^6$ vertices with 4 moves each, about $8 \cdot 10^6$ checks; memory $2RC$ `int`s, 8 MB.

On Figure 15.4, the first arrival at (0, 2) comes after 2 steps, straight through the wall at (0, 1), so the naive version fixes (0, 2) as "hammer used". Every cell to its right is first reached from (0, 2) and is also marked "used", so the wall at (1, 5) above T can no longer be broken, and the naive search returns −1. Its first arrival was the wrong one to keep: the walk around the wall, (0, 0) → (1, 0) → (2, 0) → (2, 1) → (2, 2) → (1, 2) → (0, 2), takes 6 steps but keeps the hammer. With the pairs, the search keeps both arrivals, (0, 2, 1) at 2 and (0, 2, 0) at 6, and continues (0, 3), (0, 4), (0, 5), breaks (1, 5) at step 10 and reaches T at step **11**. Without the hammer T cannot be reached at all. On 3000 random grids the pair BFS matched a brute force that removes each wall in turn and runs a plain BFS; the naive version was wrong on 88 of them, and on this grid it fails whatever order the four directions are tried in.

**Exercise 15.7.** Take 0 → 1 with cost 1, 0 → 2 with cost 0 and 2 → 1 with cost 0. Popping 0 reaches 1 with $d = 1$ (back) and 2 with $d = 0$ (front). When 2 is popped, the free edge would give 1 the cost 0, but 1 is already "reached" and is skipped. The method returns $d[1] = 1$; the correct value is 0 (path 0 → 2 → 1). In 0-1 BFS the first value given to a vertex need not be final, because a later free edge can improve it. Fix: replace the `if (d[v] != ...)` test and the next line with `int nd = d[u] + e[1]; if (nd >= d[v]) continue; d[v] = nd;`, as in *Theory Listing 15.8*. When every cost is 1 the deque only ever adds at the back, so the method is plain BFS, which is why it passed those graphs. On 4000 random graphs with costs 0 and 1 the buggy method disagreed with Bellman–Ford on 390, and on the graph of Exercise 15.4 it gives $d[2] = 1$.

**Exercise 15.8.** (1) Grid BFS, *Theory Listing 15.4*: $10^6$ cells, at most $4 \cdot 10^6$ neighbour checks, and `dist` as `int[1000][1000]` is 4 MB. Trap: check the bounds before reading the cell. (2) As `List<Integer>[]` about 26 MB, as head/next/to arrays about 8.7 MB, as measured in *Theory Section 15.1*; both fit. Trap: for an undirected graph store each edge twice. (3) An explicit stack or the BFS queue: $2 \cdot 10^5$ pushes. Trap: a recursive DFS needs $2 \cdot 10^5$ nested calls on a path, far beyond the default stack. (4) One multi-source BFS with the 1000 hospitals as sources (*Theory Theorem 15.4*): about $n + 2m = 700\,000$ steps. Trap: 1000 separate BFS runs would cost about $7 \cdot 10^8$. (5) 0-1 BFS: each cell is pushed at most twice, so at most $2 \cdot 10^6$ pushes and $8 \cdot 10^6$ edge checks. Dijkstra would also work but adds a heap factor of about $\log_2 10^6 \approx 20$. Trap: compare `nd < d[v]` on every relaxation, not "already seen". (6) Kahn's algorithm: about $n + m = 3 \cdot 10^5$ steps; possible exactly when all $10^5$ courses are output. Trap: the pair $[a, b]$ gives the edge $b \to a$. (7) Costs 1 and 2 are not "0 or $c$", so 0-1 BFS does not apply directly. Split every cost-2 edge $u \to v$ into $u \to x \to v$ through a new vertex $x$, then run plain BFS: at most $3 \cdot 10^5$ vertices and $4 \cdot 10^5$ edges. Dijkstra (Chapter 17) also works. Trap: answer only for the original vertices.

**Exercise 15.9.** Compare with *Theory Listing 15.4*. The lines that matter are pushing every source with distance 0 **before** the loop, the bounds check before the wall check, and setting `dist` when a cell is pushed.

## Chapter 16 · Analysing a Problem Before You Code

**Exercise 16.1.** (1) Objects: an array of profits. Question: count. Limits: $n = 2 \cdot 10^5$ allows O($n$ log $n$) or O($n$). Technique: prefix sums with a `HashMap<Long, Integer>` of earlier prefixes (Chapter 2), O($n$). Traps: the profits can be negative, so a sliding window does not work; the prefix sums reach $2 \cdot 10^{14}$ and the count can reach $n(n+1)/2 = 20\,000\,100\,000$, so both need `long`. (2) Objects: a graph whose vertices are students and whose edges are quarrels. Question: decide. Technique: the bipartite check by BFS colouring (Chapter 15), O($n + m$): $10^5$ vertices and $4 \cdot 10^5$ adjacency entries. Trap: the graph may be disconnected, so start a colouring from every uncoloured vertex. (3) Objects: an array of page counts. Question: optimise, "minimise the largest". Technique: binary search on the answer with a greedy check (Chapter 6). The total can be $10^5 \cdot 10^9 = 10^{14}$, so the search takes 47 rounds of an O($n$) check, about $4.7 \cdot 10^6$ steps. Traps: sums in `long`, and the lower end of the search is the largest single document, not 0. (4) Objects: a directed graph with an edge $x \to y$ for each rule. Question: construct. Technique: Kahn's algorithm (Chapter 15), O($n + m$) = $3 \cdot 10^5$ steps. Traps: the direction of the edge, and printing IMPOSSIBLE when fewer than $n$ modules are output (a cycle).

**Exercise 16.2.**

| method | bound | the work is charged to | count on the given input |
|---|---|---|---|
| loopA | $\le n$ | each step moves $l$ forward, and $l$ never passes $n$ | 6 |
| loopB | $< 2n$ | the sum $1 + 2 + 4 + \cdots + 2^j$ with $2^j < n$ is $2^{j+1} - 1$ | 1023 |
| loopC | $n(n-1)/2$ | nothing: $j$ restarts at 0 for every $i$ | 499 500 |
| loopD | $\le 2n$ | each element is pushed once and popped at most once | 14 |
| loopE | $< 2n$ | bit $i$ is set and cleared at most once per $2^i$ increments | 1994 |

In `loopA` the window shrinks by 1, 2 and 3 steps at $r = 3$, 4 and 5; in `loopD` there are 8 pushes and 6 pops. `loopC` is the quadratic one: its inner counter is declared inside the outer loop and restarts at 0, so "at most $i$ steps" adds up to $0 + 1 + \cdots + (n-1)$. In `loopA`, $l$ survives from one $r$ to the next and only moves forward, which is what *Theory Theorem 16.1* needs. For `loopE`, bit $i$ changes once every $2^i$ increments, so $n$ increments flip at most $n + n/2 + n/4 + \cdots < 2n$ bits; the exact total is $2n$ − (number of 1-bits of $n$), $2000 - 6 = 1994$ for $n = 1000$. Measured at $n = 10^6$: `loopA` with all values 1 and $k = 0$ makes 1 000 000 steps, `loopD` on an increasing array 1 999 999, and `loopE` 1 999 993.

**Exercise 16.3.**

| idea | operations or bytes | fits? | better choice |
|---|---|---|---|
| `HashMap` counting $2 \cdot 10^6$ values | at most $10^6 + 1$ keys at about 77 bytes: up to 77 MB; $2 \cdot 10^6$ merges at 0.23 s per $10^6$: about 0.46 s | yes, but slow and large | `int[1000001]`: 4 MB and about 0.04 s |
| `int[1000000][2]` | about 29 bytes per row, 29 MB | yes, but wasteful | `int[2][1000000]` or two arrays: 8 MB |
| `long[5000][5000]` | $2 \cdot 10^8$ bytes = 200 MB | no, with the JVM's own memory | two rows of `long[5000]`: 80 000 bytes |
| sorting `Integer[]` of $10^6$ | about 0.71 s | risky on a slower judge | `int[]` with `Arrays.sort`: about 0.13 s |
| $3 \cdot 10^8$ steps of `x * y % MOD` | $3 \times 0.43$ s ≈ 1.3 s | no | reduce the number of steps; the modulo is the expensive part |
| all pairs $i < j$, $n = 10^5$ | 4 999 950 000 steps, about 17 to 50 s | no | find an O($n$ log $n$) idea; keep the pairs as the stress-test brute force |
| recursive DFS, depth up to $10^5$ | $10^5$ nested calls; the default stack held between about 7 400 and 24 700 | no: `StackOverflowError` | an explicit `ArrayDeque` stack (Chapter 15) |

The times come from *Theory Table 16.3* and the stack depths from *Theory Section 16.3*, all measured on the machine used for the book; a judge can be two or three times slower, which turns "about 0.7 s" into a risk.

**Exercise 16.4.**

| code | wrong? | Java computes | fix |
|---|---|---|---|
| `long p = a * b;` | yes | 1 410 065 408 | `(long) a * b`: 10 000 000 000 |
| `int mid = (a + b) / 2;` | yes | −397 483 648 | `a + (b - a) / 2` or `(a + b) >>> 1`: 1 750 000 000 |
| `int t = a + b;` | no | 2 000 000 000 | none: it is below 2 147 483 647 (a third term would overflow) |
| `int s` summing $2 \cdot 10^5$ values of $10^9$ | yes | 552 894 464 | `long s`: 200 000 000 000 000 |
| `u * v * w % MOD` | yes | 654 148 940 | `u * v % MOD * w % MOD`: 999 999 287 (`BigInteger` agrees) |
| `(a - b) % MOD` | yes | −2 | `Math.floorMod(a - b, MOD)` or `(a - b + MOD) % MOD`: 1 000 000 005 |
| `Math.abs(a)` | yes | −2 147 483 648 | `Math.abs((long) a)`: 2 147 483 648 |
| `n * (n - 1) / 2` | yes | 704 982 704 | `(long) n * (n - 1) / 2`: 4 999 950 000 |

In the fifth line, `u * v` = 999 999 997 000 000 002 still fits in a `long`; the third factor pushes the product past $9.2 \cdot 10^{18}$. `Math.abs(Integer.MIN_VALUE)` is negative because $+2^{31}$ does not exist as an `int`.

**Exercise 16.5.**

| input | k | correct answer | method returns |
|---|---|---|---|
| $[10^9]$ | 1 | 1 000 000 000 | 0 |
| [7, 7, 7, 7, 7] | 3 | 21 | 21 |
| [1, 2, 3, 4, 5] | 3 | 12 | 12 |
| [5, 4, 3, 2, 1] | 3 | 12 | 9 |
| $[10^9, 10^9, 10^9, 10^9, 10^9]$ | 3 | 3 000 000 000 | 0 |
| $[-10^9, 10^9, -10^9, 10^9, -10^9]$ | 3 | 1 000 000 000 | 1 000 000 000 |

Three tests fail. **Bug 1:** the first window is never compared with `best`, which starts at 0. The smallest input ($n = 1$, where the second loop never runs) and the reverse-sorted array (whose best window is the first one) expose it. **Bug 2:** `sum` is an `int`; three values of $10^9$ add up to $3 \cdot 10^9$, which wraps around to −1 294 967 296, and the maximum-values test exposes it (the method then returns its starting value 0). Fix: declare `long sum`, set `long best = sum;` after the first loop, and write `sum += (long) a[i] - a[i - k];` (the difference alone, at most $2 \cdot 10^9$, happens to fit an `int`, but the cast removes the doubt). The generator has no array whose values are **all negative**. For [−5, −3, −8] with $k = 1$ the answer is −3, and any version that keeps `best = 0`, even with the first window compared, returns 0. The fixed method matched a brute force over all windows on 5000 random small arrays and on 5000 arrays with values up to $10^9$ in absolute value; the buggy one was wrong on 3058 of the small ones.

## Chapter 17 · Graphs II: Shortest Paths, Union-Find and Spanning Trees

**Exercise 17.1.** (1) `d[src] = 0;` (2) `Long.compare(x[0], y[0]);` (3) `cur[0] != d[u];` (4) `d[u] + e[1];` (5) `pq.add(new long[]{nd, e[0]})`. The subtraction overflows once two distances differ by more than $2^{31} - 1$. For $x[0] = 3\,000\,000\,000$ and $y[0] = 0$, `(int) (x[0] - y[0])` is −1 294 967 296, so the comparator claims that $3 \cdot 10^9$ is smaller than 0, and the heap order is broken.

**Exercise 17.2.**

| step | pair | action | relaxations that succeed | d[0..5] after | heap afterwards |
|---|---|---|---|---|---|
| 1 | (0, 0) | settle | d[1]: ∞ → 9, d[2]: ∞ → 2 | [0, 9, 2, ∞, ∞, ∞] | (2,2) (9,1) |
| 2 | (2, 2) | settle | d[1]: 9 → 5, d[3]: ∞ → 7, d[4]: ∞ → 11 | [0, 5, 2, 7, 11, ∞] | (5,1) (7,3) (9,1) (11,4) |
| 3 | (5, 1) | settle | d[3]: 7 → 6 (1 → 4 gives 14 and fails) | [0, 5, 2, 6, 11, ∞] | (6,3) (7,3) (9,1) (11,4) |
| 4 | (6, 3) | settle | d[4]: 11 → 8, d[5]: ∞ → 15 | [0, 5, 2, 6, 8, 15] | (7,3) (8,4) (9,1) (11,4) (15,5) |
| 5 | (7, 3) | stale | — | unchanged | (8,4) (9,1) (11,4) (15,5) |
| 6 | (8, 4) | settle | d[5]: 15 → 10 | [0, 5, 2, 6, 8, 10] | (9,1) (10,5) (11,4) (15,5) |
| 7 | (9, 1) | stale | — | unchanged | (10,5) (11,4) (15,5) |
| 8 | (10, 5) | settle | none (5 has no outgoing edges) | unchanged | (11,4) (15,5) |
| 9 | (11, 4) | stale | — | unchanged | (15,5) |
| 10 | (15, 5) | stale | — | unchanged | empty |

The distances are [0, 5, 2, 6, 8, 10]. Ten pairs were pushed: the start pair and one per successful relaxation (9 of them). Four of the ten were stale. The bound is $E + 1 = 11$. The shortest-path tree consists of each vertex's last successful relaxation: 0 → 2, 2 → 1, 1 → 3, 3 → 4 and 4 → 5. As a LeetCode 743 answer, the last vertex is reached at time 10.

**Exercise 17.3.**

| operation | result | parent[0..8] | sizes of the roots |
|---|---|---|---|
| union(0, 1) | true | [0, 0, 2, 3, 4, 5, 6, 7, 8] | 0: 2, others 1 |
| union(3, 2) | true | [0, 0, 3, 3, 4, 5, 6, 7, 8] | 0: 2, 3: 2 |
| union(2, 0) | true | [3, 0, 3, 3, 4, 5, 6, 7, 8] | 3: 4 |
| union(4, 5) | true | [3, 0, 3, 3, 4, 4, 6, 7, 8] | 3: 4, 4: 2 |
| union(6, 7) | true | [3, 0, 3, 3, 4, 4, 6, 6, 8] | 3: 4, 4: 2, 6: 2 |
| union(5, 7) | true | [3, 0, 3, 3, 4, 4, 4, 6, 8] | 3: 4, 4: 4, 8: 1 |
| union(8, 1) | true | [3, 3, 3, 3, 4, 4, 4, 6, 3] | 3: 5, 4: 4 |
| union(7, 1) | true | [3, 3, 3, 3, 3, 4, 4, 4, 3] | 3: 9 |
| union(0, 7) | false | [3, 3, 3, 3, 3, 4, 4, 3, 3] | 3: 9 |

In union(2, 0) the sizes tie (2 and 2), so the root 3 of the first argument stays the root. Path halving changed the array three times. In union(8, 1), find(1) walked 1 → 0 → 3 and set `parent[1]` = 3; the root 8 (size 1) then went under 3 (size 4), a swap. In union(7, 1), find(7) walked 7 → 6 → 4 and set `parent[7]` = 4; root 4 (size 4) went under root 3 (size 5). In the rejected union(0, 7), find(7) walked 7 → 4 → 3 and set `parent[7]` = 3. The final forest has root 3 with children 0, 1, 2, 4, 7 and 8, and vertex 4 has children 5 and 6. No vertex is deeper than 2.

**Exercise 17.4.**

| row | edge | weight | already in one group? | kept? | total | groups left |
|---|---|---|---|---|---|---|
| 4 | 2–5 | 3 | no | keep | 10 | 3 |
| 5 | 1–5 | 3 | no | keep | 13 | 2 |
| 6 | 1–2 | 4 | yes (1–5–2) | skip | 13 | 2 |
| 7 | 3–4 | 4 | no | keep | 17 | 1 |
| 8 | 0–4 | 5 | yes | skip | 17 | 1 |

| row | edge | weight | already in one group? | kept? | total | groups left |
|---|---|---|---|---|---|---|
| 9 | 0–1 | 6 | yes | skip | 17 | 1 |
| 10 | 2–6 | 6 | yes | skip | 17 | 1 |
| 11 | 4–5 | 7 | yes | skip | 17 | 1 |

The MST weighs 17 and has the edges 1–4, 5–6, 0–3, 2–5, 1–5 and 3–4. A brute force over all sets of six edges finds no other spanning tree of weight 17, so it is unique despite the tied weights. The tree path from 0 to 6 is 0–3–4–1–5–6 with weights 3, 4, 2, 3, 2, so the minimax cost is 4. The shortest path by total weight is a different route: 0–1–5–6 with length 6 + 3 + 2 = 11, whose largest step is 6.

**Exercise 17.5.** (1) False: in Exercise 17.2 vertex 1 is first pushed with 9, but its distance is 5. (2) False: the answers stay correct, because a stale pair relaxes with the current `d[u]` and every relaxation fails; only the running time grows. (3) True: with $s \to a$ (1), $s \to b$ (1), $a \to b$ (0), $b \to t$ (1), settling $b$ before $a$ loses a path (the pitfall after *Theory Theorem 17.4*). (4) False: a value improved early in a round is reused later in the same round, so one round can add several edges (*Theory Figure 17.5*). (5) False: after union(8, 1) in Exercise 17.3, `parent[5]` = 4 and `parent[1]` = 3, yet 5 and 1 are in one group; compare `find(a) == find(b)`. (6) True: by *Theory Lemma 17.6* the depth is at most $\log_2 9 \approx 3.17$, so at most 3 edges and 4 vertices on the path. (7) False: Exercise 17.4 has three edges of weight 3 and two of weight 4, and its MST is unique. (8) True: the cut property only compares the weights of two edges and never adds weights along a path, so it holds for any real weights.

**Exercise 17.6.** State: $(v, c)$ with $c = 1$ if the van has passed a station, so $2n$ vertices; give $(v, 0)$ the id $v$ and $(v, 1)$ the id $v + n$. For every road $u$–$v$ of time $w$, in both directions: $(u, 0) \to (v, 1)$ if $v$ has a station, otherwise $(u, 0) \to (v, 0)$; and $(u, 1) \to (v, 1)$. All with weight $w$. Start at $(s, 1)$ if $s$ has a station, else $(s, 0)$. The answer is $d[(t, 1)]$, or −1 if it is still ∞. The new graph has $2n$ vertices and $4m$ directed edges, so Dijkstra costs $O((n + m) \log n)$ time and $O(n + m)$ memory, with `long` distances. (Because roads are two-way, two Dijkstra runs from $s$ and from $t$ also work: the answer is the minimum over stations $c$ of dist($s$, $c$) + dist($c$, $t$).) In the example the plain shortest route is 0–1–5 with time 5, but it passes no station. Through station 2 the best is 0–2–3–5, 4 + 2 + 3 = 9; through station 6 it is 5 + 6 = 11 (0–1–4–6 and back 6–4–1–5). The answer is 9, which a Floyd–Warshall check agrees with; the layered method also matched Floyd–Warshall on 5000 random graphs.

**Exercise 17.7.** The stale check is missing. A vertex whose distance improves after its first push leaves an old pair in the heap; when that pair surfaces, the loop scans the vertex's edges again, and every tight edge adds `ways[u]` a second time. Take the undirected graph 0–1 (5), 0–2 (1), 2–1 (1), 1–3 (1) with source 0. Vertex 1 is pushed with 5, then improved to 2 through vertex 2. After 1 and 3 are settled, the stale pair (5, 1) surfaces, finds 2 + 1 = 3 = $d$[3], and adds `ways[1]` again. The method returns ways[3] = 2, but there is only one shortest path, 0–2–1–3 of length 3. Fix: insert `if (cur[0] != d[u]) continue;` after `int u = (int) cur[1];`, as in *Theory Listing 17.4*. On 3000 random small graphs the fixed method matched a brute force that enumerates all simple paths, and the buggy one was wrong on 267 of them.

**Exercise 17.8.** (1) Dijkstra, *Theory Listing 17.2*: at most $E + 1 = 200\,001$ pushes and as many polls, each about 18 comparisons, so a few million steps. Trap: a distance can reach $99\,999 \cdot 10^9 \approx 10^{14}$, so use `long`. (2) 0-1 BFS: $10^6$ cells and at most $4 \cdot 10^6$ moves, $O(V + E)$ with a deque. Trap: push to the front after a 0-move and to the back after a 1-move, and skip a cell whose distance has already improved. (3) Prim's array version, *Theory Listing 17.7*: 2000 steps of two scans each, about $8 \cdot 10^6$ operations, three arrays of 2000. Kruskal would build 1 999 000 edges (about 72 MB as `int[3]` objects) and sort them with about $4 \cdot 10^7$ comparisons. Trap: compute the distance when needed instead of storing the matrix. (4) Bellman–Ford: $V - 1 = 499$ rounds of 5000 relaxations, about $2.5 \cdot 10^6$. Trap: Dijkstra's (P1) fails with negative edges. (5) $K + 1 = 51$ Bellman–Ford rounds with a copied array (*Theory Theorem 17.5*): $51 \cdot 10^4 = 510\,000$ relaxations. Trap: in-place updates allow too many flights. (6) Union-find: start with $n$ groups and subtract 1 for every `union` that returns `true`; $2 \cdot 10^5$ operations of almost O(1) each (*Theory Theorem 17.7*). Trap: count only the successful unions. (7) Offline: sort the edges by weight and the queries by $T$, then sweep, adding edges up to each $T$ before answering (*Theory Section 17.6*): two sorts of $10^5$ items and $10^5$ unions plus $2 \cdot 10^5$ finds. Trap: answer the queries in sorted order, and print them back in input order.

**Exercise 17.9.** Compare with *Theory Listing 17.6*. The lines that matter are `parent[x] = parent[parent[x]]` before `x = parent[x]` in `find`, the swap so that the smaller tree goes under the larger, `size[a] += size[b]`, and the final check `used == n - 1`.

## Chapter 18 · Trees I: Recursion, Parent Arrays and Ancestors

**Exercise 18.1.** (1) `(1 << lg) < n`; (2) `par[v] == -1 ? v : par[v]`; (3) `up[k - 1][up[k - 1][v]]`; (4) `depth[a] - depth[b]`; (5) `up[k][a] != up[k][b]`; (6) `up[0][a]`. The recurrence for $v$ reads the rows of up[$k-1$][$v$], an ancestor of $v$. In BFS order every ancestor is processed earlier (*Theory Theorem 18.8*), so its rows are complete. In index order an ancestor with a larger number has not been filled yet, and its rows still hold 0, so jumps can land on vertex 0 instead of the true ancestor.

**Exercise 18.2.** The children lists are 3: [0, 1], 0: [2, 8], 1: [5, 6], 8: [4], 5: [9], 4: [7].

| iteration | u | children appended | order afterwards |
|---|---|---|---|
| 1 | 3 | 0, 1 | [3, 0, 1] |
| 2 | 0 | 2, 8 | [3, 0, 1, 2, 8] |
| 3 | 1 | 5, 6 | [3, 0, 1, 2, 8, 5, 6] |
| 4 | 2 | — | unchanged |
| 5 | 8 | 4 | [3, 0, 1, 2, 8, 5, 6, 4] |
| 6 | 5 | 9 | [3, 0, 1, 2, 8, 5, 6, 4, 9] |
| 7 | 6 | — | unchanged |
| 8 | 4 | 7 | [3, 0, 1, 2, 8, 5, 6, 4, 9, 7] |
| 9 | 9 | — | unchanged |
| 10 | 7 | — | unchanged |

| i | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|
| v | 7 | 9 | 4 | 6 | 5 | 8 | 2 | 1 | 0 |
| size[v] | 1 | 1 | 2 | 1 | 2 | 3 | 1 | 4 | 5 |
| par[v] | 4 | 5 | 8 | 1 | 1 | 0 | 0 | 3 | 3 |
| size[par[v]] after | 2 | 2 | 3 | 2 | 4 | 4 | 5 | 5 | 10 |

size = [5, 4, 1, 10, 2, 2, 1, 1, 3, 1] and depth = [1, 1, 2, 0, 3, 2, 2, 4, 2, 3]. The index loop reaches vertex 8 before vertex 4, so it adds size[8] = 1 to size[0] before 4 (and through it 7) has been added to size[8]. It returns size[0] = 3 and size[3] = 8; every other entry happens to be right. The cause is that 8 is the parent of the smaller number 4.

**Exercise 18.3.**

| node | gain(left) | gain(right) | l | r | bend | returns | best |
|---|---|---|---|---|---|---|---|
| 8 | 0 | 0 | 0 | 0 | 8 | 8 | 8 |
| 5 | 0 | 0 | 0 | 0 | 5 | 5 | 8 |
| 3 | 5 | 0 | 5 | 0 | 8 | 8 | 8 |
| −6 | 8 | 8 | 8 | 8 | 10 | 2 | 10 |
| −2 | 0 | 0 | 0 | 0 | −2 | −2 | 10 |
| 6 | 0 | 0 | 0 | 0 | 6 | 6 | 10 |
| −3 | 0 | 6 | 0 | 6 | 3 | 3 | 10 |

| node | gain(left) | gain(right) | l | r | bend | returns | best |
|---|---|---|---|---|---|---|---|
| 4 | −2 | 3 | 0 | 3 | 7 | 7 | 10 |
| −1 | 2 | 7 | 2 | 7 | 8 | 6 | 10 |

The answer is 10, the path 8, −6, 3, 5, which bends at −6. A brute force over all pairs of end nodes agrees. Node 4 drops its −2 child, and the root's bend (−1 + 2 + 7 = 8) loses. The diameter is 6 edges, for example 5–3–(−6)–(−1)–4–(−3)–6, which bends at the root.

**Exercise 18.4.**

| v | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| up[1] | 0 | 0 | 0 | 1 | 2 | 3 | 4 | 0 | 0 | 7 | 8 | 2 |
| up[2] | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 0 | 0 | 0 | 0 | 0 |
| up[3] | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| step | a | b | entries compared | jump? |
|---|---|---|---|---|
| lift 6 by $2 = 10_2$: up[1][6] | 4 | 10 | — | — |
| k = 3 | 4 | 10 | 0, 0 | no |
| k = 2 | 4 | 10 | 0, 0 | no |
| k = 1 | 4 → 2 | 10 → 8 | 2, 8 | yes |
| k = 0 | 2 → 1 | 8 → 7 | 1, 7 | yes |

So lca(6, 10) = up[0][1] = 0, and dist(6, 10) = 6 + 4 − 0 = 10. Here $t = 4$, so the loop subtracts the digits of $t - 1 = 3 = 11_2$, as in the proof of *Theory Theorem 18.12*. lca(6, 11): lift 6 by 2 to 4; 4 and 11 are both children of 3, so no jump, and the answer is up[0][4] = 3 with distance 4. lca(5, 9): lift 5 by 2 to 3; at $k = 1$ the entries 1 and 7 differ, so $a = 1$ and $b = 7$; at $k = 0$ both entries are 0, no jump; the answer is 0 with distance 8. `kthAncestor(11, 3)` = 1 (11 → 3 → 2 → 1), and `kthAncestor(10, 5)` = −1 because depth[10] = 4. The table and queries were checked against a brute force on 3000 random trees.

**Exercise 18.5.** (1) False: par = [−1, 3, 1, 2] has the cycle 1 → 3 → 2 → 1, and a BFS from the root reaches only one of the four vertices. Check that the BFS reaches all $n$ (*Theory Theorem 18.3*). (2) False: the parent continues the path through one child only; returning both branches would let the parent build a path that forks. In Exercise 18.3 node −6 would return 10 instead of 2, and the root would report 16 for a "path" that forks below −6. (3) False: for a single node of value −5 the answer is −5, but `best` = 0 would stay 0. (4) True: the call at $p$ returns at once, and $p$ is the LCA by *Theory Definition 18.3*. (5) False: `q.size()` changes inside the loop. On the perfect tree with values 1..7 in level order, it returns [[1, 2, 3, 4], [5, 6], [7]]. (6) True: $2^{16} < 10^5 \le 2^{17}$, and $17 \cdot 10^5 \cdot 4$ bytes = 6.8 MB. (7) False: for the chain 0–1, tin = [0, 1] and tout = [1, 1], so tout[1] = tout[0]; the test is tin[$u$] ≤ tin[$v$] ≤ tout[$u$]. (8) False: by *Theory Theorem 18.15* it is $\Sigma\, v(n - v) = 166\,666\,666\,650\,000 \approx 1.7 \cdot 10^{14}$; use `long`.

**Exercise 18.6.** mx[$k$][$v$] is the largest weight among the $2^k$ edges above $v$ (fewer if the root comes first; the root's own entry is 0). mx[0][$v$] = $w$[$v$], and mx[$k$][$v$] = max(mx[$k - 1$][$v$], mx[$k - 1$][up[$k - 1$][$v$]]), filled in BFS order next to up. In the query, every jump of $a$ by $2^k$ takes max with mx[$k$][$a$] *before* moving $a$; in the second phase each jump takes mx[$k$][$a$] and mx[$k$][$b$]; at the end add mx[0][$a$] and mx[0][$b$], the two edges into the LCA (unless $a = b$ after the first phase). Build $O(n \log n)$, query $O(\log n)$, memory $2 \cdot 17 \cdot 10^5$ `int`s, about 13.6 MB. For Figure 18.3, mx[1] = [0, 4, 7, 4, 9, 7, 6, 2, 5]. The answers: (6, 4) = 9 on the path 6–3–1–4 (weights 6, 2, 9); (7, 8) = 7 on 7–3–1–0–2–5–8 (weights 1, 2, 4, 7, 3, 5); (8, 2) = 5 on 8–5–2. A harness compared the lifting answers with a walk along the path on 60 000 random queries.

**Exercise 18.7.** (a) A chain, par[$v$] = $v - 1$, with $n = 10^5$: `fill` recurses $10^5$ deep and throws `StackOverflowError`. On the JDK used for this book it already failed on a chain of $10^4$ vertices, while a star of $10^5$ vertices (depth 1) worked. (b) par = [−1, 2, 0]: vertex 2 is the parent of 1. The index loop handles 2 first and adds size[2] = 1 to size[0]

before vertex 1 has been added to size[2], so it returns [2, 1, 2] instead of [3, 1, 2]. (c) Build the BFS order and walk it backwards, as in `subtreeSizes` of *Theory Listing 18.3*.

**Exercise 18.8.** (1) Binary lifting, *Theory Listing 18.8*: $17 \cdot 10^5 = 1.7 \cdot 10^6$ table entries to build, and at most $2 \cdot 17$ jumps per query, about $3.4 \cdot 10^6$ in total. (2) Without a table: equalise the depths and climb both vertices one step at a time, at most $10^5$ steps per query, $10^6$ in all. Depths still come from the BFS order. (3) Subtree sizes in one backward pass, *Theory Listing 18.9*: O($n$). The total reaches $1.7 \cdot 10^{14}$ on a chain, and one product `size[v] * (n - size[v])` can reach $2.5 \cdot 10^9$, so cast to `long`. (4) Recursion, *Theory Listing 18.1*: $10^4$ calls. LeetCode's tests for these problems pass with the recursive templates; do not carry that habit over to parent arrays. (5) Entry and exit times from one iterative DFS (*Theory Listing 18.6*), O($n$), then O(1) per query with $\text{tin}[u] \le \text{tin}[v] \le \text{tout}[u]$. (6) 26 root-prefix counts per vertex ($2.6 \cdot 10^5$ integers) plus an LCA per query (*Theory Theorem 18.14*): $2 \cdot 10^4 \cdot (26 + 28) \approx 1.1 \cdot 10^6$ steps. (7) `parentsFromEdges` (*Theory Listing 18.5*) and the forward depth pass: O($n$), no recursion.

**Exercise 18.9.** Compare with *Theory Listing 18.4* and *Theory Listing 18.3*. The points that matter: insert children while walking $v$ from $n - 1$ down to 0 so siblings come out in increasing order, use `order` itself as the queue, and in the size loop stop at $i = 1$ because `order[0]` is the root.

## Chapter 19 · Trees II: Tree DP, Rerooting and Small-to-Large

**Exercise 19.1.** (1) `Arrays.fill(size, 1);` (2) `n - 1;` (3) `down[v] + size[v];` (4) `ans[order[0]] = down[order[0]];` (5) `ans[par[v]] - size[v] + (n - size[v])`. Removing the edge between $v$ and its parent splits the tree into the same two parts whatever the root is: the subtree of $v$ (rooted at `order[0]`) and the rest. The formula only needs the size of the part on $v$'s side, so the original size[$v$] is correct (*Theory Theorem 19.5*).

**Exercise 19.2.**

| v | w[v] | take[v] | skip[v] | adds to take[par] | adds to skip[par] |
|---|---|---|---|---|---|
| 7 | 3 | 3 | 0 | 0 | 3 |
| 6 | 2 | 2 | 0 | 0 | 2 |
| 8 | 4 | 4 | 0 | 0 | 4 |
| 5 | 6 | 6 | 5 | 5 | 6 |
| 4 | 1 | 1 | 0 | 0 | 1 |
| 3 | 4 | 4 | 0 | 0 | 4 |
| 2 | 5 | 10 | 10 | 10 | 10 |
| 1 | 2 | 2 | 5 | 5 | 5 |
| 0 | 3 | 18 | 15 | — | — |

The best weight is max(18, 15) = 18. Walking forwards: take 0 (18 > 15); skip 1 and 2 because 0 is taken; 3, 4, 5 and 8 have skipped parents and take > skip, so take them; 6 and 7 are skipped because 5 is taken. The set is {0, 3, 4, 5, 8}, weight 3 + 4 + 1 + 6 + 4 = 18, and a brute force over all $2^9$ subsets finds it as the only optimum. Vertex 2 has take = skip = 10, but its parent is taken, so it must be skipped anyway.

**Exercise 19.3.**

| pass 1: v | 8 | 7 | 6 | 5 | 3 | 2 | 4 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| size[v] | 1 | 1 | 3 | 1 | 1 | 1 | 5 | 3 | 9 |
| down[v] | 0 | 0 | 2 | 0 | 0 | 0 | 6 | 2 | 16 |

| pass 2: v | 0 | 1 | 4 | 2 | 3 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| parent | — | 0 | 0 | 1 | 1 | 4 | 4 | 6 | 6 |

| pass 2: v | 0 | 1 | 4 | 2 | 3 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| ans[v] | 16 | 19 | 15 | 26 | 26 | 22 | 18 | 25 | 25 |

For example ans[2] = 19 − 1 + 8 = 26 and ans[6] = 15 − 3 + 6 = 18. Vertex 4 has the smallest sum, 15: its distances to 0, 1, 2, 3, 5, 6, 7, 8 are 1, 2, 3, 3, 1, 1, 2, 2. A BFS from every vertex gives the same nine sums.

**Exercise 19.4.**

| v | key | own | map of v after adding its key | handed / merged | pairs[par] after |
|---|---|---|---|---|---|
| 8 | 5 | 0 | {5:1} | handed to 4 | pairs[4] = 0 |
| 7 | 7 | 0 | {7:1} | merged into 4: its 1 entry moves, cross 0 | pairs[4] = 0 |
| 6 | 5 | 0 | {5:1} | handed to 2 | pairs[2] = 0 |
| 5 | 9 | 0 | {9:1} | merged into 2: its 1 entry moves, cross 0 | pairs[2] = 0 |
| 4 | 5 | 1 | {5:2, 7:1} | handed to 1 | pairs[1] = 1 |
| 3 | 8 | 0 | {8:1} | merged into 1: its 1 entry moves, cross 0 | pairs[1] = 1 |
| 2 | 5 | 1 | {5:2, 9:1} | handed to 0 | pairs[0] = 1 |
| 1 | 7 | 1 | {5:2, 7:2, 8:1} | merged into 0: **swap**, 0's 2 entries move, cross 2 · 2 = 4 | pairs[0] = 1 + 2 + 4 = 7 |
| 0 | 5 | 4 | {5:5, 7:2, 8:1, 9:1} | root | — |

pairs = [11, 2, 1, 0, 1, 0, 0, 0, 0]. Vertex 4 closes a pair with 8 (own = 1), and vertex 1 closes one with 7, so pairs[4] = 1 and pairs[1] = 1 + 1 = 2. The swap happened at the last merge: vertex 1's map had 3 entries and vertex 0's map (handed up from 2) had 2, so 2 entries moved instead of 3. The cross count 4 is the pairs between {2, 6} and {4, 8}, the vertices with key 5 on each side. Directly at the root: key 5 occurs at 0, 2, 4, 6, 8, which gives 10 pairs, and key 7 at 1 and 7 gives 1 more, 11 in all. The listing matched a brute force over all pairs on 4000 random trees.

**Exercise 19.5.** (1) False: if $v$ is taken, no child may be, so take[$v$] adds skip[$c$]; the max belongs in skip[$v$] (*Theory Theorem 19.1*). (2) False: ans[$v$] is computed from ans[par[$v$]], which must already be final, and only the forward walk guarantees that. (3) False: `gain` starts at 0 and is only ever raised by `Math.max`, which encodes "match $p$ to no child" as the option of gaining 0. (4) False: on the two-vertex tree 0–1 the longest chain from 0 goes through 1 itself, so up[1] becomes 1 + 1 = 2, and ecc[1] = 2 instead of 1. (5) False: the answers stay correct, but on a caterpillar every step copies the long set and the time becomes quadratic (the pitfall after *Theory Listing 19.6*). (6) True: the first leaf hands its set over, and each of the other 8 leaves moves one entry. A count in code gives 8. (7) True: by *Theory Lemma 19.9*, exactly the even-sized sets can be flipped. {1, 3} comes from operating on the edges 1–2 and 2–3, and {0, 1, 2} has odd size. (8) True: $2.6 \cdot 10^6$ `int`s are 10.4 MB, plus about 1.6 MB of row headers.

**Exercise 19.6.** Replace size[$v$] by $W[v]$, the number of parcels in the subtree of $v$, with total $W = W[\text{root}]$. Let down[$v$] = $\Sigma\, w[x] \cdot \text{dist}(v, x)$ over the subtree of $v$. Pass 1 (backwards): $W[p]$ += $W[v]$ and down[$p$] += down[$v$] + $W[v]$, because every parcel below $v$ travels one more road to reach $p$. Pass 2 (forwards): ans[root] = down[root] and ans[$v$] = ans[$p$] − $W[v]$ + ($W$ − $W[v]$): moving the warehouse from $p$ to $v$ saves one road for the $W[v]$ parcels on $v$'s side and adds one for the rest. Root the edge list with `parentsFromEdges` first; then both passes are $O(n)$ time and memory. A cost can reach about $10^4 \cdot 10^5 \cdot 10^5 = 10^{14}$, so `long` is needed. For the example, $W$ = [14, 4, 9, 1, 1, 9, 5] and down = [29, 2, 14, 0, 0, 5, 0], and the costs are ans = [29, 35, 25, 47, 47, 21, 25]. The best town is 5, with cost 21. A BFS from every town gives the same costs, and the method matched that brute force on 4000 random weighted trees.

**Exercise 19.7.** After the swap, `big` may be the child's set, but `set[p]` still points at the parent's old, smaller set, and the merged contents are lost. Take par = [−1, 0, 0, 1] with colours [1, 2, 3, 4]. Vertex 3 hands {4} to 1, and vertex 2 hands {3} to 0. Then vertex 1 has the set {2, 4}, which is larger than 0's set {3}, so the references swap and {3} is poured into {2, 4}; but `set[0]` stays {3}. The method returns [2, 2, 1, 1] instead of [4, 2, 1, 1]. Fix: add `set[p] = big;` after `big.addAll(small);`, as in *Theory Listing 19.6*. On 3000 random trees with up to 10 vertices the buggy method was wrong on 905.

**Exercise 19.8.** (1) Small-to-large sets, *Theory Listing 19.6*: at most about $n \log_2 n \approx 1.7 \cdot 10^6$ moves (*Theory Theorem 19.7*). (2) Rerooting, *Theory Listing 19.4*: two passes of $2 \cdot 10^5$ steps. Sums reach 19 999 900 000 (from the end of a chain), more than an `int` holds, so use `long`. (3) Count arrays merged upwards, $26 \cdot 10^5 = 2.6 \cdot 10^6$ additions and 10.4 MB; or *Theory Theorem 19.3* with one global array of 26 counters and O($n$) memory. (4) The two-state tree DP, *Theory Listing 19.1*: O($n$). (5) No: $(3 \cdot 10^4)^2 = 9 \cdot 10^8$ steps, far over the budget; rerooting does it in O($n$). (6) The tree drops out (*Theory Lemma 19.9*): the parity greedy of *Theory Theorem 19.10* in O($n$), with a `long` sum. (7) Rerooting with the two best chains, *Theory Listing 19.5*: three O($n$) loops.

**Exercise 19.9.** Compare with *Theory Listing 19.7*. The points that matter: add the vertex's own key (and its `same` pairs) before the hand-over, pass `pairs[v]` up in both branches, swap the references when the parent's map is smaller, add `have * e.getValue()` to `cross` before updating `big`, store `big` in `mp[p]`, and set `mp[v] = null`.

## Chapter 20 · Fenwick Trees and Segment Trees

**Exercise 20.1.** (1) `n + 1`; (2) `i < t.length`; (3) `i & -i`; (4) `i > 0`; (5) `sum(r) - sum(l - 1)`. Node $i$ covers ($i$ − lowbit($i$), $i$]. Subtracting lowbit jumps to the node whose range ends just before this one starts, so the query tiles 1..$i$ from the right. Adding lowbit jumps to the next, larger range that still contains the position, so the update reaches every node that must change (*Theory Lemma 20.3*).

**Exercise 20.2.**

| i | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| lowbit(i) | 1 | 2 | 1 | 4 | 1 | 2 | 1 | 8 | 1 | 2 | 1 | 4 |
| range | 1..1 | 1..2 | 3..3 | 1..4 | 5..5 | 5..6 | 7..7 | 1..8 | 9..9 | 9..10 | 11..11 | 9..12 |
| t[i] | 2 | 9 | 1 | 18 | 2 | 10 | 1 | 37 | 2 | 10 | 4 | 19 |

| call | nodes visited (lowbit) | values read or changed |
|---|---|---|
| add(6, −3) | 6 (2), 8 (8); next 16 > 12 stops | t[6]: 10 → 7, t[8]: 37 → 34 |
| sum(11) | 11 (1), 10 (2), 8 (8) | 4 + 10 + 34 = 48 |
| sum(4) | 4 (4) | 18 |

sum(5, 11) = 48 − 18 = 30. Directly, with $a$[6] now 5: 2 + 5 + 1 + 8 + 2 + 8 + 4 = 30. Building the tree with 12 calls of `add` gives the same $t$ as *Theory Listing 20.2*.

**Exercise 20.3.**

| l | r | l odd? | r odd? | resL | resR |
|---|---|---|---|---|---|
| 11 | 18 | yes: take t[11] = 3 | no | 3 | ∞ |
| 6 | 9 | no | yes: take t[8] = 4 | 3 | 4 |
| 3 | 4 | yes: take t[3] = 2 | no | 2 | 4 |
| 2 | 2 | loop ends | | | |

The answer is min(2, 4) = 2; directly, the minimum of 3, 8, 5, 9, 2, 7, 4 is 2. Node 8 covers positions 6..7 and node 3 covers positions 2..5. Node 2 combines node 4 (positions 6, 7, 8, 9) with node 5 (positions 0, 1), so its positions are 6, 7, 8, 9, 0, 1: not contiguous in order. By the invariant of *Theory Theorem 20.7* a query uses a node only when the node's positions are the next ones of the range in order, which never holds for node 2. The build computes node 2 (and node 1, which is out of order too), but no query reads them.

**Exercise 20.4.** The children lists are 2: [3, 4], 3: [0, 1], 4: [5, 7] and 1: [6], so the tour visits 2, 3, 0, 1, 6, 4, 5, 7.

| vertex v | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| tin[v] | 2 | 3 | 0 | 1 | 5 | 6 | 4 | 7 |
| tout[v] | 2 | 4 | 7 | 4 | 7 | 6 | 4 | 7 |

| position p | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| vertex with tin = p | 2 | 3 | 0 | 1 | 6 | 4 | 5 | 7 |

subtreeSum(3) covers positions 1..4, the vertices 3, 0, 1, 6: 2 + 5 + 1 + 7 = 15. subtreeSum(4) covers positions 5..7, the vertices 4, 5, 7: 6 + 3 + 2 = 11. 3 is an ancestor of 6 because $1 \le 4 \le 4$. 4 is not an ancestor of 6 because tin[6] = 4 < tin[4] = 5. `set(1, 10)` adds 9 at Fenwick position tin[1] + 1 = 4 (nodes 4 and 8), and subtreeSum(3) becomes 24.

**Exercise 20.5.** (1) False: a maximum has no inverse, so two prefix answers do not give a range answer, and a Fenwick tree of maxima cannot lower a value. (2) True: the proof of *Theory Theorem 20.7* uses only the sibling pairs $(2p, 2p + 1)$, never a power of two. (3) False: gcd(1, $x$) = 1 for every $x$; the identity is 0, because gcd(0, $x$) = $x$. (4) True: 65 535 = $2^{16} - 1$ has sixteen 1-bits, one node per bit. (5) False: for an odd $i$ it puts the right child first, and the merge is not commutative; in tests it gave wrong answers on 428 of 2,000 random arrays. (6) False: $v$ must also satisfy tin[$v$] ≤ tout[$u$]; in *Theory Figure 20.3*, tin[4] = 5 < tin[2] = 6, but 4 is a leaf. (7) True: min(`Long.MAX_VALUE`, $x$) = $x$, so the position never changes a combination.

**Exercise 20.6.** A node stores (*min*, *max*, *best*): the lowest price, the highest price and the best profit inside the segment. A single day $x$ is ($x$, $x$, 0). The merge of $A$ (left) with $B$ (right) is *min* = min($A$.*min*, $B$.*min*), *max* = max($A$.*max*, $B$.*max*), *best* = max($A$.*best*, $B$.*best*, $B$.*max* − $A$.*min*): the best trade either stays on one side or buys in $A$ and sells in $B$. It is **not** commutative, because buying must come before selling, so the update must recompute parents as `merge(t[2 * i], t[2 * i + 1])` and the query must keep `resL` and `resR` in order, as in *Theory Listing 20.7*. The identity is (+∞, −∞, 0), with ∞ taken as `Long.MAX_VALUE / 4` so that $B$.*max* − $A$.*min* cannot overflow when one side is the identity. Build O($n$), each operation O(log $n$). On $a$ = [7, 1, 5, 3, 6, 4, 2, 8]: the left half is (1, 7, 4), the right half is (2, 8, 6) and the root is (1, 8, 7), where 7 = 8 − 1 crosses the middle. The query for $a$[1..5] = [1, 5, 3, 6, 4] gives 5 (buy at 1, sell at 6). After $a$[1] = 9 the whole-array answer is 6 (buy at 2, sell at 8). This design matched a brute force on 29,812 random queries.

**Exercise 20.7.** When $r$ is odd, the node to take is $r - 1$ (the last node inside the half-open range), but `t[r--]` reads node $r$, which lies outside, and only then decrements. On [1, 2, 3, 4], `querySum(t, 4, 0, 3)` returns 11 instead of 6, and `querySum(t, 4, 1, 3)` returns 6 instead of 5. `query(0, 4)` passes because $r$ stays even at every level. Fix: `resR = t[--r] + resR;`. On 2,000 random queries the buggy version was wrong 685 times.

**Exercise 20.8.** (1) Prefix sums (Chapter 2): O($n + q$), about $2 \cdot 10^5$ steps. (2) A Fenwick tree with a copy of the array, adding the difference on each assignment: about $(n + q) \cdot 17 \approx 3.4 \cdot 10^6$ steps (or $n + 17q$ with *Theory Listing 20.2*). (3) The iterative segment tree with `Math.min`: $2 \cdot 10^5$ operations of at most $2 \cdot 18 = 36$ node combinations each, about $7.2 \cdot 10^6$, plus an O($n$) build. (4) One Fenwick tree on the difference array: two `add`s per update, one `sum` per query, O(log $n$) each. (5) Compression plus a Fenwick tree over ranks (*Theory Listing 20.4*): about $10^5 \cdot 17 \approx 1.7 \cdot 10^6$ steps, against $5 \cdot 10^9$ pairs for the brute force. Use `long` for the count. (6) The iterative Euler tour plus a Fenwick tree on entry times (*Theory Listing 20.8*); a recursive DFS on a path of $10^5$ vertices risks a `StackOverflowError`. (7) Neither structure of this chapter: range additions with range minimum need the lazy segment tree of Chapter 21.

**Exercise 20.9.** Compare with *Theory Listing 20.5*. For a maximum tree, `ID = Long.MIN_VALUE` and `op` returns `Math.max(a, b)`. The places that usually go wrong: the leaves are at `n + i`; the build loop runs from `n - 1` down to 1; `set` stops when `i > 1` fails; and the right end uses `t[--r]` with the node placed in front of `resR`.

## Chapter 21 · Lazy Segment Trees and the K-Parts DP

**Exercise 21.1.** (1) `mx[c] += lz[x];` (2) `lz[c] += lz[x];` (3) `lz[x] = 0;` (4) `l <= lo && hi <= r;` (5) `push(x);` (6) `mx[x] = Math.max(mx[2 * x], mx[2 * x + 1]);`. Without the push, line (6) rebuilds `mx[x]` from children that do not yet include the pending tag of $x$, so the tag disappears from `mx[x]` while `lz[x]` still holds it: the stored value is then too small by `lz[x]`.

**Exercise 21.2.** From a run of an instrumented copy of the listing:

| node | after add(1, 6, 2) | after add(0, 3, 1) | after max(2, 5) |
|---|---|---|---|
| [0,7] | 2 / 0 | 3 / 0 | 3 / 0 |
| [0,3] | 2 / 0 | 3 / 1 | 3 / 0 |
| [4,7] | 2 / 0 | 2 / 0 | 2 / 0 |
| [0,1] | 2 / 0 | 2 / 0 | 3 / 1 |
| [2,3] | 2 / 2 | 2 / 2 | 3 / 3 |
| [4,5] | 2 / 2 | 2 / 2 | 2 / 2 |
| [6,7] | 2 / 0 | 2 / 0 | 2 / 0 |
| covered nodes | [1], [2,3], [4,5], [6] | [0,3] | [2,3] → 3, [4,5] → 2 |
| calls | 11 | 3 | 7 |

The query returns 3. The array is then [1, 3, 3, 3, 2, 2, 2, 0]. The first update also writes tags on the leaves [1] and [6]; they are never read. After the second update, [2,3] stores 2 because the +1 sits in the tag of its parent [0,3], which covers it; the query max(2, 5) finds [0,3] partial and pushes the +1 into [0,1] and [2,3] before reading [2,3].

**Exercise 21.3.** Rows 4–7, from a run of the method:

| i | a[i − 1] | +1 on j = | T[0..7] after the add | dp[2][i] |
|---|---|---|---|---|
| 4 | 3 | 0..3 | −∞, 4, 4, 3, 3, 3, 3, 3 | 4 |
| 5 | 1 | 2..4 | −∞, 4, 5, 4, 4, 3, 3, 3 | 5 |
| 6 | 3 | 4..5 | −∞, 4, 5, 4, 5, 4, 3, 3 | 5 |
| 7 | 1 | 5..6 | −∞, 4, 5, 4, 5, 5, 4, 3 | 5 |

The answer is 5. At $i = 7$, three split points reach 5: $j = 2$ gives [2, 1] + [2, 3, 1, 3, 1] = 2 + 3; $j = 4$ gives [2, 1, 2, 3] + [1, 3, 1] = 3 + 2; $j = 5$ gives [2, 1, 2, 3, 1] + [3, 1] = 3 + 2. Row 5 adds only on 2..4 because the previous 1 is at index 1.

**Exercise 21.4.** (1) False: a node misses the tags of its ancestors; in *Theory Figure 21.1*, [4,5] stores 3 but $a$[4..5] has maximum 5. (2) False: with `n = 2`, add(0, 1, 5) then max(0, 0) returns 0 without the push (Exercise 21.7). (3) True: up to four calls per level, two of them partial (*Theory Theorem 21.4*); add(3, 12, 1) on 16 elements has four calls on each of the levels 2, 3 and 4. (4) False: $n = 6$ already uses index 13 ≥ 12. (5) True: dp[$k$][$i$] reads only row $k − 1$. (6) True: every box then has as many distinct values as elements, so every split totals $n$ (confirmed on 2000 random permutations). (7) False: position `last[x]` already contains a copy of $x$ and must not gain 1. In a random test this version disagreed with the baseline on 340 of 1000 cases; one failing case is [3, 1, 1, 1, 1, 2, 1, 3, 3, 3] with $K = 1$, where it returns 4 instead of 3.

**Exercise 21.5.** (1) “set 7” (add 2, then set 4; add 3 makes set 7; set 1; add 6 makes set 7); the maximum is 7. A run of the class gave tag 7 with `isSet` true and max 7. (2) 5 and 6: the two tags do not commute. (3) Tag +7 and sum 7 · 6 = 42. (4) “set 5”: the root’s tag is newer, and set overrides the older add 7; the query returns 5.

**Exercise 21.6.** For $j \le p_2$, $x$ already occurred at least twice: $\Delta = 0$. For $p_2 < j \le p_1$, $x$ occurred exactly once and now twice: $\Delta = -1$. For $p_1 < j \le i - 1$, $x$ is new: $\Delta = +1$. So each step is two range adds, +1 on $[p_1 + 1, i - 1]$ and −1 on $[p_2 + 1, p_1]$ (only when $p_1 \ge 0$), followed by one range max. The tree stores dp[$k - 1$][$j$] + score($a$[$j..i - 1$]); per value you keep the last two positions. Time $O(K\, n \log n)$, memory $O(n)$. For $a$ = [1, 2, 1, 1, 3, 2] at $i = 4$: $p_1 = 2$, $p_2 = 0$, so +1 on $j$ = 3..3 and −1 on $j$ = 1..2 (a direct count gives the changes 0, −1, −1, +1 for $j$ = 0..3). The answer for $K = 2$ is 4, from [1, 2] + [1, 1, 3, 2] = 2 + 2 or [1, 2, 1] + [1, 3, 2] = 1 + 3. This design was checked against an exhaustive search on 4000 random arrays.

**Exercise 21.7.** `max` never pushes. With `n = 2`: add(0, 1, 5) covers the root, which stores 5 with tag 5; max(0, 0) finds the root partial, goes to leaf [0] without pushing and returns 0 instead of 5. Fix: call `push(x);` in `max` after

the covered check, before computing `mid`. The bug hides when only leaves carry tags, as with the point updates of Chapter 20 (3000 of 3000 random runs correct), and when every query asks for the whole array (also 3000 of 3000). In the sample, the two whole-array queries stop at the root, and the query max(3, 4) passes only through nodes that hold no pending tag, so the missing push changes nothing.

**Exercise 21.8.** (1) Lazy tree per layer, *Theory Listing 21.6*: at most $50 \cdot (3 \cdot 35\,000 + 1) \cdot 63 \approx 3.3 \cdot 10^8$ calls (about 0.6 s measured); the baseline would make about $3.06 \cdot 10^{10}$ steps. (2) Baseline *Theory Listing 21.5*: 1 909 600 inner steps. (3) Difference array (Chapter 2): $O(n + m) \approx 2 \cdot 10^5$ steps. (4) *Theory Listing 21.2*: at most 67 calls per operation for $n = 10^5$, so about $1.34 \cdot 10^7$ calls. (5) Lazy **min** tree per layer with the range add of *Theory Section 21.7*: at most $100 \cdot (3 \cdot 35\,000 + 1) \cdot 63 \approx 6.6 \cdot 10^8$ calls. (6) Try all $2^{19} = 524\,288$ ways to place the cuts, about $10^7$ steps with an $O(n)$ cost per split, and use it as the brute force in a stress test.

**Exercise 21.9.** Compare with *Theory Listing 21.1*. `push` is called in exactly two places: in `add` and in `max`, after the covered check and before the recursive calls. The arrays have $4n$ entries.

## Chapter 22 · Backtracking, Bitmask DP and Digit DP

**Exercise 22.1.** (1) `dp[0] = 0`; (2) `Integer.bitCount(mask)`; (3) `(mask & (1 << job)) != 0`; (4) `mask | (1 << job)`; (5) `dp[(1 << n) - 1]`. Increasing order is valid by *Theory Theorem 22.4*: every transition adds a bit, and a proper superset has a larger number, because adding disjoint bits produces no carries.

**Exercise 22.2.**

| node | chosen | remain | what happens |
|---|---|---|---|
| 1 | {} | 8 | tries 1, 3, 4, 6, 7 |
| 2 | {1} | 7 | tries 3, 4, 6, 7 |
| 3 | {1,3} | 4 | tries 4; stops at 6 > 4 |
| 4 | {1,3,4} | 0 | solution |
| 5 | {1,4} | 3 | stops at 6 > 3 |
| 6 | {1,6} | 1 | stops at 7 > 1 |
| 7 | {1,7} | 0 | solution |
| 8 | {3} | 5 | tries 4; stops at 6 > 5 |
| 9 | {3,4} | 1 | stops at 6 > 1 |
| 10 | {4} | 4 | stops at 6 > 4 |
| 11 | {6} | 2 | stops at 7 > 2 |
| 12 | {7} | 1 | no values left |

The search prints [1, 3, 4] and [1, 7] and visits 12 nodes; without the `break` it would visit all $2^5 = 32$ subsets. The `break` is safe because the array is sorted and positive: once $a[i] >$ `remain`, every later value is larger too, so no extension can reach the target.

**Exercise 22.3.**

| mask | j = 0 | j = 1 | j = 2 | j = 3 | best |
|---|---|---|---|---|---|
| 0110 | – | (1,6) | (1,6) | – | (1,6) |
| 0111 | (2,5) | (2,6) | (2,4) | – | (2,4) |
| 1000 | – | – | – | (1,3) | (1,3) |
| 1001 | (2,5) | – | – | (2,3) | (2,3) |
| 1010 | – | (1,5) | – | (1,5) | (1,5) |

| mask | j = 0 | j = 1 | j = 2 | j = 3 | best |
|---|---|---|---|---|---|
| 1011 | (2,5) | (2,5) | – | (2,3) | (2,3) |
| 1100 | – | – | (1,7) | (1,7) | (1,7) |
| 1101 | (2,5) | – | (2,7) | (2,7) | (2,5) |
| 1110 | – | (2,2) | (2,4) | (2,3) | (2,2) |
| 1111 | (2,7) | (2,7) | (2,7) | (2,7) | (2,7) |

The fewest sessions is 2, for example {5, 2} and {4, 3}; a backtracking oracle agrees. Row 1101 shows the second component at work: all three candidates use two sessions, and (2, 5), "4 + 3 first, then 5", wins.

**Exercise 22.4.**

| position | digit of N | allowed smaller digits | free completions | contribution | tight path continues? |
|---|---|---|---|---|---|
| 0 | 5 | 0, 1, 2, 4 (4 digits) | $9^3 = 729$ | 2916 | yes, with 5 |
| 1 | 3 | 0, 1, 2 (3 digits) | $9^2 = 81$ | 243 | no: the digit 3 is forbidden |
| 2 | 7 | – | – | – | – |
| 3 | 2 | – | – | – | – |
| end | | | | *N* itself: 0 | |

So $f(5372) = 2916 + 243 = 3159$: the tight path dies at position 1, because every $x$ that keeps the prefix 53 contains a 3, and so 5372 itself is not counted. For 999 every position allows 8 smaller digits: $8 \cdot 81 + 8 \cdot 9 + 8 \cdot 1 + 1 = 648 + 72 + 8 + 1 = 729$ (which is $9^3$, as it must be: all three-digit strings over the 9 allowed digits). The count in [1000, 5372] is $3159 - 729 = 2430$. A direct count by computer gives the same three numbers.

**Exercise 22.5.** (1) False: a skipped value creates no node either way; `continue` only makes the loop run longer (on this input both visit 12 nodes). (2) True: it has $\lfloor e \cdot 10! \rfloor = 9{,}864{,}101$ nodes. (3) False: $011 = 3 < 100 = 4$, but 011 is not a submask of 100. The converse holds (*Theory Theorem 22.4*). (4) False: dp[*mask*] is the minimum over all ways to assign the first $|mask|$ workers to those jobs; the order is not part of the state. (5) False: the loop stops when $s$ becomes 0, so the empty submask is never visited. (6) True: keyed by (*pos*, *rem*, tight) the table is correct, but it saves nothing, because each tight state is reached once. (7) True: with $L = 0$ the formula reads $f(R) - f(-1)$, and the count of $[0, -1]$, the empty range, is 0.

**Exercise 22.6.** State: dp[*mask*] = the least cost of pairing up exactly the students in *mask*. Transition: let $i$ be the lowest student not in *mask*; for every other $j$ not in *mask*, $\text{dp}[mask \cup \{i, j\}] \le \text{dp}[mask] + c[i][j]$. Base dp[0] = 0, increasing masks, answer $\text{dp}[2^n - 1]$. Student $i$ must be paired with someone, and pairing them now loses nothing, because the order in which the pairs are formed does not change the cost; insisting on the lowest one builds each pairing exactly once instead of in every order. Time $O(2^n \cdot n)$, about $2^{20} \cdot 20 \approx 2.1 \cdot 10^7$ steps at most, against $19 \cdot 17 \cdot \cdots \cdot 1 = 654{,}729{,}075$ ways to split 20 students into pairs. For the instance, the best pairing is 0–1, 2–4, 3–5 with cost 4 + 3 + 2 = 9; the 15 pairings of 6 students cost between 9 and 23, and the DP and a brute force both return 9.

**Exercise 22.7.** For $N = 23$ and $D = 2$ it returns 15; the right answer is 12 (0, 2, 4, 6, 8, 11, 13, 15, 17, 19, 20, 22). Every $N$ from 20 to 27 fails. The memo is shared by tight and free states. At position 0 the free branches for the first digits 0 and 1 store the counts of (1, 0) and (1, 1), five each; the tight branch with first digit 2 reaches the key (1, 0) and reads 5, although only 20 and 22 remain below 23. For $N$ from 10 to 19 the tight branch reaches the key (1, 1), which no free branch has stored yet, and when $N$ ends in 9 the tight count equals the free count; that is why 0..19 and 99 pass. Fix: read and write the memo only when `tight` is false, as in *Theory Listing 22.7*: `if (!tight && memo[pos][rem] != -1) ...` and `if (!tight) memo[pos][rem] = res;`.

**Exercise 22.8.** (1) Backtracking over permutations, pruning an item that differs by 1 from the last one: at most $\lfloor e \cdot 8! \rfloor = 109{,}601$ nodes. (2) Assignment DP, *Theory Listing 22.3*: $20 \cdot 2^{19} = 10{,}485{,}760$ relaxations and 4 MB. (3) dp[*mask*][*last*], *Theory Listing 22.4*: $2^{18} \cdot 18^2 \approx 8.5 \cdot 10^7$ steps. A path has at most 17 edges of at most $10^6$, so $1.7 \cdot 10^7$ fits in an

`int`; the table takes $2^{18} \cdot 18 \cdot 4$ bytes ≈ 18.9 MB. (4) Submask DP as in *Theory Listing 22.6*, with `ok` in place of `indep`: $3^{16} \approx 4.3 \cdot 10^7$ steps, after computing `ok` once for all $2^{16}$ groups. (5) Digit DP with the state (position, $x$ mod 7, digit sum mod 5, tight): about $19 \cdot 7 \cdot 5 \cdot 10 \approx 6{,}650$ digit steps per call, and $f(R) - f(L-1)$. (6) None of this chapter's methods: $2^{40} \approx 1.1 \cdot 10^{12}$ subsets is far too many. Meet in the middle, from the "Complete search" chapter of the *Competitive Programmer's Handbook*, splits the values into two halves of $2^{20}$ subsets each.

**Exercise 22.9.** Compare with *Theory Listing 22.5*. The places that matter: `cnt[0] = 1` with `last[0] = 0`, the new session `c++; t = task[j];` when `t > T`, and the comparison `c < cnt[mask] || (c == cnt[mask] && t < last[mask])`. Without its second part the method returns 3 instead of 2 for [1, 3, 2, 2, 2] and $T = 5$.

## Chapter 23 · Four Problems, Partial Marks and Mock 3

**Exercise 23.1.** The ratios are A 0.6, B 0.45, C 0.5, D 0.7 and E 0.2, so the ratio order is D, A, C, B, E with finishing times 15, 45, 90, 110 and 170. Σ $w \cdot C$ is 6022.5 for the ratio order and 6322.5 for A, B, C, D, E (finishing times 30, 50, 95, 110, 170); a brute force over all 120 orders confirms that 6022.5 is the minimum. Secured by minute 60: 28.5 against 27; by minute 90: 51 against 27; by minute 120: 60 in both. With 150 working minutes, E (ending at 170) is unfinished in both orders. The lemma does decide D and B: D is shorter ($15 \le 20$) and worth more ($10.5 \ge 9$), so D first is never behind at any moment.

**Exercise 23.2.** Tier 1, no restriction beyond small sizes: walk up $k$ parents per query, O($q \cdot$ depth); for $n, q \le 3000$ that is at most $3000 \cdot 3000 = 9 \cdot 10^6$ steps. Detect it from $n$ and $q$. Tier 2, the tree is a path (par[$v$] = $v - 1$ for every $v \ge 1$, checked in one O($n$) loop): the ancestors of $u$ are $u - 1$, ..., $u - k$, so prefix sums answer each query in O(1). Tier 3, the full solution: sums from the root (Chapter 18) give the sum of a vertical path as rootSum[$u$] − rootSum[parent of the $k$-th ancestor], and binary lifting finds the $k$-th ancestor in O(log $n$); about $(n + q) \cdot 18 \approx 7.2 \cdot 10^6$ steps for $n = q = 2 \cdot 10^5$. Sums need `long` (up to $2 \cdot 10^5$ values of $10^9$). In a mock, write tier 1 first: about ten lines, nearly certain, and it is the oracle for testing the others; then tier 3 if the reading-phase estimate gives it a good ratio, or tier 2 if binary lifting is shaky. Walking without a restriction at the full limits would be up to $2 \cdot 10^5 \cdot 2 \cdot 10^5 = 4 \cdot 10^{10}$ steps.

**Exercise 23.3.** With par = [−1, 2, 0] (vertex 2 hangs below the root, vertex 1 below vertex 2) the loop handles $v = 2$ before $v = 1$, so vertex 1's size reaches vertex 2 after vertex 2 has already been added to the root. It returns [2, 1, 2]; the correct sizes are [3, 1, 2]. The check `check(size[0] == n, "root size " + size[0] + " != n")` after the loop catches it: in 20,000 random trees rooted at 0 with shuffled labels, the method was wrong on 13,269 and the check fired on all 13,269. A check `par[v] < v` for every $v$ fires on exactly the same trees (all 20,000 agreed): when a parent is numbered above its child, it is processed first, and the child's size never reaches the root. Fix: add sizes in reverse BFS order (Chapter 18), which finishes every child before its parent whatever the numbering; the fixed method matched a brute force on 20,000 random trees.

**Exercise 23.4.**

| Try | Leaf deleted | Parent array after deletion | Buggy | True | Kept? |
|---|---|---|---|---|---|
| 1 | 7 | [−1, 0, 1, 1, 3, 1, 2] | 3 | 4 | yes |
| 2 | 6 | [−1, 0, 1, 1, 3, 1] | 3 | 3 | no |
| 3 | 5 | [−1, 0, 1, 1, 3, 2] | 3 | 4 | yes |
| 4 | 5 | [−1, 0, 1, 1, 3] | 3 | 3 | no |
| 5 | 4 | [−1, 0, 1, 1, 2] | 3 | 3 | no |

The start has buggy 4 and true 5. The final tree is [−1, 0, 1, 1, 3, 2], the tree of *Theory Figure 23.3*. Its only leaves are 4 and 5, and deleting either makes the two diameters agree, so a full pass keeps nothing and the loop ends (*Theory Lemma 23.4*).

**Exercise 23.5.** (1) True: 9/15 = 0.6 per minute against 8.75/50 = 0.175, and the brute force is also shorter and worth more, so it is never behind. (2) False: spent minutes are gone whatever you choose; only the minutes still needed enter the ratio. (3) False: `assert` runs only with the `-ea` flag, which you normally cannot set on a judge. (4) False:

Codeforces compares standard output only, although heavy debug output still costs time. (5) False: on correct tiers the answers are the same, and the brute force costs at most about $4.5 \cdot 10^6$ pair checks per test, far below the time limit. (6) False: under Plan B, Mock 3 is dropped, 28 November is the Day 29 rest routine and 29 November is the exam.

## Chapter 24 · Infosys Repeat Problems I

**Exercise 24.1.** (1) `System.in`; (2) `fs.nextInt()`; (3) `N + 1`; (4) `cnt[fs.nextInt()]++`. After reading, $N = 10$, $K = 2$, $M = 5$ and cnt[1..10] = [0, 1, 1, 0, 0, 1, 1, 0, 1, 0]. Nothing changes if the values arrive on one line: `next()` returns one token at a time and reads a new line only when the current one is used up, so the program prints 5 either way.

**Exercise 24.2.**

| power | bonus | e before | fight or stop | e after |
|---|---|---|---|---|
| 1 | 2 | 4 | fight | 6 |
| 4 | 0 | 6 | fight | 6 |
| 5 | 4 | 6 | fight | 10 |
| 6 | 1 | 10 | fight | 11 |
| 9 | 3 | 11 | fight | 14 |
| 13 | 2 | 14 | fight | 16 |
| 20 | 7 | 16 | stop | 16 |

The program prints 6. To beat all seven, the experience before the last fight, $e + 2 + 0 + 4 + 1 + 3 + 2 = e + 12$, must reach 20, so $e \geq 8$; the earlier fights need only $e \geq 3$. The smallest value is $e = 8$, and running the program with $e = 7$ and $e = 8$ gives 6 and 7.

**Exercise 24.3.**

| gap after house | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $P_i$ | 3 | −2 | 2 | 1 | 3 |
| $\lvert P_i \rvert$ | 3 | 2 | 2 | 1 | 3 |
| direction | ← | → | ← | ← | ← |

The total work is 3 + 2 + 2 + 1 + 3 = 11, and the brute force over all pairings of the 9 sold bottles also gives 11. A positive $P_i$ means houses 0..$i$ are short, so bottles come in from the right (←); at gap 1, houses 0 and 1 have 2 bottles to spare, so 2 bottles go right (→). House 5 is the last house, so there is no gap after it; its running sum would be the total, 0.

**Exercise 24.4.**

| $s \setminus v$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 12 |
| 2 | 0 | 1 | 1 | 2 | 1 | 3 | 1 | 3 | 2 | 3 | 1 | 5 | 23 |
| 3 | 0 | 0 | 0 | 1 | 0 | 2 | 0 | 3 | 1 | 2 | 0 | 7 | 16 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 3 | 4 |

C(4, 0..3) = 1, 4, 6, 4, so the count is $12 \cdot 1 + 23 \cdot 4 + 16 \cdot 6 + 4 \cdot 4 = 12 + 92 + 96 + 16 = 216$. *Theory Listing 24.7* prints 216, and a brute force that lists every valid array agrees; the count is below 10000, so the modulus changes nothing here. A chain with 4 values ending at $v$ needs three prime factors of $v$, counted with multiplicity (Chapter 14), and

the only such $v \le 12$ are $8 = 2 \cdot 2 \cdot 2$ and $12 = 2 \cdot 2 \cdot 3$. The three chains ending at 12 are 1, 2, 4, 12 and 1, 2, 6, 12 and 1, 3, 6, 12.

**Exercise 24.5.**

| bit $b$ | ones $c_b$ | keep[$b$] | flip[$b$] |
|---|---|---|---|
| 3 | 0 | 0 | 40 |
| 2 | 1 | 4 | 16 |
| 1 | 3 | 6 | 4 |
| 0 | 2 | 2 | 3 |

| candidate | $X$ in binary | $X$ | sum |
|---|---|---|---|
| $X = K$ | 1011 | 11 | 40 + 4 + 4 + 3 = 51 |
| branch at bit 3 | 0101 | 5 | 0 + 16 + 6 + 3 = 25 |
| branch at bit 1 | 1001 | 9 | 40 + 4 + 6 + 3 = 53 |
| branch at bit 0 | 1010 | 10 | 40 + 4 + 4 + 2 = 50 |

The answer is 53 at $X = 9$; trying every $X$ from 0 to 11 agrees. The free choice of every bit is $1101_2 = 13$ with sum 65, but $13 > K$, so it is not allowed. Bit 2 is the problem: $K$ has a 0 there, so a 1 at bit 2 is only possible after branching at bit 3, which costs the 40 of bit 3.

**Exercise 24.6.** The window charges 1 for a busy day, whatever its number of obligations. With $N = 5$, $K = 1$ and two obligations on day 3 it prints 5, but the answer is 2: one cancellation cannot free day 3. A stress test against the brute force of Theory Chapter 24 finds a failure on its first input, $N = 6$, $K = 1$ and days [4, 3, 5, 3], where it prints 3 instead of 2. The fix is to charge the count: `used += cnt[r];` and `while (used > K) used -= cnt[l++];`, as in *Theory Listing 24.5*.

**Exercise 24.7.** (1) False: $10000 = 2^4 \cdot 5^4$, and every factorial from 20! on is 0 modulo 10000; use Pascal's rule. (2) True: $3 \cdot 6667 = 20001 \equiv 1$, because gcd(3, 10000) = 1. (3) False: $e = 4$, powers [3, 1, 2], bonuses [2, 0, −3] gives 2 with the greedy and 3 with the best order. (4) True: that is *Theory Lemma 24.7*. (5) False: for $E = 30$ and drains 4, 9, 2, 7, smallest first needs 7 exercises (2, 2, 4, 4, 7, 7, 9) and largest first needs 4. (6) False: with the first half buying $10^4$ each and the second half selling $10^4$ each, the answer is 25,000,000,000,000. (7) True: `next()` skips line ends and reads a new line only when the current one has no tokens left.

**Exercise 24.8.** Compare with `stress()` in *Theory Listing 24.1*. The parts that must be there: a fixed seed (`new Random(1)`), so a failure can be repeated; a generator that makes only legal inputs, for example $n$ = 1 + `rnd.nextInt(7)` exercises, drains 1 + `rnd.nextInt(10)` (positive, as the statement requires) and $E$ = `rnd.nextInt(70) - 5`, which includes $E \le 0$ and values too large to drain; calls to both methods on the same input; and a print of the input and both answers at the first mismatch, followed by `return`. With these ranges the brute force tries at most $3^7 = 2187$ ways per input. The theory chapter's test used these ranges: 20,000 inputs with no mismatch, 6,672 of them with answer −1 and 1,674 with answer 0.

## Chapter 25 · Mock 4: The Official Infosys Sample Paper

**Exercise 25.1.** Factorisations and kernels: $72 = 2^3 \cdot 3^2 \to 2$; $50 = 2 \cdot 5^2 \to 2$; $98 = 2 \cdot 7^2 \to 2$; $45 = 3^2 \cdot 5 \to 5$; $20 = 2^2 \cdot 5 \to 5$; $8 = 2^3 \to 2$; $1 \to 1$; $49 = 7^2 \to 1$; 3 054 243 → 3 (the cofactor $1009^2$ is a square and vanishes); 1 032 247 → 1 032 247 (the cofactor $1013 \cdot 1019$ is not a square and stays); 4 128 988 → 1 032 247 ($2^2$ vanishes, the cofactor stays); 1 018 081 → 1. The groups are kernel 2: {72, 50, 98, 8}, kernel 5: {45, 20}, kernel 1: {1, 49, 1 018 081}, kernel 1 032 247: {1 032 247, 4 128 988}, kernel 3: {3 054 243}. The number of square products is C(4, 2) + C(2, 2) + C(3, 2) + C(2, 2) + 0 = 6 + 1 + 3 + 1 = 11, which a direct check of all 66 products confirms.

**Exercise 25.2.** The kernels of nodes 1 to 8 are 2, 5, 2, 5, 5, 3, 3, 2.

| child → parent | child's map | parent's map before | swap? | cross | pairs[parent] after | parent's map after |
|---|---|---|---|---|---|---|
| 6 → 3 | {2:1, 3:2} | {2:1} | yes | 1 | 2 | {2:2, 3:2} |
| 2 → 1 | {5:3} | {2:1} | no | 0 | 3 | {2:1, 5:3} |
| 3 → 1 | {2:2, 3:2} | {2:1, 5:3} | no | 2 | 7 | {2:3, 3:2, 5:3} |

beauty = 7, 3, 2, 0, 0, 1, 0, 0 for nodes 1 to 8, and the answer is 13; a brute force over all subtrees and pairs gives 13 too. The swap happens at 6 → 3: node 3's map {2:1} has one key and node 6's has two, so the parent's single entry is poured into the child's map, which becomes node 3's map. At 3 → 1 both maps have two keys, so there is no swap and the child's two entries move; key 2 meets 2 copies against 1, giving the cross count $1 \cdot 2 = 2$.

**Exercise 25.3.**

| update | array after | leaves t[4..7] | t[2] | t[3] | g | notDiv | rule | answer |
|---|---|---|---|---|---|---|---|---|
| (3, 45) | [18, 30, 45] | 18 30 45 0 | 6 | 45 | 3 | 0 | drop-one | NO |
| (1, 9) | [9, 30, 45] | 9 30 45 0 | 3 | 45 | 3 | 0 | drop-one | YES |
| (2, 7) | [9, 7, 45] | 9 0 45 0 | 9 | 45 | 9 | 1 | $g \neq p$ | NO |
| (3, 12) | [9, 7, 12] | 9 0 12 0 | 9 | 12 | 3 | 1 | `notDiv` > 0 | YES |

The output is 2, the same as the brute force over all proper subsets. The drop-one check runs twice. After (3, 45) it computes gcd(30, 45) = 15, gcd(18, 45) = 9 and gcd(18, 30) = 6, none equal to 3, so the answer is NO although the whole array has gcd 3. After (1, 9) it computes gcd(30, 45) = 15, gcd(9, 45) = 9 and gcd(9, 30) = 3, and stops with YES.

**Exercise 25.4.**

| element | cand[0] | cand[1] | cand[2] | cand[3] |
|---|---|---|---|---|
| A[2] = 1 | 2 | 1 | 0 | 0 |
| A[3] = 2 | 0 | 0 | 3 | 2 |
| A[4] = 2 | 4 | 3 | 3 | 2 |
| best | 4 | 3 | 3 | 2 |

The answer is 2, the best entry with XOR ≥ 3; the bitmask brute force agrees. One optimal subsequence is 1, 2 (indices 0 and 3, XOR 3). The longest non-decreasing subsequence 1, 1, 2, 2 has XOR 0 < 3, and the best lengths with XOR 1 and 2 (3 each, for example 1, 2, 2 and 1, 1, 2) are also below $M$. Only XOR 3 qualifies, and its best is 2.

**Exercise 25.5.** Easy: for every node, collect its subtree with an explicit stack, then for every pair test whether the `long` product is a perfect square, correcting `Math.sqrt` with two `while` loops; $O(\Sigma \text{ size}^2)$, about $n^3/3$ on a chain, so a chain of about 1,000 nodes or a random tree of about $10^4$ nodes. Medium: after each update, try every mask from 1 to $2^n - 2$, compute the gcd of the chosen values and stop at the first gcd equal to $p$; about $q \cdot 2^n \cdot n$ steps, so $n \approx 16$ with 100 updates. Hard: every mask from 1 to $2^N - 1$, check non-decreasing and the XOR; $O(2^N \cdot N)$, about $N = 24$ on a sorted array, and guard against `1 << n` with $n \geq 32$. Complex: backtrack over the edges (*Parent*[$v$], $v$), taking an edge only when both ends are unused; for each matching build every root-to-leaf string with the flipped values and test each pattern with `String.contains`; keep the largest count and, on ties, the fewest flips; $N \leq 15$. Compare your versions with *Theory Listing 25.1* and *Theory Listing 25.5*.

**Exercise 25.6.** (1) Input: run every sample before writing logic, and trust the samples over the format text. (2) Stack: on parent-array trees, build a BFS order and loop over it backwards, never recurse. (3) Overflow: compute products of values up to $10^9$ as `(long) a * b`. (4) Idea (the DP's evaluation order): compute every candidate for an element before writing any of them. (5) Edge: test $n = 1$, where the only shorter subsequence is empty. (6) Misread: "exactly $p$" excludes $2p$, so underline every "exactly", "at most" and "strictly" while reading. (7) Time: the Complex problem gets only the time left after the other three have submissions.

## Chapter 26 · The Repair Session and the Hard XOR Repeats

**Exercise 26.1.** No single answer: check your table against three tests. The old rule and the new rule differ, and the new one names a line of code or a check you can perform ("cast the first factor of every product of two `int`s to `long`", "print the parsed input to standard error and compare it with the first sample before writing the logic", "run $n = 1$, all values equal and the largest values before submitting"), not an intention ("be careful with overflow"). The "differences found" column names concrete lines of the template, such as a loop bound or the order of two statements, and says whether the reason sentence was right. Every re-solve was timed against the plan's limits (Easy 20, Medium 40, Hard 60 minutes). A failed re-solve is a new error-log entry with the re-solve date three days later, Day 29.

**Exercise 26.2.** (1) `Integer.MAX_VALUE / 2`; (2) `cur[0] = 0`; (3) `Math.min(cur[x], cur[x ^ v] + 1)`; (4) the swap `int[] t = cur; cur = next; next = t`; (5) `f[x] <= cap` (with `f` the returned array). With `Integer.MAX_VALUE`, the expression `cur[x ^ v] + 1` overflows to `Integer.MIN_VALUE` whenever $x \oplus v$ is unreachable, and that negative value wins the minimum (Exercise 26.5 shows the damage).

**Exercise 26.3.**

| | x = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| f0 | 0 | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| f1 (a = 3) | 0 | ∞ | ∞ | 1 | ∞ | ∞ | ∞ | ∞ |
| f2 (a = 5) | 0 | ∞ | ∞ | 1 | ∞ | 1 | 2 | ∞ |
| f3 (a = 6) | 0 | ∞ | ∞ | 1 | ∞ | 1 | 1 | ∞ |
| f4 (a = 1) | 0 | 1 | 2 | 1 | 2 | 1 | 1 | 2 |

The answer is 7 = 6 ⊕ 1, two elements. In row f3, 6 drops from 2 (the set {3, 5}) to 1 (the element 6 itself). The rank is 3: 5 and 3 enter the basis, 6 = 5 ⊕ 3 is rejected, and 1 enters. Since 3 > cap = 2, `solve` runs the knapsack branch, but the answer equals the basis maximum 7, because 7 happens to be reachable with two elements. A rank above the cap means only that the basis cannot *promise* the answer. `solve` returns 7, as does a brute force.

**Exercise 26.4.**

| bit | x_b | k_b | action | subtree taken: prefix, range of s | sums tested (mark primes) |
|---|---|---|---|---|---|
| 3 | 0 | 0 | XOR bit must be 0: go to child 0 (prefix 0) | none | none |
| 2 | 1 | 1 | take child 1, then go to child 0 (prefix 00) | 01: *s*[3..5], the values 4, 5, 6 | 11 prime, 12, 13 prime |
| 1 | 1 | 0 | XOR bit must be 0: go to child 1 (prefix 001) | none | none |
| 0 | 1 | 1 | take child 1, then go to child 0 (prefix 0010) | 0011: *s*[2..2], the value 3 | 10 |

The query finds 4 candidates with XOR below 5 (7 ⊕ 4 = 3, 7 ⊕ 5 = 2, 7 ⊕ 6 = 1, 7 ⊕ 3 = 4) and 2 prime sums, so the count goes from 3 to 5. The values 1 and 2 are never looked at: 7 ⊕ 1 = 6 and 7 ⊕ 2 = 5 are not below 5, and the walk stops in the leaf of 2 without taking it. The query for 12 adds nothing, so the program prints 5: the pairs (4, 7), (1, 2), (6, 7), (6, 5) and (3, 2), with sums 11, 3, 13, 11 and 5.

**Exercise 26.5.** After the first pass ($v = 1$) the row is [−2147483648, 1, −2147483648, −2147483648]: for $x = 0$, 2 and 3 the term `cur[x ^ v] + 1` is `Integer.MAX_VALUE + 1`, which wraps to `Integer.MIN_VALUE`, and the minimum takes it. Even $x = 0$ is overwritten. Every negative entry passes `cur[x] <= cap`, so the method returns 3 instead of 2. Fix: fill the array with `Integer.MAX_VALUE / 2` instead. On 5,000 random inputs with even $N \le 12$ the buggy version was wrong 663 times.

**Exercise 26.6.** State: $g[k][x]$ = the number of subsets of the elements seen so far with exactly $k$ elements and XOR $x$, starting from $g[0][0] = 1$. Transition for a new element $v$: $g[k][x] \mathrel{+}= g[k-1][x \oplus v]$ for every $k \ge 1$ and every $x$, modulo $10^9 + 7$ (take $v$ or not). In place, loop $k$ downwards from $c$ to 1: row $k$ reads only row $k - 1$, which has not been updated yet in this pass, exactly as the capacity loop of Chapter 11 runs downwards. The order of $x$ does not

matter, because each write goes to row $k$ and each read comes from row $k - 1$. Answer: $\Sigma$ over $k \le c$ of $g[k][T]$. Cost: $N \cdot c \cdot 2^{10} \le 40 \cdot 40 \cdot 1024 = 1{,}638{,}400$ updates and $(c + 1) \cdot 2^{10}$ numbers. For [1, 2, 3, 1] and $T = 0$: with $c = 2$ the answer is 2 (the empty set and the two 1s); with $c = 3$ it is 4 (adding {1, 2, 3} with either copy of 1). This design matched a brute force over all subsets on 3,000 random cases.

**Exercise 26.7.** (1) True: the rank is at most $20 \le N/2$, so by *Theory Theorem 26.2* the basis maximum is reachable within the cap. (2) False: for [3, 5, 6, 1] the rank is $3 > 2$, yet the answer is the basis maximum 7 (Exercise 26.3). (3) False: two rows of $2^{20}$ `int`s, 8 MB, are enough. (4) True: the values below a node share a prefix, so they lie in an interval, and in sorted order they are consecutive (*Theory Lemma 26.3*). (5) False: then every pair has XOR below $K$, the trie lists all $n(n - 1)/2$ pairs and adds its own overhead. (6) False: the sums go up to $2 \cdot \max(A)$, so the sieve must too. (7) True: a path of 15 vertices has 987 matchings, a star 15, and 10,000 random trees with 15 vertices had at most 923.

## Chapter 27 · The Exam Playbook and Mock 5

**Exercise 27.1.** There is no single right card; it must be yours. A model to compare with:

*Time.* 0–15 read all, notes, rank by sureness. 15–60 surest. 60–120 second (minute 100: brute force and move on). 120–165 hard: brute force if no idea in 15 minutes, then optimise; with four problems, both brute forces first, both submitted by 150, then optimise one. 165–180 no new code, check every submission compiles.

*Rules.* Start the surest problem at 15. No idea after 15 minutes: fallback submitted. Wrong answer and no new explanation for 15 minutes: stress test for 10, then move on. Minute 100: Q2 finished or fallback in. Minute 150: everything has a submission. A passing solution is never touched again. Minute 165: no new code.

*Checklist.* Format, samples, edges ($n = 1$, equal, max), `long` and mod, input order as in the samples and stub unchanged, test-case resets, recursion depth, flush and no debug prints, right version.

*Personal rules* (example from a log with overflow, misread and off-by-one entries): cast the first factor of every product to `long`; write the output format of the first sample before coding; test $n = 1$ before every submission.

**Exercise 27.2.** Four items fail.

- Item 5, input: the program reads all $n$ values from one line, but the samples put each value on its own line. On the input `2 / 3 / 1 / 1 / 1 / 2 / 1 / 1` (one number per line) the second `st.nextToken()` throws `NoSuchElementException`. Fix: read with FastScanner, whose `next()` ignores line breaks.
- Item 8, output plumbing: there is no `out.flush()`, so the program prints nothing at all, even on the sample. Fix: `out.flush();` after the loop.
- Item 6, many test cases: `seen` is static and never cleared, so the second test case sees the first one's counts. With the input and flush fixed, the sample above prints 3 and 7 instead of 3 and 1. Fix: create the map inside the `while` loop (its cost is proportional to the values of that test case only).
- Item 4, types: `pairs` is an `int`. For $n = 2 \cdot 10^5$ equal values the answer is $2 \cdot 10^5 \cdot (2 \cdot 10^5 - 1)/2 = 19{,}999{,}900{,}000$, and the program prints −1,474,936,480. Fix: `long pairs`.

With the four fixes the method agrees with an $O(n^2)$ brute force on 5,000 random arrays. Items 1–3, 7 and 9 pass: the format is one number per line, there is no recursion, and there is only one version.

**Exercise 27.3.**

1. Start Q3: it is the problem you are surest of, and the rule at minute 15 says "whatever its number" (row 1 of the rules; the 0–15 row of the time table says to rank by sureness, not labels).
2. Stress-test against the brute force you already have; the all-equal case is small enough to compare by hand too. The rule is "a wrong answer and no new explanation": you have spent less than 15 minutes on this bug, so keep at it, using the oracle, until minute 60 at most.
3. Write and submit Q2's brute force now, then move to Q3 (row "minute 100"). Writing it takes 10 minutes or so, and it is still the cheapest marks on Q2.

4. The four-problem plan: brute forces for Q3 and Q4 first, about 15 minutes each, both submitted by minute 150, then optimise the one with the clearest next step, which is probably Q3. Starting the 40-minute Q3 idea now would leave Q4 blank at minute 150.
5. Keep working on the full solution until 165: the minute-150 rule is met because every problem has a submission. If the full solution does not pass your tests by 165, the brute force stays as the submission.
6. Do nothing. Q2 passes, so it is not touched again, and after minute 165 no new code is written.

**Exercise 27.4.**

| outcome | weighted score | ≥ 65? |
|---|---|---|
| (a) | 20 + 30 + 10 = 60 | no |
| (b) | 20 + 18 + 20 = 58 | no |
| (c) | 20 + 30 + 20 = 70 | yes |
| (d) | 10 + 30 + 25 = 65 | yes, exactly |

Part 2: in (a), Q3 needed (65 − 50)/50 = 30 % of its tests. Part 3: brute force first loses when $T - b < f \le T$, that is $25 < f \le 40$, and loses $F - \beta = 35$ marks there. The marks for (plan F, plan B): $f = 20$: (50, 50); 25: (50, 50); 30: (50, 15); 40: (50, 15); 41: (0, 15); 60: (0, 15).

**Exercise 27.5.** A model row. *Limits:* $n, q \le 10^5$, prices and budgets $\le 10^9$ (too large to index an array), counts $\le 10^5$ fit an `int`. *Idea:* sort the prices once; for each budget the answer is the upper bound of the budget, the first index whose price is larger (Chapter 6). *Complexity:* $O(n \log n + q \log n)$, about $3.4 \cdot 10^6$ comparisons. *Brute force:* for each customer, count the prices ≤ budget in a loop, $O(nq)$, which finishes for $n \cdot q$ up to about $10^7$, for example $n = q =$ 3,000. *Edges:* a budget below every price (answer 0) or above every price (answer $n$), repeated prices equal to the budget (upper bound, not lower bound), $n = 1$. *Sure:* 5 of 5, and a good candidate for the first problem.

**Exercise 27.6.**

1. Misread statement. Rule: before coding, write the output of the first sample next to your own trace of it, including whether positions start at 0 or 1.
2. Stack overflow. Rule: any traversal of a tree or graph with more than $10^4$ vertices uses a BFS order or an explicit stack, never recursion.
3. Ran out of time. Rule: minute 100, Q2 finished or its brute force submitted; minute 150, every problem has a submission.
4. Overflow. Rule: every sum and product is `long`, and every solution is run once on the maximum size with maximum values before submitting.
5. Wrong idea. Rule: a greedy is either proved by an exchange argument or stress-tested against a brute force before it is submitted.
6. Input reading. Rule: check the reading order against every sample before writing the logic, and trust the samples over the format text (the plan's Lesson 2).

## Chapter 28 · Consolidation: Recognising Patterns and Recalling Templates

**Exercise 28.1.** Compare with the exam file of Theory Chapter 27 and the FastScanner of Theory Chapter 1. The items that cost marks when wrong are the checklist's: a missing `out.flush()` prints nothing at all; a tokenizer refilled with `if` instead of `while` fails on a blank line; `next()` without the `null` check throws at the end of the input; a missing `throws IOException` does not compile. If the file took more than about five minutes, write it once more before you go to bed and once in the warm-up on the exam morning.

**Exercise 28.2.** There is no single right table. The plan's target is 40 minutes for the 15 templates, 160 seconds each on average. Every row marked "wrong line" or "missing line" in a template of *Theory Table 28.5*, and every template that took more than five minutes, goes on the one-page sheet with the detail that went wrong.

**Exercise 28.3.**

1. Previous strictly smaller element: monotonic stack (Chapter 7), O($n$). Signal: "most recent earlier day with a lower price".
2. Prefix XOR with a hash map of earlier prefix values (Chapters 2 and 8): a block XORs to zero exactly when its two end prefixes are equal. O($n$) expected. Signal: "contiguous blocks" plus "XOR".
3. Bitmask DP over (set visited, last sight) (Chapter 22): $2^{14} \cdot 14 \cdot 14 = 3{,}211{,}264$ steps. Signal: $n \le 14$ and "visit every sight once".
4. BFS order from the parent array, walked backwards to add each salary to the parent (Chapter 18), O($n$), no recursion. Signal: a parent array and "everyone below".
5. Binary search on the answer $T$ (Chapter 6): the check $\Sigma \lfloor T/t[i] \rfloor \ge m$ is monotone in $T$. The answer is at most $m \cdot \min t \le 10^{18}$, so about 60 checks of O($n$), $6 \cdot 10^6$ steps. Use `long`, and stop adding once the sum reaches $m$: $10^5$ terms of up to $10^{18}$ would overflow even a `long`. Signal: "how soon" with a monotone check.
6. Dijkstra (Chapter 17): non-negative weights, about $(n + m) \log n \approx 5.1 \cdot 10^6$ steps; distances up to $10^5 \cdot 10^9$ need `long`.
7. Lazy segment tree with range add and range maximum (Chapter 21): O($(n + q) \log n$) $\approx 3.4 \cdot 10^6$ steps. Signal: range updates mixed with range queries.
8. Digit DP (Chapter 22) with the state (position, last digit or "not started", tight), answering count($R$) − count($L$ − 1): about $19 \cdot 11 \cdot 2 \cdot 10 = 4{,}180$ steps. Signal: $R \le 10^{18}$ and a digit property.
9. Interval DP (Chapter 14): the last merge splits the row into two parts, so cost[$l$][$r$] = min over the split $k$ of cost[$l$][$k$] + cost[$k$ + 1][$r$] + sum($l$..$r$). O($n^3$) = $2.7 \cdot 10^7$. Signal: $n \le 300$ and "merge neighbours".
10. Two-string DP (Chapter 12): the answer is $|s| + |t| - 2 \cdot \mathrm{LCS}(s, t)$, an O($nm$) = $9 \cdot 10^6$ table. Signal: two strings and "turn one into the other".
11. Union-find (Chapter 17): start with $n$ groups and subtract one for every union that joins two different groups. Nearly O(1) per event.
12. Small-to-large merging of colour sets (Chapter 19): each colour moves O(log $n$) times, O($n \log n$) expected in total, over a BFS order without recursion. Signal: "for every node … in its subtree" with a set-valued answer.

**Exercise 28.4.**

| | Target | Steps at the limit | Family |
|---|---|---|---|
| 1 | O($2^n \cdot n$) | $16 \cdot 2^{16} = 1{,}048{,}576$ | subsets, bitmask DP (Chapters 9, 22) |
| 2 | O($n^3$) | $400^3 = 6.4 \cdot 10^7$ | interval DP (Chapter 14) |
| 3 | O($nm$) | $3000 \cdot 3000 = 9 \cdot 10^6$ | two-string DP (Chapter 12) |
| 4 | O($(n + q) \log n$) | $2 \cdot 10^5 \cdot 17 = 3.4 \cdot 10^6$ | Fenwick or segment tree (Chapter 20) |
| 5 | O($\Sigma\, n \log n$) | $2 \cdot 10^5 \cdot 18 = 3.6 \cdot 10^6$ | the band of the sum: sorting, heaps, binary search |
| 6 | O($\sqrt{N}$) | $\sqrt{10^{12}} = 10^6$ | divisors, trial division (Chapter 10) |
| 7 | O($n! \cdot n$) | $9! \cdot 9 = 3{,}265{,}920$ | all orders (Chapters 9, 22) |
| 8 | O($n^2$) | $2000^2 = 4 \cdot 10^6$ | all pairs, 2-D DP (Chapters 11, 12) |

Line 4's brute force costs $n \cdot q = 10^{10}$ steps, far too slow. In line 5, clearing $2 \cdot 10^5$ entries in each of $10^4$ test cases is $2 \cdot 10^9$ writes, a time-limit failure even though $\Sigma\, n$ is small.

**Exercise 28.5.** *Part 1.* Sorted, the weights are [1, 2, 3, 3, 4, 5, 6].

| boat | i | j | p[i] + p[j] | boat holds |
|---|---|---|---|---|
| 1 | 0 | 6 | 7 | {6} |
| 2 | 0 | 5 | 6 | {1, 5} |
| 3 | 1 | 4 | 6 | {2, 4} |

| boat | i | j | p[i] + p[j] | boat holds |
|---|---|---|---|---|
| 4 | 2 | 3 | 6 | {3, 3} |

Four boats; the fifth row stays empty because the loop ends with $i = 3 > j = 2$. Both lower bounds are 4: the total weight is 24 and $\lceil 24/6 \rceil = 4$, and $\lceil 7/2 \rceil = 4$. So the answer is optimal, and the brute force agrees.

*Part 2.* [1, 1, 2, 2] with *limit* = 3: the greedy pairs {1, 1}, then 2 + 2 > 3 leaves {2} and {2}, three boats; {1, 2} and {1, 2} need only two. A search over every input with at most three people and *limit* ≤ 8 finds no failure, so the smallest failures have four people; on 10,000 random inputs with up to nine people the greedy is wrong on 2,694. The cause is a **wrong idea**: pairing the two lightest people spends a light person who could have carried a heavy one, the opposite of the exchange argument in *Theory Theorem 28.1*. Rule: a greedy is either proved by an exchange argument or stress-tested against a brute force before it is submitted.

**Exercise 28.6.** Three causes appear twice: overflow / mod (entries 1 and 8), off-by-one (entries 3 and 4) and wrong idea (entries 6 and 7). The scores are 3, 3, 2, 5, 0, 5, 3 and 4. Entry 7 (LC 787) comes first because it is due on Day 29. The highest of the rest are entries 4 (LC 315, logged on Day 20) and 6 (LC 2447, Day 10), both with 5, in that order because entry 4 is more recent, and then entry 8 (LC 2338) with 4. The four re-solves are LC 787, LC 315, LC 2447 and LC 2338.

**Exercise 28.7.** A complete page has the four parts of *Theory Section 28.7*: the detail lines of every template rated *Again* or *Shaky* today, with chapter numbers (for example "Fenwick: `i += i & -i` to add, `i -= i & -i` to sum, 1-indexed, Chapter 20"); the shape of any longer template rated *Again*; one rule per error-log entry, with today's failed re-solves at the top; and a note that the playbook card and the map's signal columns are to be read. If a speed drill overran, write the step that overran and the minute it ended.

## Chapter 29 · Rest and Logistics: The Day Before the Exam

**Exercise 29.1.** The plan's sources state two rows: phones, calculators and smartwatches were banned at past in-person rounds, so plan to hand them in (**S**, though the instructions on the day say where they go), and the languages offered are C, C++, Java, Python and JavaScript (**S**). Every other row is **I**. The plan is built for 6 December, but only the invite confirms the date; the reporting time, the venue, what to bring, identity documents, a watch, scratch paper, food and water are not stated in the sources at all. The answer column is yours.

**Exercise 29.2.** *Part 1.* Arrival 07:50 (08:30 − 40 minutes), leaving 06:45 (07:50 − 65 minutes), waking 05:25 (06:45 − 80 minutes), in bed by 21:25 for eight hours or 22:25 for seven, and no problems or screens from 20:25. *Part 2* is yours; check it with the same three subtractions.

**Exercise 29.3.** Compare with *Theory Listing 29.1* and check four things: it fits on one side; the top has the playbook times (read until 15, the surest problem until 60, the second until 120 with a brute force at 100, the hard problem until 165, every problem submitted by 150, checks only after 165) and three personal rules; the bottom has only templates you rated *Again* or *Shaky*, each cut to its deciding lines with a chapter number; and nothing on it is new to you.

**Exercise 29.4.**

1. Sliding window with letter counts (Chapter 3), $O(n)$: the condition "at most two distinct letters" stays true when the window shrinks.
2. Topological order by Kahn's algorithm (Chapter 15), $O(n + m)$: if fewer than $n$ steps come out, there is a cycle and no order exists.
3. Subset enumeration (Chapters 9 and 22): with $n \le 20$ there are $2^{20} = 1{,}048{,}576$ splits, and weights up to $10^9$ rule out a knapsack over sums.
4. Lowest common ancestors by binary lifting (Chapter 18): the distance is $\text{depth}(u) + \text{depth}(v) - 2 \cdot \text{depth}(\text{lca}(u, v))$, $O(\log n)$ per query.
5. Sort the start times and the end times; the answer for moment $t$ is the number of starts $\le t$ minus the number of ends $< t$, two binary searches per moment (Chapter 6). The times are too large for a difference array indexed by time (Chapter 2).

6. Loops over multiples with inclusion–exclusion on the gcd (Chapter 10): count the pairs whose gcd is a multiple of each $d$ from the number of values divisible by $d$, then subtract the counts for the multiples of $d$, from the largest $d$ down. One pass over all multiples up to $10^6$ is $\Sigma \lfloor 10^6/d \rfloor = 13{,}970{,}034$ steps, and the method needs two.

**Exercise 29.5.** (b), (c), then (d) and (e) together, problem by problem, then (f), (a), (g), (h). Actions (b) and (c) come before minute 0; (d), (e) and (f) are minutes 0–15 of the playbook; (a), (g) and (h) start the surest problem.

APPENDIX C

# Error log

Every mistake goes here with the rule that would have prevented it. Causes: misread statement, wrong idea, overflow / mod, off-by-one, TLE / complexity, input reading, edge case, stack overflow, ran out of time. Re-solve each entry three days later.

| Date | Problem | Topic | Cause | Rule that prevents it | Re-solve |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

| Date | Problem | Topic | Cause | Rule that prevents it | Re-solve |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

| Date | Problem | Topic | Cause | Rule that prevents it | Re-solve |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

APPENDIX D

# Mock scores and SP checkpoints

Copy each mock's result here so the trend is visible on one page. The checkpoints are estimates of what is on track for the Specialist Programmer role, not Infosys's published cut-offs (it publishes none).

| Mock | Day | On track if | Q1 % | Q2 % | Q3 % | Q4 % | Score | Weakest topic |
|---|---|---|---|---|---|---|---|---|
| Mock 1 · Phase 1 topics | 9 | Q1 full; Q2 ≥ half | | | | | | |
| Mock 2 · number theory and DP added | 16 | Q1+Q2 full; ≥ 50 | | | | | | |
| Mock 3 · four problems | 23 | ≥ 60; brute force on all | | | | | | |
| Mock 4 · official Infosys sample paper | 25 | grade, then walk-throughs | | | | | | |
| Mock 5 · dress rehearsal | 27 | ≥ 65 | | | | | | |

| If you are below the checkpoint after… | Do this |
|---|---|
| Mock 1 (Day 9) | Skip bonus problems in week 2 and redo the core problems of your weakest day. |
| Mock 2 (Day 16) | Same, and spend the review block on DP problems from Days 11–14. |
| Mock 3 (Day 23) | Days 24–26: repeats and error-log repair only, no new topics. |
| Mock 5 (Day 27) | On exam day, lock in Q1 and Q2 first and use partial marks on Q3. |

APPENDIX E

# Notes and scratch paper